

Cardiac Computational Modeling

From Ion Channels to Tissue Solvers

A Visual Textbook for Engine V5.4

Cardiac Computational Modeling Research

February 2026

Contents

PART I SINGLE CELL DYNAMICS

1. The Hodgkin–Huxley Revolution
 - 1.1 The Membrane as a Circuit
 - 1.2 Gating Variables: m , h , and n
 - 1.3 The Sodium Hypothesis and Positive Feedback
 - 1.4 Activation vs. Inactivation: The Refractory Period
2. From Neurons to Heart Cells
 - 2.1 Why the Heart Needs a Longer Action Potential
 - 2.2 The L-type Calcium Current and the Plateau
 - 2.3 Minimal Cardiac Models: FitzHugh–Nagumo to Fenton–Karma
3. Anatomy of the Cardiac Action Potential
 - 3.1 Phase 0: The Sodium Upstroke
 - 3.2 Phase 1: Early Repolarization (I_{to})
 - 3.3 Phase 2: The Plateau — A Delicate Balance
 - 3.4 Phase 3: Repolarization — The Potassium Takeover
 - 3.5 Phase 4: Rest and the Role of I_{K1}
4. Intracellular Calcium: The Master Regulator
 - 4.1 The Dyadic Cleft — Where Electrical Meets Mechanical
 - 4.2 Calcium-Induced Calcium Release — Local Control Theory
 - 4.3 The Ryanodine Receptor — Gatekeeper of the SR
 - 4.4 SERCA2a and Phospholamban — The Pump and Its Brake
 - 4.5 The Sodium-Calcium Exchanger — Dual-Mode Transport
 - 4.6 Calcium Buffering — The 1% Rule
 - 4.7 T-Tubule Remodeling — When Structure Fails
 - 4.8 Emerging Frontiers: Imaging, Modeling, and Therapy
5. The ten Tusscher–Panfilov 2006 Model
 - 5.1 18 State Variables: What and Why
 - 5.2 The Complete Current Inventory
 - 5.3 Voltage-Gated Depolarizing Currents
 - 5.4 Voltage-Gated Repolarizing Currents
 - 5.5 Pumps, Exchangers, and Background Currents
 - 5.6 Currents During Each AP Phase — A Visual Map
 - 5.7 Endo, Epi, and M-Cells: Transmural Heterogeneity
 - 5.8 Calcium Handling and CICR
6. The O'Hara–Rudy 2011 Model
 - 6.1 41 State Variables: What and Why
 - 6.2 The Complete Current Inventory
 - 6.3 Voltage-Gated Depolarizing Currents
 - 6.4 Voltage-Gated Repolarizing Currents
 - 6.5 Pumps, Exchangers, and Background Currents

- 6.6 The CaMKII Signaling Pathway
- 6.7 Currents During Each AP Phase — A Visual Map
- 6.8 Endo, Epi, and M-Cells: Transmural Heterogeneity
- 6.9 Calcium Handling and CICR
- 6.10 Comparison: ORd vs. TTP06

PART II TISSUE-LEVEL MONODOMAIN MODELING

1. The Monodomain Equation

- 1.1 The Cable Equation — One Cell in a Row
- 1.2 From Cable to Tissue: The Monodomain PDE
- 1.3 What the Conductivity Tensor Encodes
- 1.4 The Operator Splitting Philosophy
- 1.5 Semi-Discrete Form: $M \cdot dV/dt = -K \cdot V + F$
- 1.6 Spatial Discretization — FDM and FEM
- 1.7 Boundary Conditions
- 1.8 Choosing Your Method: A Decision Framework
- 1.9 Summary and Bridge to Chapters 8–10

2. Operator Splitting: Divide and Conquer

- 2.1 Godunov (First-Order) Splitting
- 2.2 Strang (Second-Order) Splitting
- 2.3 The Rush–Larsen Method for Ionic ODEs
- 2.4 Putting It All Together

3. Explicit Diffusion Solvers

- 3.1 Forward Euler and the CFL Condition
- 3.2 RK2: The Midpoint Method
- 3.3 RK2: Discrete Matrix Form
- 3.4 RK4: The Workhorse
- 3.5 RK4: Discrete Matrix Form
- 3.6 Stability Regions: A Visual Guide
- 3.7 When to Use Explicit Methods

4. Implicit Diffusion Solvers

- 4.1 Why Go Implicit?
- 4.2 BDF1 (Backward Euler): The Simplest Implicit Method
- 4.3 Crank–Nicolson: The Sweet Spot
- 4.4 BDF2: Higher-Order Implicit
- 4.5 Solving the Linear System: PCG and Preconditioners
- 4.6 When to Use Implicit Methods

PART III THE BIDOMAIN EXTENSION

1. Why Bidomain? — The Physical Picture

- 1.1 Two Domains, One Membrane
- 1.2 When Monodomain Fails
- 1.3 The Bidomain Advantage

2. The Bidomain Equations

- 2.1 Building the Coupled System

- 2.2 The Parabolic–Elliptic Couple
- 2.3 When Bidomain Reduces to Monodomain
- 3. Spatial Discretization for Bidomain
 - 3.1 From One Stiffness Matrix to Two
 - 3.2 The Bidomain Weak Form
 - 3.3 The 2×2 Block System
- 4. Bidomain Time Integration
 - 4.1 Operator Splitting Applied to Bidomain
 - 4.2 Time Steppers: Crank–Nicolson vs. SDIRK2
 - 4.3 IMEX Methods
 - 4.4 Why Fully Explicit Methods Fail
 - 4.5 Explicit Stabilized Methods (RKC)
- 5. Linear Solvers for the Block System
 - 5.1 The Saddle-Point Structure
 - 5.2 Block Preconditioners and AMG
 - 5.3 Krylov Methods and the Null-Space Problem
- 6. Implementation Roadmap for Engine V5.4
 - 6.1 Architecture Extensions
 - 6.2 Recommended Solver Stack
 - 6.3 Performance Expectations

PART IV LATTICE-BOLTZMANN METHODS

- 1. Why LBM? — Discovering a New Paradigm Through the Heat Equation
 - 1.1 The Heat Equation Revisited: A Hidden Redistribution
 - 1.2 From Weighted Averages to Distribution Functions
 - 1.3 The Direction Matrix: Lattice Geometry
 - 1.4 Collision: The Art of Local Relaxation
 - 1.5 Averaging and Flux Conservation
 - 1.6 The Eigen-Angle Matrix: Anisotropic Diffusion
 - 1.7 The Transformation Matrix M : From Distributions to Moments
 - 1.8 The Chapman–Enskog Bridge: From Lattice to PDE
 - 1.9 Boundary Conditions: Bounce-Back
 - 1.10 From Heat Equation to Cardiac Diffusion
- 2. Lattice-Boltzmann Methods for Monodomain
 - 2.1 From PDEs to Particle Distributions
 - 2.2 The D2Q5 and D3Q7 Lattices
 - 2.3 BGK Collision and Streaming
 - 2.4 MRT Collision: Anisotropic Diffusion
 - 2.5 Bounce-Back Boundary Conditions
 - 2.6 The Complete LBM-EP Timestep
 - 2.7 LBM vs. Classical Methods
 - 2.8 GPU Acceleration
- 3. Lattice-Boltzmann Methods for Bidomain
 - 3.1 The Bidomain Challenge: One Parabolic + One Elliptic

- 3.2 Strategy 1: Pseudo-Time Relaxation
- 3.3 Strategy 2: Hybrid LBM–Classical Solver
- 3.4 The Dual-Lattice Architecture
- 3.5 Performance and Implementation Outlook

Appendices

1. A. A Visual Guide to Differential Equations

PART I

Single Cell Dynamics

*From the squid giant axon to the human ventricular myocyte — how ion channels
generate the electrical signals that drive every heartbeat.*

The Hodgkin–Huxley Revolution

In 1952, Alan Hodgkin and Andrew Huxley did something no one thought possible: they wrote down a mathematical model of the nerve impulse that was so precise it predicted experiments that hadn't been done yet. Every cardiac cell model used today — from the simplest to the most complex — is a direct descendant of their work.

1.1 The Membrane as a Circuit

A cell membrane is, at its core, a thin lipid bilayer — roughly 7 nm of oily molecules separating two conducting salt solutions. This structure has a remarkable electrical consequence: the membrane acts as a *capacitor*. Like two metal plates separated by an insulator, the membrane stores charge — positive ions lining up on one side, negative on the other.

But the membrane is not a perfect insulator. Embedded within it are *ion channels* — protein pores that allow specific ions (Na^+ , K^+ , Ca^{2+} , Cl^-) to flow through when open. Each channel type acts like a variable resistor: its conductance changes with voltage, time, and chemical signals. At any instant, the total current flowing across the membrane is the sum of the capacitive current (charging the membrane) and the ionic current (flowing through channels):

$$I_m = C_m \frac{dV_m}{dt} + I_{\text{ion}}(V_m, t) \quad (1.1)$$

Here $C_m \approx 1 \mu\text{F}/\text{cm}^2$ is the specific membrane capacitance (remarkably constant across all cell types), V_m is the transmembrane potential, and I_{ion} is the total ionic current — the quantity that Hodgkin and Huxley set out to understand.

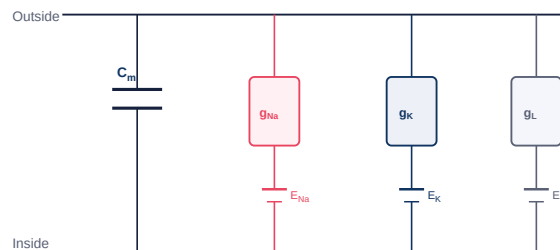


Figure 1.1. The Hodgkin–Huxley equivalent circuit. The membrane capacitance C_m sits in parallel with voltage-dependent conductances for Na^+ , K^+ , and a leak current. Each ionic branch has a variable conductance g_x in series with a Nernst battery E_x .

The key insight of Figure 1.1 is that each ionic species contributes its own current $I_x = g_x(V_m - E_x)$, where E_x is the *reversal potential* — the voltage at which the electrochemical gradient for that ion vanishes. For sodium, $E_{\text{Na}} \approx +65$ mV; for potassium, $E_K \approx -88$ mV. The enormous 150 mV difference between these two batteries is the energy source that powers every action potential.

Intuition

Think of each ion channel as a door with a bouncer who checks the voltage. At rest ($V_m \approx -85$ mV), the Na^+ doors are nearly all shut and the K^+ doors are mostly open. The cell sits quietly at a voltage close to E_K . But if something pushes V_m above about -55 mV, the Na^+ doors suddenly swing open — and the positive feedback loop that creates an action potential begins.

1.2 Gating Variables: m, h, and n

Hodgkin and Huxley's genius was not the circuit diagram — that was known. It was their decision to treat each conductance as a product of independent "gates," each opening and closing with its own voltage-dependent kinetics. For the squid axon, they found:

$$g_{Na} = \bar{g}_{Na} \cdot m^3 \cdot h, \quad g_K = \bar{g}_K \cdot n^4$$

Here $\bar{g}_{Na}$ and $\bar{g}_K$ are the *maximal conductances* — what you'd measure if every channel were fully open. The gating variables m , h , and n each range between 0 (all gates closed) and 1 (all gates open). They obey first-order kinetics:

$$\frac{dx}{dt} = \alpha_x(V_m)(1-x) - \beta_x(V_m)x, \quad x \in \{m, h, n\} \quad (1.2)$$

where α_x and β_x are voltage-dependent rate constants that Hodgkin and Huxley measured experimentally from the squid giant axon. This equation says something beautifully simple: each gate transitions between a closed state (population $1-x$) and an open state (population x), with forward rate α_x and backward rate β_x .

It is often more revealing to rewrite equation (1.2) in *relaxation form*:

$$\frac{dx}{dt} = \frac{x_\infty(V_m) - x}{\tau_x(V_m)} \quad (1.3)$$

where $x_\infty = \alpha_x/(\alpha_x + \beta_x)$ is the *steady-state* value (where the gate would end up if voltage were held constant forever) and $\tau_x = 1/(\alpha_x + \beta_x)$ is the *time constant* (how fast it gets there). This form makes the physics transparent: the gate exponentially approaches its target $x_\infty(V_m)$ with time constant $\tau_x(V_m)$.

△ Key Insight

The relaxation form (1.3) is not just prettier notation — it is the foundation of the **Rush–Larsen method**, the standard numerical integrator for gating variables in cardiac simulations. We will meet it again in Chapter 8 when we discuss operator splitting.

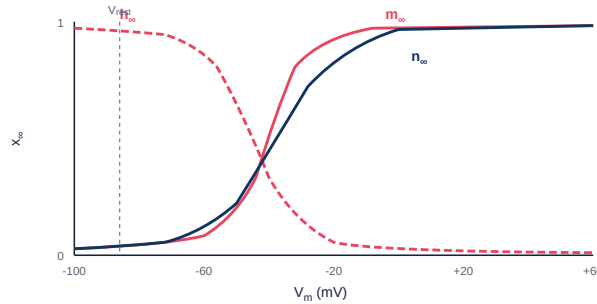


Figure 1.2. Steady-state gating curves for the HH model. At rest, $m_\infty \approx 0$ (Na^+ activation off), $h_\infty \approx 0.6$ (Na^+ inactivation partially available), and $n_\infty \approx 0.3$ (K^+ partially open). The Na^+ activation curve m_∞ is notably steeper than n_∞ , explaining why sodium channels open faster than potassium channels — the key to the action potential upstroke.

What do these three gates mean physically? The variable m is the sodium *activation* gate: it controls how quickly Na^+ channels open when the membrane depolarizes. The variable h is the sodium *inactivation* gate: it shuts Na^+ channels back down, even if the voltage stays high. The variable n is the potassium *activation* gate: it opens K^+ channels slowly during depolarization to repolarize the cell.

The powers (m^3 and n^4) reflect the cooperativity of channel gating — Hodgkin and Huxley interpreted them as the number of independent “particles” that must simultaneously be in the right position for the channel to conduct. Modern structural biology has confirmed this: voltage-gated channels have four voltage-sensing domains, and the sodium channel’s fast gate involves three of them.

1.3 The Sodium Hypothesis and Positive Feedback

The action potential is a self-amplifying event, and understanding *why* it fires requires tracing a feedback loop. Suppose a small stimulus pushes V_m slightly above the threshold of about -55 mV. Here is what happens:

Step 1: The small depolarization causes m_∞ to increase (the m_∞ curve in Figure 1.2 is steeply rising near threshold). Gates start opening: m rises toward m_∞ .

Step 2: As m increases, the sodium conductance $g_{Na} = \bar{g}_{Na} m^3 h$ grows rapidly. Na^+ ions rush into the cell (driven by the large electrochemical gradient $V_m - E_{Na} \approx -55 - 65 = -120$ mV).

Step 3: The inward Na^+ current further depolarizes the membrane, which causes even more m -gates to open — and the process accelerates explosively.

Intuition

This is *positive feedback*: depolarization opens Na^+ gates, which cause more depolarization, which opens more gates. It is the biological equivalent of a spark igniting gasoline. Once started, the upstroke is essentially unstoppable — V_m shoots from -85 mV toward $E_{Na} \approx +65$ mV in less than 1 ms. This is the “all-or-nothing” nature of the action potential.

What stops this runaway? Two mechanisms, operating on different timescales:

Fast shutdown — inactivation (h): While m opens in under 0.5 ms, the inactivation gate h closes with a time constant of 1–5 ms. As $h \rightarrow 0$, the product $m^3 h \rightarrow 0$ even though m is large. The Na^+ current self-terminates. This is the fundamental asymmetry: m is fast, h is slower.

Slow recovery — potassium (n): The K^+ gate n opens even more slowly ($\tau_n \approx 5$ –10 ms). As n^4 grows, the outward K^+ current $I_K = \bar{g}_K n^4 (V_m - E_K)$ overwhelms the now-extinct Na^+ current and pulls V_m back toward E_K .

1.4 Activation vs. Inactivation: The Refractory Period

After an action potential, the cell enters a *refractory period* during which it cannot fire again. This is essential for ordered wave propagation — without it, waves could travel backward and the heart would fibrillate. The refractory period has a clear gating-variable explanation:

Immediately after the action potential, $h \approx 0$ (Na^+ channels are inactivated) and n is still elevated (K^+ channels are open). Even if a new stimulus arrives, there are no Na^+ channels available to open — the h gate needs time to recover (h must return toward $h_\infty(V_{rest}) \approx 0.6$). This recovery takes 5–10 ms in nerve, and 200–300 ms in ventricular cardiac cells.

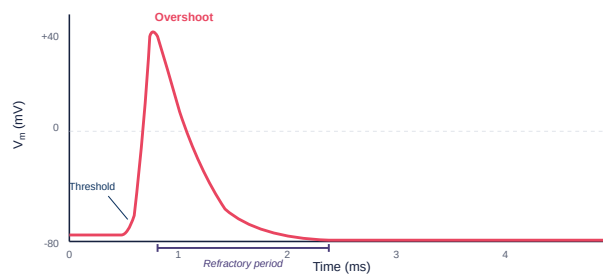


Figure 1.3. The nerve action potential (squid axon). The upstroke is driven by Na^+ influx ($m^3 h$ activating), the repolarization by Na^+ inactivation ($h \rightarrow 0$) and delayed K^+ activation (n^4 increasing). The refractory period corresponds to the time needed for h to recover.

Engine V5.4 Connection

In Engine V5.4, every ionic model inherits from the `IonicModel` abstract base class (`cardiac_sim/ionic/base.py`), which requires: `compute_Iion(V, ionic_states)` to compute total ionic current, `step(V, ionic_states, dt)` to advance the full model, and separate methods `compute_gate_steady_states()` and `compute_gate_time_constants()` so that the Rush–Larsen solver can update gates externally. V is always stored separately from ionic states. The Hodgkin–Huxley structure — membrane current as sum of conductances times driving forces, gating

variables following first-order kinetics — is exactly the pattern this interface encodes. Understanding HH means understanding the API contract of every model in the engine.

The complete Hodgkin–Huxley system is four coupled ordinary differential equations: one for V_m (the circuit equation) and three for the gates (m , h , n). It was the first biophysically detailed cell model and remains the conceptual template for all that followed. But for the heart, we need something quite different — as we’ll see in the next chapter.

From Neurons to Heart Cells

The nerve action potential is a brief spark — over in 2 milliseconds. The cardiac action potential is a sustained flame, lasting 200–400 ms. This chapter explains why the heart demands such a profoundly different electrical signature, and what new biophysics makes it possible.

2.1 Why the Heart Needs a Longer Action Potential

A nerve fiber has one job: transmit a binary signal from point A to point B as fast as possible. It fires and recovers quickly so it can fire again. But a cardiac myocyte has a very different job: it must *contract*, and contraction takes time. The heart needs the electrical signal to outlast the mechanical event so that every cell stays activated long enough to squeeze in synchrony.

If the cardiac AP were as short as a nerve spike, the cell would repolarize before it finished contracting. Worse, a new electrical impulse could arrive and re-activate parts of the heart while other parts were still contracting — leading to chaotic, uncoordinated contraction. The long AP duration serves as a *built-in safety mechanism*: by keeping the cell depolarized for hundreds of milliseconds, it prevents premature re-excitation.

Intuition

Think of it this way: the nerve AP is like a camera flash — bright, instantaneous, done. The cardiac AP is like holding a light switch on for a deliberate period. The heart needs this "held" state because mechanical contraction is slow compared to electrical activation. The electrical plateau is the heart's way of saying "stay contracted until I tell you to relax."

2.2 The L-type Calcium Current and the Plateau

So how does the heart stretch a 2 ms spike into a 300 ms plateau? The answer is a current that neurons barely use: the *L-type calcium current*, I_{CaL} . This inward current activates during depolarization (like Na^+), but it inactivates *much more slowly* — on a timescale of 50–200 ms rather than 1–2 ms.

During the plateau phase (Phase 2 of the cardiac AP), the membrane sits at roughly 0 to +20 mV because two opposing forces are nearly balanced:

$$\underbrace{I_{CaL}}_{\text{inward, depolarising}} \approx - \underbrace{I_{Kr} + I_{Ks}}_{\text{outward, repolarising}}$$

The L-type Ca^{2+} current holds the voltage up, while slowly activating delayed-rectifier K^+ currents (I_{Kr} and I_{Ks}) try to pull it down. This tug-of-war creates a nearly flat plateau that can last over 200 ms. Eventually, as I_{CaL} inactivates (both via voltage and via rising intracellular Ca^{2+}), the K^+ currents win and repolarization accelerates.

Key Insight

The plateau is a *fragile equilibrium*. A small change in any current during Phase 2 can dramatically alter AP duration. This is why drug-induced QT prolongation (blocking I_{Kr}) is so dangerous — it tips the balance toward inward current, lengthens the plateau, and can trigger the lethal arrhythmia *torsades de pointes*. Understanding this balance is central to computational drug safety testing.

The L-type calcium current also serves a second critical role: it triggers the release of a much larger amount of calcium from the sarcoplasmic reticulum (SR) — the cell's internal calcium store. This process, called *calcium-induced calcium release* (CICR), amplifies the Ca^{2+} signal roughly 10-fold and drives actual muscle contraction. We will see this calcium handling machinery in detail when we study the TTP06 model in Chapter 5.

2.3 Minimal Cardiac Models: FitzHugh–Nagumo to Fenton–Karma

Before diving into the full complexity of modern ionic models, it is instructive to see how the cardiac AP can be captured with just 2–3 variables. These minimal models sacrifice biophysical detail for mathematical tractability, but they capture the essential shape.

The FitzHugh–Nagumo Model (2 variables)

In 1961, Richard FitzHugh realized that the HH equations could be distilled to their essence. The key insight: the fast variable (Na^+ activation) and the slow variable (recovery) are what matter most. He proposed:

$$\frac{dv}{dt} = v - \frac{v^3}{3} - w + I_{\text{ext}}, \quad \frac{dw}{dt} = \epsilon(v + a - bw) \quad (2.1)$$

Here v is a rescaled voltage, w is a slow recovery variable (combining h and n into a single “how recovered is the cell” variable), and $\epsilon \ll 1$ ensures that w moves much slower than v . The cubic nonlinearity $v - v^3/3$ creates the bistability needed for excitation: v has two stable branches (resting and excited) and the system jumps between them.

FitzHugh–Nagumo captures excitability, threshold behavior, and refractoriness — but it produces a triangular AP, not the flat plateau of a cardiac cell.

The Fenton–Karma Model (3 variables)

In 1998, Flavio Fenton and Alain Karma introduced a three-variable model specifically designed for cardiac tissue. The key addition: a slow inward current that produces the plateau.

$$\frac{dV}{dt} = -(I_{\text{f}}(V, v) + I_{\text{so}}(V) + I_{\text{ai}}(V, w)) \quad (2.2)$$

The three currents map directly to cardiac physiology: I_{f} is a fast inward current (like Na^+ creating the upstroke), I_{so} is a slow outward current (like K^+ causing repolarization), and I_{ai} is a slow inward current (like Ca^{2+} sustaining the plateau). With just three variables (V, v, w), the Fenton–Karma model reproduces realistic AP shapes, spiral waves, and even alternans — all phenomena seen in real cardiac tissue.

⚙ Engine V5.4 Connection

Minimal models like Fenton–Karma are not toys. In Engine V5.4, they serve as essential *validation benchmarks*: if the monodomain solver produces the correct spiral-wave dynamics with a 3-variable model, you can have confidence in the diffusion numerics before scaling up to TTP06 (18 variables) or ORd (40 variables). They are also used for parameter sweeps and machine-learning studies where millions of simulations are needed.

The progression from HH (4 variables, nerve) → FitzHugh–Nagumo (2 variables, generic excitable) → Fenton–Karma (3 variables, cardiac) → full ionic models (18–40+ variables, human ventricle) is the story of matching model complexity to the scientific question being asked. In the next chapter, we zoom in on the full cardiac action potential and dissect each phase.

Anatomy of the Cardiac Action Potential

The ventricular action potential is traditionally divided into five phases (0 through 4). Each phase has a dominant current, a clear biophysical story, and direct consequences for the heart's electrical and mechanical function. Understanding these phases is the Rosetta Stone for reading any cardiac cell model. Chapter 5 provides the complete mathematical formulations for all 12 membrane currents in the TTP06 model; here we build the qualitative intuition first.

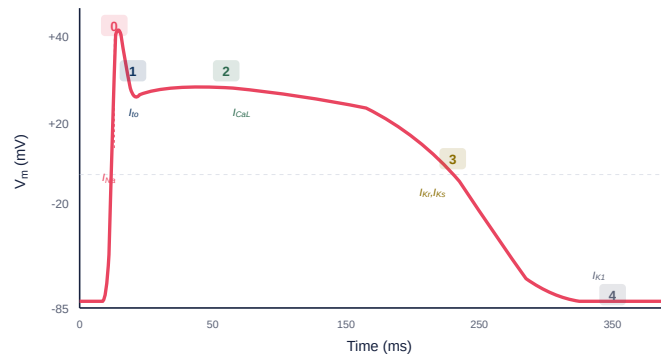


Figure 3.1. The five phases of the human ventricular action potential with dominant currents. Phase 0: Na^+ upstroke. Phase 1: transient outward K^+ (notch). Phase 2: Ca^{2+} - K^+ plateau. Phase 3: K^+ -driven repolarization. Phase 4: resting potential maintained by IK1 .

3.1 Phase 0: The Sodium Upstroke

Phase 0 is the fastest event in cardiac electrophysiology. When an arriving wavefront brings V_m above threshold (≈ -65 mV), sodium channels undergo the same explosive positive feedback we saw in the HH model: m -gates open rapidly, Na^+ floods inward, and V_m rockets from -85 mV to about $+40$ mV in roughly 1–2 ms.

The maximum rate of rise, $dV/dt_{\max}$, is a clinical measure of sodium channel availability. In healthy ventricular tissue, $dV/dt_{\max} \approx 200\text{--}300$ V/s. This is also what determines *conduction velocity* — the faster the upstroke, the larger the local current source that can depolarize neighboring cells, and the faster the wave propagates. In the monodomain equation (Chapter 7), this translates directly to the source term.

3.2 Phase 1: Early Repolarization (I_{to})

Immediately after the peak, a brief notch appears — a small dip in voltage before the plateau. This is caused by the *transient outward* potassium current I_{to} , which activates rapidly during depolarization and inactivates within tens of milliseconds.

Phase 1 may look like a minor detail, but it has major consequences. The depth of the notch varies across the ventricular wall: epicardial cells (outer surface) have large I_{to} and a deep notch, while endocardial cells (inner surface) have little I_{to} and almost no notch. This *transmural gradient* in AP morphology creates voltage differences across the wall that are visible on the ECG — the J-wave and T-wave both depend on it.

3.3 Phase 2: The Plateau — A Delicate Balance

The plateau is the heart's signature. For 150–250 ms, the membrane voltage hovers between -10 and $+20$ mV, sustained by the near-cancellation of two opposing current groups:

Inward (depolarizing): The L-type calcium current I_{CaL} is the dominant plateau current. Ca^{2+} ions enter through slowly inactivating channels, holding the voltage up. A smaller late sodium current (I_{NaL}) also contributes, especially in disease states.

Outward (repolarizing): Two delayed-rectifier potassium currents slowly activate: I_{Kr} (rapid component, carried by hERG channels) and I_{Ks} (slow component). The inward-rectifier I_{K1} is largely blocked at plateau voltages due to its voltage dependence — it will

become important again in Phase 3–4.

△ Key Insight

The plateau balance explains why the cardiac AP duration (APD) is so sensitive to perturbations. Block 30% of I_{Kr} (as many common drugs do), and the plateau suddenly lacks enough repolarizing current. APD extends, the QT interval on the ECG lengthens, and the risk of arrhythmia rises. This is why *in silico* drug testing with models like TTP06 and ORd has become a regulatory focus (the CiPA initiative).

3.4 Phase 3: Repolarization — The Potassium Takeover

As I_{CaL} gradually inactivates (driven both by voltage-dependent inactivation and by rising intracellular Ca^{2+} concentration), the balance tips irreversibly toward outward current. The K^+ currents I_{Kr} and I_{Ks} , which have been slowly building, now dominate. As the voltage falls, I_{K1} — the strong inward rectifier that was blocked at plateau voltages — begins to unblock and adds its large outward conductance to the repolarization.

Phase 3 repolarization is *regenerative*: as V_m falls, more I_{K1} unblocks, which pulls V_m down further, which unblocks more I_{K1} . This positive feedback (now outward) ensures that repolarization, once started, proceeds rapidly to completion. The membrane returns to its resting potential of about -85 mV.

3.5 Phase 4: Rest and the Role of I_{K1}

At rest, the membrane potential sits near $E_K \approx -88$ mV because I_{K1} dominates the conductance. The “inward rectifier” name refers to a peculiar property: I_{K1} passes large outward current at voltages just below E_K , but essentially zero current at plateau voltages. This voltage-dependent block (caused by intracellular magnesium and polyamines plugging the channel pore) is what allows the plateau to exist — without it, K^+ would short-circuit the plateau and the AP would collapse.

Intuition

Think of I_{K1} as a two-state switch. At rest, it is “on” — a strong anchor holding V_m near E_K . During the plateau, it switches “off” — stepping aside to let the delicate Ca^{2+}/K^+ balance play out. Then in Phase 3, it switches back “on” to slam the door shut and restore the resting potential. No other current has this dual role.

With the five phases clearly in mind, we now turn to the single most important ion in cardiac physiology: calcium. The next chapter is devoted entirely to the intracellular calcium machinery — from the nanoscale architecture of the dyadic cleft to the pathological remodeling that drives heart failure and arrhythmia.

Intracellular Calcium: The Master Regulator

Calcium is the singular ion that links electrical excitation to mechanical contraction. But this link is not simple: it involves nanoscale signaling domains, stochastic release events, multi-protein regulatory complexes, and feedback loops that span timescales from microseconds to minutes. This chapter provides a modern, in-depth account of cardiac calcium handling — from the architecture of the dyadic cleft to the pathological remodeling seen in heart failure.

4.1 The Dyadic Cleft — Where Electrical Meets Mechanical

Excitation-contraction (EC) coupling occurs at *dyads* — nanoscopic junctions formed where transverse tubules (T-tubules) invaginate to within 10–20 nm of the junctional sarcoplasmic reticulum (jSR). This tiny gap — the *dyadic cleft* — is the reaction chamber where the electrical signal is transduced into a calcium signal. During an action potential, calcium concentration within the cleft surges from a resting ~ 100 nM to ~ 10 μ M, a 100-fold increase confined to a space smaller than a virus particle.

The cleft is held together by **junctophilin-2 (JPH2)**, a structural protein that physically anchors the T-tubule membrane to the jSR membrane. JPH2 maintains the critical 10–20 nm separation required for efficient local control coupling. As we will see in Section 4.7, the loss of JPH2 in heart failure is one of the earliest and most consequential structural failures.

Calcium Release Units and Nanoscale Organization

The functional unit of calcium release is the *calcium release unit* (CRU), also called a *couplon*. A CRU consists of a cluster of type-2 ryanodine receptors (RyR2) on the jSR membrane, positioned directly beneath a group of L-type calcium channels (Cav1.2) on the T-tubule membrane. A typical human ventricular myocyte contains approximately 10,000–20,000 CRUs.

Recent super-resolution imaging — using dSTORM (direct stochastic optical reconstruction microscopy), DNA-PAINT, and most recently MINFLUX microscopy — has revealed the nanoscale architecture of these clusters in unprecedented detail. Key findings include:

RyR2 cluster heterogeneity: A typical CRU contains 4–20 RyR2 tetramers (16–80 individual subunits), but cluster sizes vary widely. RyR2 channels function cohesively as a CRU only when their nearest-neighbor distance (NND) is less than ~ 100 –150 nm. Beyond this threshold, coupling between channels is lost and the release event becomes unreliable.

Sparse channel activation: Despite clusters containing dozens of channels, only 2–4 RyR2 channels typically open simultaneously during a single elementary release event under basal conditions. Beta-adrenergic stimulation increases this to 5–8 channels, providing a mechanism for graded calcium release amplification.

L-type channel organization: Super-resolution studies using STED and DNA-PAINT (2024) revealed that Cav1.2 and Cav1.3 channels form loose clusters of ~ 9 molecules within ~ 120 nm domains, arranged in an irregular pattern with significant intermolecular spacing — not densely packed. This loose arrangement reflects the functional need for positioning (close to RyR2) rather than crowding.

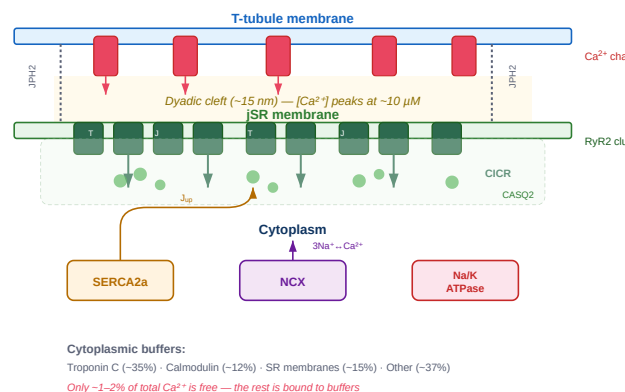


Figure 4.1. Architecture of the dyadic cleft. L-type Ca^{2+} channels on the T-tubule membrane face RyR2 clusters on the jSR, separated by ~ 15 nm. Junctophilin-2 (JPH2) anchors the two membranes. Triadin (T) and junctin (J) link RyR2 to luminal CASQ2 polymers. SERCA2a pumps Ca^{2+} back into the SR; NCX extrudes Ca^{2+} across the sarcolemma. Only 1–2% of cytoplasmic Ca^{2+} is free.

4.2 Calcium-Induced Calcium Release — Local Control Theory

Calcium Sparks: The Elementary Release Event

A *calcium spark* is the optical signature of the concerted opening of a small group (typically 2–10) of RyR2 channels within a single CRU. Discovered by Cheng, Lederer, and Bhatt in 1993 using confocal line-scanning and the fluorescent indicator fluo-3, sparks are now understood as the fundamental building blocks of the whole-cell calcium transient.

Each spark has well-defined characteristics: a peak amplitude of $0.5\text{--}2\ \mu\text{M}$ above baseline, a spatial half-width of $1\text{--}3\ \mu\text{M}$, a rise time of 2–5 ms, and a full-width-at-half-maximum (FWHM) duration of 15–50 ms. During a normal heartbeat, the action potential triggers $\sim 10,000$ sparks nearly simultaneously across the myocyte; their spatial and temporal summation produces the global calcium transient that drives contraction.

Intuition

Think of CRUs as thousands of tiny fireworks launchers spread throughout the cell. When the action potential arrives, it does not light them all at once with a single fuse. Instead, each launcher has its own independent trigger — an L-type channel — that fires with a certain probability ($\sim 30\text{--}40\%$ per beat). The global calcium transient is the *sum* of thousands of probabilistic, independent sparks. This is why calcium release is *graded*: more L-type current recruits more sparks, producing a larger transient — without the all-or-nothing behavior you might expect from positive feedback.

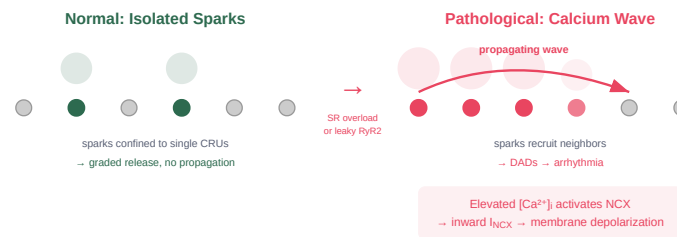


Figure 4.2. The spark-to-wave transition. Left: Under normal conditions, individual sparks (green) are spatially isolated — positive feedback is confined within each CRU. Right: With SR overload or leaky RyR2, sparks recruit neighboring CRUs, generating a propagating calcium wave that activates NCX and triggers DADs.

Local Control vs. Common Pool

The *local control* theory, established in the 1990s and continuously refined through 2025, resolves a paradox that troubled the field for decades: how can CICR be both *high-gain* (a tiny calcium trigger from L-type channels produces a large SR release) and *graded* (release is proportional to trigger size, not all-or-nothing)?

The answer is spatial isolation. Each RyR2 cluster senses *local* calcium concentration within its dyadic cleft ($\sim 10\ \mu\text{M}$ during activation), not the global cytosolic concentration ($\sim 1\ \mu\text{M}$). Positive feedback — calcium from one RyR2 opening its neighbors — is confined within the CRU because the cleft's tiny volume ($\sim 10^{-18}$ L) means even picoampere currents produce micromolar concentrations locally, while rapid diffusion dilutes the signal before it reaches neighboring CRUs $\sim 2\ \mu\text{m}$ away.

This contrasts with the older *common pool* model, which assumed a single well-mixed cytoplasmic calcium compartment. Common pool models predict all-or-nothing release — once any RyR2 opens, the resulting calcium elevation should trigger all other RyR2s. Experimentally, this is not what happens: sparks occur at discrete locations without propagating, and the total release scales smoothly with L-type current magnitude. Local control explains both observations.

SR Load: The Central Regulator of Release

The SR calcium content ($[Ca^{2+}]_{SR}$, typically 1–2 mM total, of which only $\sim 20\text{--}50\ \mu\text{M}$ is free — the rest bound to calsequestrin) is the single most powerful determinant of release properties. Spark frequency increases *exponentially* with SR load: doubling the load can increase spontaneous release 5–10-fold. Spark amplitude increases more modestly, while the probability that a given L-type channel opening will trigger a spark (the *coupling fidelity*) rises steeply with load.

This load-dependence creates an inherent negative feedback: release depletes the store, which reduces the probability of further release. In healthy cells, this feedback stabilizes the system. In disease, when SR load is chronically elevated or RyR2 sensitivity is pathologically increased, the feedback can be overwhelmed — sparks merge into waves, and arrhythmias follow.

The Spark-to-Wave Transition

Under pathological conditions — elevated SR load, increased diastolic calcium, enhanced RyR2 sensitivity — isolated sparks can recruit their neighbors, generating *calcium waves* that propagate through the myocyte at $\sim 100\ \mu\text{m/s}$. The wave front advances by saltatory conduction: a spark at one CRU raises local calcium enough to trigger sparks at neighboring CRUs $\sim 2\ \mu\text{m}$ away. The critical radius for wave propagation is $\sim 2.7\ \mu\text{m}$ — below this, the calcium elevation dissipates before reaching the next CRU.

Calcium waves are the primary trigger for *delayed afterdepolarizations* (DADs). As the wave sweeps through the cell, the elevated cytoplasmic calcium activates the sodium-calcium exchanger (NCX) in forward mode, generating a net inward current that depolarizes the membrane. If this depolarization reaches threshold during diastole, a triggered action potential fires — the cellular mechanism behind many clinical arrhythmias.

4.3 The Ryanodine Receptor — Gatekeeper of the SR

RyR2 is the largest known ion channel ($\sim 2.2\ \text{MDa}$ as a homotetramer) and the primary gate controlling calcium release from the SR. Its regulation involves a remarkable convergence of signals: cytoplasmic calcium (activation), luminal calcium (load sensing), phosphorylation by multiple kinases, oxidative modifications, and protein–protein interactions.

Phosphorylation by PKA and CaMKII

Two kinases dominate RyR2 regulation, each targeting a distinct serine residue with different physiological and pathological consequences:

PKA at Serine 2808: During beta-adrenergic stimulation (exercise, stress), PKA phosphorylates RyR2 at S2808, increasing the channel's sensitivity to calcium activation. The channel opens more frequently and requires less trigger calcium. This is the acute "fight-or-flight" enhancement of calcium release. Phosphatase 1 (PP1) rapidly reverses the phosphorylation when adrenergic tone subsides.

CaMKII at Serine 2814: CaMKII, activated by calcium/calmodulin, phosphorylates RyR2 at S2814 with more potent effects on spark frequency than PKA. Critically, CaMKII also increases the channel's sensitivity to *luminal* calcium, amplifying the load-dependent release mechanism. This creates a positive feedback loop: more calcium $\rightarrow$ more CaMKII activation $\rightarrow$ more RyR2 phosphorylation $\rightarrow$ more calcium release.

▲ Key Insight

Recent work (2023–2025) has established that CaMKII phosphorylation at S2814 — not PKA phosphorylation at S2808 — is the primary driver of pro-arrhythmic calcium leak in chronic heart failure. A 2025 study in *Nature Cardiovascular Research* found an unexpected twist: preventing RyR2 phosphorylation reduced calcium leak but paradoxically reactivated the sodium current I_{Na} , suggesting a previously unknown crosstalk between RyR2 status and sarcolemmal channel regulation. This has implications for anti-arrhythmic drug design.

Luminal Calcium Sensing: The CASQ2–Triadin–Junctin Complex

RyR2 possesses an intrinsic ability to sense the calcium concentration *inside* the SR, independent of cytoplasmic calcium. This luminal sensing is mediated by a multi-protein complex anchored to the SR luminal face of RyR2:

Calsequestrin 2 (CASQ2) is the primary SR calcium storage protein, binding $\sim 40\text{--}50$ calcium ions per molecule across a broad affinity range ($K_d \approx 0.1\text{--}10\ \text{mM}$). At low luminal $[Ca^{2+}]$ ($< 1\ \text{mM}$), CASQ2 exists as a monomer and directly inhibits RyR2 via triadin

and junctin — a molecular "brake" that prevents release from a depleted store. As luminal calcium rises through intermediate concentrations (3–5 mM), CASQ2 undergoes progressive oligomerization: monomer → dimer → polymer. The polymeric form dissociates from the triadin–junctin bridge, *relieving* RyR2 inhibition and permitting calcium release.

Triadin (primarily the 95 kDa junctional isoform) and **junctin** are integral SR membrane proteins that serve as molecular scaffolds linking CASQ2 to RyR2. Their ablation causes complete loss of functional CRUs, severe arrhythmias, and cardiomyopathy — underscoring their essential structural role. Mutations in triadin and junctin are now recognized causes of catecholaminergic polymorphic ventricular tachycardia (CPVT).

Intuition

The CASQ2 sensing mechanism works like a pressure gauge. When the SR "tank" is nearly empty (low luminal calcium), CASQ2 monomers bind to the RyR2 "valve" and hold it shut — preventing futile release from a depleted store. As the tank fills (SERCA pumps calcium back in), CASQ2 polymers form and detach from the valve, allowing it to open on the next trigger. This self-regulating mechanism ensures that release magnitude tracks store content — a form of automatic gain control.

The "Leaky RyR" Hypothesis and Heart Failure

Andrew Marks and colleagues demonstrated that RyR2 dysfunction is central to heart failure pathophysiology through a mechanism now termed the *leaky RyR hypothesis*. In failing hearts, chronic neurohormonal activation leads to persistent hyperphosphorylation of RyR2 at both S2808 (PKA) and S2814 (CaMKII). This, combined with oxidative modification (S-nitrosylation, sulfhydryl oxidation), increases the channel's diastolic open probability — creating a sustained *calcium leak* from the SR even when the cell should be at rest.

The consequences cascade: elevated diastolic $[Ca^{2+}]_i$ (~200–300 nM vs. normal 50–100 nM) impairs relaxation, activates calcium-dependent proteases (calpains), and drives maladaptive hypertrophic signaling via calcineurin/NFAT. Simultaneously, the increased forward-mode NCX activity needed to extrude the leaked calcium brings excess sodium into the cell, raising $[Na^+]_i$ and further compromising SERCA efficiency. The leaked SR calcium is also unavailable for systolic release, weakening contraction.

Store-Overload-Induced Calcium Release (SOICR)

When SR calcium load exceeds a critical threshold ($[Ca^{2+}]_{SR} > \sim 1.5$ mM free), spontaneous calcium release occurs without any electrical trigger — a phenomenon called SOICR. At such high loads, CASQ2 inhibition is completely relieved, and the tiny amount of calcium that inevitably leaks through any channel is amplified by CICR into full-scale spontaneous waves.

SOICR is the cellular mechanism behind digitalis toxicity (Na/K-ATPase inhibition → sodium overload → reverse NCX → calcium overload → SOICR → arrhythmia), and it is a primary trigger for CPVT — a genetic arrhythmia syndrome caused by RyR2 or CASQ2 mutations that lower the load threshold for spontaneous release. Carvedilol, a nonselective beta-blocker, uniquely suppresses SOICR by directly reducing RyR2 open duration, independent of its beta-blocking properties — making it particularly effective for CPVT and heart failure arrhythmias.

4.4 SERCA2a and Phospholamban — The Pump and Its Brake

The sarco/endoplasmic reticulum calcium-ATPase 2a (SERCA2a) is the workhorse of diastolic calcium removal, responsible for 70–90% of cytoplasmic calcium clearance in human ventricular myocytes. Understanding its regulation is essential for understanding both normal relaxation and the diastolic dysfunction that characterizes heart failure.

SERCA2a Kinetics

SERCA2a transports 2 Ca^{2+} ions per ATP hydrolyzed, with a K_m of ~0.5–1 μ M for calcium — meaning it operates at roughly half-maximal velocity at resting diastolic $[Ca^{2+}]$. The pump consumes an estimated 30–40% of the myocyte's total ATP budget during active beating — a staggering metabolic cost that underscores calcium cycling's importance. A typical human ventricular myocyte contains ~1,000–2,000 SERCA2a molecules per unit SR surface.

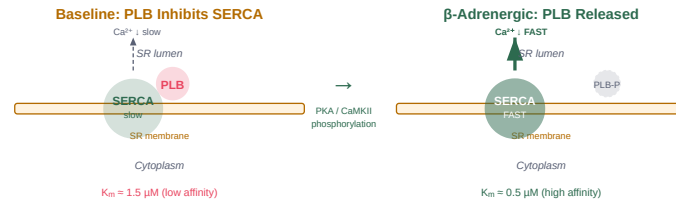


Figure 4.3. The SERCA2a/phospholamban pump-brake system. Left: At baseline, PLB inhibits SERCA2a by reducing its calcium affinity (high K_m), resulting in slow calcium reuptake. Right: After PKA phosphorylation (β -adrenergic stimulation), PLB dissociates, restoring SERCA2a's full affinity and dramatically accelerating calcium reuptake — the molecular basis of lusitropy.

Phospholamban: The Molecular Brake

Phospholamban (PLB) is a 52-amino-acid transmembrane protein that constitutively *inhibits* SERCA2a in its unphosphorylated state. It reduces SERCA2a's calcium affinity 2–3-fold (shifting K_m from $\sim 0.5 \mu M$ to $\sim 1.5 \mu M$), dramatically slowing calcium reuptake at diastolic calcium concentrations. PLB is the reason that at baseline, SERCA2a operates far below its maximum capacity — a reserve that can be unlocked on demand.

During beta-adrenergic stimulation, PKA phosphorylates PLB at serine-16 (and CaMKII at threonine-17), causing PLB to dissociate from SERCA2a. This relieves inhibition and dramatically accelerates calcium reuptake — the molecular basis of the *lusitropic* (relaxation-enhancing) effect of sympathetic stimulation. The coordinated phosphorylation of both RyR2 (more release) and PLB (faster reuptake) ensures that the enhanced systolic transient is followed by faster relaxation, allowing the heart to fill adequately at elevated heart rates.

Key Insight

The CUPID trials (Calcium Upregulation by Percutaneous Administration of Gene Therapy in Cardiac Disease) attempted to restore SERCA2a expression in failing hearts using AAV1-delivered gene therapy. Early-phase results were encouraging, but the pivotal CUPID 2 trial (2016) did not meet its primary endpoint. Current research has shifted toward upstream approaches: rather than adding more pump protein, investigators are targeting phosphatase-1 inhibitors (e.g., AB-1002) to relieve PLB-mediated inhibition of the *endogenous* SERCA2a — a more physiologically regulated strategy.

4.5 The Sodium-Calcium Exchanger — Dual-Mode Transport

The sodium-calcium exchanger NCX1 is a bidirectional transporter that exchanges $3 Na^+$ for $1 Ca^{2+}$ across the sarcolemma. The direction of transport depends on the electrochemical gradients of both ions and the membrane potential — making NCX a voltage-sensitive, context-dependent player in calcium homeostasis.



Figure 4.4. The two operating modes of the Na^+/Ca^{2+} exchanger (NCX). Left: Forward mode — dominant during diastole and late repolarization — extrudes Ca^{2+} while importing Na^+ , generating a net inward (depolarizing) current. Right: Reverse mode — briefly active during early depolarization — imports Ca^{2+} and exports Na^+ , generating a net outward current. The reversal potential depends on both $[Na^+]$ and $[Ca^{2+}]$ gradients plus V_m .

Forward Mode (Calcium Extrusion)

Under normal diastolic conditions, NCX operates in *forward mode*: $3 Na^+$ ions enter the cell down their concentration gradient, providing the energy to extrude $1 Ca^{2+}$ against its gradient. This accounts for ~ 10 – 30% of diastolic calcium removal (with SERCA2a

handling the remainder). Because the exchange is electrogenic (net transfer of one positive charge inward), forward-mode NCX generates a small depolarizing current — normally negligible, but significant when calcium is elevated.

Reverse Mode and Arrhythmogenesis

When intracellular sodium is elevated (as in heart failure or digitalis toxicity) or the membrane is strongly depolarized, NCX reverses: Na^+ exits and Ca^{2+} enters. This reverse-mode operation contributes to calcium overload. More importantly, during spontaneous calcium waves, the large forward-mode NCX current (extruding the wave-elevated calcium) generates substantial inward current — 100–500 pA when summed over the cell — sufficient to depolarize the membrane and trigger delayed afterdepolarizations (DADs). This is the primary mechanism linking calcium instability to electrical arrhythmia.

Recent cryo-EM structures of human NCX1 (2023) revealed distinct inactivated and activated conformations, with a sodium-dependent inactivation mechanism that serves as an intrinsic "brake" when intracellular sodium is chronically elevated. A 2024 study showed that gain-of-function NCX1 mutations paradoxically depress contractility by excessive calcium extrusion, demonstrating that too much of either forward or reverse NCX activity is maladaptive.

The Calcium Clock in Pacemaker Cells

In sinoatrial node (SAN) cells, a fundamentally different calcium cycling mechanism contributes to automaticity. During diastole, spontaneous calcium sparks from the SR activate NCX in forward mode, generating a ramp-like inward current that gradually depolarizes the membrane toward threshold. This "calcium clock" works in concert with the classical "membrane clock" (funny current I_f , T-type calcium current) to set heart rate. Disruption of the calcium clock — through RyR2 inhibition or SERCA2a depression — slows or abolishes pacemaker activity, demonstrating that spontaneous calcium cycling is not merely a byproduct but an active driver of the heartbeat.

4.6 Calcium Buffering — The 1% Rule

Perhaps the most underappreciated fact in cardiac calcium biology is that **only ~1–2% of total intracellular calcium exists as free Ca^{2+} ions**. The remaining 98–99% is bound to proteins. This massive buffering has profound consequences for calcium signaling: it drastically slows effective calcium diffusion (to ~10–30 $\mu\text{m}^2/\text{s}$, roughly 10× slower than free aqueous diffusion) and shapes the kinetics of every calcium-dependent process.

The buffer capacity β — defined as the ratio of total calcium change to free calcium change — ranges from ~20 to 100 depending on the cell compartment. During a typical beat, free $[\text{Ca}^{2+}]_i$ rises from ~100 nM to ~1–2 μM , a 10–20-fold increase. But the total calcium flux required to achieve this is 20–100 times larger, because most of the released calcium is immediately captured by buffers.

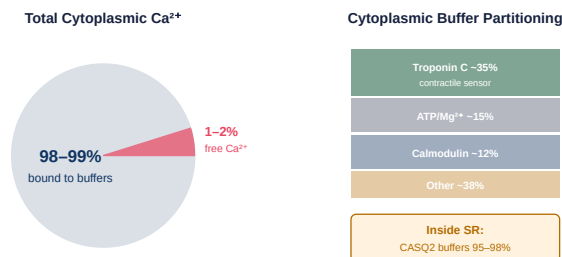


Figure 4.5. Calcium buffering in the cardiomyocyte. Left: Only 1–2% of total cytoplasmic calcium is free; the rest is bound to buffer proteins. Right: Partitioning among the major cytoplasmic buffers. Troponin C (the contractile sensor) is the largest single buffer pool. Inside the SR, calsequestrin-2 (CASQ2) binds 95–98% of luminal calcium.

The Major Buffers

Troponin C (~35% of cytoplasmic buffering): The regulatory calcium sensor of the contractile apparatus. Each troponin C molecule has 4 calcium-binding sites (2 regulatory with $K_d \approx 1\text{--}10 \mu\text{M}$, 2 structural with $K_d \approx 100 \text{ nM}$). With ~10–15 million troponin C molecules per myocyte, this represents the largest single buffer pool. Calcium binding to the regulatory sites triggers the conformational cascade (tropomyosin shift → myosin binding site exposure → cross-bridge cycling) that produces contraction. Thus troponin C is simultaneously a buffer, a sensor, and the effector of the entire contractile process.

Calmodulin (~12% of buffering): A ubiquitous calcium sensor with 4 EF-hand binding motifs (K_d range 100 nM to 5 μ M). Calmodulin activates CaMKII, calcineurin, and numerous other enzymes — serving as the cell's primary calcium-to-kinase signal transducer. With ~1–2 million molecules per myocyte and faster on/off kinetics than troponin C, calmodulin is particularly responsive to rapid calcium transients.

Calsequestrin 2 (~95–98% of SR luminal buffering): Inside the SR, CASQ2 binds ~40–50 calcium ions per molecule, maintaining total SR calcium at 1–2 mM while keeping free SR calcium at only ~20–50 μ M. This enormous buffer capacity allows the SR to function as a high-capacity store without the osmotic stress that millimolar free calcium would create.

Other contributors: ATP and its magnesium complexes (~15%), mitochondrial uptake (<5% acutely but important for long-term calcium homeostasis), and various smaller calcium-binding proteins collectively account for the remaining buffering capacity.

Mathematical Note

In computational models, calcium buffering is typically handled by either *rapid-equilibrium approximation* (assuming buffer binding/unbinding is instantaneous, valid when buffer kinetics are much faster than calcium transient kinetics) or *dynamic buffering ODEs* (explicitly tracking bound and free buffer pools). TTP06 uses a rapid-equilibrium approach for most buffers, while ORd tracks selected buffer states dynamically. The choice matters: dynamic buffering adds state variables but captures the delay between calcium rise and buffer saturation — important for accurate spark modeling.

4.7 T-Tubule Remodeling — When Structure Fails

The T-tubule network consists of regular sarcolemmal invaginations (~0.2–0.5 μ m diameter) that penetrate deep into the myocyte at ~2 μ m intervals, aligned with the Z-lines of the sarcomere. This regular architecture ensures that every CRU across the cell volume is within ~1 μ m of a T-tubule, enabling near-synchronous calcium release during the action potential.

T-Tubule Loss in Heart Failure

One of the earliest and most consequential structural changes in the failing myocardium is progressive loss and disorganization of the T-tubule network. Advanced heart failure can show 50–80% reduction in T-tubular density, with the remaining tubules becoming tortuous, dilated, and misaligned.

The primary mechanism is **calpain-mediated cleavage of junctophilin-2**. Elevated diastolic calcium in failing cells activates the protease calpain, which cleaves JPH2 — the very protein that anchors T-tubules to the jSR. Once this anchor is lost, the T-tubule membrane retracts from the SR surface, physically separating L-type channels from RyR2 clusters.

Orphaned RyR2 and Dyssynchronous Release

As T-tubules withdraw, the RyR2 clusters they once faced become *orphaned* — physically uncoupled from their L-type channel triggers. These orphaned RyR2s cannot be activated by the normal CICR mechanism (the L-type channel calcium signal no longer reaches them). Instead, they can only be activated secondarily, by the slower wave-like diffusion of calcium from neighboring intact dyads. The result is *dyssynchronous calcium release*: surface dyads fire promptly with the action potential, while interior orphaned clusters fire with a variable delay of 10–100 ms.

This dyssynchrony has dual consequences. First, the contractile force is reduced — not all sarcomeres are activated simultaneously. Second, the orphaned RyR2s, no longer under the tight local control of a nearby L-type channel, often exhibit increased sensitivity to luminal calcium and are prone to spontaneous opening. They become sources of diastolic calcium leak and spontaneous waves — contributing to both systolic dysfunction and arrhythmia susceptibility.

⚙️ Engine V5.4 Connection

From a modeling perspective, the T-tubule remodeling story highlights a limitation of standard ionic models like TTP06 and ORd: they assume a single, well-coupled dyadic compartment. Capturing the spatial heterogeneity of heart failure — some CRUs coupled, some orphaned, some partially coupled — requires either multi-compartment extensions or spatially explicit subcellular models. This

remains an active area of computational research, with multiscale models (2021–2024) integrating nanoscale dyadic dynamics into whole-cell and tissue-level simulations.

4.8 Emerging Frontiers: Imaging, Modeling, and Therapy

Super-Resolution Imaging of Living Cardiomyocytes

The past five years have seen transformative advances in visualizing calcium handling machinery in intact, living cells. **MINFLUX microscopy** (2024–2025) can now resolve individual RyR2 subunits and determine their 3D orientation within clusters — approaching atomic-level structural information in the native cellular environment. Live-cell photoactivated localization microscopy (PALM, 2022) has correlated nanoscale RyR2 cluster configuration with functional calcium spark properties, revealing that heart failure induces fragmentation of large RyR2 clusters into numerous smaller groupings with increased nearest-neighbor distances and impaired cooperative gating.

Next-Generation Calcium Indicators

Genetically encoded calcium indicators continue to evolve. The jRCaMP1b family (2023) achieves submillisecond rise times with the highest sensitivity yet — enabling real-time nanodomain calcium imaging. The NEMOer sensor family (2025) was specifically designed for endoplasmic/sarcoplasmic reticulum calcium imaging, overcoming the limited dynamic range of previous luminal indicators. These tools are beginning to provide direct experimental access to the $[Ca^{2+}]_{SR}$ dynamics that models have long predicted but experiments could not resolve.

Graph-Theoretic Prediction of Spark Fidelity

A striking computational advance (2023) uses graph theory to predict calcium spark probability directly from nanoscale cluster architecture. By constructing an adjacency graph of RyR2 positions within a CRU (nodes = channels, edges = functional coupling), researchers showed that the *maximum eigenvalue* of this adjacency matrix predicts spark probability with remarkable accuracy. This bridges the gap between structure (super-resolution images) and function (calcium spark recordings), explaining why some RyR2 clusters reliably produce sparks while others are functionally "silent."

With this deep understanding of calcium handling in hand, we are now prepared to examine the quantitative ionic models that encode these mechanisms computationally. The next chapter dissects the ten Tusscher–Panfilov 2006 model — 18 state variables that capture the calcium cycling machinery alongside every ionic current we have discussed.

The ten Tusscher–Panfilov 2006 Model

TTP06 is the workhorse model for human ventricular electrophysiology. With 18 state variables — 12 gating variables and 6 ionic concentrations — it captures every phase of the action potential we dissected in Chapter 3, plus the calcium cycling machinery that drives contraction.

5.1 18 State Variables: What and Why

A natural question when encountering TTP06 for the first time is: *why 18 variables?* The answer lies in the variety of currents and intracellular processes that shape the cardiac AP. Each current requires gates, and each ion species requires a concentration variable to track changes over multiple beats.

The 18 states fall into two natural groups:

12 gating variables (each between 0 and 1, following Hodgkin–Huxley-style kinetics):

Gate	Current	Role	Speed
m	I_{Na}	Na^+ activation	< 1 ms
h, j	I_{Na}	Na^+ fast and slow inactivation	1–100 ms
d	I_{CaL}	Ca^{2+} activation	1–5 ms
f, f_2	I_{CaL}	Ca^{2+} voltage inactivation	20–300 ms
f_{Ca}	I_{CaL}	Ca^{2+} -dependent inactivation	2–50 ms
r	I_{to}	Transient outward activation	1–5 ms
s	I_{to}	Transient outward inactivation	5–50 ms
x_{r1}, x_{r2}	I_{Kr}	Rapid delayed rectifier	10–100 ms
x_s	I_{Ks}	Slow delayed rectifier	100–500 ms

6 concentration variables (dynamic, changing each beat):

Variable	Description	Typical Value
$[Na^+]_i$	Intracellular sodium	~ 10 mM
$[K^+]_i$	Intracellular potassium	~ 138 mM
$[Ca^{2+}]_i$	Cytoplasmic calcium (free)	~ 0.0001 mM
$[Ca^{2+}]_{SR}$	SR calcium store	~ 1 – 3 mM
$[Ca^{2+}]_{ss}$	Subspace calcium (near L-type channels)	~ 0.0001 mM
R'	SR release recovery variable	~ 1

The distinction between $[Ca^{2+}]_i$, $[Ca^{2+}]_{ss}$, and $[Ca^{2+}]_{SR}$ reflects the *compartmentalization* of calcium handling. Calcium entering through L-type channels first reaches a tiny subspace near the SR release channels, triggering CICR locally before diffusing into the bulk cytoplasm.

5.2 The Complete Current Inventory

TTP06 models 12 distinct membrane currents plus 4 internal calcium fluxes. Every membrane current contributes to the total ionic current:

$$I_{\text{ion}} = \underbrace{I_{\text{Na}} + I_{\text{CaL}}}_{\text{depolarizing}} + \underbrace{I_{\text{to}} + I_{\text{Kr}} + I_{\text{Ks}} + I_{\text{K1}} + I_{\text{pK}}}_{\text{repolarizing}} + \underbrace{I_{\text{NaK}} + I_{\text{NaCa}} + I_{\text{pCa}}}_{\text{pumps/exchangers}} + \underbrace{I_{\text{bNa}} + I_{\text{bCa}}}_{\text{background}} \quad (5.1)$$

This equation is the heart of the model: at every timestep, the simulator evaluates all 12 terms, sums them, and uses the total to update V_m . The sign convention follows physiology — positive current is outward (repolarizing), negative current is inward (depolarizing). The table below provides a complete reference, mapping each current to its AP phase, physiological role, and clinical significance.

Current	Type	Ion	AP Phase	Direction	Clinical Relevance
I_{Na}	Voltage-gated channel	Na^+	0 (upstroke)	Inward	Conduction velocity; Class I antiarrhythmics
I_{CaL}	Voltage-gated channel	Ca^{2+}	2 (plateau)	Inward	EC coupling trigger; Ca^{2+} channel blockers (verapamil)
I_{to}	Voltage-gated channel	K^+	1 (notch)	Outward	Transmural heterogeneity; Brugada syndrome
I_{Kr}	Voltage-gated channel	K^+	3 (repolarization)	Outward	Primary drug target; Long QT syndrome (hERG block)
I_{Ks}	Voltage-gated channel	K^+	3 (repolarization reserve)	Outward	Rate adaptation; LQT1; adrenergic modulation
I_{K1}	Inward rectifier	K^+	4 (rest) / 3 (late)	Outward	Resting potential; Andersen-Tawil syndrome (LQT7)
I_{pK}	Voltage-gated channel	K^+	2 (plateau)	Outward	Small contribution to repolarization reserve
I_{NaK}	ATP-driven pump	Na^+/K^+	All	Net outward	Gradient maintenance; digitalis target
I_{NaCa}	Exchanger	$\text{Na}^+/\text{Ca}^{2+}$	2–3 / 4	Both*	Calcium extrusion; DAD generation; heart failure
I_{pCa}	ATP-driven pump	Ca^{2+}	All	Outward	Fine calcium homeostasis
I_{bNa}	Background leak	Na^+	All	Inward	Resting Na^+ influx; balanced by I_{NaK}
I_{bCa}	Background leak	Ca^{2+}	All	Inward	Resting Ca^{2+} influx; balanced by pumps

* I_{NaCa} operates in *forward mode* (inward, extruding Ca^{2+}) during late repolarization and diastole, and in *reverse mode* (outward, bringing Ca^{2+} in) briefly during early depolarization.

5.3 Voltage-Gated Depolarizing Currents

Fast Sodium Current (I_{Na}) — Phase 0

The fast sodium current is responsible for the action potential upstroke and determines conduction velocity in tissue. It follows the classical HH formulation with three activation gates (m) and two inactivation gates (fast h and slow j):

$$I_{\text{Na}} = \bar{g}_{\text{Na}} m^3 h j (V_m - E_{\text{Na}}) \quad (5.2)$$

The maximal conductance $\bar{g}_{\text{Na}} = 14.838$ nS/pF is one of the largest in the model. The m^3 activation is ultrafast ($\tau_m < 1$ ms), providing the explosive upstroke. The dual inactivation (h fast, j slow) ensures that sodium channels close within a few milliseconds and remain

closed throughout the plateau — creating the absolute refractory period. Physiologically, I_{Na} determines the maximum rate of rise $dV/dt_{max} \approx 250\text{--}350$ V/s, which is a direct clinical measure of sodium channel availability. Class I antiarrhythmic drugs (lidocaine, flecainide) work by reducing $\bar{g}_{Na}$, slowing conduction.

L-Type Calcium Current (I_{CaL}) — Phase 2

The L-type calcium current sustains the plateau and triggers excitation-contraction coupling. Unlike I_{Na} , it uses the Goldman–Hodgkin–Katz (GHK) formulation rather than an Ohmic driving force, because Ca^{2+} has a very low intracellular concentration and a 2+ valence:

$$I_{CaL} = P_{CaL} d f f_2 f_{Ca} \cdot 4 \frac{F^2}{RT} (V_m - 15) \frac{0.25 [Ca^{2+}]_{ss} e^{2F(V_m-15)/RT} - [Ca^{2+}]_o}{e^{2F(V_m-15)/RT} - 1} \quad (5.3)$$

Note three features that distinguish this from a simple Ohmic current. First, TTP06 uses a *permeability* $P_{CaL} = 3.98 \times 10^{-5}$ cm/s rather than a conductance — the natural parameter for the GHK formulation. Second, the voltage is shifted by -15 mV ($V_m - 15$), reflecting the experimental observation that the current–voltage relationship of L-type channels peaks near $+15$ mV rather than 0 mV. Third, the intracellular calcium carries an activity coefficient of 0.25, while the extracellular side uses bare $[Ca^{2+}]_o$ — this asymmetry accounts for the different ionic environments on each side. The four gates represent distinct biophysical mechanisms: d is voltage activation ($\tau_d \sim 1\text{--}5$ ms), f and f_2 are voltage-dependent inactivation (slow, $\tau \sim 20\text{--}300$ ms — this determines how long the plateau lasts), and f_{Ca} is *calcium-dependent inactivation* (CDI), which responds to the local subspace calcium $[Ca^{2+}]_{ss}$. CDI is the primary negative feedback mechanism that limits calcium entry: as CICR raises subspace calcium, f_{Ca} inactivates the channel, shutting off the trigger.

Intuition

The GHK equation replaces the simple $(V_m - E_{Ca})$ driving force with a nonlinear function of voltage and concentrations on both sides. Why? For ions with large concentration gradients ($[Ca^{2+}]_o/[Ca^{2+}]_i \approx 20,000$) and multivalent charge, the Ohmic approximation badly overestimates current at large depolarizations. The GHK formula — derived from the Nernst–Planck electrodiffusion equation — accounts for the exponential concentration profile across the membrane and gives physically correct saturation behavior.

5.4 Voltage-Gated Repolarizing Currents

Transient Outward Current (I_{to}) — Phase 1

The transient outward K^+ current creates the Phase 1 notch. It activates rapidly during depolarization (r gate, $\tau \sim 1\text{--}5$ ms) and inactivates within tens of milliseconds (s gate, $\tau \sim 5\text{--}50$ ms):

$$I_{to} = \bar{g}_{to} r s (V_m - E_K) \quad (5.4)$$

The maximal conductance $\bar{g}_{to}$ varies dramatically between cell types: 0.294 nS/pF in epicardial cells (deep notch) vs. 0.073 nS/pF in endocardial cells (minimal notch). This transmural gradient is the single largest source of AP shape variation across the ventricular wall and produces the J-point deflection on the surface ECG. In Brugada syndrome, enhanced I_{to} in right ventricular epicardium creates an exaggerated notch that can trigger reentrant arrhythmias.

Rapid Delayed Rectifier (I_{Kr}) — Phase 3

I_{Kr} is the primary repolarization current, carried by hERG (Kv11.1) channels. It has unusual kinetics — fast inactivation at positive voltages, rapid recovery from inactivation during repolarization — that produce a characteristic “resurgent” current during Phase 3:

(5.5)

$$I_{K1} = \bar{g}_{K1} \sqrt{[K^+]_o / 5.4} x_{r1} x_{r2} (V_m - E_K)$$

The $\sqrt{[K^+]_o / 5.4}$ factor captures the experimental observation that hERG conductance scales with the square root of extracellular potassium. The two gating variables x_{r1} (activation, $\tau \sim 10\text{--}100$ ms) and x_{r2} (inactivation, fast) together create the voltage-dependent rectification that makes I_{K1} small during the plateau but large during Phase 3. Clinically, I_{K1} is the most important current in cardiac pharmacology: hERG block is the leading cause of drug-induced Long QT syndrome, and the CiPA initiative focuses heavily on predicting hERG interactions *in silico*.

Slow Delayed Rectifier (I_{Ks}) — Repolarization Reserve

I_{Ks} activates very slowly ($\tau_{x_s} \sim 100\text{--}500$ ms) and provides a "safety net" when I_{K1} is compromised:

$$I_{Ks} = \bar{g}_{Ks} x_s^2 (V_m - E_{Ks}) \quad (5.6)$$

Note that I_{Ks} uses the potassium–sodium reversal potential E_{Ks} (computed from the Goldman equation for K^+ and Na^+ permeability) rather than the pure potassium reversal E_K , because the KvLQT1/minK channel complex has slight Na^+ permeability. The x_s^2 activation gives the current a sigmoid-like voltage dependence with a slower foot than I_{K1} . Under normal conditions, I_{Ks} contributes modestly to repolarization. Its importance emerges when I_{K1} is blocked — then I_{Ks} becomes the backup repolarization pathway. This is the concept of *repolarization reserve*: a healthy heart can tolerate partial block of one K^+ channel because others compensate. Loss of repolarization reserve (e.g., combined $I_{K1} + I_{Ks}$ reduction) dramatically increases arrhythmia risk.

Inward Rectifier (I_{K1}) — Phase 4 and Late Phase 3

$$I_{K1} = \bar{g}_{K1} \sqrt{[K^+]_o / 5.4} x_{K1,\infty}(V_m) (V_m - E_K) \quad (5.7)$$

I_{K1} is unique among TTP06 currents: it has *no dynamic gating variable*. Its rectification is instantaneous, captured entirely by the steady-state function $x_{K1,\infty}(V_m)$, which transitions from ~ 1 below -80 mV to ~ 0 above -40 mV. Physiologically, this reflects the block of Kir2.1 channels by intracellular Mg^{2+} and polyamines at depolarized voltages. The result is a current that behaves as a strong "anchor" at rest (holding V_m near $E_K \approx -88$ mV), vanishes during the plateau (allowing the I_{CaL}/I_{Kr} balance to play out), and then reactivates during Phase 3 to accelerate the final phase of repolarization.

Plateau Potassium Current (I_{pK})

$$I_{pK} = \bar{g}_{pK} \frac{1}{1 + e^{(25 - V_m)/5.98}} (V_m - E_K) \quad (5.8)$$

I_{pK} is a small outward K^+ current active during the plateau. It has no dynamic gates — the sigmoidal voltage dependence is instantaneous. Its contribution to total current is minor ($\bar{g}_{pK} = 0.0146$ nS/pF), but it participates in the repolarization reserve and affects APD at the margins.

5.5 Pumps, Exchangers, and Background Currents

The six voltage-gated currents above shape the AP waveform on a beat-to-beat basis. But the cell must also maintain its ionic gradients over thousands of beats. This is the job of the pumps, exchangers, and background leak currents — a group that is often overlooked but is essential for long-term stability.

Na^+/K^+ ATPase (I_{NaK}) — The Gradient Keeper

(5.9)

$$I_{\text{NaK}} = \frac{P_{\text{NaK}} [\text{K}^+]_o [\text{Na}^+]_i}{([\text{K}^+]_o + K_{m\text{K}})([\text{Na}^+]_i + K_{m\text{Na}})} \cdot \frac{1}{1 + 0.1245 e^{-0.1 F V_m / RT} + 0.0353 e^{-F V_m / RT}}$$

The Na^+/K^+ -ATPase pumps 3 Na^+ ions out and 2 K^+ ions in per ATP hydrolyzed, generating a net outward current. This pump is what maintains the sodium and potassium gradients that all voltage-gated currents depend on. Without it, gradients would run down within minutes and the cell would depolarize. The current is voltage-dependent (less effective at positive voltages), saturates with intracellular Na^+ ($K_{m\text{Na}} = 40$ mM) and extracellular K^+ ($K_{m\text{K}} = 1$ mM), and is the target of cardiac glycosides (digoxin/digitalis): by blocking I_{NaK} , intracellular Na^+ rises, which reduces the driving force for the $\text{Na}^+/\text{Ca}^{2+}$ exchanger, indirectly increasing intracellular Ca^{2+} and strengthening contraction (positive inotropy).

$\text{Na}^+/\text{Ca}^{2+}$ Exchanger (I_{NaCa}) — The Bidirectional Bridge

$$I_{\text{NaCa}} = \frac{k_{\text{NaCa}} ([\text{Na}^+]_i^3 [\text{Ca}^{2+}]_o e^{\gamma F V_m / RT} - \alpha [\text{Na}^+]_o^3 [\text{Ca}^{2+}]_i e^{(\gamma-1) F V_m / RT})}{(K_{m,\text{Na}}^3 + [\text{Na}^+]_o^3)(K_{m,\text{Ca}} + [\text{Ca}^{2+}]_o)(1 + k_{\text{sat}} e^{(\gamma-1) F V_m / RT})} \quad (5.10)$$

The NCX exchanges 3 Na^+ for 1 Ca^{2+} and can operate in either direction depending on the electrochemical gradients. The numerator contains two terms: the first (forward) drives Na^+ in and Ca^{2+} out; the second (reverse, scaled by $\alpha = 2.5$) drives Na^+ out and Ca^{2+} in. The factor α sets the relative strength of reverse-mode transport. The denominator has three components: the K_m saturation terms for Na^+ and Ca^{2+} , and a voltage-dependent saturation factor $1 + k_{\text{sat}} e^{(\gamma-1) F V_m / RT}$ that prevents the exchanger from running at unphysically high rates at extreme potentials. In **forward mode** (dominant during late repolarization and diastole): Na^+ enters, Ca^{2+} is extruded $\rightarrow$ net inward current. This is the primary mechanism for removing calcium from the cytoplasm after each beat (contributing $\sim 28\%$ of calcium removal in human ventricular myocytes, with SERCA handling the remaining $\sim 70\%$). In **reverse mode** (brief, during early depolarization): Na^+ exits, Ca^{2+} enters $\rightarrow$ net outward current. The voltage-dependent partition factor $\gamma \approx 0.35$ determines the relative influence of voltage on the forward vs. reverse rates.

I_{NaCa} is critically important in disease. In heart failure, elevated $[\text{Na}^+]_i$ reduces forward-mode NCX activity, impairing calcium removal and contributing to diastolic dysfunction. In conditions of calcium overload, the inward current generated by forward-mode NCX during diastole can trigger *delayed afterdepolarizations* (DADs) — a major arrhythmia mechanism (Chapter 4, Section 4.5).

Sarcolemmal Ca^{2+} Pump (I_{pCa})

$$I_{\text{pCa}} = \frac{\bar{g}_{\text{pCa}} [\text{Ca}^{2+}]_i}{K_{\text{pCa}} + [\text{Ca}^{2+}]_i} \quad (5.11)$$

The plasma membrane Ca^{2+} -ATPase (PMCA) provides a secondary route for calcium extrusion — slower than NCX but with higher affinity ($K_{\text{pCa}} = 0.0005$ mM). It contributes only $\sim 1\text{--}2\%$ of total calcium removal per beat but is important for fine-tuning the diastolic calcium level over many beats.

Background Leak Currents (I_{bNa} , I_{bCa})

$$I_{\text{bNa}} = \bar{g}_{\text{bNa}} (V_m - E_{\text{Na}}), \quad I_{\text{bCa}} = \bar{g}_{\text{bCa}} (V_m - E_{\text{Ca}}) \quad (5.12)$$

These small Ohmic currents represent the passive leak of Na^+ and Ca^{2+} through non-specific channels. Their conductances are tiny ($\bar{g}_{\text{bNa}} = 2.9 \times 10^{-4}$, $\bar{g}_{\text{bCa}} = 5.92 \times 10^{-4}$ nS/pF), but they serve a critical bookkeeping role: they provide the steady inward Na^+ and Ca^{2+} leak that the pumps (I_{NaK} , I_{pCa} , NCX) must counterbalance. Without these background currents, the model would slowly drift to non-physiological concentrations over hundreds of beats.

△ Key Insight

The balance between background leak currents and pump/exchanger currents is what determines the long-term stability of ionic concentrations. If you modify pump parameters (e.g., reducing I_{NaK} to simulate ischemia), you must simulate hundreds of beats to reach a new steady state — the first few beats will look normal, but gradual Na^+ accumulation will progressively alter the AP shape. This is why cardiac modelers must always run to steady state before collecting data.

5.6 Currents During Each AP Phase — A Visual Map



Figure 5.1. Activity map of all 12 TTP06 membrane currents across the five AP phases. Bar width represents the temporal extent; opacity represents relative magnitude. Red bars are inward (depolarizing), blue bars are outward (repolarizing), gray bars are housekeeping currents active throughout the cycle.

Mathematical Note

All gating variables in TTP06 satisfy the same relaxation ODE we derived in equation (1.3): $\frac{dx}{dt} = (x_\infty - x)/\tau_x$. This uniformity is what makes the Rush–Larsen method (Chapter 8) so powerful: you can integrate *every* gate with the same exponential formula $x^{n+1} = x_\infty + (x^n - x_\infty) e^{-\Delta t/\tau_x}$, regardless of how stiff the kinetics are. The concentration variables ($[\text{Na}^+]_i$, $[\text{K}^+]_i$, etc.) do *not* follow relaxation kinetics and must be integrated with Forward Euler or a similar method.

Mathematical Note

All gating variables in TTP06 satisfy the same relaxation ODE we derived in equation (1.3): $\frac{dx}{dt} = (x_\infty - x)/\tau_x$. This uniformity is what makes the Rush–Larsen method (Chapter 8) so powerful: you can integrate *every* gate with the same exponential formula $x^{n+1} = x_\infty + (x^n - x_\infty)e^{-\Delta t/\tau_x}$, regardless of how stiff the kinetics are. The concentration variables ($[\text{Na}^+]_i$, $[\text{K}^+]_i$, etc.) do *not* follow relaxation kinetics and must be integrated with Forward Euler or a similar method.

5.7 Endo, Epi, and M-Cells: Transmural Heterogeneity

One of TTP06's most important features is its ability to represent three distinct cell types across the ventricular wall, each with a different AP morphology:

Epicardial cells (outer wall): Large I_{to} → deep Phase 1 notch → shorter AP. **Endocardial cells** (inner wall): Small I_{to} → minimal notch → moderate AP duration. **M-cells** (mid-wall): Small I_{Ks} → very long plateau → longest AP duration.

These differences arise from changing just a few maximal conductance values ($\tilde{g}_{\text{to}}$, $\tilde{g}_{\text{Ks}}$) and some kinetic parameters. The fact that the same model structure, with minor parameter adjustments, reproduces three qualitatively different AP shapes is a testament to how well TTP06 captures the underlying biophysics.

⚙️ Engine V5.4 Connection

In Engine V5.4, the TTP06 implementation lives in the `cardiac_sim/ionic/ttp06/` package: `model.py` (the `TTP06Model` class), `currents.py` (all 12 current functions), `gating.py` (steady-states and time constants), `calcium.py` (concentration update equations), and `parameters.py` (the `StateIndex` enum and `TTP06Parameters` dataclass). `V` is stored separately; the 18 ionic states live in a tensor of shape `(n_cells, 18)`. The `cell_type` parameter (`CellType.ENDO`, `CellType.EPI`, `CellType.M_CELL`) selects endo/epi/M-cell variants by switching maximal

⚙️ Engine V5.4 Connection

In Engine V5.4, the TTP06 implementation lives in the `cardiac_sim/ionic/ttp06/` package: `model.py` (the `TTP06Model` class), `currents.py` (all 12 current functions), `gating.py` (steady-states and time constants), `calcium.py` (concentration update equations), and `parameters.py` (the `StateIndex` enum and `TTP06Parameters` dataclass). `V` is stored separately; the 18 ionic states live in a tensor of shape `(n_cells, 18)`. The `cell_type` parameter (`CellType.ENDO`, `CellType.EPI`, `CellType.M_CELL`) selects endo/epi/M-cell variants by switching maximal

conductances and I_{to} kinetics. A single tissue simulation can represent the full transmural gradient by assigning different cell types to different spatial regions — essential for realistic ECG simulation.

5.8 Calcium Handling and CICR

The calcium subsystem is perhaps the most physiologically important part of TTP06, because calcium is the link between electrical excitation and mechanical contraction. The model tracks calcium through three compartments connected by fluxes:

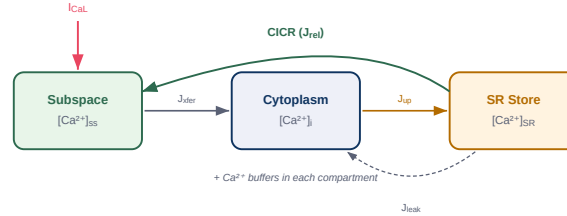


Figure 4.1. Calcium cycling in TTP06. The L-type current I_{CaL} brings Ca^{2+} into the subspace, triggering CICR from the SR (J_{rel}). Ca^{2+} diffuses to the cytoplasm (J_{diff}), drives contraction, then is pumped back into the SR by SERCA (J_{up}).

The CICR mechanism works like a trigger and amplifier:

Trigger: L-type calcium channels open during Phase 2, admitting a small amount of Ca^{2+} into the subspace. This raises $[Ca^{2+}]_{ss}$.

Amplification: The elevated $[Ca^{2+}]_{ss}$ activates ryanodine receptors (RyR) on the SR membrane, which release a much larger bolus of Ca^{2+} from the SR store into the subspace. This is the "release" flux J_{rel} .

Diffusion and contraction: Ca^{2+} diffuses from the subspace to the bulk cytoplasm (J_{diff}), where it binds to troponin C and triggers the contractile machinery of the sarcomere.

Recovery: The SERCA pump (J_{up}) actively transports Ca^{2+} from the cytoplasm back into the SR, refilling the store for the next beat. A small leak (J_{leak}) allows some Ca^{2+} to trickle back out.

Getting this calcium balance right is critical: if the SR is overloaded, spontaneous Ca^{2+} release can trigger delayed afterdepolarizations (DADs) and arrhythmias. If the SR is depleted, contractile force drops and the AP shape distorts. The three-compartment model in TTP06 captures these dynamics with reasonable fidelity while remaining computationally tractable for tissue-scale simulations.

The O’Hara–Rudy 2011 Model

The O’Hara–Rudy dynamic (ORd) model was built from an unprecedented dataset of undiseased human ventricular myocyte recordings. With 41 state variables, 15 membrane currents, and a CaMKII signaling pathway that confers rate adaptation, it represents the current gold standard for human ventricular electrophysiology simulation. This chapter provides the same comprehensive treatment given to TTP06 in Chapter 5 — every current, every equation, organized by physiological function.

6.1 41 State Variables: What and Why

ORd has more than twice the state variables of TTP06 (41 vs. 18). The additional complexity arises from three sources: (1) separate fast and slow inactivation gates for most currents, (2) CaMKII-phosphorylated variants of several gates, and (3) two additional ionic compartments (subspace sodium and potassium). Every state variable has a clear biophysical justification.

29 gating variables (each between 0 and 1, following Hodgkin–Huxley-style kinetics):

Gate(s)	Current	Role	CaMKII variant?
m	I_{Na}	Fast Na^+ activation	No
h_f, h_s	I_{Na}	Fast and slow inactivation	h_{sp} (slow, phosphorylated)
j	I_{Na}	Recovery from inactivation	j_p
m_L	I_{NaL}	Late Na^+ activation	No
h_L	I_{NaL}	Late Na^+ inactivation	h_{Lp}
d	I_{CaL}	Ca^{2+} activation	No
f_f, f_s	I_{CaL}	Fast and slow voltage inactivation	f_{fp}
$f_{Ca,f}, f_{Ca,s}$	I_{CaL}	Fast and slow Ca^{2+} -dependent inactivation	$f_{Ca,fp}$
j_{Ca}	I_{CaL}	Ca^{2+} -dependent recovery	No
n_{Ca}	I_{CaL}	CaMKII phosphorylation transition	—
a	I_{to}	Transient outward activation	a_p
i_F, i_S	I_{to}	Fast and slow inactivation	i_{Fp}, i_{Sp}
$x_{r,f}, x_{r,s}$	I_{Kr}	Fast and slow activation	No
x_{s1}, x_{s2}	I_{Ks}	Slow delayed rectifier gates	No
x_{K1}	I_{K1}	Inward rectifier (now dynamic!)	No

A key design principle of ORd is the *CaMKII-phosphorylated population* concept. For currents modulated by CaMKII (I_{Na} , I_{NaL} , I_{CaL} , I_{to}), the model maintains separate gating variables for the phosphorylated channel population. The total current is then a weighted average: $(1 - \Phi) \cdot I_{normal} + \Phi \cdot I_{phosphorylated}$, where Φ is the CaMKII-dependent phosphorylation fraction. This doubles the gate count for affected currents but allows rate-dependent modulation of channel kinetics.

8 concentration variables (dynamic, changing each beat):

Variable	Description	Typical Value	New vs. TTP06?
$[Na^+]_i$	Intracellular sodium (bulk)	~ 7 mM	Same concept
$[Na^+]_{ss}$	Subspace sodium	~ 7 mM	New

$[K^+]_i$	Intracellular potassium (bulk)	~ 145 mM	Same concept
$[K^+]_{ss}$	Subspace potassium	~ 145 mM	New
$[Ca^{2+}]_i$	Cytoplasmic calcium (free)	~ 0.0001 mM	Same concept
$[Ca^{2+}]_{ss}$	Subspace calcium	~ 0.0001 mM	Same concept
$[Ca^{2+}]_{nSR}$	Network SR calcium	~ 1.2 mM	Split from $[Ca^{2+}]_{SR}$
$[Ca^{2+}]_{jSR}$	Junctional SR calcium	~ 1.2 mM	Split from $[Ca^{2+}]_{SR}$

Additional state variables:

Variable	Description	Role
$CaMK_{trap}$	CaMKII trapped (autophosphorylated) fraction	Long-term kinase memory; drives rate adaptation
V_m	Membrane potential	Primary electrical variable
$J_{rel,NP}$	SR release flux (non-phosphorylated)	RyR release dynamics
$J_{rel,CaMK}$	SR release flux (CaMKII-phosphorylated)	Enhanced RyR release

The most important structural differences from TTP06 are the addition of subspace Na^+ and K^+ compartments (reflecting the fact that these ions also have local concentrations near the membrane), the split of the SR into network and junctional compartments (SERCA pumps into the nSR; RyR releases from the jSR), and the CaMKII trapped state that provides long-term "memory" of recent heart rate.

Intuition

Why does ORd track subspace Na^+ and K^+ ? In TTP06, only Ca^{2+} had a subspace compartment because its concentration can change dramatically during a single beat ($\times 10$ – 100 locally). Na^+ and K^+ concentrations change by much less ($< 1\%$), so a single bulk compartment seemed sufficient. But ORd found that even small Na^+ gradients near the membrane affect NCX and Na^+/K^+ -ATPase activity — the subspace compartments improve accuracy when these transporters are the focus.

6.2 The Complete Current Inventory

ORd models 15 distinct membrane currents. Every current contributes to the total ionic current:

$$\begin{aligned}
 I_{ion} = & \underbrace{I_{Na} + I_{NaL} + I_{CaL}}_{\text{depolarizing}} + \underbrace{I_{to} + I_{Kr} + I_{Ks} + I_{K1} + I_{Kb}}_{\text{repolarizing}} \\
 & + \underbrace{I_{NaK} + I_{NaCa,i} + I_{NaCa,ss} + I_{pCa}}_{\text{pumps/exchangers}} + \underbrace{I_{Nab} + I_{Cab}}_{\text{background}} + \underbrace{I_{ClCa} + I_{Clb}}_{\text{chloride}}
 \end{aligned}
 \tag{6.1}$$

Compared to TTP06's 12 currents, ORd adds I_{NaL} (late sodium), splits I_{NaCa} into two populations, adds chloride currents (I_{ClCa} , I_{Clb}), and replaces the plateau potassium current I_{pK} with a background potassium current I_{Kb} . The L-type calcium current is further decomposed into three ionic components ($I_{CaL,Ca}$, $I_{CaL,Na}$, $I_{CaL,K}$). The table below provides the complete reference.

Current	Type	Ion	AP Phase	Direction	Clinical Relevance
I_{Na}	Voltage-gated channel	Na^+	0 (upstroke)	Inward	Conduction velocity; Class I antiarrhythmics
I_{NaL}	Voltage-gated channel	Na^+	2 (plateau)	Inward	NEW: APD modulation; ranolazine target; LQT3

I_{CaL}	Voltage-gated channel	$Ca^{2+}/Na^{+}/K^{+}$	2 (plateau)	Inward	EC coupling trigger; 3 ionic components; CaMKII-modulated
I_{to}	Voltage-gated channel	K^{+}	1 (notch)	Outward	Transmural heterogeneity; Brugada syndrome
I_{Kr}	Voltage-gated channel	K^{+}	3 (repolarization)	Outward	hERG block; drug-induced LQTS; CiPA assessment
I_{Ks}	Voltage-gated channel	K^{+}	3 (repolarization reserve)	Outward	Rate adaptation (CaMKII); LQT1; β -adrenergic
I_{K1}	Inward rectifier	K^{+}	4 (rest) / 3 (late)	Outward	Resting potential; now with dynamic gate
I_{Kb}	Background leak	K^{+}	All	Outward	Replaces TTP06's I_{pK} ; small repolarization
I_{NaK}	ATP-driven pump	Na^{+}/K^{+}	All	Net outward	Thermodynamic cycle model; digitalis target
$I_{NaCa,i}$	Exchanger	Na^{+}/Ca^{2+}	2–3 / 4	Both*	Bulk cytoplasm NCX; calcium extrusion
$I_{NaCa,ss}$	Exchanger	Na^{+}/Ca^{2+}	2–3 / 4	Both*	NEW: Subspace NCX; local Ca^{2+} control
I_{pCa}	ATP-driven pump	Ca^{2+}	All	Outward	Fine calcium homeostasis (PMCA)
I_{Nab}	Background leak	Na^{+}	All	Inward	Resting Na^{+} influx; balanced by I_{NaK}
I_{Cab}	Background leak	Ca^{2+}	All	Inward	Resting Ca^{2+} influx; GHK formulation
I_{ClCa}	Ca^{2+} -activated channel	Cl^{-}	1–2	Inward†	NEW: Contributes to Phase 1 notch and plateau
I_{Clb}	Background leak	Cl^{-}	All	Inward†	NEW: Background Cl^{-} leak

* I_{NaCa} operates in forward mode (inward, extruding Ca^{2+}) during late repolarization and diastole, and in reverse mode (outward, bringing Ca^{2+} in) briefly during early depolarization. † Cl^{-} currents are depolarizing (inward by convention) because Cl^{-} efflux = inward current, with $E_{Cl} \approx -40$ mV.

6.3 Voltage-Gated Depolarizing Currents

Fast Sodium Current (I_{Na}) — Phase 0

The fast sodium current retains the classical HH structure from TTP06, but with two key upgrades: refitted kinetics from undiseased human data, and CaMKII phosphorylation of the slow inactivation gate. The total current blends non-phosphorylated and phosphorylated channel populations:

$$I_{Na} = \bar{g}_{Na} m^3 [(1 - \Phi_{Na}) h_f j + \Phi_{Na} h_p j_p] (V_m - E_{Na}) \quad (6.2)$$

Here Φ_{Na} is the CaMKII phosphorylation fraction for the sodium channel, and the composite inactivation gate h is a weighted sum of fast and slow components: $h = A_{hf} h_f + (1 - A_{hf}) h_s$, with the weight A_{hf} depending on voltage (fast inactivation dominates at depolarized potentials). The phosphorylated population uses $h_p = A_{hf} h_f + (1 - A_{hf}) h_{sp}$, where h_{sp} has a shifted voltage dependence — CaMKII phosphorylation slows recovery from inactivation, contributing to rate-dependent sodium channel availability. The maximal conductance $\bar{g}_{Na} = 75$ mS/ μ F is substantially larger than TTP06's 14.838 nS/pF (note different units), reflecting the refitting to human data.

Late Sodium Current (I_{NaL}) — Phase 2

The late sodium current is the most important addition in ORd relative to TTP06. It represents a small but persistent Na^{+} influx that flows throughout the plateau — generated by sodium channels that briefly reopen or fail to fully inactivate:

(6.3)

$$I_{\text{NaL}} = \bar{g}_{\text{NaL}} m_L [(1 - \Phi_{\text{NaL}}) h_L + \Phi_{\text{NaL}} h_{Lp}] (V_m - E_{\text{Na}})$$

The maximal conductance $\bar{g}_{\text{NaL}} = 0.0075$ mS/ μF is roughly 1/10,000th of peak I_{Na} , but because I_{NaL} flows for 200+ ms (vs. <2 ms for peak I_{Na}), it carries significant charge and contributes approximately 5% of APD. The activation gate m_L is similar to the fast sodium m gate but with slightly slower kinetics. The inactivation gate h_L has a time constant of ~ 200 ms — orders of magnitude slower than the fast sodium h gate — explaining why this current persists through the plateau.

CaMKII phosphorylation of I_{NaL} (h_{Lp}) shifts the inactivation, increasing the late current at faster heart rates. This creates a positive feedback loop: faster rate $\rightarrow$ more Ca^{2+} entry $\rightarrow$ more CaMKII activation $\rightarrow$ more I_{NaL} $\rightarrow$ more Na^+ entry $\rightarrow$ potentially dangerous if unchecked. Clinically, this pathway is relevant to arrhythmia in heart failure.

△ Key Insight

I_{NaL} is the primary target of **ranolazine**, an antianginal drug that selectively inhibits late sodium current without significantly affecting peak I_{Na} . This specificity is impossible to study computationally without explicitly modeling I_{NaL} — one of the main reasons TTP06 is inadequate for drug safety assessment. In Long QT syndrome type 3 (LQT3), gain-of-function mutations in SCN5A increase I_{NaL} by 2–5 fold, dangerously prolonging the plateau. ORd can reproduce this pathology by simply scaling $\bar{g}_{\text{NaL}}$.

L-Type Calcium Current (I_{CaL}) — Phase 2

The L-type calcium current in ORd is substantially more complex than in TTP06. It is decomposed into three ionic components, reflecting the imperfect selectivity of the Cav1.2 channel:

$$I_{\text{CaL}} = I_{\text{CaL,Ca}} + I_{\text{CaL,Na}} + I_{\text{CaL,K}} \quad (6.4)$$

Each component uses the GHK flux equation with its respective permeability. The calcium component (by far the largest) is:

$$I_{\text{CaL,Ca}} = P_{\text{Ca}} d [(1 - \Phi_{\text{CaL}}) f j_{\text{Ca}} f_{\text{Ca}} + \Phi_{\text{CaL}} f_p j_{\text{Ca}} f_{\text{Ca,p}}] \cdot \Psi_{\text{Ca}}(V_m, [\text{Ca}^{2+}]_{\text{ss}}) \quad (6.5)$$

where Ψ_{Ca} is the GHK flux: $\Psi_{\text{Ca}} = 4 \frac{F^2}{RT} V_m \frac{[\text{Ca}^{2+}]_{\text{ss}} e^{2FV_m/RT} - 0.341 [\text{Ca}^{2+}]_o}{e^{2FV_m/RT} - 1}$, and the composite gates are: $f = A_{ff} f_f + (1 - A_{ff}) f_s$ (voltage inactivation, blending fast and slow components) and $f_{\text{Ca}} = A_{\text{Ca},f} f_{\text{Ca},f} + (1 - A_{\text{Ca},f}) f_{\text{Ca},s}$ (calcium-dependent inactivation).

The CaMKII-phosphorylated population (Φ_{CaL}) uses separate gates f_p and $f_{\text{Ca,p}}$ with shifted kinetics — phosphorylation facilitates channel reopening and slows CDI, increasing calcium influx at faster rates. This CaMKII-mediated enhancement of I_{CaL} is counterbalanced by increased SERCA activity (also CaMKII-dependent), maintaining calcium homeostasis while strengthening contraction — a key component of the positive force-frequency relationship.

The Na^+ and K^+ components follow the same gating but use sodium and potassium permeabilities ($P_{\text{Na}} = 0.00125 P_{\text{Ca}}$ and $P_{\text{K}} = 3.574 \times 10^{-4} P_{\text{Ca}}$) and their respective GHK fluxes. Though individually small, these components contribute to the ion balance over many beats.

Mathematical Note

Why split I_{CaL} into three components? In TTP06, only Ca^{2+} permeation was modeled because Ca^{2+} is the dominant charge carrier. But when tracking Na^+ and K^+ concentrations dynamically over thousands of beats (as ORd does with its subspace compartments), even the small Na^+ and K^+ fluxes through L-type channels contribute to concentration drift. Ignoring them causes slow, insidious accumulation errors that manifest as AP duration changes after hundreds of beats — exactly the kind of subtle bug that plagues long-running simulations.

6.4 Voltage-Gated Repolarizing Currents

Transient Outward Current (I_{to}) — Phase 1

The transient outward current in ORd has separate fast and slow inactivation gates (replacing TTP06's single s gate), plus CaMKII-phosphorylated variants:

$$I_{to} = \bar{g}_{to} [(1 - \Phi_{to}) a_i + \Phi_{to} a_p i_p] (V_m - E_K) \quad (6.6)$$

where $i = A_{iF} i_F + (1 - A_{iF}) i_S$ blends fast and slow inactivation, with the weight A_{iF} depending on voltage. The CaMKII-phosphorylated inactivation $i_p = A_{iF} i_{FP} + (1 - A_{iF}) i_{SP}$ has shifted kinetics. The maximal conductance $\bar{g}_{to}$ retains the dramatic transmural gradient: 0.02 mS/ μ F in endocardium vs. 0.08 mS/ μ F in epicardium and M-cells.

Rapid Delayed Rectifier (I_{Kr}) — Phase 3

I_{Kr} in ORd uses refitted kinetics from undiseased human hERG data. Like TTP06, it has two activation components, but ORd blends fast and slow activation:

$$I_{Kr} = \bar{g}_{Kr} \sqrt{[K^+]_o / 5.4} x_r r_{Kr}(V_m) (V_m - E_K) \quad (6.7)$$

Here $x_r = A_{x_{r,f}} x_{r,f} + (1 - A_{x_{r,f}}) x_{r,s}$ is a weighted blend of fast and slow activation, and $r_{Kr}(V_m)$ is an instantaneous rectification factor that captures the voltage-dependent inactivation of hERG channels. The $\sqrt{[K^+]_o / 5.4}$ factor preserves the extracellular K^+ dependence from TTP06. Notably, I_{Kr} is *not* CaMKII-modulated in ORd — its kinetics are purely voltage-dependent. This is the key current for the CiPA (Comprehensive *in vitro* Proarrhythmia Assay) initiative, which uses ORd as its reference model for predicting drug-induced arrhythmia risk.

Slow Delayed Rectifier (I_{Ks}) — Repolarization Reserve

$$I_{Ks} = \bar{g}_{Ks} K_{s,Ca} x_{s1} x_{s2} (V_m - E_{Ks}) \quad (6.8)$$

A key difference from TTP06 is the calcium-sensitivity factor $K_{s,Ca} = 1 + 0.6 / (1 + (3.8 \times 10^{-5} / [Ca^{2+}]_i)^{1.4})$, which increases I_{Ks} when intracellular calcium is elevated. At resting calcium (~ 100 nM), $K_{s,Ca} \approx 1$ and the current is near baseline. During systolic calcium peaks ($\sim 1 \mu$ M), $K_{s,Ca}$ increases to ~ 1.6 , boosting repolarization reserve precisely when the cell needs it most. Combined with CaMKII modulation of $\bar{g}_{Ks}$ at faster heart rates, this creates robust rate-dependent enhancement of repolarization reserve. The reversal potential E_{Ks} remains a mixed Na^+/K^+ potential, computed from the Goldman equation. The maximal conductance shows strong transmural variation: $\bar{g}_{Ks} = 0.0034$ mS/ μ F (baseline) in endocardial cells, scaled **1.4×** in epicardial cells, with reduced values in M-cells explaining the prolonged M-cell APD and the T-wave morphology on the ECG.

Inward Rectifier (I_{K1}) — Phase 4 and Late Phase 3

$$I_{K1} = \bar{g}_{K1} \sqrt{[K^+]_o} x_{K1} r_{K1}(V_m) (V_m - E_K) \quad (6.9)$$

Unlike TTP06 — where I_{K1} used only an instantaneous steady-state function — ORd gives I_{K1} a *dynamic* gating variable x_{K1} with its own time constant (though still fast, $\tau \sim 1\text{--}10$ ms). The instantaneous rectification factor $r_{K1}(V_m)$ captures the polyamine block at depolarized voltages. This formulation better reproduces the terminal repolarization phase and the resting membrane potential measured in human ventricular myocytes.

Background Potassium Current (I_{Kb})

$$I_{Kb} = \bar{g}_{Kb} x_{Kb}(V_m)(V_m - E_K) \quad (6.10)$$

I_{Kb} replaces TTP06's I_{pK} . The gating is instantaneous (a sigmoidal function of voltage). Like its predecessor, it provides a small outward current during the plateau.

6.5 Pumps, Exchangers, and Background Currents

Na⁺/K⁺ ATPase (I_{NaK}) — Thermodynamic Cycle Model

The Na⁺/K⁺-ATPase in ORd represents the most significant single-equation change from TTP06. Instead of the Hill-type empirical formulation, ORd uses a thermodynamically constrained cycle model that explicitly accounts for the pump's conformational states:

$$I_{NaK} = 30 \frac{z_{Na} \cdot a_1 \cdot k_{1p} - z_{Na} \cdot b_1 \cdot k_{1m}}{1 + \frac{a_1 k_{1p}}{b_2 k_{2m}} + \frac{b_1 k_{1m}}{a_2 k_{2p}} + \frac{a_2 k_{2p}}{b_3 k_{3m}} + \frac{b_2 k_{2m}}{a_3 k_{3p}} + \frac{a_3 k_{3p}}{b_4 k_{4m}} + \frac{b_3 k_{3m}}{a_4 k_{4p}}} \quad (6.11)$$

The rate constants a_i , b_i , k_{ip} , k_{im} depend on $[Na^+]_i$, $[K^+]_o$, ATP, and V_m through microscopic detailed balance. This formulation satisfies the second law of thermodynamics — the pump cannot generate flux against its electrochemical gradient without ATP hydrolysis, a constraint that the simpler TTP06 formulation does not enforce. The practical consequence is more accurate pump behavior under extreme conditions (ischemia, low ATP, hyperkalemia) where the driving forces are close to equilibrium.

Na⁺/Ca²⁺ Exchanger — Split Populations

ORd explicitly models two NCX populations operating in different calcium environments:

$$I_{NaCa} = I_{NaCa,i} + I_{NaCa,ss} \quad (6.12)$$

where $I_{NaCa,i}$ (80% of NCX) senses bulk cytoplasmic $[Ca^{2+}]_i$ and $[Na^+]_i$, while $I_{NaCa,ss}$ (20% of NCX) senses subspace $[Ca^{2+}]_{ss}$ and $[Na^+]_{ss}$. Each uses a thermodynamic formulation:

$$I_{NaCa,x} = 0.8 \bar{g}_{NaCa} \cdot \frac{k_{3p} k_{4pp} [Na^+]_x^3 - k_{3pp} k_{4p} [Ca^{2+}]_x}{(1 + Q_{10})(h_1 \cdot h_2 + h_3)(1/h_7 + h_8)} \quad (6.13)$$

The split is physiologically motivated: NCX molecules near the dyadic cleft "see" a very different calcium signal (high, fast transient from CICR) than those in the bulk cytoplasm (lower, slower rise). In TTP06's single-population NCX, forward-mode extrusion is driven by a spatially averaged $[Ca^{2+}]_i$ that underestimates the local signal near the release sites. The subspace NCX population responds to the actual local calcium in the dyad, providing more accurate calcium cycling and a better match to experimental measurements of NCX current during the action potential.

Sarcolemmal Ca²⁺ Pump (I_{pCa})

$$I_{pCa} = \frac{\bar{g}_{pCa} [Ca^{2+}]_i}{K_{m,pCa} + [Ca^{2+}]_i} \quad (6.14)$$

The PMCA formulation is essentially unchanged from TTP06: a Michaelis–Menten-style saturating function of cytoplasmic calcium. It contributes roughly 1–2% of total calcium extrusion per beat.

Background Leak Currents (I_{Nab} , I_{Cab})

$$I_{\text{Nab}} = \bar{g}_{\text{Nab}} (V_m - E_{\text{Na}}) \quad (6.15)$$

$$I_{\text{Cab}} = P_{\text{Cab}} \cdot \Psi_{\text{Ca}}(V_m, [\text{Ca}^{2+}]_i) \quad (6.16)$$

The sodium background current retains TTP06's Ohmic formulation. However, the calcium background current in ORd switches from an Ohmic to a GHK formulation (Ψ_{Ca}), providing more physically correct behavior at the very low intracellular Ca^{2+} concentrations where the Ohmic approximation breaks down.

Chloride Currents (I_{ClCa} , I_{Clb}) — New in ORd

$$I_{\text{ClCa}} = \bar{g}_{\text{ClCa}} \frac{[\text{Ca}^{2+}]_{\text{ss}}}{K_{d,\text{ClCa}} + [\text{Ca}^{2+}]_{\text{ss}}} (V_m - E_{\text{Cl}}) \quad (6.17)$$

$$I_{\text{Clb}} = \bar{g}_{\text{Clb}} (V_m - E_{\text{Cl}}) \quad (6.18)$$

ORd introduces chloride currents, absent from TTP06. The calcium-activated chloride current I_{ClCa} is gated by subspace calcium and contributes to the Phase 1 notch alongside I_{to} . With $E_{\text{Cl}} \approx -40$ mV (between rest and plateau), this current is depolarizing at rest and repolarizing during the plateau, adding nuance to the Phase 1–2 transition. The background chloride leak I_{Clb} is small but ensures chloride balance.

⚙ Engine V5.4 Connection

Engine V5.4's ORd implementation omits the chloride currents (I_{ClCa} and I_{Clb}). Their combined contribution is typically <1% of total ionic current, and many benchmark ORd implementations (including the CiPA reference) also exclude them. If chloride sensitivity is needed for a specific study, these currents can be added as post-hoc corrections without modifying the core state variable system.

6.6 The CaMKII Signaling Pathway

The calcium/calmodulin-dependent protein kinase II (CaMKII) signaling pathway is the largest conceptual addition in ORd and the primary mechanism underlying rate adaptation. CaMKII is activated by calcium/calmodulin binding and can autophosphorylate to maintain activity even after calcium levels fall — creating a molecular "memory" of recent heart rate.

The active CaMKII fraction in ORd is:

$$\text{CaMK}_{\text{active}} = \text{CaMK}_{\text{bound}} + \text{CaMK}_{\text{trap}} \quad (6.19)$$

where $\text{CaMK}_{\text{bound}} = \text{CaMK}_0 (1 - \text{CaMK}_{\text{trap}}) / (1 + K_m / [\text{Ca}^{2+}]_{\text{ss}})$ is the fraction bound to calcium/calmodulin (instantaneous, set by current subspace Ca^{2+}), and $\text{CaMK}_{\text{trap}}$ is the autophosphorylated fraction that decays slowly:

$$(6.20)$$

$$\frac{d \text{CaMK}_{\text{trap}}}{dt} = \alpha_{\text{CaMK}} \text{CaMK}_{\text{bound}} \text{CaMK}_{\text{active}} - \beta_{\text{CaMK}} \text{CaMK}_{\text{trap}}$$

The time constant for CaMKII trapping is on the order of seconds — much slower than any gating variable. This creates a hierarchical timescale structure:

Milliseconds: Gate kinetics (m, h, d, f, etc.) — respond within a single AP. **Hundreds of milliseconds:** Concentration changes ($[\text{Ca}^{2+}]_i$, $[\text{Na}^+]_i$) — settle over a few beats. **Seconds to minutes:** CaMKII trapping — requires dozens to hundreds of beats to equilibrate.

Active CaMKII phosphorylates four main targets in ORd:

Target	Effect of CaMKII Phosphorylation	Consequence for AP
I_{CaL}	Slowed voltage inactivation; enhanced recovery	Increased Ca^{2+} entry → stronger contraction
I_{NaL}	Slowed inactivation → increased late Na^+	Slight APD prolongation at fast rates
$\text{RyR} / J_{\text{rel}}$	Increased open probability and release fraction	Larger Ca^{2+} transient; potential for diastolic leak
SERCA / J_{up}	Faster Ca^{2+} reuptake into SR	Faster relaxation; maintains SR load at fast rates

The net effect of CaMKII at faster heart rates is: more calcium entry per beat (via enhanced I_{CaL}), faster calcium reuptake (via enhanced SERCA), larger SR release (via enhanced RyR sensitivity), and slightly increased late sodium influx. The enhanced SERCA activity dominates the APD effect — by removing calcium from the cytoplasm faster, it reduces the duration of the calcium transient and the plateau currents that depend on it, resulting in APD shortening. This is how ORd achieves rate adaptation.

⚠ Key Insight

CaMKII makes ORd simulations significantly more expensive than TTP06. With 41 state variables (vs. 18), the per-cell cost roughly doubles. But the bigger issue is the slow CaMKII equilibration time: reaching steady state after a rate change requires 300–600 beats (2–5 minutes of simulated time), compared to 50–100 beats for TTP06. For tissue-scale studies where CaMKII effects are not the primary focus, TTP06 remains the pragmatic choice.

6.7 Currents During Each AP Phase — A Visual Map



Figure 6.1. Activity map of all ORd membrane currents across the five AP phases. Bar width represents temporal extent; opacity represents relative magnitude. Red bars are inward (depolarizing), blue bars are outward (repolarizing), gray bars are housekeeping currents, gold bars are chloride currents. Compare with Figure 5.1 for TTP06 — note the addition of I_{NaL} (persistent through the plateau), split NCX populations, and chloride currents.

Mathematical Note

As in TTP06, all gating variables in ORd satisfy the relaxation ODE $dx/dt = (x_{\infty} - x)/\tau_x$. The Rush–Larsen exponential integrator (Chapter 8) applies to every gate, regardless of CaMKII phosphorylation status. The key numerical difference is that ORd has 29 gates vs. TTP06’s 12, so the Rush–Larsen evaluation cost roughly doubles. The concentration ODEs remain non-relaxation and require explicit or implicit integration.

6.8 Endo, Epi, and M-Cells: Transmural Heterogeneity

Like TTP06, ORd supports three transmural cell types. However, ORd’s transmural differences are more nuanced, affecting more parameters:

Parameter	Endocardial	Epicardial	M-Cell	Physiological Basis
$\bar{g}_{to}$	0.02	0.08 (4×)	0.08 (4×)	Large epi/M-cell notch; minimal endo notch
P_{CaL}	1×	1.2×	2.5×	Enhanced plateau in M-cells
$\bar{g}_{Kr}$	1×	1.3×	0.8×	M-cell APD prolongation
$\bar{g}_{Ks}$	0.0034	0.00476 (1.4×)	Reduced	M-cell APD prolongation (reduced repol. reserve)
$\bar{g}_{K1}$	1×	1.2×	1.3×	Resting potential variation
$\bar{g}_{NaL}$	1×	0.6×	1×	Reduced late Na^+ in epicardium
I_{NaCa}	1×	1.1×	1.4×	Enhanced Ca^{2+} extrusion in M-cells
I_{NaK}	1×	0.9×	0.7×	Reduced pump activity in M-cells

The M-cell phenotype in ORd arises from the combination of reduced I_{Ks} (less repolarization reserve), reduced I_{Kr} (**0.8×**), enhanced I_{CaL} (**2.5×**), and reduced I_{NaK} (**0.7×**). These changes collectively produce the longest APD of any cell type — a feature that is important for generating the T-wave on the surface ECG. The transmural APD gradient (endo < epi < M-cell) drives the voltage gradient that produces the terminal T-wave inscription.

⚙ Engine V5.4 Connection

In Engine V5.4, both TTP06 and ORd implement the same `IonicModel` interface. The `cell_type` parameter selects transmural variant by switching maximal conductances. A simulation can mix cell types spatially — endocardial cells on the inner surface, epicardial on the outer surface, M-cells in between — reproducing the full transmural gradient with a single model class.

6.9 Calcium Handling: Two SR Compartments

ORd refines the calcium subsystem beyond TTP06’s three-compartment model (subspace, cytoplasm, SR) by splitting the SR into two functionally distinct regions:

Network SR (nSR): The bulk of the SR volume. SERCA pumps calcium from the cytoplasm into the nSR. Calcium diffuses freely within the nSR toward the junctional terminals.

Junctional SR (jSR): The terminal cisternae directly apposed to the T-tubule membrane, containing the RyR2 clusters. This compartment empties rapidly during CICR and must refill from the nSR between beats.

The calcium fluxes in ORd are:

J_{up} (SERCA2a, cytoplasm $\rightarrow$ nSR): CaMKII-enhanced pump with phospholamban relief. At faster rates, CaMKII increases the maximum pump rate, enabling faster reuptake.

J_{tr} (nSR $\rightarrow$ jSR diffusion): A simple first-order transfer with time constant $\tau_{tr} = 60$ ms. This creates a delay between SERCA refilling and jSR recovery — the jSR content lags behind the nSR, explaining why premature beats (before full jSR recovery) produce weaker calcium transients.

J_{rel} (RyR, jSR $\rightarrow$ subspace): CaMKII-enhanced release. The release flux is proportional to the jSR-subspace calcium gradient and is activated by subspace calcium (CICR trigger). CaMKII phosphorylation increases the RyR open probability, producing larger releases at faster rates. The ORd release model blends non-phosphorylated and CaMKII-phosphorylated populations, with the total release $J_{rel} = (1 - \Phi) J_{rel,NP} + \Phi J_{rel,CaMK}$.

J_{xfer} (subspace $\rightarrow$ cytoplasm diffusion): First-order transfer with time constant $\tau_{xfer} = 8$ ms. Combined with the subspace Na^+ and K^+ diffusion, this maintains the subspace as a distinct microenvironment that equilibrates with the bulk cytoplasm on a timescale of tens of milliseconds.

J_{leak} (nSR $\rightarrow$ cytoplasm): A small passive leak proportional to $[Ca^{2+}]_{nSR}$. This leak is the starting point for modeling diastolic SR calcium leak in heart failure — by increasing the leak rate, one can simulate the RyR2 dysfunction that drives spontaneous calcium waves and DADs.

6.10 TTP06 vs. ORd at a Glance

The following table summarizes the key differences between TTP06 and ORd for quick reference:

Feature	TTP06	ORd
State variables	18	41
Membrane currents	12	15 (+ chloride)
Data source	Mixed species/cell lines	Undiseased human ventricle
Late Na^+ current	Not modeled	Explicit I_{NaL}
CaMKII signaling	Not modeled	Full pathway; rate adaptation
SR compartments	1 (single SR)	2 (nSR + jSR)
NCX populations	1 (bulk)	2 (bulk + subspace)
Na^+/K^+ pump	Hill-type empirical	Thermodynamic cycle
Subspace ions	Ca^{2+} only	Ca^{2+} , Na^+ , K^+
Beats to steady state	$\sim 50\text{--}100$	$\sim 300\text{--}600$ (CaMKII)
Cost per cell per step	1 $\times$ (baseline)	$\sim 2\text{--}2.5\times$
Best for	Wave dynamics, large-scale 3D, prototyping	Drug safety (CiPA), rate-dependent phenomena

⚙️ Engine V5.4 Connection

Engine V5.4 implements both models behind the same `IonicModel` interface. Switching from TTP06 to ORd requires changing a single configuration parameter — the spatial solver, mesh, operator splitting strategy, and output pipeline remain identical. The `step(V, ionic_states, dt)` method handles the internal complexity, whether updating TTP06's 18 or ORd's 40 ionic state variables (`V` stored separately in both). The `RushLarsenSolver` calls each model's `compute_gate_steady_states()` and `compute_gate_time_constants()` to perform the exponential update, and `compute_concentration_rates()` for Forward Euler — all defined identically on the `IonicModel` ABC. This abstraction is what allows the same tissue solver to work with any ionic model.

With both ionic models thoroughly characterized — from 18-variable TTP06 to 41-variable ORd — we now have the single-cell building blocks that will be embedded at every spatial point in the tissue. In Part II, we add the spatial dimension: diffusion, the monodomain equation, and the numerical methods that make tissue-scale simulation possible.

PART II

Tissue-Level Monodomain Modeling

Connecting millions of cells with diffusion — from the reaction–diffusion PDE to the numerical solvers that make whole-heart simulation possible.

The Monodomain Equation

A single cell model tells you what happens at one point. To simulate a wave of activation spreading across the heart, we need to couple millions of these cells through the physics of electrical diffusion. The monodomain equation is this coupling — and it is the equation that Engine V5.4 was built to solve.

7.1 From Single Cell to Tissue: Adding Diffusion

Imagine you have a row of light bulbs connected by wires. If you turn on one bulb in the middle, electrical current flows through the wires to its neighbors, and they start to glow too — then *their* neighbors light up, and so on down the line. That is exactly what happens in cardiac tissue. One cell depolarizes (the "bulb turns on"), and current flows through gap junctions (the "wires") to neighboring cells, triggering them in sequence. This chain reaction is the propagating action potential.

In Chapter 1, we wrote the membrane current equation for a single cell: $C_m dV_m/dt = -I_{ion}$. This says "the voltage changes because ions flow through channels." But in tissue, there is a second source of current: *electrotonic current* from neighboring cells, flowing through gap junctions.

When cell A depolarizes before cell B, a voltage gradient exists between them, and current flows from A to B through the intracellular pathway. This is just Ohm's law — voltage difference divided by resistance gives current — written in continuous form:

$$\mathbf{J}_i = -\mathbf{D}_i \nabla V_m$$

where $\mathbf{D}_i$ is the *effective intracellular conductivity tensor* — it encodes both the gap-junction coupling strength and the geometric anisotropy of the tissue. The divergence of this current, $\nabla \cdot (\mathbf{D}_i \nabla V_m)$, gives the net current arriving at each point from its neighbors. Adding this to the single-cell equation gives us the monodomain equation.

7.2 The Reaction–Diffusion Formulation

Before writing the equation, let us understand what it will say in plain language. At every point in the tissue, the voltage changes for exactly two reasons: (1) ion channels at that point open and close, pushing the voltage up or down — this is the *reaction*, the local chemistry; (2) current leaks in from depolarized neighbors or out to resting neighbors through gap junctions — this is the *diffusion*, the spatial coupling. The monodomain equation is simply the mathematical statement that "voltage change = reaction + diffusion."

Written out, the monodomain equation is:

$$\chi C_m \frac{\partial V_m}{\partial t} = \nabla \cdot (\mathbf{D} \nabla V_m) - \chi I_{ion}(V_m, \mathbf{w}) + I_{stim} \quad (7.1)$$

This is a *reaction–diffusion* equation — one of the most important classes of PDEs in mathematical biology. The "reaction" is the ionic current I_{ion} (local chemistry at each cell), and the "diffusion" is $\nabla \cdot (\mathbf{D} \nabla V_m)$ (spatial coupling between cells). The stimulus current I_{stim} represents external pacing.

Intuition

The monodomain equation says: "at every point in the tissue, the voltage changes for two reasons — (1) ion channels open and close locally (the reaction), and (2) current leaks in from depolarized neighbors or out to resting neighbors (the diffusion)." These two processes compete: the reaction creates sharp voltage fronts, while the diffusion smooths them out. The balance determines the conduction velocity — typically 60–80 cm/s along fibers in healthy ventricular tissue.

The state vector $\mathbf{w}$ contains all the gating variables and concentrations from the ionic model (18 for TTP06, 40 for ORd). These evolve by their own ODEs at each spatial point:

$$\frac{\partial \mathbf{w}}{\partial t} = \mathbf{g}(V_m, \mathbf{w}) \tag{7.2}$$

Note that $\mathbf{w}$ has *no diffusion term* — gating variables do not diffuse from cell to cell. Only voltage (and in the bidomain model, extracellular potential) spreads spatially. This locality of the ionic state is what makes operator splitting so natural.

7.3 Conductivity, Fibers, and Anisotropy

Think of cardiac tissue like a bundle of straws, all pointing roughly the same direction. If you push water into one straw, it flows easily along the straw's length but very poorly sideways into neighboring straws. Heart muscle works the same way: myocytes are long, thin cells connected end-to-end by gap junctions. Electrical current zips along the fiber direction about *three times faster* than it crosses fibers. This directional preference is called *anisotropy*, and it shapes the entire wavefront: elliptical in a tissue slab, spiraling in the ventricle wall where fibers rotate transmurally from epicardium to endocardium.

Mathematically, a single number (scalar conductivity) cannot capture "fast in one direction, slow in another." We need a *tensor* — essentially a 2×2 or 3×3 matrix that converts a voltage gradient in any direction into the resulting current in that direction. If $\mathbf{f}$ is the local fiber direction (a unit vector), the conductivity tensor is:

$$\mathbf{D} = d_{\parallel} \mathbf{f} \mathbf{f}^T + d_{\perp} (\mathbf{I} - \mathbf{f} \mathbf{f}^T) \tag{7.3}$$

where $d_{\parallel}$ and $d_{\perp}$ are the conductivities along and across fibers, respectively. Typical values are $d_{\parallel} \approx 0.174$ and $d_{\perp} \approx 0.019$ mm²/ms, giving an anisotropy ratio of about 3:1 in conduction velocity (since $CV \propto \sqrt{d}$). In 3D, a sheet-normal direction adds a third axis with even lower conductivity.

7.4 Spatial Discretization: What It Means Physically

The continuous diffusion operator $\nabla \cdot (\mathbf{D} \nabla V_m)$ answers a simple physical question: *how much net electrical current is flowing into this point from its neighbors?* Spatial discretization replaces this continuous question with a discrete one: given the voltage at every mesh point, compute the net current arriving at each point from its neighbors. This computation turns out to be a matrix-vector product.

Think of it this way. If node $\mathbf{i}$ is at -85 mV (resting) and its neighbor $\mathbf{j}$ is at $+20$ mV (depolarized), there is a voltage difference of 105 mV driving current from $\mathbf{j}$ to $\mathbf{i}$ through the intracellular resistance. The amount of current depends on the conductivity between them and the distance. Summing up these pairwise contributions from all neighbors of node $\mathbf{i}$ gives the net diffusive current — and that sum is exactly one row of the matrix-vector product $\mathbf{K} \cdot \mathbf{V}$.

Three classical methods exist for building this matrix. Each approaches the problem from a different mathematical direction, but all three produce the same object: a *sparse matrix* that encodes how current flows through the tissue mesh. The matrix is sparse because each node couples only to its immediate neighbors — a node in the middle of a 3D mesh might have 6–26 neighbors out of millions of total nodes.

Method	Core Idea	Mesh Requirement	Best For
FDM	Replace derivatives with neighbor differences	Regular Cartesian grid	Tissue slabs, GPU computing, fast prototyping
FEM	Reformulate as integral (weak form)	Any: triangles, tetrahedra, hexahedra	Realistic heart geometries, curved boundaries
FVM	Balance fluxes across control-volume faces	Any with orthogonal dual mesh	Sharp conductivity jumps (scar/healthy boundary)

We will work through each method with the same running philosophy: *show the matrix*. For each method, we will build the actual stiffness matrix for a tiny example mesh, inspect its structure, and verify that it makes physical sense. Then in Section 7.9, we will connect these matrices back to the ionic models from Chapters 5–6, showing how TTP06's 12 currents or ORd's 15 currents feed into the full system.

7.5 Finite Difference Method (FDM)

The Central Idea

FDM is the most direct approach: place nodes on a regular grid with uniform spacing h , and approximate the second derivative using the values at neighboring grid points. The fundamental approximation in 1D is:

$$d \frac{\partial^2 V}{\partial x^2} \Big|_i \approx \frac{d}{h^2} (V_{i-1} - 2V_i + V_{i+1})$$

This is second-order accurate (error $\sim O(h^2)$), derived from Taylor expansion. But the deeper insight is *geometric*: the expression $V_{i-1} - 2V_i + V_{i+1}$ measures how much V_i deviates from the average of its neighbors. If V_i is higher than the average $\frac{1}{2}(V_{i-1} + V_{i+1})$, the Laplacian is negative — current flows outward, pulling V_i down. If V_i is lower than the average, current flows inward, pushing V_i up. Diffusion always acts to smooth out voltage differences.

Worked Example: 5-Node 1D Cable

Consider a tiny 1D cardiac cable with 5 nodes, spacing $h = 0.2$ mm, and uniform conductivity $d = 0.1$ mm²/ms. The no-flux (Neumann) boundary condition means no current flows out of the cable ends.

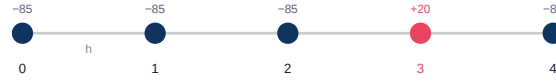


Figure 7.1. A 5-node 1D cable with node 3 depolarized to +20 mV (wavefront). All other nodes at resting potential −85 mV.

The FDM stencil $[1, -2, 1]$ is applied at each interior node. At boundary nodes (0 and 4), the no-flux condition $\partial V / \partial x = 0$ modifies the stencil to $[-1, 1]$ (the "missing" neighbor is replaced by mirroring). Collecting all stencils into a matrix, we get the **stiffness matrix** $\mathbf{K}$:

$$\mathbf{K} = \frac{d}{h^2} \begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & 1 \end{pmatrix} = 2.5 \begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & 1 \end{pmatrix}$$

Three properties of $\mathbf{K}$ to notice: (1) it is *symmetric* — the coupling from node i to j equals the coupling from j to i ; (2) it is *tridiagonal* — each node couples only to its immediate neighbors; (3) every row sums to zero — applying a uniform voltage $\mathbf{V} = c\mathbf{1}$ gives $\mathbf{K} \cdot \mathbf{V} = \mathbf{0}$, meaning diffusion does nothing to a spatially constant voltage (as it should).

Now let us compute the diffusion term $-\mathbf{K} \cdot \mathbf{V}$ for the wavefront scenario $\mathbf{V} = [-85, -85, -85, +20, -85]^T$ (mV):

$$-\mathbf{K} \cdot \mathbf{V} = -2.5 \begin{pmatrix} 1(-85) + (-1)(-85) \\ (-1)(-85) + 2(-85) + (-1)(-85) \\ (-1)(-85) + 2(-85) + (-1)(20) \\ (-1)(-85) + 2(20) + (-1)(-85) \\ (-1)(20) + 1(-85) \end{pmatrix} = -2.5 \begin{pmatrix} 0 \\ 0 \\ -190 \\ 210 \\ -105 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 475 \\ -525 \\ 262.5 \end{pmatrix}$$

Read this result physically:

Node 2 (-85 mV, next to wavefront): receives **+475** mV/ms of diffusive current — a large positive push from the depolarized node 3. This is the electrotonic current that will soon trigger node 2's sodium channels, propagating the wave to the left.

Node 3 ($+20$ mV, the wavefront): loses **-525** mV/ms — current flows out to both resting neighbors. The depolarized cell is the "source" that drives the wavefront forward.

Node 4 (-85 mV, behind the wavefront): receives **+262.5** mV/ms from node 3. Only half the push that node 2 gets, because node 4 has only one neighbor (boundary).

Nodes 0-1 (far from wavefront): zero diffusive current — all neighbors have the same voltage, so there is nothing to diffuse.

Intuition

This example reveals the core mechanism of wave propagation. The depolarized region (node 3) acts as a current source, pushing electrotonic current into the resting neighbors (nodes 2 and 4). When this current is strong enough to bring a neighbor above threshold (~ -65 mV), the neighbor's sodium channels fire explosively, depolarizing it and extending the wavefront. The matrix **K** encodes *all* of this neighbor-to-neighbor coupling.

The 5-Point and 9-Point Stencils (2D)

In two dimensions with isotropic conductivity **d**, the Laplacian is approximated by the **5-point stencil**:

$$d \nabla^2 V|_{i,j} \approx \frac{d}{h^2} (V_{i+1,j} + V_{i-1,j} + V_{i,j+1} + V_{i,j-1} - 4V_{i,j})$$

Same idea as 1D: the value at the center minus the average of its four axis-aligned neighbors. The stiffness matrix now has 5 nonzeros per row (the center node and its four neighbors). On an $N \times N$ grid, **K** is $N^2 \times N^2$ — potentially millions of rows — but with only $\sim 5N^2$ nonzero entries. This extreme sparsity is what makes large-scale cardiac simulation tractable.

When the conductivity tensor **D** has off-diagonal entries (because cardiac fibers are rotated relative to the grid axes):

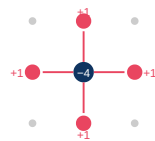
$$\mathbf{D} = \begin{pmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{pmatrix}$$

the 5-point stencil can only handle the diagonal terms D_{xx} and D_{yy} . The cross-derivative $D_{xy} \partial^2 V / \partial x \partial y$ requires the **9-point stencil**, which includes diagonal neighbors:

$$D_{xy} \frac{\partial^2 V}{\partial x \partial y} \Big|_{i,j} \approx \frac{D_{xy}}{4h^2} (V_{i+1,j+1} - V_{i+1,j-1} - V_{i-1,j+1} + V_{i-1,j-1})$$

In 3D, the full anisotropic operator requires a **27-point stencil** — the node plus all 26 neighbors in a $3 \times 3 \times 3$ cube. The stiffness matrix then has up to 27 nonzeros per row.

5-Point (Isotropic)



9-Point (Anisotropic)

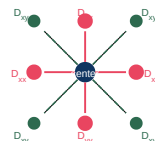


Figure 7.2. FDM stencils for the 2D diffusion operator. Left: The 5-point stencil uses axis-aligned neighbors only (isotropic or axis-aligned anisotropic). Right: The 9-point stencil adds diagonal neighbors (green) for the cross-derivative D_{xy} terms from rotated fiber fields.

Key Insight

When to use FDM: FDM is fastest, simplest to implement, and cache-friendly — ideal for regular geometries (tissue slabs, rectangular patches), GPU computation, and rapid prototyping. Its limitation is that it *requires a structured Cartesian grid*. You cannot fit a regular grid to the curved walls of the left ventricle without staircase artifacts at the boundary. For realistic anatomical geometries, use FEM or FVM instead.

7.6 Finite Element Method (FEM)

Why FEM? The Geometry Problem

Real hearts are not rectangular. The left ventricle is a thick-walled ellipsoidal shell with curved boundaries, trabeculations, and papillary muscles. To represent this geometry faithfully, we need meshes made of triangles (2D) or tetrahedra (3D) that can conform to any shape. FDM cannot do this. FEM can.

The mathematical trick that enables unstructured meshes is a change of perspective. Instead of approximating derivatives pointwise (as FDM does), FEM reformulates the PDE in *weak form* — multiplying by a test function and integrating. This seemingly abstract step has a profound practical consequence: it lowers the smoothness requirement on the solution, allowing us to use simple piecewise-linear "hat functions" on arbitrary triangulations.

Weak Form: What Integration by Parts Really Does

Before diving into the math, let us understand the idea in plain language. We want to ask: "does my approximate solution satisfy the PDE?" But checking this point-by-point (like FDM does) fails on triangular meshes because our piecewise-linear hat functions have *kinks* — the second derivative is undefined at element boundaries. So instead, we check the PDE *on average*, by multiplying both sides by a smooth "test function" v and integrating over the whole domain. If the average checks out for *every* possible test function, the solution must be correct. This is the weak form.

The magic trick is *integration by parts* (the multidimensional version of the product rule from calculus). It shifts one derivative from V_m onto the test function v , so we only need first derivatives of both — and first derivatives of our hat functions are perfectly well-defined (they are just constant slopes on each triangle). This is why FEM can use simple piecewise-linear approximations on arbitrary meshes, while FDM needs smooth grids.

Now let us see this concretely. Start with the monodomain diffusion term. Multiply by a test function v , integrate over the domain Ω , and apply integration by parts (the divergence theorem):

$$\underbrace{\int_{\Omega} \chi C_m \frac{\partial V_m}{\partial t} v \, dx}_{\text{time derivative}} + \underbrace{\int_{\Omega} \nabla V_m \cdot (\mathbf{D} \nabla v) \, dx}_{\text{diffusion (after IBP)}} = - \underbrace{\int_{\Omega} \chi I_{\text{ion}} v \, dx}_{\text{ionic current}} + \underbrace{\int_{\Omega} I_{\text{stim}} v \, dx}_{\text{stimulus}}$$

The crucial step is the integration by parts on the diffusion term. The original PDE contains $\nabla \cdot (\mathbf{D} \nabla V_m)$ — a second derivative of V_m . After integration by parts, we have $\nabla V_m \cdot (\mathbf{D} \nabla v)$ — only *first* derivatives of both V_m and v . The boundary integral vanishes because of the no-flux condition (physically: no current flows out of the heart). This matters because first derivatives of piecewise-linear functions are well-defined (they are piecewise constant), while second derivatives are not.

From Integrals to Matrices

Expand the voltage as $V_m(\mathbf{x}, t) = \sum_j V_j(t) \varphi_j(\mathbf{x})$, where each φ_j is a "hat function" that equals 1 at node j and 0 at all other nodes. Choose $v = \varphi_i$ for each node i . The integrals become matrix entries:

$$\mathbf{M} \dot{\mathbf{V}}_m + \mathbf{K} \mathbf{V}_m = \mathbf{F}$$

with two matrices and one vector:

Mass matrix $M_{ij} = \int_{\Omega} \chi C_m \varphi_i \varphi_j d\mathbf{x}$ — comes from the time derivative. Physically, it weights each node by the "volume" it represents. It is symmetric, positive definite, and sparse.

Stiffness matrix $K_{ij} = \int_{\Omega} \nabla \varphi_i \cdot (\mathbf{D} \nabla \varphi_j) d\mathbf{x}$ — encodes the diffusive coupling. This is the FEM analogue of the FDM matrix we built above. It is symmetric, positive semi-definite (row sums = 0 under Neumann BC), and sparse.

Load vector $F_i = - \int_{\Omega} \chi I_{\text{ion}} \varphi_i d\mathbf{x} + \int_{\Omega} I_{\text{stim}} \varphi_i d\mathbf{x}$ — contains the ionic current and stimulus. This is where Chapters 5–6 connect: at each node, TTP06 or ORd evaluates I_{ion} , which becomes an entry in $\mathbf{F}$.

Worked Example: 4-Node, 2-Triangle Mesh

Consider the simplest 2D mesh: a unit square $[0, h] \times [0, h]$ divided into two triangles. We use isotropic conductivity d and spacing h .

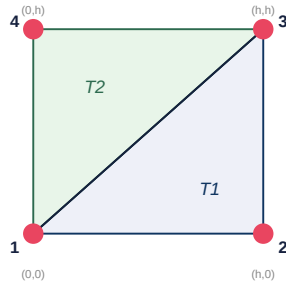


Figure 7.3. A minimal 2D FEM mesh: 4 nodes, 2 triangles (T_1 with nodes 1-2-3, T_2 with nodes 1-3-4). The diagonal splits the square into two right triangles of area $h^2/2$.

FEM assembles the global stiffness matrix element-by-element. For each triangle, we compute a small *element stiffness matrix* (3×3 for a triangle with 3 nodes), then "scatter" its entries into the correct positions in the global matrix.

For a triangle with vertices at $\mathbf{x}_a, \mathbf{x}_b, \mathbf{x}_c$ and area A , the gradients of the linear basis functions are constant vectors: $\nabla \varphi_a = \frac{1}{2A} (y_b - y_c, x_c - x_b)^T$ (and cyclic permutations). The element stiffness entry is:

$$K_{ab}^e = A \nabla \varphi_a \cdot (d \nabla \varphi_b) = \frac{d}{4A} [(y_b - y_c)(y_c - y_a) + (x_c - x_b)(x_a - x_c)]$$

Triangle T_1 (nodes 1, 2, 3) with vertices $(0, 0), (h, 0), (h, h)$, area = $h^2/2$:

$$\mathbf{K}^{T_1} = \frac{d}{2} \begin{pmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{pmatrix} \quad (\text{for nodes 1, 2, 3})$$

Triangle T_2 (nodes 1, 3, 4) with vertices $(0, 0), (h, h), (0, h)$, area = $h^2/2$:

$$\mathbf{K}^{T_2} = \frac{d}{2} \begin{pmatrix} 1 & 0 & -1 \\ 0 & 1 & -1 \\ -1 & -1 & 2 \end{pmatrix} \quad (\text{for nodes 1, 3, 4})$$

Global assembly: Add element matrices into the global 4×4 matrix. Node 1 appears in both triangles, so its diagonal entry gets contributions from both. Node 2 appears only in T_1 ; node 4 only in T_2 . The result:

$$\mathbf{K}_{\text{global}} = \frac{d}{2} \begin{pmatrix} 2 & -1 & 0 & -1 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ -1 & 0 & -1 & 2 \end{pmatrix}$$

Check: every row sums to zero (conservation under Neumann BC). The matrix is symmetric and positive semi-definite with a single zero eigenvalue (the constant mode). Physically, each node couples to its connected neighbors with equal weight $-d/2$ — which makes sense because the mesh is symmetric and the conductivity is isotropic.

Notice what happened during assembly: the element stiffness matrices *overlap* at shared nodes. Node 1's diagonal entry is $d/2 + d/2 = d$ (contributions from both triangles). Node 3's diagonal entry is also $d/2 + d/2 = d$. The off-diagonal entries come from whichever element(s) connect the two nodes. Nodes 2 and 4 are *not* directly connected by any element edge, so $\mathbf{K}_{24} = \mathbf{0}$ — even though they are geometric neighbors across the diagonal.

The Mass Matrix: Consistent vs. Lumped

The mass matrix for a linear triangle with area A has entries:

$$M_{ab}^e = \frac{\chi C_m A}{12} \times \begin{cases} 2 & \text{if } a = b \\ 1 & \text{if } a \neq b \end{cases}$$

For our two-triangle mesh (each with area $h^2/2$), the assembled **consistent mass matrix** is:

$$\mathbf{M}_{\text{consistent}} = \frac{\chi C_m h^2}{12} \begin{pmatrix} 4 & 1 & 2 & 1 \\ 1 & 2 & 1 & 0 \\ 2 & 1 & 4 & 1 \\ 1 & 0 & 1 & 2 \end{pmatrix}$$

This matrix is full (no zero entries for connected nodes) and couples each node to its neighbors even for the time derivative. This coupling is mathematically correct but computationally inconvenient: for explicit time stepping, you would need to solve $\mathbf{M} \dot{\mathbf{V}} = \mathbf{b}$ at every timestep, which requires inverting $\mathbf{M}$.

The **lumped mass matrix** avoids this by concentrating each row's total mass onto the diagonal:

$$\mathbf{M}_{\text{lumped}} = \frac{\chi C_m h^2}{3} \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

The diagonal entries reflect the area "owned" by each node: nodes 1 and 3 (shared by two triangles) each own $2 \times h^2/6 = h^2/3$ of area, while nodes 2 and 4 (each in one triangle) own $h^2/6$. With a lumped mass matrix, explicit time stepping becomes trivial: $\dot{\mathbf{V}}_i = \mathbf{b}_i / M_{ii}$ — just a pointwise division, no linear solve required.

Mathematical Note

Mass lumping introduces an $O(h^2)$ error — the same order as the spatial discretization itself. So for P1 elements, lumping does not degrade the overall accuracy order. This is why lumped mass is the default choice in cardiac simulation: it preserves the convergence rate while eliminating the mass-matrix solve. The only case where consistent mass matters is when using higher-order elements (P2 or above), where lumping can reduce the accuracy benefit.

Element Types

For cardiac simulations, **P1 elements** (linear triangles in 2D, linear tetrahedra in 3D) are the workhorse, providing $O(h^2)$ accuracy with minimal overhead. The gradients are constant within each element, making the integrals trivial. **P2 elements** (quadratic) achieve $O(h^3)$ accuracy but at $\sim 8\times$ the memory cost. For the monodomain problem — where the ionic model, not the linear solve, is the bottleneck — P1 elements with mesh refinement are generally preferred.

⚠ Key Insight

When to use FEM: FEM is the method of choice for realistic anatomical heart geometries. It handles any mesh shape (triangles, tetrahedra, hexahedra), naturally accommodates curved boundaries, and has rigorous mathematical convergence theory. Its main cost is the assembly step (computing element stiffness matrices and scattering into the global matrix) and the need for a mass matrix. For structured Cartesian grids, FDM is faster; for problems dominated by sharp conductivity jumps, FVM may be more appropriate.

⚙ Engine V5.4 Connection

Engine V5.4 supports P1 FEM on triangular (2D) and tetrahedral (3D) meshes. The mass matrix can be either consistent or lumped (diagonal). The stiffness matrix $\mathbf{K}$ is assembled element-by-element and stored in CSR (Compressed Sparse Row) format. Both explicit solvers (which need only $\mathbf{KV}$ products) and implicit solvers (which solve $(\mathbf{M} + \alpha \mathbf{K})\mathbf{V} = \mathbf{b}$) operate through the same assembled matrices.

7.7 Finite Volume Method (FVM)

The Conservation Principle

FVM starts from the most physically intuitive formulation: conservation. Around each node $\mathbf{i}$, draw a *control volume* $\Omega_{\mathbf{i}}$ — the region of space “owned” by that node. Integrate the monodomain equation over $\Omega_{\mathbf{i}}$:

$$\chi C_m |\Omega_{\mathbf{i}}| \frac{dV_{\mathbf{i}}}{dt} = \underbrace{\sum_{j \in \mathcal{N}(\mathbf{i})} F_{ij}}_{\text{net flux from neighbors}} - \chi |\Omega_{\mathbf{i}}| I_{\text{ion},\mathbf{i}}$$

where $|\Omega_{\mathbf{i}}|$ is the volume (or area, in 2D) of the control cell, $\mathcal{N}(\mathbf{i})$ is the set of neighbors sharing a face with cell $\mathbf{i}$, and F_{ij} is the flux from cell j into cell i . The key insight: the flux leaving cell i into cell j is *exactly equal and opposite* to the flux entering cell j from cell i : $F_{ij} = -F_{ji}$. This ensures that total charge is conserved to machine precision — no charge is created or destroyed at any face.

Two-Point Flux Approximation (TPFA)

The flux between cells $\mathbf{i}$ and $\mathbf{j}$ is approximated using only the voltages at the two cell centers:

$$F_{ij} = \sigma_{ij} \frac{V_j - V_i}{|\mathbf{x}_j - \mathbf{x}_i|} A_{ij}$$

where A_{ij} is the area of the shared face and σ_{ij} is the effective conductivity at the face. When the two cells have different conductivities ($\sigma_i \neq \sigma_j$), the correct effective value is the **harmonic mean**:

$$\sigma_{ij} = \frac{2\sigma_i \sigma_j}{\sigma_i + \sigma_j}$$

This is the formula for two resistors in series — the face lies between the two cells, so the total resistance is the sum of half-cell resistances. Importantly, if one cell has zero conductivity (scar tissue), $\sigma_{ij} = 0$ regardless of the other cell's conductivity. This makes FVM naturally correct at sharp tissue boundaries.

Worked Example: 1D, 4-Cell FVM

Consider four control volumes in 1D with uniform cell width h and uniform conductivity σ . Each face has area 1 (unit cross-section). The flux from cell j to cell i is $F_{ij} = \sigma (V_j - V_i)/h$.

The balance equation for cell i (interior) is: $h \chi C_m dV_i/dt = \sigma(V_{i-1} - V_i)/h + \sigma(V_{i+1} - V_i)/h$, which simplifies to:

$$\chi C_m \frac{dV_i}{dt} = \frac{\sigma}{h^2} (V_{i-1} - 2V_i + V_{i+1})$$

This is *identical* to the FDM formula! On a uniform 1D grid with uniform conductivity, FVM and FDM produce the same stiffness matrix. The difference emerges in two situations: (1) non-uniform grids, where FVM correctly accounts for variable cell sizes through the control-volume areas, and (2) discontinuous conductivity, where the harmonic mean gives the physically correct flux while FDM's arithmetic average does not.

Key Insight

When to use FVM: FVM excels when *conservation* is paramount and when conductivity has sharp jumps (e.g., scar tissue abutting healthy myocardium). The harmonic-mean flux ensures physically correct behavior at material interfaces. The trade-off is that the two-point flux approximation requires the mesh to be "K-orthogonal" (control-volume faces perpendicular to the line connecting cell centers) for full accuracy with anisotropic conductivity. On non-orthogonal meshes, more complex multi-point flux approximations are needed.

7.8 Choosing Your Method: A Practical Guide

The choice between FDM, FEM, and FVM is ultimately a practical engineering decision. No method is universally best — each has a domain where it excels. The following table summarizes the trade-offs:

Criterion	FDM	FEM	FVM
Mesh type	Structured Cartesian only	Any (triangles, tets, hex)	Any with dual mesh
Geometry fidelity	Staircase boundaries	Exact curved boundaries	Good with Voronoi cells
Implementation	Simplest (arrays + stencil)	Most complex (assembly)	Moderate (flux loops)
Anisotropy	9/27-pt stencil for off-diagonal	Natural (in weak form)	Needs K-orthogonal mesh
Conservation	Approximate	Approximate (exact in weak sense)	Exact by construction
Sharp interfaces	Poor (averaging artifacts)	Good (per-element D)	Excellent (harmonic mean)
GPU friendliness	Excellent (regular access)	Good (element-level parallelism)	Good (face-level parallelism)
Memory per node	Lowest	Moderate (M + K storage)	Low
Mass matrix	Not needed (M = I)	Required (can be lumped)	Diagonal by construction
Accuracy order (P1)	$O(h^2)$	$O(h^2)$	$O(h^2)$

In practice, the cardiac simulation community uses all three:

FDM dominates rapid prototyping, GPU-accelerated tissue slabs, and machine-learning training datasets where millions of simulations are needed on simple geometries.

FEM dominates patient-specific simulations on MRI-derived anatomical meshes — the standard for clinical applications and the default in major cardiac simulation packages (openCARP, FEniCS-based solvers, deal.II frameworks).

FVM is used for specialized problems involving sharp material interfaces (fibrosis modeling, scar characterization) and in Lattice Boltzmann-based solvers (Chapters 17–18) where conservation is the defining principle.

7.9 Connecting the Pieces: From PDE to Matrix System

We have spent the last five sections building spatial discretization tools — stiffness matrices from FDM, FEM, and FVM. Now comes the payoff: we will derive, step by step, the central matrix equation that the simulator actually solves. The goal is to show you exactly where every term comes from, so that equation (7.5) below feels *inevitable* rather than pulled from thin air.

Here is the roadmap. We start from the continuous PDE (equation 7.1), break it into pieces, replace each continuous piece with its discrete counterpart, and collect everything into one matrix equation. By the end, you will see that equation (7.5) is just a compact way of writing "voltage changes at every node because of ion channels (local) plus current from neighbors (spatial)."

Step 1: Start from the Continuous PDE

Recall the monodomain equation (7.1):

$$\chi C_m \frac{\partial V_m}{\partial t} = \nabla \cdot (\mathbf{D} \nabla V_m) - \chi I_{\text{ion}}(V_m, \mathbf{w}) + I_{\text{stim}}$$

This is a continuous equation — it holds at every point $\mathbf{x}$ in the tissue simultaneously. A computer cannot handle "every point" (there are infinitely many), so we pick a finite set of N points (the mesh nodes) and ask: what does this equation look like at node i ?

Step 2: Sample at Node i (the Semi-Discrete Idea)

Let $V_{m,i}(t)$ denote the voltage at node i as a function of time. The time derivative is simple — it just becomes $dV_{m,i}/dt$, an ordinary derivative because we are tracking a single node's voltage over time. The ionic current is also simple — it only depends on local quantities, so it becomes $I_{\text{ion}}(V_{m,i}, \mathbf{w}_i)$ evaluated entirely at node i . The tricky term is the diffusion operator $\nabla \cdot (\mathbf{D} \nabla V_m)$, because it involves spatial derivatives — information from neighboring nodes. This is what sections 7.5–7.7 taught us to handle.

Step 3: Replace the Diffusion Operator with a Matrix Product

Every spatial discretization method does the same thing: it replaces the continuous diffusion operator at node i with a *weighted sum* of voltages at node i and its neighbors:

$$[\nabla \cdot (\mathbf{D} \nabla V_m)]_i \approx - \sum_{j=1}^N K_{ij} V_{m,j}$$

The minus sign is a convention: we define $\mathbf{K}$ so that it is *positive* semi-definite (the natural sign for a "stiffness" matrix). The entry K_{ij} encodes how strongly node j influences node i through diffusion — this is exactly what we computed in the FDM stencil (Section 7.5), the FEM element assembly (Section 7.6), or the FVM flux balance (Section 7.7). If node j is not a neighbor of node i , then $K_{ij} = 0$, which is why $\mathbf{K}$ is sparse.

In vector notation, gathering all N node voltages into $\mathbf{V}_m = (V_{m,1}, V_{m,2}, \dots, V_{m,N})^T$:

$$\text{Diffusion at all nodes} \approx -\mathbf{K} \mathbf{V}_m$$

Step 4: Handle the Time Derivative (the Mass Matrix)

For FDM and FVM, the time derivative is straightforward. At node i , the left-hand side of the PDE is simply $\chi C_m dV_{m,i}/dt$, and we can write $\chi C_m \dot{\mathbf{V}}_m$ for the whole vector. In matrix language, this is $\mathbf{M} \dot{\mathbf{V}}_m$ where $\mathbf{M}$ is a diagonal matrix with $M_{ii} = \chi C_m |\Omega_i|$ and $|\Omega_i|$ is the volume associated with node i (for FDM on a uniform grid, $|\Omega_i| = h^d$, so $\mathbf{M}$ is proportional to the identity).

For FEM, the mass matrix arises naturally from the weak form (Section 7.6). The time-derivative integral $\int_{\Omega} \chi C_m (\partial V_m / \partial t) \varphi_i d\mathbf{x}$ produces the entry $M_{ij} = \int_{\Omega} \chi C_m \varphi_i \varphi_j d\mathbf{x}$. This is a *consistent* mass matrix — it is not diagonal because the hat functions of neighboring nodes overlap. In practice, we often *lump* it (replace each row with its row sum on the diagonal) so that $\mathbf{M}$ becomes diagonal and trivially invertible, just like FDM's.

In all cases, $\mathbf{M}$ has the physical meaning: it weights each node's time derivative by the "volume of tissue" that node represents.

Step 5: Gather the Ionic Current into a Load Vector

The ionic current and stimulus terms are purely local — they don't involve spatial derivatives. At node i , they contribute:

$$F_i(t) = -\chi |\Omega_i| I_{\text{ion}}(V_{m,i}, \mathbf{w}_i) + |\Omega_i| I_{\text{stim},i}(t)$$

Here $I_{\text{ion}}(V_{m,i}, \mathbf{w}_i)$ is the total ionic current at node i — this is equation (5.1) for TTP06 (12 currents, 17 state variables) or equation (6.1) for ORd (15 currents, 40 state variables). The state vector $\mathbf{w}_i$ lives entirely at node i : none of its components (gates, concentrations, CaMKII states) diffuse. Collecting all N entries gives the *load vector* $\mathbf{F}(t)$.

The weighting factor $|\Omega_i|$ is the "volume" associated with node i : for FDM on a uniform grid, $|\Omega_i| = h^d$; for lumped FEM, it equals the lumped mass diagonal entry divided by χC_m ; for FVM, it is the control-volume size.

Step 6: Assemble the Semi-Discrete System

Now we put steps 3, 4, and 5 together. The continuous PDE at node i was:

$$\underbrace{\chi C_m \frac{dV_{m,i}}{dt}}_{\text{mass} \times \text{time deriv}} = - \underbrace{\sum_j K_{ij} V_{m,j}}_{\text{diffusion}} + \underbrace{(-\chi I_{\text{ion},i} + I_{\text{stim},i}) |\Omega_i|^{-1} \cdot |\Omega_i|}_{\text{ionic} + \text{stimulus}}$$

Stacking all N nodes into vectors and matrices:

$$\mathbf{M} \dot{\mathbf{V}}_m = -\mathbf{K} \mathbf{V}_m + \mathbf{F}(t)$$

(7.5)

This is it — the *semi-discrete* monodomain system. "Semi-discrete" because space has been discretized (the matrices $\mathbf{M}$ and $\mathbf{K}$) but time is still continuous (the time derivative $\dot{\mathbf{V}}_m$). Every term has a clear origin:

Term	Origin	What it encodes
$\mathbf{M}$	Time-derivative integral (PDE left-hand side)	Volume weight per node; $\mathbf{I}$ for FDM, diagonal for lumped FEM/FVM
$\mathbf{K}$	Diffusion operator $\nabla \cdot (\mathbf{D} \nabla \mathbf{V}_m)$	Spatial coupling: stencil (FDM), element assembly (FEM), flux balance (FVM)
$\mathbf{F}(t)$	Ionic current + stimulus (PDE right-hand side)	Local chemistry from TTP06/ORd at each node; no spatial derivatives

Intuition

Equation (7.5) says exactly the same thing as the continuous PDE (7.1), but in a language a computer can execute: "at each node, the voltage rate of change equals the diffusive current arriving from neighbors (encoded by the matrix $-\mathbf{K} \mathbf{V}_m$) plus the ionic current produced locally (encoded by $\mathbf{F}$)." The matrices are just bookkeeping — a structured way to store the neighbor weights that sections 7.5–7.7 computed.

One Full Timestep: How It All Fits Together

To make this concrete, here is what happens during a single timestep Δt using Godunov operator splitting (Chapter 8), FDM spatial discretization, and TTP06 ionic model, on a mesh with N nodes:

Step 1 — Reaction (at every node independently): For each node $i = 1, \dots, N$, evaluate TTP06's 12 currents from the current voltage $V_{m,i}^n$ and state variables $\mathbf{w}_i^n$:

$$I_{\text{ion},i} = I_{\text{Na},i} + I_{\text{CaL},i} + I_{\text{to},i} + \dots \quad (12 \text{ terms from eq. 5.1})$$

Update the voltage: $V_{m,i}^* = V_{m,i}^n + \Delta t (-I_{\text{ion},i}/C_m)$. Update the 17 state variables using Rush–Larsen for gates and Forward Euler for concentrations: $\mathbf{w}_i^{n+1} = \text{advance}(\mathbf{w}_i^n, V_{m,i}^n, \Delta t)$. This step is *embarrassingly parallel* — each node is independent.

Step 2 — Diffusion (the matrix-vector product): Starting from the reaction-updated voltages $\mathbf{V}_m^*$, solve: $\mathbf{M}(\mathbf{V}_m^{n+1} - \mathbf{V}_m^*)/\Delta t = -\mathbf{K} \mathbf{V}_m^{n+1}$ (for implicit diffusion) or $\mathbf{V}_m^{n+1} = \mathbf{V}_m^* + (\Delta t/\chi C_m)(-\mathbf{K} \mathbf{V}_m^*)$ (for explicit diffusion with lumped mass). For FDM, this is just the stencil applied to each grid point — 5 multiplies and adds in 2D.

Advance to next timestep: Set $\mathbf{V}_m^n \leftarrow \mathbf{V}_m^{n+1}$ and repeat. On a mesh with 1 million nodes, this means evaluating TTP06 one million times per timestep (Step 1), then performing one sparse matrix-vector product of size 10^6 (Step 2). With $\Delta t = 0.02$ ms and a simulation length of 500 ms, this is **25,000** timesteps $\times 10^6$ ionic evaluations = **2.5×10^{10}** TTP06 calls. This is why efficient ionic model implementations and spatial discretizations matter.

⚙ Engine V5.4 Connection

This separation of spatial and temporal concerns is a deliberate design choice in Engine V5.4. All three discretizations (FDM, FEM, FVM) implement the `SpatialDiscretization` ABC, which exposes `apply_diffusion(V)` for explicit solvers and `get_diffusion_operators(dt, scheme)` for implicit solvers. You can switch from FDM to FEM without changing a single line of the solver code. The ionic model plugs in through the `IonicModel` interface — switching from TTP06 (18 ionic states + V) to ORd (40 ionic states + V) changes only the reaction step, not the diffusion step. Chapters 9 and 10 describe the time integrators; Chapter 14 shows how this framework extends to the bidomain's 2×2 block system.

Operator Splitting: Divide and Conquer

The monodomain equation couples a stiff, nonlinear ODE system (ion channels) with a linear diffusion PDE. Trying to solve them simultaneously would be nightmarish. Operator splitting breaks the problem into two simpler subproblems — and it works remarkably well.

8.1 Why Split? The Stiffness Argument

Imagine you are cooking a stir-fry. The vegetables need to sizzle at high heat for 30 seconds (fast process), while the rice simmers gently for 20 minutes (slow process). Trying to cook both in the same pot at the same temperature is hopeless — you either burn the rice or undercook the vegetables. The smart approach: cook them in separate pots, each at the right temperature, then combine at the end. Operator splitting does exactly this for the monodomain equation: the ionic channels (fast, local, nonlinear) get their own "pot," and the diffusion (slow, global, linear) gets another.

The fundamental problem is *timescale separation*. During the upstroke (Phase 0), the sodium gate m changes with a time constant of ~ 0.1 ms, while the diffusion length scale requires timesteps of ~ 0.01 ms for stability. But during the plateau (Phase 2), everything moves slowly — $\tau \sim 50\text{--}200$ ms. Any method that tries to handle both regimes in a single system must either use tiny timesteps everywhere (wasting computation during the plateau) or use implicit methods for the full coupled system (requiring the solution of a massive nonlinear system at each step).

Operator splitting exploits the structure of the problem. Write equation (7.1) as:

$$\frac{\partial V_m}{\partial t} = \underbrace{\frac{1}{\chi C_m} \nabla \cdot (\mathbf{D} \nabla V_m)}_{\mathcal{D}(V_m)} + \underbrace{\frac{-I_{\text{ion}}(V_m, \mathbf{w})}{C_m}}_{\mathcal{R}(V_m, \mathbf{w})}$$

Instead of solving $\partial V_m / \partial t = \mathcal{D} + \mathcal{R}$ as one equation, we solve them sequentially: first advance the reaction $\mathcal{R}$ (purely local ODE at each point), then advance the diffusion $\mathcal{D}$ (linear PDE, no ionic coupling).

8.2 Godunov (First-Order) Splitting

The simplest splitting method advances from t^n to $t^{n+1} = t^n + \Delta t$ in two stages:

Stage 1 — Reaction: At every spatial point, solve the ODE system $dV_m/dt = \mathcal{R}(V_m, \mathbf{w})$ and $d\mathbf{w}/dt = \mathbf{g}(V_m, \mathbf{w})$ from t^n to t^{n+1} , using V_m^n as the initial condition.

Stage 2 — Diffusion: Starting from the updated V_m^* (output of Stage 1), solve $\partial V_m / \partial t = \mathcal{D}(V_m)$ from t^n to t^{n+1} .

Godunov splitting is first-order accurate in time: the splitting error is $O(\Delta t)$. For cardiac simulations with $\Delta t = 0.01\text{--}0.05$ ms, this error is generally acceptable because the spatial and temporal discretization errors are of similar magnitude.

8.3 Strang (Second-Order) Splitting

For higher accuracy, Strang splitting uses a symmetric sequence:

Half-step reaction ($\Delta t/2$) → Full-step diffusion (Δt) → Half-step reaction ($\Delta t/2$).

The symmetry cancels the leading-order splitting error, yielding $O(\Delta t^2)$ accuracy. The cost is modestly higher (two reaction solves instead of one, though each is half as long), but the improved accuracy can allow larger timesteps for the same error tolerance.

Mathematical Note

Strang splitting is second-order because of a *symmetry argument*. If the error of $\mathcal{R} \rightarrow \mathcal{D}$ is $+c\Delta t^2$ for some constant c , then the error of $\mathcal{D} \rightarrow \mathcal{R}$ is $-c\Delta t^2$ (the error changes sign when you swap the order). By sandwiching $-\mathcal{R}(\Delta t/2) \rightarrow \mathcal{D}(\Delta t) \rightarrow \mathcal{R}(\Delta t/2)$ — the first-order errors cancel, leaving $O(\Delta t^3)$ local error and $O(\Delta t^2)$ global error.

8.4 The Rush–Larsen Method for Ionic Variables

During the reaction substep, we must integrate the gating ODEs $d\mathbf{x}/dt = (\mathbf{x}_\infty - \mathbf{x})/\tau_{\mathbf{x}}$. Forward Euler would require very small substeps for gates with fast kinetics (small $\tau_{\mathbf{x}}$). The Rush–Larsen method sidesteps this entirely by noting that for *fixed voltage*, the gating ODE has an exact exponential solution:

$$\mathbf{x}^{n+1} = \mathbf{x}_\infty(V_m^n) + (\mathbf{x}^n - \mathbf{x}_\infty(V_m^n)) e^{-\Delta t/\tau_{\mathbf{x}}(V_m^n)} \quad (8.1)$$

This is unconditionally stable for any $\Delta t > 0$: even if $\tau_{\mathbf{x}} = 0.01$ ms (the fastest Na^+ gate during the upstroke), the exponential simply drives $\mathbf{x}$ to $\mathbf{x}_\infty$ without oscillation. For slow gates ($\tau_{\mathbf{x}} \gg \Delta t$), the exponential reduces to Forward Euler, so no accuracy is lost.

Intuition

Rush–Larsen is like solving the "easy part" of the ODE exactly and only approximating the "hard part" (the voltage dependence of $\mathbf{x}_\infty$ and $\tau_{\mathbf{x}}$). Since these parameters change slowly compared to the gate dynamics during an upstroke, the approximation is excellent. It transforms the stiffest part of the problem — fast gating — from a numerical headache into a single exponential evaluation per gate per timestep.

⚙️ Engine V5.4 Connection

In Engine V5.4, the splitting and ionic integration are cleanly separated. The `SplittingStrategy ABC` (`solver/splitting/base.py`) owns an `IonicSolver` and a `DiffusionSolver` and calls them in the appropriate order. `GodunovSplitting` calls `ionic_solver.step(state, dt)` then `diffusion_solver.step(state, dt)`. `StrangSplitting` calls ionic at `dt/2`, diffusion at `dt`, ionic at `dt/2`.

The `RushLarsenSolver` (`solver/ionic_time_stepping/rush_larsen.py`) is a subclass of the `IonicSolver ABC`. Its `step()` method follows a specific order: (1) compute I_{ion} from the current state, (2) compute $\mathbf{x}_\infty$ and $\tau_{\mathbf{x}}$ from the *old* voltage, (3) Forward Euler on voltage, (4) Rush–Larsen exponential update on gates, (5) Forward Euler on concentrations. A separate `ForwardEulerIonicSolver` is available for debugging. Concentration variables always use Forward Euler because they do not follow relaxation kinetics.

Explicit Diffusion Solvers

After the reaction substep updates the ionic variables, the diffusion substep must advance $\mathbf{V}_m$ under pure diffusion. Explicit time-stepping methods — from the workhorse Forward Euler to the high-accuracy RK4 — form a family of increasing sophistication. This chapter walks through each one, starting from intuition and building to implementation-ready equations.

9.1 Forward Euler and the CFL Condition

The simplest idea for advancing the diffusion equation is also the most intuitive: look at the current state, compute how fast things are changing, and take a step forward. This is *Forward Euler* — the computational equivalent of “if I’m heading north at 60 mph, in one hour I’ll be 60 miles north.” It works beautifully as long as you don’t take steps that are too large. Take too big a step, and the solution oscillates wildly and blows up — this limit is called the *CFL condition*, and it is the fundamental price of simplicity.

Here is the core idea: at each timestep, we evaluate the right-hand side of the diffusion equation (the spatial Laplacian acting on $\mathbf{V}_m$), multiply by Δt , and add the result to the current state. We never look ahead — only at where we *are*. That is what makes it “explicit”: the new state is computed entirely from the old state, with no system of equations to solve.

In the semi-discrete form from equation (7.5), the diffusion substep is the ODE system $\mathbf{M} d\mathbf{V}/dt = -\mathbf{K} \mathbf{V}$. Forward Euler approximates the time derivative by a forward difference:

$$\mathbf{V}^{n+1} = \mathbf{V}^n + \Delta t \mathbf{M}^{-1}(-\mathbf{K} \mathbf{V}^n) \quad (9.1)$$

When using FDM with lumped mass ($\mathbf{M} = \mathbf{I}$), this simplifies to $\mathbf{V}^{n+1} = \mathbf{V}^n - \Delta t \mathbf{K} \mathbf{V}^n$. Each grid point’s update depends only on its neighbors at time level n — no global communication, no matrix solve. This is why Forward Euler is trivially parallelizable and ideal for GPU computation.

The price is the *CFL stability condition*. For a regular grid with spacing h and diffusivity d , Forward Euler is stable only when:

$$\Delta t \leq \frac{h^2}{2 n_{\text{dim}} d_{\text{max}}} \quad (9.2)$$

where n_{dim} is the spatial dimension and d_{max} is the largest eigenvalue of the conductivity tensor. For typical cardiac parameters ($h = 0.2$ mm, $d_{\text{max}} = 0.174$ mm²/ms, 2D), this gives $\Delta t \leq 0.057$ ms — tight, but workable for the $\Delta t = 0.01$ – 0.05 ms range that the ionic model already requires.

Intuition

Worked CFL example. Suppose we want to simulate a 2D cardiac sheet with $h = 0.1$ mm (a fine mesh) and longitudinal conductivity $d_L = 0.174$ mm²/ms. Then $\Delta t_{\text{max}} = (0.1)^2 / (2 \times 2 \times 0.174) = 0.01 / 0.696 = 0.0144$ ms. That is below the typical ionic timestep of 0.02 ms — so for fine meshes, the diffusion stability constraint becomes the bottleneck, not the ionic kinetics. This is exactly why Chapter 10’s implicit methods exist.

Key Insight

The CFL condition scales as h^2 . Halve the mesh spacing and the maximum timestep drops by a factor of four. For fine 3D meshes ($h = 0.1$ mm), this can force Δt below what the ionic solver needs, making explicit methods the bottleneck. This is the primary motivation for implicit methods (Chapter 10).

9.2 Why Go Beyond Forward Euler?

Imagine driving from New York to Boston. Forward Euler checks your speedometer *once* at the start and assumes that speed holds for the entire trip. If the road curves, you end up in the ocean. A smarter driver checks the speedometer multiple times along the way — at the start, at the midpoint, near the end — and uses all those readings to estimate where they actually end up. That is what Runge–Kutta methods do: *sample the slope multiple times within each timestep* and combine the samples into a much better estimate of the true solution.

The key insight is that Forward Euler's error comes from assuming the derivative is constant across the entire step. In reality, the derivative changes as $\mathbf{V}_m$ changes. By evaluating the derivative at intermediate points within the step, we can capture this curvature. More evaluations = better curvature tracking = higher-order accuracy. The family of methods that does this systematically is called **Runge–Kutta (RK) methods**, named after Carl Runge and Martin Kutta who developed them in the early 1900s.

Every explicit RK method follows the same recipe: (1) evaluate the right-hand side at several carefully chosen points, (2) take a weighted average of those evaluations, (3) use that average as the effective slope for the timestep. The differences between RK1 (Forward Euler), RK2, and RK4 are simply *how many* sample points they use and *how* they weight them.

9.3 RK2 — Heun's Method (The Two-Sample Correction)

RK2 is like a rough draft followed by a revision. You start by making the same guess Forward Euler would (the "predictor") — but instead of stopping there, you evaluate the slope at that predicted point too, and then average the two slopes. The result is far more accurate because you have sampled the derivative at both ends of the interval instead of just one.

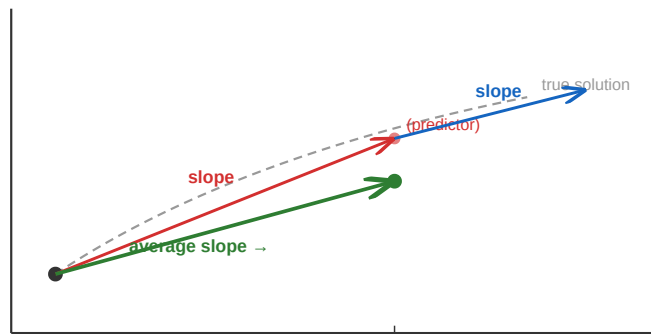


Figure 9.1 — RK2 (Heun's method) visualized. The red arrow is the Forward Euler predictor (k_1 slope). The blue arrow is the slope evaluated at the predicted point (k_2). The green arrow is the average of both slopes — which lands much closer to the true solution (dashed gray).

Formally, RK2 (Heun's method) advances from $\mathbf{V}^n$ to $\mathbf{V}^{n+1}$ in two stages. Let $f(\mathbf{V}) = \mathbf{M}^{-1}(-\mathbf{K}\mathbf{V})$ denote the right-hand side of the diffusion ODE:

$k_1 = \Delta t f(\mathbf{V}^n)$	(slope at start)
$k_2 = \Delta t f(\mathbf{V}^n + k_1)$	(slope at FE predictor)
$\mathbf{V}^{n+1} = \mathbf{V}^n + \frac{1}{2}(k_1 + k_2)$	(average both slopes)

(9.3)

That final line is the key: $\mathbf{V}^{n+1}$ is *not* the Forward Euler predictor $\mathbf{V}^n + k_1$. Instead, we go back to $\mathbf{V}^n$ and use the average of the two slopes, $\frac{1}{2}(k_1 + k_2)$. This is where the name "predictor–corrector" comes from: k_1 predicts, and the averaging with k_2 corrects.

Expanding $f(\mathbf{V}) = \mathcal{L}\mathbf{V}$ where $\mathcal{L}$ is the discrete diffusion operator (i.e., `apply_diffusion(V)` from Chapter 7), the matrix form computed at each timestep is:

(9.3b)

$$\begin{aligned} \mathbf{k}_1 &= \mathcal{L} \mathbf{V}^n \\ \mathbf{k}_2 &= \mathcal{L}(\mathbf{V}^n + \Delta t \mathbf{k}_1) \\ \mathbf{V}^{n+1} &= \mathbf{V}^n + \frac{\Delta t}{2} (\mathbf{k}_1 + \mathbf{k}_2) \end{aligned}$$

Note that each $\mathbf{k}_i$ here is a *bare* rate vector (units of mV/ms) — the factor Δt is applied only at the final update. This is the convention used by the solver code and is mathematically equivalent to the standard form (9.3) where $\mathbf{k}_i = \Delta t \cdot \mathbf{f}(\mathbf{V})$.

Mathematical Note

Why exactly $\frac{1}{2}$? Heun's method is designed so that the Taylor expansion of $\mathbf{V}^{n+1}$ matches the true solution through $\mathcal{O}(\Delta t^2)$. Forward Euler matches only through $\mathcal{O}(\Delta t)$. By evaluating $\mathbf{f}$ at the predicted point and averaging, we capture the second-order term $\frac{1}{2} \Delta t^2 \mathbf{f}'(\mathbf{V}^n) \mathbf{f}(\mathbf{V}^n)$ that Forward Euler misses. The coefficient $\frac{1}{2}$ is not arbitrary — it is the unique weight that achieves this cancellation for the two-stage Heun form.

Intuition

Worked example. Consider $d\mathbf{V}/dt = -2\mathbf{V}$ with $\mathbf{V}^0 = 1.0$ and $\Delta t = 0.1$. The exact solution at $t = 0.1$ is $e^{-0.2} = 0.81873$.

Forward Euler: $\mathbf{V}^1 = 1.0 + 0.1 \times (-2 \times 1.0) = 0.800$. Error = 0.01873.

RK2 (Heun):

$$\mathbf{k}_1 = 0.1 \times (-2 \times 1.0) = -0.2$$

$$\hat{\mathbf{V}} = 1.0 + (-0.2) = 0.8$$

$$\mathbf{k}_2 = 0.1 \times (-2 \times 0.8) = -0.16$$

$$\mathbf{V}^1 = 1.0 + \frac{1}{2}(-0.2 + (-0.16)) = 1.0 - 0.18 = 0.820$$
. Error = 0.00127.

RK2's error (0.00127) is **15× smaller** than Forward Euler's (0.01873) for the same timestep — and it only cost one extra function evaluation.

9.4 RK4 — The Classical Four-Stage Method

If RK2 is checking your speedometer twice, RK4 checks it *four times*: at the start, twice at the midpoint (using two different estimates), and once at the end. It then combines all four readings with a specific weighted average that is tuned to capture the curvature of the solution remarkably well. RK4 is so accurate that it has been the default ODE integrator in science and engineering since the 1900s — a century of trust.

The genius of RK4 is its *midpoint sampling*. Rather than just checking the endpoints (like RK2), it takes two independent samples at $t^n + \Delta t/2$. Why the midpoint? Because the midpoint rule for integration is second-order accurate all by itself, and by doing it twice (with refined estimates), the errors cancel to fourth order. The final weighted average — giving the midpoint samples twice the weight of the endpoints — is not a coincidence. It mirrors Simpson's rule from numerical integration, which achieves fourth-order accuracy by weighting the endpoints and midpoint as $\frac{1}{6}, \frac{4}{6}, \frac{1}{6}$.

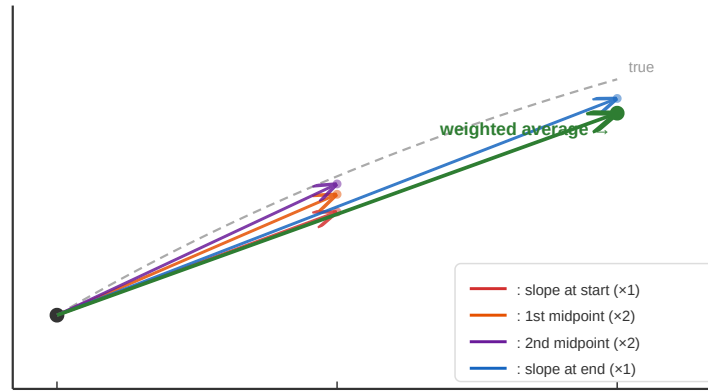


Figure 9.2 — The four stages of RK4. Each colored line shows a slope evaluation; the green result is the weighted average $\frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$. The midpoint slopes (k_2, k_3) get double weight, mirroring Simpson's rule.

The classical RK4 method evaluates the right-hand side $f(V) = M^{-1}(-KV)$ at four carefully chosen points:

$$\begin{array}{ll}
 k_1 = \Delta t f(V^n) & (\text{slope at } t^n) \\
 k_2 = \Delta t f(V^n + \frac{1}{2} k_1) & (\text{1st midpoint, using } k_1 \text{ to get there}) \\
 k_3 = \Delta t f(V^n + \frac{1}{2} k_2) & (\text{2nd midpoint, using } k_2 \text{ to get there}) \\
 k_4 = \Delta t f(V^n + k_3) & (\text{slope at } t^{n+1}, \text{ using } k_3 \text{ to get there}) \\
 V^{n+1} = V^n + \frac{1}{6} (k_1 + 2k_2 + 2k_3 + k_4) & (\text{weighted average})
 \end{array}$$

(9.4)

Read the stages as a conversation: k_1 says "here's the slope at the start." k_2 says "okay, I used your slope to walk to the midpoint — here's the slope *there*." k_3 says "let me try the midpoint again using k_2 's improved estimate." k_4 says "and here's the slope at the far end, using k_3 ." The final weighted average gives the midpoint samples (k_2, k_3) double weight because the midpoint carries more information about the interval's curvature — exactly as in Simpson's 1-4-1 quadrature rule.

In discrete matrix form with $\mathcal{L}$ as the spatial operator:

$$\begin{array}{l}
 \mathbf{k}_1 = \mathcal{L} V^n \\
 \mathbf{k}_2 = \mathcal{L} (V^n + \frac{\Delta t}{2} \mathbf{k}_1) \\
 \mathbf{k}_3 = \mathcal{L} (V^n + \frac{\Delta t}{2} \mathbf{k}_2) \\
 \mathbf{k}_4 = \mathcal{L} (V^n + \Delta t \mathbf{k}_3) \\
 V^{n+1} = V^n + \frac{\Delta t}{6} (\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)
 \end{array}$$

(9.4b)

Again, each $\mathbf{k}_i$ is a bare rate (mV/ms), not pre-multiplied by Δt . The factor Δt appears only at the final update, which avoids four unnecessary multiplications per step.

Mathematical Note

Why these specific weights? The weights $\frac{1}{6}(1, 2, 2, 1)$ are not chosen by trial and error. They come from matching the Taylor expansion of $V(t^n + \Delta t)$ through the Δt^4 term. There are 8 "order conditions" that a 4th-order method must satisfy (corresponding to the 8 rooted trees with up to 4 vertices in Butcher's theory). The classical RK4 tableau — with its midpoint-endpoint structure — is the simplest 4-stage method satisfying all 8 conditions. No 3-stage explicit method can achieve 4th-order accuracy.

Intuition

Worked example. Same test: $dV/dt = -2V$, $V^0 = 1.0$, $\Delta t = 0.1$. Exact answer: $e^{-0.2} = 0.818731$.

$$k_1 = 0.1 \times (-2 \times 1.0) = -0.200$$

$$k_2 = 0.1 \times (-2 \times (1.0 - 0.100)) = 0.1 \times (-1.800) = -0.180$$

$$k_3 = 0.1 \times (-2 \times (1.0 - 0.090)) = 0.1 \times (-1.820) = -0.182$$

$$k_4 = 0.1 \times (-2 \times (1.0 - 0.182)) = 0.1 \times (-1.636) = -0.1636$$

$$V^1 = 1.0 + \frac{1}{6}(-0.200 + 2(-0.180) + 2(-0.182) + (-0.1636))$$

$$= 1.0 + \frac{1}{6}(-1.0876) = 1.0 - 0.18127 = 0.81873$$

Error = 0.000001 — five orders of magnitude smaller than Forward Euler, for just $4\times$ the cost. This is the power of fourth-order convergence.

9.5 Stability Regions: How Large Can Δt Be?

Every explicit method has a "safe zone" — a region in the complex plane where the timestep is small enough that the solution stays bounded. Step outside this zone, and the solution blows up exponentially. The size and shape of this safe zone determines the practical maximum timestep.

For the test equation $dV/dt = \lambda V$ (where λ is an eigenvalue of the diffusion operator), one timestep gives $V^{n+1} = R(z) V^n$ where $z = \lambda \Delta t$ and $R(z)$ is the *stability function*. The method is stable when $|R(z)| \leq 1$. For diffusion, λ is real and negative (eigenvalues of $-\mathbf{K}$), so we only need the intersection of the stability region with the negative real axis.

The stability functions for each method are:

Forward Euler: $R(z) = 1 + z$ RK2 (Heun): $R(z) = 1 + z + \frac{1}{2}z^2$ RK4: $R(z) = 1 + z + \frac{1}{2}z^2 + \frac{1}{6}z^3 + \frac{1}{24}z^4$	stable for $z \in [-2, 0]$ stable for $z \in [-2, 0]$ stable for $z \in [-2.785, 0]$
--	---

(9.5)

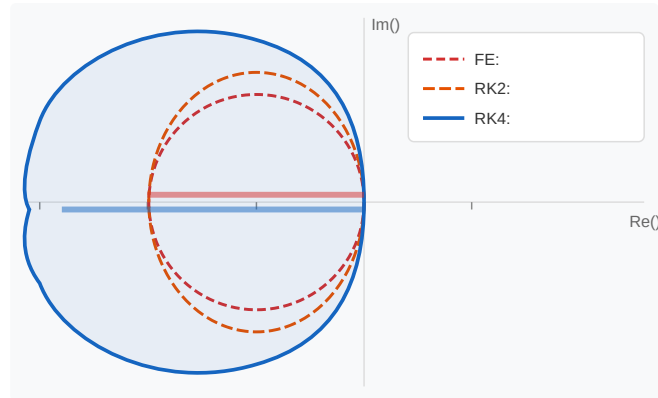


Figure 9.3 — Stability regions in the complex $z = \lambda \Delta t$ plane. The method is stable when z lies inside the boundary. Along the negative real axis (which is what matters for diffusion), RK4 allows $\sim 39\%$ larger timesteps than FE/RK2.

A critical observation: **RK2 and Forward Euler have the same stability limit along the real axis ($|z| \leq 2$)**. So RK2 cannot take larger timesteps than Forward Euler for pure diffusion — its advantage is purely in *accuracy per step*, not stability. RK4, however, extends the real-axis limit to $|z| \leq 2.785$, allowing about 39% larger steps. In terms of the CFL condition:

$$\Delta t_{\text{RK4}} \leq \frac{2.785 h^2}{2 n_{\text{dim}} d_{\text{max}}} \approx 1.39 \Delta t_{\text{FE}} \quad (9.6)$$

9.6 Cost vs. Accuracy: Choosing Your Explicit Method

Each RK method trades cost for accuracy. The question is: for a given error tolerance, which method minimizes total work? The answer depends on whether you are *stability-limited* (the CFL condition forces small Δt) or *accuracy-limited* (you could take larger Δt but the error would be too large).

Method	f evals/step	Order	Real-axis stability	$\Delta t_{\max}$ (relative)	Best when...
Forward Euler	1	1	$[-2, 0]$	1.0×	CFL-limited (fine mesh, ionic $\Delta t \gg$ CFL)
RK2 (Heun)	2	2	$[-2, 0]$	1.0×	Accuracy matters, not quite CFL-limited
RK4	4	4	$[-2.785, 0]$	1.39×	Smooth solution, accuracy-limited regime

The practical verdict for cardiac simulation: Most cardiac simulations are CFL-limited — the sharp wavefront during depolarization demands small Δt regardless of accuracy concerns. In this regime, Forward Euler is usually the best choice: the CFL condition forces a small enough Δt that first-order accuracy is adequate, and the cost per step is minimal. RK4 shines in accuracy-limited regimes (coarse mesh, smooth solutions) or when coupled with adaptive timestepping, which Chapter 10 does not cover but is an active research direction.

9.7 When Explicit Methods Shine

Despite the CFL constraint, explicit methods remain competitive in several important scenarios:

The mesh is coarse enough that the CFL limit does not bite. For $h = 0.2\text{--}0.5$ mm (common in 2D studies), the allowed Δt is comparable to or larger than what the ionic solver needs anyway.

GPU parallelism is critical. Explicit methods are embarrassingly parallel — each cell's update is independent. No linear system needs to be solved, no global communication is required. On modern GPUs with thousands of cores, this parallelism translates directly to speed.

Memory is limited. Explicit methods store only the current state and the Laplacian evaluation — no preconditioner, no Krylov vectors, no mass matrix factorization.

⚙ Engine V5.4 Connection

In Engine V5.4, all three explicit methods inherit from a `DiffusionSolver ABC` (`solver/diffusion_time_stepping/base.py`). Each calls `self._spatial.apply_diffusion(V)` — the $\mathcal{L}$ operator — which encapsulates FDM, FEM, or FVM (Chapter 7). Swapping between spatial methods is a one-line configuration change; the time integrator only sees the operator $\mathcal{L}$. Explicit solvers need no pre-built matrices; only the implicit solvers (Chapter 10) build `DiffusionOperators`.

🔑 Key Insight

Memory consideration for RK4. Each $\mathbf{k}_i$ is a full-length voltage vector (one float per mesh node). RK4 stores 4 of these simultaneously, plus the original $\mathbf{V}^n$. For a 3D mesh with 10^6 nodes at 8 bytes/float, that is $5 \times 8 \text{ MB} = 40 \text{ MB}$ — trivial on a CPU, but meaningful on a GPU where shared memory is limited. Forward Euler needs only one extra vector ($2 \times 8 \text{ MB}$). This is another reason why Forward Euler dominates GPU cardiac codes.

Implicit Diffusion Solvers

When meshes are fine or three-dimensional, the CFL constraint makes explicit methods impractical. Implicit methods trade the simplicity of a matrix-free update for unconditional stability — at the cost of solving a linear system at every timestep.

10.1 Why Implicit? The CFL Wall

Explicit methods (Chapter 9) are like steering a boat by looking only at where you *are*. If the river is calm, this works fine. But in rapids (fine meshes, fast diffusion), the boat oscillates wildly because you are always reacting to where you *were*, not where you are *going*. Implicit methods fix this by peeking ahead: they ask “where will I be at the end of this step?” and solve for that future state directly. This costs more per step (you must solve a system of equations), but the payoff is unconditional stability — you can take any step size without blowing up.

The fundamental issue is that the CFL condition (equation 9.2) scales as h^2 : halve the mesh spacing and the maximum timestep drops by a factor of four. For a 3D mesh with $h = 0.1$ mm ($d_{\max} = 0.174$ mm²/ms), the CFL limit is $\Delta t \leq 0.004$ ms — far below the 0.01–0.05 ms range the ionic solver prefers. Running 5–12 explicit diffusion substeps per ionic step wastes enormous effort. Implicit methods eliminate this constraint entirely: they are *unconditionally stable*, meaning Δt is limited only by accuracy, not by stability. In practice, this lets us match the diffusion timestep to the ionic timestep, halving or more the total simulation time.

The price is that at each timestep, the new voltage $\mathbf{V}^{n+1}$ appears on *both sides* of the equation — so we must solve a linear system $\mathbf{A} \mathbf{V}^{n+1} = \mathbf{b}$ to find it. This is where the machinery of Chapter 7 pays off: the stiffness matrix $\mathbf{K}$ and mass matrix $\mathbf{M}$ from the spatial discretization become the building blocks of $\mathbf{A}$ and $\mathbf{b}$.

10.2 Backward Euler (BDF1): The Simplest Implicit Method

If Forward Euler asks “where am I now, and where does the slope say I should go?”, Backward Euler asks the reverse: “where do I end up such that the slope *there* points back to where I started?” This backward-looking logic makes it inherently stable — it cannot overshoot because it is defined by the destination, not the departure.

Formally, Backward Euler evaluates the diffusion operator at the *new* time level:

$$\mathbf{V}^{n+1} = \mathbf{V}^n + \Delta t \mathcal{D}(\mathbf{V}^{n+1}) \quad (10.1)$$

Since $\mathbf{V}^{n+1}$ appears on both sides, this is an *implicit* equation. To solve it, we substitute the semi-discrete diffusion operator from Chapter 7: $\mathbf{M} d\mathbf{V}/dt = -\mathbf{K} \mathbf{V}$. Replacing $d\mathbf{V}/dt$ with the backward difference $(\mathbf{V}^{n+1} - \mathbf{V}^n)/\Delta t$ and rearranging:

$$\underbrace{(\mathbf{M} + \Delta t \mathbf{K})}_{\mathbf{A}_{\text{lhs}}} \mathbf{V}^{n+1} = \underbrace{\mathbf{M}}_{\mathbf{B}_{\text{rhs}}} \mathbf{V}^n \quad (10.2)$$

This is the exact linear system the code solves. The left-hand side matrix $\mathbf{A}_{\text{lhs}} = \mathbf{M} + \Delta t \mathbf{K}$ is symmetric positive definite (SPD) for the standard monodomain problem with Neumann boundary conditions, so the conjugate gradient method converges reliably. The right-hand side is simply $\mathbf{B}_{\text{rhs}} \cdot \mathbf{V}^n$ — a single sparse matrix-vector product.

Backward Euler is first-order accurate ($O(\Delta t)$) and *L-stable* — meaning it not only stays bounded but actively *damps* high-frequency modes. This aggressive damping is useful after discontinuous events (stimulus application, wavefront collision) where oscillations would otherwise pollute the solution. It is also the natural first step for multi-step methods like BDF2.

For FDM with lumped mass ($\mathbf{M} = \mathbf{I}$), the system simplifies to $(\mathbf{I} + \Delta t \mathbf{K}) \mathbf{V}^{n+1} = \mathbf{V}^n$.

10.3 Crank–Nicolson: Second-Order Implicit

Backward Euler looks only at the destination; Forward Euler looks only at the start. Crank–Nicolson *averages the two* — evaluating the diffusion operator at both the old and new time levels and taking the midpoint. This averaging is what gives it second-order accuracy: by sampling both ends of the interval, it captures the curvature that either method alone would miss.

The Crank–Nicolson method (also called the trapezoidal rule) applies the diffusion operator at the half-way point between old and new:

$$\mathbf{V}^{n+1} = \mathbf{V}^n + \frac{\Delta t}{2} [\mathcal{D}(\mathbf{V}^n) + \mathcal{D}(\mathbf{V}^{n+1})] \quad (10.3)$$

Substituting $\mathcal{D}(\mathbf{V}) = -\mathbf{M}^{-1}\mathbf{K}\mathbf{V}$, multiplying through by $\mathbf{M}$, and collecting terms:

$$\underbrace{\left(\mathbf{M} + \frac{\Delta t}{2}\mathbf{K}\right)}_{\mathbf{A}_{\text{lhs}}} \mathbf{V}^{n+1} = \underbrace{\left(\mathbf{M} - \frac{\Delta t}{2}\mathbf{K}\right)}_{\mathbf{B}_{\text{rhs}}} \mathbf{V}^n \quad (10.4)$$

Compare with Backward Euler (10.2): the only difference is the $\frac{1}{2}$ on the $\mathbf{K}$ terms and the appearance of $\mathbf{K}$ on the right-hand side. Both sides now involve the stiffness matrix, but the computation is the same: one sparse matrix-vector product for the RHS, then a linear solve with $\mathbf{A}_{\text{lhs}}$.

Crank–Nicolson is second-order accurate ($\mathcal{O}(\Delta t^2)$) and A-stable — unconditionally stable, but *not* L-stable. The practical consequence: at very sharp wavefronts, CN can produce mild $2\Delta t$ oscillations (the Crank–Nicolson “ringing” artifact). These oscillations do not grow (A-stability) but they can be aesthetically unpleasant and may trigger spurious ionic model activation. For most cardiac simulations with $\Delta t \leq 0.05$ ms, the oscillations are negligible.

Intuition

Worked example. Consider a 5-node 1D cable (FDM, lumped mass, $\mathbf{M} = \mathbf{I}$) with stiffness matrix $\mathbf{K}$ from Chapter 7 and $\Delta t = 0.02$ ms. The CN matrices are:

$$\mathbf{A}_{\text{lhs}} = \mathbf{I} + 0.01\mathbf{K}, \quad \mathbf{B}_{\text{rhs}} = \mathbf{I} - 0.01\mathbf{K}$$

At each timestep: (1) compute $\mathbf{b} = \mathbf{B}_{\text{rhs}} \cdot \mathbf{V}^n$ (sparse matvec, $\sim 5N$ flops), then (2) solve $\mathbf{A}_{\text{lhs}} \cdot \mathbf{V}^{n+1} = \mathbf{b}$ (CG iteration, 10–30 iterations $\times \sim 5N$ flops per iteration). Compare with Forward Euler: one matvec per step, but requiring $\Delta t \leq h^2/(2d) \approx 0.003$ ms — meaning CN can take $\sim 7\times$ larger steps for the same total cost per unit simulation time.

Key Insight

Both $\mathbf{A}_{\text{lhs}}$ and $\mathbf{B}_{\text{rhs}}$ are precomputed once at the start of the simulation (or when Δt changes) and reused at every timestep. They are sparse matrices with the same sparsity pattern as $\mathbf{K}$. This one-time assembly cost is amortized over thousands of timesteps.

10.4 BDF2: Second-Order with Strong Damping

BDF2 is like Crank–Nicolson’s tougher cousin. Both are second-order accurate, but BDF2 achieves this by using *two previous time levels* — $\mathbf{V}^n$ and $\mathbf{V}^{n-1}$ — rather than averaging the diffusion operator. The payoff is *L-stability*: BDF2 aggressively damps stiff transients that would cause CN to ring. For the steep cardiac wavefront (~ 0.3 mm wide, crossing a grid cell in ~ 0.01 ms), this damping is usually beneficial.

The BDF2 formula replaces the first-order backward difference with a second-order version using two history levels:

(10.5)

$$\frac{3\mathbf{V}^{n+1} - 4\mathbf{V}^n + \mathbf{V}^{n-1}}{2\Delta t} = \mathcal{D}(\mathbf{V}^{n+1})$$

The left side is a second-order backward difference approximation to $d\mathbf{V}/dt$ at time t^{n+1} , using the three points $\{t^{n-1}, t^n, t^{n+1}\}$. Multiplying through by $2\Delta t\mathbf{M}$ and substituting $\mathcal{D} = -\mathbf{M}^{-1}\mathbf{K}$:

$$\underbrace{(3\mathbf{M} + 2\Delta t\mathbf{K})}_{\mathbf{A}_{\text{lhs}}} \mathbf{V}^{n+1} = \underbrace{4\mathbf{M}\mathbf{V}^n - \mathbf{M}\mathbf{V}^{n-1}}_{\mathbf{B}_{\text{rhs}}} \quad (10.6)$$

The right-hand side now involves *two* previous voltage vectors: $4\mathbf{M}\mathbf{V}^n - \mathbf{M}\mathbf{V}^{n-1}$. This is the price of being a two-step method: the solver must store the previous timestep's voltage $\mathbf{V}^{n-1}$ in addition to the current $\mathbf{V}^n$. Since $\mathbf{V}^{n-1}$ is not available at $t = 0$, the first step must use BDF1 (Backward Euler) as a bootstrap — a one-step method that requires only $\mathbf{V}^0$.

Mathematical Note

BDF2 is A-stable but (despite its strong damping) technically not L-stable — that distinction belongs to BDF1. However, BDF2's damping of high-frequency modes is much stronger than CN's. The stability function $R(z) = (4 - 1/(1 + 2z/3))$ satisfies $|R(\infty)| = 1/3$, compared to $|R(\infty)| = 1$ for CN. In practice, this means BDF2 suppresses the "ringing" at sharp wavefronts that CN can exhibit.

10.5 Comparing the Three Methods

Method	Order	$\mathbf{A}_{\text{lhs}}$	$\mathbf{B}_{\text{rhs}} \cdot \mathbf{V}^n$	Stability	Best when...
BDF1	1	$\mathbf{M} + \Delta t\mathbf{K}$	$\mathbf{M}\mathbf{V}^n$	L-stable	First step; after discontinuities
CN	2	$\mathbf{M} + \frac{\Delta t}{2}\mathbf{K}$	$(\mathbf{M} - \frac{\Delta t}{2}\mathbf{K})\mathbf{V}^n$	A-stable	Default choice; smooth solutions
BDF2	2	$3\mathbf{M} + 2\Delta t\mathbf{K}$	$4\mathbf{M}\mathbf{V}^n - \mathbf{M}\mathbf{V}^{n-1}$	A-stable, strongly damping	Sharp wavefronts; large Δt

All three methods share the same computational skeleton: precompute $\mathbf{A}_{\text{lhs}}$ and $\mathbf{B}_{\text{rhs}}$ once, then at each timestep perform one sparse matrix-vector product (the RHS) followed by one linear solve (the LHS). The linear solve dominates the cost — typically 60–80% of wall-clock time in an implicit monodomain simulation.

10.6 Linear Solvers: PCG, Chebyshev, and FFT

The linear system $\mathbf{A}_{\text{lhs}}\mathbf{V}^{n+1} = \mathbf{b}$ must be solved efficiently at every timestep. Since $\mathbf{A}_{\text{lhs}}$ is SPD (for Neumann boundary conditions), the conjugate gradient method is the natural choice. Engine V5.4 offers three solver strategies, each suited to different hardware and problem configurations:

PCG (Preconditioned Conjugate Gradient) with Jacobi preconditioning is the default. It is robust, well-understood, and converges in $\mathcal{O}(\sqrt{\kappa})$ iterations, where κ is the condition number of the system. For the monodomain problem, $\kappa \propto h^{-2}$, so finer meshes require more iterations — but typically 10–30 suffice for the tolerance levels needed in cardiac simulation.

Chebyshev iteration is a fixed-polynomial method that requires no inner products — making it ideal for GPU computation, where global reductions (inner products) are expensive. It requires an estimate of the eigenvalue bounds of $\mathbf{A}_{\text{lhs}}$, but these can be precomputed once. The Chebyshev solver achieves zero-synchronization operation: every thread computes independently for a fixed number of iterations.

FFT/DCT solver applies when the domain is a regular rectangle with uniform spacing. The discrete Laplacian diagonalizes under a cosine transform (for Neumann BC), giving a direct solve: transform $\rightarrow$ divide by eigenvalues $\rightarrow$ inverse transform. This is $\mathcal{O}(N \log N)$ and exact to machine precision — no iterations needed. It is the fastest option when applicable, but limited to rectangular domains with constant conductivity.

⚠ Key Insight

The choice of linear solver often dominates the total simulation cost. In a typical implicit monodomain simulation, 60–80% of wall-clock time is spent in the linear solver. This is why the Chebyshev (GPU-friendly) and FFT (direct) solvers exist in V5.4 alongside PCG — each targets a different performance bottleneck.

⚙ Engine V5.4 Connection

In Engine V5.4, the implicit solvers (`CrankNicolsonSolver`, `BDF1Solver`, `BDF2Solver`) each inherit from `DiffusionSolver` and additionally own a `LinearSolver`. At construction, each calls `spatial.get_diffusion_operators(dt, scheme)` with `scheme = "CN"`, `"BDF1"`, or `"BDF2"` — this returns a `DiffusionOperators` object containing the precomputed sparse matrices $\mathbf{A}_{lhs}$ and $\mathbf{B}_{rhs}$. The `step()` method is then identical for all three: compute $rhs = B_{rhs} @ V$, solve $A_{lhs} @ V_{new} = rhs$. BDF2 additionally stores V_{prev} and subtracts $M @ V_{prev}$ from the RHS, with an automatic fallback to BDF1 for the first timestep.

PART III

The Bidomain Extension

When the monodomain assumption breaks down — modeling both intracellular and extracellular domains for defibrillation, unipolar electrograms, and heterogeneous tissue.

Why Bidomain? — The Physical Picture

Before we write a single equation, let us build a physical picture of what is happening inside a beating heart — and why two "domains" are needed to describe it honestly.

11.1 Currents in a Living Heart

Cardiac tissue is not a simple conducting wire. It is a dense, three-dimensional mesh of individual cells (myocytes), each wrapped in a lipid bilayer membrane that acts as both a capacitor and a nonlinear switch. Between the cells lies a bath of salty fluid — the *extracellular space*. Inside each cell, the cytoplasm forms another conducting medium — the *intracellular space*. The two spaces are connected electrically by thousands of gap junctions that stitch neighboring cells together.

When a wave of electrical activation sweeps through the heart, current flows simultaneously in **both** spaces:

- **Intracellular current $\mathbf{J}_i$** travels from cell to cell through gap junctions, following the fiber direction preferentially (anisotropy ratio $\approx 10:1$).
- **Extracellular current $\mathbf{J}_e$** flows through the interstitial fluid, also anisotropic but with a milder ratio ($\approx 2.5:1$).
- **Transmembrane current I_m** crosses the cell membrane, carried by ion channels, pumps, and membrane capacitance.

The crucial observation is that *these anisotropy ratios are not equal*. Current takes different preferred paths in the two spaces. This mismatch is what the monodomain model cannot capture — and what the bidomain model was invented to describe.

11.2 Divergence and Conservation: The Language of Current Flow

To build the bidomain equations, we need one fundamental physical law: **conservation of charge**. In a region of tissue, charge cannot appear or disappear — it can only flow in or out. The mathematical way to say "what flows out must equal what is produced inside" is the *divergence*.

Intuition

What is divergence, intuitively? Imagine placing a tiny balloon at a point in a fluid. If the fluid is flowing outward from that point, the balloon inflates — the divergence is positive. If the fluid converges inward, the balloon deflates — the divergence is negative. For electric current, $\nabla \cdot \mathbf{J}$ measures how much current is "created" at each point (positive) or "absorbed" (negative).

Now apply this to our two domains. In the intracellular space, current $\mathbf{J}_i$ flows from cell to cell. At any point, some of that current may leak out of the intracellular space across the membrane into the extracellular space. That leaking current *is* the transmembrane current I_m . Conservation says:

$$\nabla \cdot \mathbf{J}_i = -\chi I_m \quad (11.1)$$

The negative sign tells us: if transmembrane current flows *outward* (from inside the cell to outside), it is a *sink* for intracellular current — current is leaving the intracellular space.

Similarly, in the extracellular space, that same transmembrane current is now a *source* — current arriving from inside:

$$\nabla \cdot \mathbf{J}_e = +\chi I_m \quad (11.2)$$

Here χ is the surface-to-volume ratio of the cells (how much membrane area per unit volume of tissue — typically 1000–1500 cm⁻¹). It converts a current per unit membrane area (I_m in $\mu\text{A}/\text{cm}^2$) into a current per unit volume.

△ Key Insight

The conservation identity. If you add equations (11.1) and (11.2), the transmembrane currents cancel exactly:

$$\nabla \cdot \mathbf{J}_i + \nabla \cdot \mathbf{J}_e = 0$$

This is simply the statement that *total current is conserved*: every charge that leaves the intracellular space enters the extracellular space, and vice versa. No charge is created or destroyed inside the tissue. This identity is the backbone of the entire bidomain framework.

11.3 From One Domain to Two

In the **monodomain** model, we make a simplifying assumption: the ratio of intracellular to extracellular conductivity is the same in every direction. Under this assumption, the extracellular potential ϕ_e becomes spatially uniform (or can be set to zero), and we need to track only one unknown — the transmembrane potential V_m . One PDE, one unknown. Simple and fast.

But real cardiac tissue violates this assumption. The intracellular space is strongly anisotropic (current flows about 10 times faster along fibers than across them), while the extracellular space is only mildly anisotropic ($\approx 2.5:1$). This **unequal anisotropy ratio** means the extracellular potential is *not* uniform — it develops complex spatial patterns of its own.

This matters in practice whenever we need to model:

- **Defibrillation** — an external electric field interacts with the tissue through the extracellular space.
- **Virtual electrode polarization** — field-induced regions of depolarization and hyperpolarization that cannot exist in the monodomain picture.
- **ECG generation** — body-surface potentials are driven by extracellular currents.
- **External stimulation** — electrode stimuli enter through ϕ_e .

To capture these phenomena, we must solve for **both** V_m and ϕ_e . We need the *bidomain* model: two coupled equations, two unknowns, one beautiful (and computationally expensive) system.

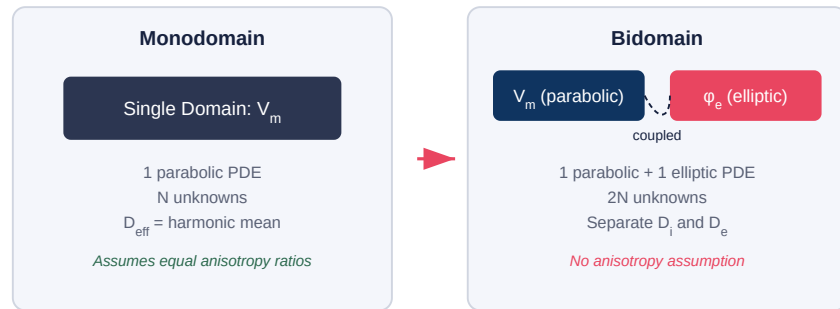


Figure 12.1. Comparison of monodomain and bidomain representations. The bidomain model introduces a second unknown (ϕ_e) and a second equation (elliptic), coupled through the intracellular conductivity tensor.

The Bidomain Equations

We now derive the bidomain equations step by step, building on the conservation laws of Chapter 12 and the monodomain equation familiar from Engine V5.4.

12.1 The Monodomain Equation (Review)

The monodomain equation, which Engine V5.4 solves, describes how the transmembrane potential V_m evolves in time:

$$\chi C_m \frac{\partial V_m}{\partial t} = -\chi I_{\text{ion}}(V_m, \mathbf{w}) + \nabla \cdot (\mathbf{D} \nabla V_m) + I_{\text{stim}} \quad (12.1)$$

Reading each term from left to right:

- **Left-hand side:** rate of change of membrane charge ($\chi C_m \partial V_m / \partial t$). The membrane is a capacitor.
- $-\chi I_{\text{ion}}$: ionic currents through membrane channels — these are nonlinear functions of V_m and gating variables $\mathbf{w}$.
- $\nabla \cdot (\mathbf{D} \nabla V_m)$: diffusion of voltage through the tissue. The tensor $\mathbf{D}$ encodes the *effective* conductivity — a blend of intracellular and extracellular contributions.
- I_{stim} : externally applied stimulus current.

This is a single **parabolic** PDE (it has a time derivative and a second-order spatial derivative). It is well-suited to standard diffusion solvers, LBM, or finite-element methods.

12.2 Building the Bidomain System

Let us now be more precise. In each domain, Ohm's law gives us the current density:

$$\mathbf{J}_i = -\mathbf{D}_i \nabla \phi_i, \quad \mathbf{J}_e = -\mathbf{D}_e \nabla \phi_e$$

where ϕ_i and ϕ_e are the intracellular and extracellular potentials, and $\mathbf{D}_i, \mathbf{D}_e$ are their respective conductivity tensors.

The transmembrane potential is defined as their difference:

$$V_m = \phi_i - \phi_e$$

The transmembrane current per unit volume is the sum of capacitive and ionic components:

$$\chi I_m = \chi C_m \frac{\partial V_m}{\partial t} + \chi I_{\text{ion}}(V_m, \mathbf{w})$$

Now substitute into the conservation laws from Chapter 12. From the intracellular conservation $\nabla \cdot \mathbf{J}_i = -\chi I_m$:

$$\nabla \cdot (\mathbf{D}_i \nabla \phi_i) = \chi C_m \frac{\partial V_m}{\partial t} + \chi I_{\text{ion}}$$

Since $\phi_i = V_m + \phi_e$, we can rewrite $\nabla \phi_i = \nabla V_m + \nabla \phi_e$, giving us:

$$\chi C_m \frac{\partial V_m}{\partial t} = -\chi I_{\text{ion}}(V_m, \mathbf{w}) + \nabla \cdot (\mathbf{D}_i \nabla V_m) + \nabla \cdot (\mathbf{D}_i \nabla \phi_e) + I_{\text{stim}}^i \quad (12.2)$$

This is the **parabolic equation** of the bidomain system. Compare it to the monodomain equation (12.1): the key difference is the *extra coupling term* $\nabla \cdot (\mathbf{D}_i \nabla \phi_e)$, which accounts for how the extracellular potential gradient drives intracellular current.

For the second equation, add both conservation laws: $\nabla \cdot \mathbf{J}_i + \nabla \cdot \mathbf{J}_e = 0$. Expanding:

$$0 = \nabla \cdot (\mathbf{D}_i \nabla V_m) + \nabla \cdot [(\mathbf{D}_i + \mathbf{D}_e) \nabla \phi_e] + I_{\text{stim}}^e \quad (12.3)$$

This is the **elliptic equation**. It has *no time derivative* — it is a constraint that must hold at every instant. Think of it as saying: "at each moment, the total current flowing into and out of every point in the tissue must balance."

Intuition

Why is the second equation elliptic? Because the extracellular space has no "memory" — it equilibrates instantaneously (on the timescale of cardiac propagation). There is no capacitor in the extracellular space storing charge over time. The extracellular potential adjusts immediately to satisfy current conservation. This makes ϕ_e a *slave variable*: it is completely determined by the current state of V_m at each instant.

12.3 The Parabolic–Elliptic Couple

Equations (12.2) and (12.3) together form the bidomain system. Let us highlight their structure:

Equation	Type	Unknown	Character
(12.2) — Membrane dynamics	Parabolic	V_m (primary)	Evolves in time; has $\partial V_m / \partial t$
(12.3) — Current conservation	Elliptic	ϕ_e (constraint)	No time derivative; instantaneous balance

The two equations are **bidirectionally coupled**: the parabolic equation depends on ϕ_e (through $\nabla \cdot (\mathbf{D}_i \nabla \phi_e)$), and the elliptic equation depends on V_m (through $\nabla \cdot (\mathbf{D}_i \nabla V_m)$). Neither can be solved independently.

Additionally, the gating variables $\mathbf{w}$ evolve according to a system of ODEs:

$$\frac{d\mathbf{w}}{dt} = \mathbf{h}(V_m, \mathbf{w}) \quad (12.4)$$

These ODEs are *local* — they depend only on V_m and $\mathbf{w}$ at each point, not on spatial derivatives. This is good news: the ionic solver from Engine V5.4 (e.g., Rush–Larsen exponential integrator) can be reused without modification.

12.4 When Bidomain Reduces to Monodomain

Under the **equal anisotropy ratio** assumption — meaning $\mathbf{D}_e = \lambda \mathbf{D}_i$ for some positive scalar λ — the elliptic equation (12.3) degenerates. With this assumption, the combined conductivity $(\mathbf{D}_i + \mathbf{D}_e) = (1 + \lambda) \mathbf{D}_i$, and the elliptic equation becomes $\nabla \cdot (\mathbf{D}_i \nabla [V_m + (1 + \lambda)\phi_e]) = 0$, which implies ϕ_e is essentially constant (or zero with appropriate boundary conditions). The parabolic equation then reduces to the monodomain form with an effective conductivity:

$$\mathbf{D}_{\text{eff}} = \frac{\mathbf{D}_i \mathbf{D}_e}{\mathbf{D}_i + \mathbf{D}_e}$$

In real cardiac tissue, however, the anisotropy ratios are markedly different:

Domain	σ_L (S/cm)	σ_T (S/cm)	Ratio σ_L/σ_T
Intracellular	0.3 – 0.5	0.03 – 0.05	$\approx 10:1$
Extracellular	0.3 – 0.6	0.15 – 0.3	$\approx 2.5:1$

Because $10:1 \neq 2.5:1$, the equal anisotropy assumption fails, and the full bidomain system is needed.

12.5 Conductivity Tensors and Fiber Architecture

In each domain, the conductivity tensor is expressed in local fiber coordinates as a diagonal matrix:

$$\mathbf{D}_d^{\text{local}} = \begin{pmatrix} \sigma_d^L & 0 & 0 \\ 0 & \sigma_d^T & 0 \\ 0 & 0 & \sigma_d^T \end{pmatrix}, \quad d \in \{i, e\}$$

where σ_d^L and σ_d^T are the longitudinal (along-fiber) and transverse (cross-fiber) conductivities. In an anatomically realistic heart model, the fibers rotate transmurally (about 120° from epicardium to endocardium), so the local tensor must be rotated into global coordinates via:

$$\mathbf{D}_d^{\text{global}}(\mathbf{x}) = \mathbf{R}(\theta(\mathbf{x})) \mathbf{D}_d^{\text{local}} \mathbf{R}(\theta(\mathbf{x}))^T$$

This makes the conductivity a *spatially varying* tensor field — one of the major implementation challenges for any bidomain solver.

⚙ Engine V5.4 Connection

Engine V5.4 already handles fiber-aligned conductivity tensors for the monodomain model. For bidomain, the same rotation machinery can be reused — we simply need to maintain *two* tensor fields ($\mathbf{D}_i$ and $\mathbf{D}_e$) instead of one effective $\mathbf{D}$.

Spatial Discretization

Chapter 7 developed the spatial discretization toolkit — FDM, FEM, FVM — for a single diffusion operator. Here we apply those same methods to the bidomain system, where two coupled PDEs produce a fundamentally different algebraic structure: a 2×2 block matrix system with $2N$ unknowns.

13.1 From One Stiffness Matrix to Two

In the monodomain model (Chapter 7), spatial discretization produces a single stiffness matrix $\mathbf{K}$ assembled from the effective conductivity tensor $\mathbf{D}$. The bidomain model has *two* conducting domains — intracellular and extracellular — each with its own conductivity tensor. Applying the same discretization machinery (whether FDM, FEM, or FVM) to each domain separately produces two stiffness matrices:

Intracellular stiffness $\mathbf{K}_i$: assembled from $\mathbf{D}_i$ using exactly the same procedure as the monodomain $\mathbf{K}$, with entries $(\mathbf{K}_i)_{jk} = \int_{\Omega} \nabla \varphi_j \cdot (\mathbf{D}_i \nabla \varphi_k) d\mathbf{x}$ for FEM, or the corresponding stencil matrix $\mathbf{L}_i$ for FDM.

Extracellular stiffness $\mathbf{K}_e$: assembled identically but with $\mathbf{D}_e$. The combined stiffness $\mathbf{K}_i + \mathbf{K}_e$ corresponds to the bulk conductivity tensor $\mathbf{D}_i + \mathbf{D}_e$.

⚙ Engine V5.4 Connection

Engine V5.4's spatial discretization infrastructure already assembles a single stiffness matrix $\mathbf{K}$ (or Laplacian $\mathbf{L}$) from a conductivity tensor field. Extending to bidomain requires calling this same assembly routine *twice* — once with $\mathbf{D}_i$ and once with $\mathbf{D}_e$ — then combining the results into the 2×2 block format described below.

13.2 The Bidomain Weak Form

For FEM specifically, the bidomain weak form involves both unknowns ($\mathbf{V}_m$ and ϕ_e) simultaneously. Multiplying the parabolic equation by test function $\mathbf{v} \in \mathbf{H}^1(\Omega)$ and integrating by parts:

$$\begin{aligned} \int_{\Omega} \chi C_m \frac{\partial \mathbf{V}_m}{\partial t} \cdot \mathbf{v} d\mathbf{x} + \int_{\Omega} \nabla \mathbf{V}_m \cdot (\mathbf{D}_i \nabla \mathbf{v}) d\mathbf{x} + \int_{\Omega} \nabla \phi_e \cdot (\mathbf{D}_i \nabla \mathbf{v}) d\mathbf{x} \\ = - \int_{\Omega} \chi I_{\text{ion}} \mathbf{v} d\mathbf{x} + \int_{\Omega} I_{\text{stim}}^i \mathbf{v} d\mathbf{x} \end{aligned}$$

For the elliptic equation, multiply by $q \in H^1(\Omega)$:

$$\begin{aligned} \int_{\Omega} \nabla \phi_e \cdot [(\mathbf{D}_i + \mathbf{D}_e) \nabla q] d\mathbf{x} + \int_{\Omega} \nabla \mathbf{V}_m \cdot (\mathbf{D}_i \nabla q) d\mathbf{x} \\ = - \int_{\Omega} I_{\text{stim}}^e q d\mathbf{x} \end{aligned}$$

Notice the critical structural feature: the intracellular conductivity $\mathbf{D}_i$ appears in *both* equations — it is the coupling agent between $\mathbf{V}_m$ and ϕ_e . The extracellular conductivity $\mathbf{D}_e$ appears only in the elliptic equation. Expanding in the finite element basis produces the semi-discrete system:

$$\begin{pmatrix} \mathbf{M} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{pmatrix} \begin{pmatrix} \dot{\mathbf{V}}_m \\ \dot{\phi}_e \end{pmatrix} + \begin{pmatrix} \mathbf{K}_i & \mathbf{K}_i \\ \mathbf{K}_i & \mathbf{K}_i + \mathbf{K}_e \end{pmatrix} \begin{pmatrix} \mathbf{V}_m \\ \phi_e \end{pmatrix} = \begin{pmatrix} \mathbf{F}_V \\ \mathbf{F}_{\phi} \end{pmatrix} \quad (13.1)$$

Mathematical Note

Notice the elegant structure: $\mathbf{K}_i$ appears in *every* block of the 2×2 stiffness matrix — it is the "glue" coupling $\mathbf{V}_m$ and ϕ_e . The extracellular stiffness $\mathbf{K}_e$ appears only in the (2,2) block, strengthening the elliptic equation. The mass matrix $\mathbf{M}$ (from Chapter 7) appears only in the (1,1) position because only $\mathbf{V}_m$ has a time derivative.

13.3 The 2×2 Block System

Regardless of the spatial discretization method (FDM, FEM, or FVM — all described in Chapter 7), the final algebraic system at each time step has the same form:

$$\underbrace{\begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{pmatrix}}_{\mathbf{A}} \begin{pmatrix} \mathbf{V}_m \\ \phi_e \end{pmatrix} = \begin{pmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \end{pmatrix} \quad (13.2)$$

With the typical identification (after applying a semi-implicit time stepper like Crank–Nicolson):

Block	Definition	Properties
$\mathbf{A}_{11}$	$\mathbf{M}/\Delta t + \mathbf{K}_i$	Symmetric positive definite (SPD)
$\mathbf{A}_{12}$	$\mathbf{K}_i$	Coupling: intracellular stiffness
$\mathbf{A}_{21}$	$\mathbf{K}_i$	$= \mathbf{A}_{12}^T$ (symmetric)
$\mathbf{A}_{22}$	$\mathbf{K}_i + \mathbf{K}_e$	SPD (combined stiffness)

The total system has $2N$ unknowns (where N is the number of spatial nodes) — *double* the monodomain system's N unknowns. This is the fundamental object that all subsequent solver strategies (Chapter 16) must handle efficiently. The system is **symmetric** but **indefinite** — a saddle-point problem that requires specialized linear algebra. This is a qualitative departure from the monodomain, where the implicit system is SPD and solvable by conjugate gradient (Chapter 10).

$$\begin{bmatrix} \begin{array}{c} \mathbf{A}_{11} \\ M/\Delta t + K_i \\ \text{(SPD)} \end{array} & \begin{array}{c} \mathbf{A}_{12} \\ K_i \\ \text{(coupling)} \end{array} \\ \begin{array}{c} \mathbf{A}_{21} \\ K_i \\ \text{(coupling)} \end{array} & \begin{array}{c} \mathbf{A}_{22} \\ K_i + K_e \\ \text{(SPD)} \end{array} \end{bmatrix} \begin{bmatrix} \mathbf{V}_m \\ \phi_e \end{bmatrix} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \end{bmatrix}$$

Figure 14.1. The 2×2 block structure of the bidomain linear system. The diagonal blocks are SPD; the off-diagonal coupling blocks are the intracellular stiffness $\mathbf{K}_i$. The full system is symmetric but indefinite.

Time Integration and Operator Splitting

Chapter 8 introduced operator splitting for the monodomain equation — separating the local reaction (ionic ODEs) from the global diffusion (PDE). The same splitting framework applies to the bidomain model, but the diffusion sub-step is fundamentally different: instead of a single SPD system, it requires solving the 2×2 block system from Chapter 14.

14.1 Operator Splitting Applied to Bidomain

The splitting strategy from Chapter 8 carries over directly. In Godunov (first-order) splitting, each time step has two stages: (1) advance the reaction locally using Rush–Larsen (Section 8.4), then (2) solve the diffusion sub-step. The difference is entirely in step 2. For the monodomain, the diffusion sub-step solves a single SPD linear system. For the bidomain, it solves the full 2×2 block system:

$$\begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{pmatrix} \begin{pmatrix} \mathbf{V}_m^{n+1} \\ \phi_e^{n+1} \end{pmatrix} = \begin{pmatrix} \mathbf{b}_1(\mathbf{V}_m^*) \\ \mathbf{b}_2 \end{pmatrix}$$

where $\mathbf{V}_m^*$ comes from the reaction sub-step. Strang (second-order) splitting uses the same symmetric sandwich — half-reaction, full-diffusion, half-reaction — as described in Section 8.3. Research has shown that Strang splitting has better *stability* properties than Godunov for bidomain, because the symmetric arrangement prevents splitting error from accumulating unidirectionally. The recommendation is to use Strang splitting for production bidomain simulations.

The critical new question for bidomain is: *what time stepper do we use for the diffusion sub-step?* The monodomain's implicit solvers (Crank–Nicolson, BDF — Chapter 10) are starting points, but the bidomain's saddle-point structure and sharp wavefronts demand more careful choices.

14.2 Time Steppers: Crank–Nicolson vs. SDIRK2

Within the diffusion sub-step, we need an implicit time integrator for the 2×2 block system. Two standard choices are:

Crank–Nicolson (CN)

The trapezoidal rule applied to the diffusion step:

$$\frac{\mathbf{M}}{\Delta t} (\mathbf{V}_m^{n+1} - \mathbf{V}_m^*) = \frac{1}{2} [\mathcal{L}(\mathbf{V}_m^{n+1}, \phi_e^{n+1}) + \mathcal{L}(\mathbf{V}_m^*, \phi_e^n)]$$

where $\mathcal{L}$ represents the spatial diffusion operator. CN is second-order and A-stable, but it can produce **spurious oscillations** near sharp wave fronts — a serious concern in cardiac electrophysiology, where the action potential upstroke is extremely steep ($dV/dt \sim 300$ V/s).

SDIRK2 (Singly Diagonally Implicit Runge–Kutta, order 2)

The SDIRK2 method uses two implicit stages with the same diagonal coefficient $\gamma = 1 - \sqrt{2}/2 \approx 0.2929$:

$$\begin{aligned} \text{Stage 1: } \mathbf{k}_1 &= \mathcal{L}(\mathbf{V}_m^n + \gamma \Delta t \mathbf{k}_1) && \text{(implicit solve)} \\ \text{Stage 2: } \mathbf{k}_2 &= \mathcal{L}(\mathbf{V}_m^n + (1-\gamma) \Delta t \mathbf{k}_1 + \gamma \Delta t \mathbf{k}_2) && \text{(implicit solve)} \\ \mathbf{V}_m^{n+1} &= \mathbf{V}_m^n + \Delta t [(1-\gamma) \mathbf{k}_1 + \gamma \mathbf{k}_2] \end{aligned}$$

SDIRK2 is second-order, L-stable (meaning it *damps* spurious high-frequency oscillations rather than preserving them), and requires two linear solves per step — each of the same size and structure as the CN solve, but with a smaller effective time step ($\gamma \Delta t$ instead of

Δt).

Key Insight

Recommendation: SDIRK2 is preferred over Crank–Nicolson for bidomain. The extra linear solve per step (two instead of one) is a modest cost increase ($\approx 1.5\times$), but L-stability eliminates the spurious oscillations that plague CN near sharp wave fronts. This is especially important for bidomain, where the coupling to ϕ_e can amplify such oscillations.

14.3 IMEX Methods

An alternative to operator splitting is the Implicit-Explicit (IMEX) approach, which treats different terms with different discretizations within a single unified step:

- **Explicit:** ionic current I_{ion} (nonlinear, local, less stiff)
- **Implicit:** diffusion operator (linear, global, stiff)

IMEX avoids the splitting error entirely. The general form writes the semi-discrete system as $d\mathbf{u}/dt = \mathbf{f}_E(\mathbf{u}) + \mathbf{f}_I(\mathbf{u})$, where $\mathbf{f}_E$ is treated explicitly (ionic currents) and $\mathbf{f}_I$ implicitly (diffusion + elliptic coupling). Because the diffusion operator is *linear*, the implicit part requires only a linear solve — no Newton iteration.

IMEX Euler (SBDF1) — first-order, L-stable, self-starting:

$$\frac{\chi C_m}{\Delta t} \mathbf{M}(\mathbf{V}_m^{n+1} - \mathbf{V}_m^n) + \mathbf{K}_i \mathbf{V}_m^{n+1} + \mathbf{K}_i \phi_e^{n+1} = -\chi \mathbf{M} I_{\text{ion}}^n$$

SBDF2 — second-order, strong damping, recommended for production:

$$\frac{\chi C_m}{\Delta t} \mathbf{M}(\frac{3}{2} \mathbf{V}_m^{n+1} - 2\mathbf{V}_m^n + \frac{1}{2} \mathbf{V}_m^{n-1}) + \mathbf{K}_i \mathbf{V}_m^{n+1} + \mathbf{K}_i \phi_e^{n+1} = -\chi \mathbf{M}(2 I_{\text{ion}}^n - I_{\text{ion}}^{n-1})$$

Other second-order IMEX multistep variants include **CN-FE** (Crank–Nicolson for diffusion, forward Euler for reaction) and **CN-AB** (Crank–Nicolson with Adams–Bashforth extrapolation for ionic terms). Ethier & Bourgault (2008) conducted a systematic stability analysis and found that **SBDF2 provides the strongest damping** of high-frequency error — making it preferred for cardiac applications where sharp wave fronts produce high-frequency error components. CN-FE should generally be avoided for bidomain because it can produce spurious oscillations near the action potential upstroke, similar to the Crank–Nicolson issue discussed in Section 15.2.

IMEX Runge–Kutta methods offer a single-step (self-starting) alternative. An s -stage IMEX-RK method uses an explicit tableau for ionic terms and a DIRK (diagonally implicit Runge–Kutta) tableau for diffusion, requiring s linear solves per step. The stability regions are generally larger than IMEX multistep methods, but the extra linear solves per step make them more expensive for bidomain.

Key Insight

IMEX vs. operator splitting: IMEX eliminates splitting error entirely, while operator splitting (Godunov/Strang) introduces $O(\Delta t)$ / $O(\Delta t^2)$ splitting error. However, operator splitting is simpler to implement, allows specialized ionic solvers (Rush–Larsen), and permits different time steps for reaction and diffusion. For production codes, operator splitting with SDIRK2 remains the most common choice. IMEX (especially SBDF2) is a strong alternative when splitting error is a concern.

14.4 Why Fully Explicit Methods Fail

A natural question arises: can we avoid implicit solves entirely and use fully explicit time integration for the bidomain model? The answer is **no**, for a fundamental structural reason: the elliptic equation for ϕ_e has no time derivative.

Recall the bidomain system. The parabolic equation (for $\mathbf{V}_m$) has a $\partial \mathbf{V}_m / \partial t$ term that can in principle be marched forward explicitly. But the elliptic equation — $\mathbf{0} = \nabla \cdot ((\mathbf{D}_i + \mathbf{D}_e) \nabla \phi_e) + \nabla \cdot (\mathbf{D}_i \nabla \mathbf{V}_m)$ — is a *constraint* that must be satisfied at every instant. There is no “marching forward” possible for a constraint equation. It always requires solving a linear system:

$$(\mathbf{K}_i + \mathbf{K}_e) \phi_e = -\mathbf{K}_i \mathbf{V}_m \quad (15.1)$$

This is the single most important distinction between bidomain and monodomain. In the monodomain model, there is only a single parabolic PDE, and fully explicit methods (including LBM) can handle the entire system. In the bidomain model, **at least one implicit linear solve per time step is unavoidable**.

Even if we accept the mandatory elliptic solve, we can ask: can the *parabolic* part be treated explicitly? Yes — but at a severe cost. Puwal & Roth (2007) derived the forward Euler stability condition for the bidomain parabolic equation. The CFL condition takes the standard parabolic form:

$$\Delta t \leq \frac{\chi C_m \Delta x^2}{2 d D_{\max}} \quad (15.2)$$

where d is the spatial dimension and $D_{\max}$ is the maximum effective diffusion coefficient derived from the bidomain conductivity tensors. For typical cardiac parameters with $\Delta x = 0.1$ mm in 3D, this gives $\Delta t_{\text{explicit}} \lesssim 0.5$ μs . By contrast, semi-implicit methods allow $\Delta t \approx 50$ μs — a factor of **100 $\times$ larger**. Weber dos Santos et al. (2004) numerically verified this: for $\Delta x = 3.3$ μm , forward Euler required $\Delta t \leq 0.07$ μs while a semi-implicit scheme permitted $\Delta t > 100\times$ larger.

Because $\Delta t \propto \Delta x^2$, refining the mesh makes the penalty worse quadratically. For production-quality meshes, the explicit CFL restriction is 25–100 $\times$ more severe than what accuracy alone requires.

⚙ Engine V5.4 Connection

Engine V5.4's explicit diffusion solvers (RK2, RK4) work well for monodomain because the CFL restriction for a single parabolic PDE is manageable. For bidomain, these same explicit methods would require prohibitively small time steps for the parabolic equation, *and* they cannot handle the elliptic equation at all. The implicit diffusion solvers (CN, BDF1, BDF2) are the correct starting point for bidomain extension.

14.5 Explicit Stabilized Methods (RKC)

Runge–Kutta–Chebyshev (RKC) methods offer a middle ground between fully explicit and implicit approaches. An RKC method with s stages is fully explicit (no linear solves for the parabolic part), but its stability domain grows *quadratically* with s :

$$\beta_s \approx 0.65 s^2$$

where β_s is the stability interval length. Standard forward Euler has $\beta_1 = 2$; an RKC method with $s = 20$ stages achieves $\beta_{20} \approx 260$ — effectively relaxing the CFL restriction by a factor of 130 $\times$. Each stage requires only a matrix-vector product (cheap, parallelizable), and the total number of stages adapts automatically to the stiffness.

For the bidomain parabolic equation, RKC would work as follows: (1) solve the elliptic constraint for ϕ_e (implicit, unavoidable), (2) advance $\mathbf{V}_m$ with s RKC stages (explicit), (3) update ionic variables with Rush–Larsen. The elliptic solve is still required, but the parabolic part avoids any implicit linear solve.

Rosilho de Souza et al. (2024) recently proposed the **emRKC** (explicit exponential multirate stabilized) method for cardiac electrophysiology, combining multirate RKC for diffusion with Rush–Larsen exponential integration for gating variables. This method **outperforms the standard IMEX baseline** for the monodomain model and is inherently more parallel. However, it has been demonstrated only for the monodomain model so far. Extending emRKC to the bidomain parabolic equation is conceptually straightforward, though the elliptic solve will still dominate the cost (82–90% of total runtime in typical bidomain simulations).

⚠ Key Insight

The spectrum of explicitness for bidomain: At one extreme, fully implicit methods (backward Euler, CBDF2) treat everything implicitly, requiring nonlinear Newton solves. At the other, “maximally explicit” methods treat the parabolic PDE with RKC and only solve the elliptic constraint implicitly. In between, IMEX methods (SBDF2, operator splitting) treat ionic currents explicitly and diffusion implicitly. For most production bidomain codes, IMEX/operator-splitting represents the best trade-off — but RKC-based approaches are an active area of research with potential GPU advantages.

Linear Solvers for the Block System

The linear solve is the computational bottleneck of the bidomain model, consuming 70–85% of total compute time. Choosing the right solver and preconditioner is critical for practical performance.

15.1 The Saddle-Point Structure

The 2×2 block system (15.2) is **symmetric but indefinite**. This means its eigenvalues include both positive and negative values — a hallmark of saddle-point problems that arise throughout computational physics (Stokes flow, constrained optimization, mixed FEM). The key implications are:

- **Conjugate Gradient (CG) cannot be used** on the full system — CG requires positive definiteness.
- **MINRES** (Minimum Residual) is the natural Krylov solver for symmetric indefinite systems.
- **GMRES** works for any system but requires more memory (stores the full Krylov basis).
- The **condition number** of the bidomain system ranges from 10^8 to 10^{10} for realistic meshes — *preconditioning is absolutely essential*.

15.2 Block Preconditioners

The 2×2 block structure suggests a natural family of preconditioners that exploit the block decomposition rather than treating the system as a monolithic matrix.

Block Diagonal Preconditioner

$$\mathbf{P}_{\text{diag}} = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{0} \\ \mathbf{0} & \mathbf{A}_{22} \end{pmatrix}$$

Application: solve two independent SPD systems ($\mathbf{A}_{11}^{-1}\mathbf{x}_1$ and $\mathbf{A}_{22}^{-1}\mathbf{x}_2$) in each preconditioning step. Simple to implement but ignores the off-diagonal coupling, leading to 50–200 outer iterations.

Block LDU (Triangular) Preconditioner

A more sophisticated choice uses the **Schur complement**:

$$\mathbf{S} = \mathbf{A}_{22} - \mathbf{A}_{21} \mathbf{A}_{11}^{-1} \mathbf{A}_{12}$$

The lower-triangular preconditioner is:

$$\mathbf{P}_{\text{LDU}} = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{0} \\ \mathbf{A}_{21} & \mathbf{S} \end{pmatrix}$$

With *exact* inverses of $\mathbf{A}_{11}$ and $\mathbf{S}$, the preconditioned system converges in at most 2–3 Krylov iterations. In practice, we approximate:

- $\mathbf{A}_{11}^{-1}$ using one AMG V-cycle for the mass-shifted stiffness matrix.
- $\mathbf{S}^{-1}$ using one AMG V-cycle for an approximation of the Schur complement (often $\tilde{\mathbf{S}} \approx \mathbf{K}_e$ or $\tilde{\mathbf{S}} \approx \mathbf{K}_i + \mathbf{K}_e$).

With these approximations, convergence typically requires **10–30 outer iterations** — a dramatic improvement over unpreconditioned solves.

Mathematical Note

The Schur complement has a physical interpretation. It represents the "effective" equation for ϕ_e after eliminating V_m . Computing it exactly is as expensive as solving the original system, but good approximations are cheap: for instance, $\tilde{\mathbf{S}} \approx \mathbf{K}_e$ works well because the mass-matrix contribution in $\mathbf{A}_{11}$ makes $\mathbf{A}_{11}^{-1} \approx (\Delta t / \mathbf{M})$ for small Δt , and the correction term $\mathbf{K}_i (\Delta t / \mathbf{M}) \mathbf{K}_i$ is small.

15.3 Algebraic Multigrid (AMG)

AMG is the workhorse preconditioner for elliptic and elliptic-like systems in cardiac simulations. It constructs a hierarchy of progressively coarser "meshes" (algebraically, not geometrically) and uses smoothing + coarse-grid correction cycles to damp errors at all spatial frequencies simultaneously.

For the bidomain system, AMG is applied at the **sub-block level**:

- **$\mathbf{A}_{11}$ block:** AMG handles the mass-shifted intracellular stiffness efficiently. Convergence in 1–2 V-cycles.
- **$\mathbf{A}_{22}$ or $\mathbf{S}$ block:** AMG handles the combined stiffness (elliptic problem for ϕ_e). This is the *bottleneck* — the elliptic solve typically consumes 60–80% of total linear solver time.

Available AMG implementations for GPU-accelerated bidomain:

Library	GPU Support	Best For
AMGX (NVIDIA)	Native CUDA	GPU-only workloads; best performance on NVIDIA hardware
Hypr (LLNL)	CUDA, HIP, SYCL	Large-scale CPU/GPU clusters; mature and robust
PyAMG (Python)	CPU only	Prototyping; integrates well with PyTorch via scipy
Trilinos ML/MueLu	Partial (Kokkos)	Full solver ecosystem; advanced smoothers

15.4 Krylov Methods: MINRES, GMRES, and Friends

The outer Krylov solver iterates on the preconditioned system until the residual falls below a tolerance (typically 10^{-6} to 10^{-8}).

MINRES

The mathematically optimal choice for symmetric indefinite systems. It minimizes the residual norm over the Krylov subspace at each iteration. Requires a **symmetric** preconditioner (block diagonal qualifies; block LDU does not, unless symmetrized). Memory-efficient: stores only 3 vectors.

GMRES

Works for any system (no symmetry requirement). Allows non-symmetric preconditioners like block LDU. However, memory grows linearly with iteration count (each iteration stores one vector). A restart parameter (typically 30–50) limits memory at the cost of slower convergence.

Flexible GMRES (FGMRES)

A variant that allows the preconditioner to *change* at each iteration — useful when AMG V-cycles produce slightly different results each time (as happens with some adaptive smoothers). Memory cost is 2× GMRES.

△ Key Insight

Recommended solver stack for bidomain:

Outer solver: FGMRES(30) or MINRES

Preconditioner: Block LDU with AMG V-cycles for sub-blocks

Expected iterations: 10–30 per time step

Expected speedup over unpreconditioned: 50–100×

15.5 The Null-Space Problem

The elliptic equation for ϕ_e has homogeneous Neumann boundary conditions everywhere (no current leaves the tissue). This means ϕ_e is determined *only up to an additive constant* — the matrix $\mathbf{A}_{22}$ has a one-dimensional null space spanned by the constant vector $\mathbf{1}$.

This must be handled explicitly. Common approaches:

1. **Pin one node:** set $(\phi_e)_k = 0$ for some node k . Computationally simplest; physically reasonable (choose a "ground" reference).
2. **Integral constraint:** enforce $\int_{\Omega} \phi_e \, d\mathbf{x} = 0$ via a Lagrange multiplier. More elegant but adds one extra equation.
3. **Deflation:** project the constant mode out of the Krylov subspace at each iteration. Algebraically clean; no modification to the matrix.

⚙ Engine V5.4 Connection

For Engine V5.4 extension, we recommend the simplest approach: **pin one node** to zero. This is easy to implement (replace one row/column of $\mathbf{A}_{22}$ with the identity) and has negligible impact on solution accuracy for cardiac-sized domains.

Implementation Roadmap for Engine V5.4

This chapter maps the mathematical framework from Chapters 12–15 onto the concrete software architecture of Engine V5.4, identifying what can be reused, what must be extended, and what must be built new.

16.1 Architecture Extensions

Engine V5.4 is built around four abstract interfaces: `SpatialDiscretization`, `DiffusionSolver`, `LinearSolver`, and `IonicSolver`. The bidomain extension requires changes primarily in the first three:

Component	Monodomain (Current)	Bidomain (Extension)
State Vector	$[V_m]$ — N unknowns	$[V_m, \phi_e]$ — $2N$ unknowns
Conductivity	Single D_{eff}	Separate D_i, D_e
Stiffness Matrix	One K	Two: $K_i, K_e \rightarrow 2 \times 2$ block
Linear Solver	PCG (SPD system)	FGMRES/MINRES (indefinite system)
Preconditioner	AMG or IC(0)	Block LDU + AMG sub-blocks
Ionic Solver	Rush–Larsen	Rush–Larsen (<i>unchanged</i>)
LBM Solver	1 lattice (D3Q7/MRT)	2 lattices + pseudo-time for ϕ_e
Operator Splitting	Strang splitting	Strang splitting (<i>unchanged</i>)

16.2 Recommended Solver Stack

Based on the analysis in Chapters 13–15, the recommended solver configuration for a production bidomain engine is:

⚙ Engine V5.4 Connection

Time Integration: Strang splitting (reaction–diffusion–reaction)

Reaction Solver: Rush–Larsen (reuse from V5.4)

Diffusion Time Stepper: SDIRK2 (L-stable, second-order)

Outer Linear Solver: FGMRES(30) with tolerance 10^{-6}

Preconditioner: Block LDU with Schur complement approximation

Sub-block Solver: AMG V-cycle (AMGX for GPU, Hypre for CPU)

Null-Space Handling: Pin one node of ϕ_e to zero

Expected Performance: 10–30 FGMRES iterations per time step; 10–20× slower than monodomain

16.3 Performance Expectations

The bidomain model is inherently more expensive than monodomain. Realistic expectations:

Metric	Monodomain	Bidomain	Ratio
Unknowns per node	1	2	2×

Linear system size	N	$2N$	$2\times$
Linear solve cost per step	1 PCG solve	2 AMG solves + coupling	$5\text{--}10\times$
Total wall time (with optimal solver)	Baseline	$10\text{--}20\times$ baseline	$10\text{--}20\times$
Memory	Baseline	$3\text{--}4\times$ baseline	$3\text{--}4\times$

For a typical 3D cardiac mesh with $N = 10^6$ nodes, the monodomain solve takes approximately 1–5 seconds per heartbeat on a modern GPU. The bidomain solve would take approximately 10–100 seconds per heartbeat — still tractable for research and many clinical applications.

End of Parts I–III. The full research documentation, including detailed code examples and implementation guides, is available in the `Research/Bidomain/` folder of the Engine V5.4 repository.

PART IV

Lattice-Boltzmann Methods

A radically different paradigm for solving diffusion equations — no matrices, no linear solves, just fictitious particles streaming and colliding on a lattice. From the heat equation to cardiac monodomain and bidomain, LBM trades algebraic machinery for embarrassingly parallel data movement.

The Lattice-Boltzmann Method: From Kinetic Theory to Computation

In Chapters 13–16, we discretized partial differential equations. We started with physics at the macroscopic scale — fluid flow, heat diffusion, electrical signal propagation — and asked: how do we approximate the derivatives on a computer? Those methods (FDM, FEM, FVM) work brilliantly and are the industrial standard. But there is another path, one that begins not with a PDE, but with something more fundamental: the motion of individual particles, governed by kinetic theory. This chapter introduces the Lattice-Boltzmann Method (LBM). Unlike classical discretizations that approximate a PDE, LBM simulates a discrete analogue of the Boltzmann equation — a statement about how particles move and collide. The miracle is that when you correctly simulate the microscopic collisions, the macroscopic behavior (flow, diffusion, electrical conduction) emerges automatically. For cardiac problems, especially when spatial resolution is tight or parallelization is critical, LBM offers a compelling alternative to classical methods. We will build this from first principles: the Boltzmann equation, its simplification to the BGK collision operator, the discretization of velocity space onto a lattice, and the final algorithm. By chapter's end, you will understand not just what LBM computes, but why the method works — and how to tune it for cardiac electrophysiology.

17.1 The Thermodynamic Foundation: Phase Space and the Equilibrium State

Intuition

Core Insight: Every numerical method we have studied so far — FDM, FEM, FVM — starts from a macroscopic PDE and discretizes it. LBM takes a radically different route. It starts from a much deeper level: the statistical mechanics of particles. If you correctly model how particles distribute themselves in velocity space and how that distribution evolves, the macroscopic behavior — diffusion, fluid flow, heat conduction — emerges automatically. To understand LBM, we must first understand the space it lives in and the equilibrium it relaxes toward.

Position Space: What You Already Know

In every method we have encountered so far, the unknown is a scalar field: $V(\mathbf{x}, t)$ for voltage, $T(\mathbf{x}, t)$ for temperature, $\phi(\mathbf{x}, t)$ for a generic concentration. At each point $\mathbf{x}$ in the domain, we store **one number** — the local value of the field. This is position space: a map from location to field value. Position space is intuitive. You can picture a heat map of the heart where each pixel has a color representing voltage. FDM puts a number on each grid point. FEM interpolates between nodes. FVM averages over cells. In all cases, the fundamental object is simple: one value per spatial location. But this simplicity comes at a cost. The macroscopic PDE — the diffusion equation, the monodomain equation — is a coarse-grained description. It tells you how the *average* behavior evolves, but it has already thrown away information about what individual particles are doing. To build LBM, we need to recover some of that microscopic information. That means working in a richer space.

Velocity Space: What Are the Particles Doing?

Imagine standing at a single point in cardiac tissue and watching every ion that passes through. Some ions are moving fast to the right. Some are drifting slowly upward. Some are nearly stationary, vibrating in place. If you could catalog every ion and record its velocity, you would not get a single number — you would get a **distribution**. This distribution is the velocity-space view at a single point. It answers the question: "Of all the particles at position $\mathbf{x}$, how many have velocity $\mathbf{c}$?" In a hot gas, the distribution is wide — particles are moving at all speeds in all directions. In a cold gas, the distribution is narrow — most particles are clustered near zero velocity. In a gas with bulk flow, the distribution is shifted — its peak is no longer at zero but at the flow velocity $\mathbf{u}$. The crucial point: **at every single location in physical space, there is an entire distribution of velocities**. Not one velocity, not an average velocity — a full spread. The shape of this spread encodes the thermodynamic state at that point: temperature (width of the spread), bulk flow (shift of the peak), pressure (area under the curve).

Phase Space: Position and Velocity Together

Phase space is the combined space where you track both where a particle is AND how fast it is going. In two spatial dimensions, a particle's full state is four numbers: (x, y, c_x, c_y) — two for position, two for velocity. In three dimensions, it is six numbers: (x, y, z, c_x, c_y, c_z) . The **distribution function** $f(\mathbf{x}, \mathbf{c}, t)$ lives in this combined space. It is a function of position, velocity, and time simultaneously. Its value tells you: at position $\mathbf{x}$ at time t , the density of particles with velocity $\mathbf{c}$.

$$f : (\mathbf{x}, \mathbf{c}, t) \longrightarrow \text{particle density at } (\mathbf{x}, \mathbf{c}) \text{ at time } t$$

(17.1)

This is the fundamental object of kinetic theory. Compare it to the macroscopic field:

	Macroscopic (FDM/FEM)	Kinetic (LBM)
Unknown	$\phi(\mathbf{x}, t)$ — one number per point	$f(\mathbf{x}, \mathbf{c}, t)$ — a distribution per point
Lives in	Position space (D dimensions)	Phase space ($2D$ dimensions)
Information	Average field value	Full velocity distribution
Storage per node	1 value	Q values (one per velocity direction)

The extra information in the velocity distribution is not wasted. It is exactly what allows LBM to avoid assembling and solving matrix systems — the transport physics is encoded in the distribution itself, and the collision-streaming algorithm updates it locally.

The Equilibrium State: Where Does the Distribution Settle?

Here is the central question of this section — and arguably the most important idea in all of LBM: **When a system of particles reaches thermal equilibrium, what shape does the velocity distribution take?** The answer has been known since the 19th century, and it is one of the great results of statistical mechanics. James Clerk Maxwell and Ludwig Boltzmann proved that the equilibrium velocity distribution of a gas is a **Gaussian** (a bell curve) in velocity space:

$$f^{\text{eq}}(\mathbf{c}) = \frac{\rho}{(2\pi c_s^2)^{D/2}} \exp\left(-\frac{|\mathbf{c} - \mathbf{u}|^2}{2c_s^2}\right)$$

(17.2)

This is the **Maxwell-Boltzmann distribution**. Every symbol has a physical meaning:

- ρ — the total density of particles at this location. This is the macroscopic quantity we are ultimately tracking (voltage, temperature, concentration).
- $\mathbf{u}$ — the bulk (average) velocity of the particles. For diffusion problems, $\mathbf{u} = \mathbf{0}$ — there is no net flow, and the bell curve is centered at the origin.
- c_s — a parameter that controls the **width** of the bell curve. Physically, it is related to temperature: a higher c_s means a wider spread (hotter gas, faster particles on average). In LBM, c_s will become the **lattice speed of sound**, connecting the microscopic model to macroscopic transport properties.
- D — the spatial dimension (2 for 2D, 3 for 3D). The normalization $(2\pi c_s^2)^{D/2}$ ensures the total probability integrates to ρ .

What the Gaussian Tells Us: Three Physical Claims

The Maxwell-Boltzmann distribution (17.2) is not just a formula — it makes three specific physical claims about what equilibrium looks like. Each claim corresponds to a different "order" of the distribution, and each will turn out to be critically important when we build the lattice method.

Zeroth Order — Conservation

Integrate the distribution over all velocities:

$$\int f^{\text{eq}}(\mathbf{c}) d\mathbf{c} = \rho$$

(17.3)

The total amount of “stuff” is ρ . This is conservation of mass (or charge, or energy, depending on what ρ represents). The equilibrium does not create or destroy anything — it only redistributes velocities. No matter how the particles rearrange their speeds and directions, the total count stays the same.

First Order — Symmetry of Flux

Compute the average velocity by weighting with $\mathbf{c}$:

$$\int \mathbf{c} f^{\text{eq}}(\mathbf{c}) d\mathbf{c} = \rho \mathbf{u} \quad (17.4)$$

For a gas at rest ($\mathbf{u} = \mathbf{0}$), the mean velocity is zero. The bell curve is symmetric — as many particles move left as right, up as down. There is no net flux. For diffusion problems in cardiac EP, this is exactly the equilibrium condition: at rest, the distribution is centered and symmetric.

Second Order — Isotropic Variance

Compute the velocity covariance (the “spread” of the bell curve):

$$\int (c_\alpha - u_\alpha)(c_\beta - u_\beta) f^{\text{eq}}(\mathbf{c}) d\mathbf{c} = \rho c_s^2 \delta_{\alpha\beta} \quad (17.5)$$

The variance is **isotropic** — the bell curve has the same width in every direction. The $\delta_{\alpha\beta}$ (Kronecker delta: 1 if $\alpha = \beta$, 0 otherwise) means there is no preferred axis. The parameter c_s^2 is the variance of the Gaussian, and it is the same along $\mathbf{x}$, $\mathbf{y}$, and $\mathbf{z}$. This isotropy is a physical claim: at thermodynamic equilibrium, no direction is special. The random thermal motion of particles is equally vigorous in all directions. This will later impose constraints on the lattice weights and will be the reason why the **standard LBM lattice produces isotropic diffusion by default**.

Intuition

Three orders, three physics: The Gaussian equilibrium encodes conservation (zeroth order), zero net flux (first order), and isotropic variance (second order). These are not abstract mathematical properties — they are the defining characteristics of thermodynamic equilibrium. Every lattice-Boltzmann scheme must respect all three, because they are the physical laws that the scheme must recover at the macroscopic level. In Section 17.4, we will see that these three conditions are exactly the constraints that determine the lattice weights w_i and velocities $\mathbf{c}_i$.

Two Cases: Rest and Flow

The Maxwell-Boltzmann distribution (17.2) handles two physically distinct situations: **Case 1: System at rest ($\mathbf{u} = \mathbf{0}$)**. The bell curve is centered at the origin of velocity space. No bulk motion — particles jiggle randomly in all directions with equal probability. This is the relevant case for **diffusion**: cardiac tissue conducts electrical signals by ion diffusion, not bulk fluid flow. The equilibrium distribution simplifies to:

$$f^{\text{eq}}(\mathbf{c}) = \frac{\rho}{(2\pi c_s^2)^{D/2}} \exp\left(-\frac{|\mathbf{c}|^2}{2c_s^2}\right) \quad (17.6)$$

a symmetric Gaussian centered at the origin. This is the equilibrium that our cardiac LBM will use. **Case 2: System with bulk flow ($\mathbf{u} \neq \mathbf{0}$)**. The bell curve is shifted — its peak is at $\mathbf{u}$ instead of at the origin. More particles are moving in the flow direction than against it. This is the relevant case for **fluid dynamics** (Navier-Stokes). The distribution is still a Gaussian, but the center has moved. The shift breaks the first-order symmetry: $\int \mathbf{c} f^{\text{eq}} d\mathbf{c} = \rho \mathbf{u} \neq \mathbf{0}$.

Mathematical Note

Why $\mathbf{c}_s$ is the bridge between micro and macro: The parameter $\mathbf{c}_s$ plays a dual role. In the continuous Maxwell-Boltzmann distribution, it is the **thermal velocity** — the standard deviation of the velocity Gaussian, set by the temperature of the gas ($\mathbf{c}_s^2 = k_B T/m$). When we discretize onto a lattice, $\mathbf{c}_s$ becomes the **lattice speed of sound** ($\mathbf{c}_s^2 = \mathbf{c}^2/3$ where $\mathbf{c} = \Delta \mathbf{x}/\Delta t$). This is not a coincidence: the Chapman-Enskog analysis (Section 17.2) shows that the lattice speed of sound must equal the Gaussian's variance parameter for the discrete scheme to recover the correct macroscopic transport equations. The connection $\mathbf{c}_s^2 = \mathbf{c}^2/3$ is a fixed consequence of the lattice geometry and the quadrature exactness conditions (Section 17.4).

Why the Equilibrium Is the Heart of LBM

In FDM, FEM, and FVM, the physics enters through the PDE — you discretize the diffusion equation or the monodomain equation directly. In LBM, **the physics enters through the equilibrium distribution**. The entire method can be summarized in one sentence: *Relax the velocity distribution toward the Maxwell-Boltzmann equilibrium, and the correct macroscopic PDE emerges automatically.* This is why we have spent this section building the equilibrium from physical principles rather than just writing down a formula. The Gaussian is not an arbitrary choice — it is the unique distribution that satisfies the constraints of thermodynamic equilibrium (maximum entropy subject to conservation of mass, momentum, and energy). Every property of LBM — the lattice weights, the velocity sets, the collision operator, the transport coefficients — traces back to this Gaussian. In the sections that follow, we will:

- **Section 17.2:** Write down the equation that governs how $\mathbf{f}$ evolves toward this equilibrium (the Boltzmann equation), and simplify it for practical computation (the BGK collision operator).
- **Section 17.3:** Discretize velocity space onto a lattice, turning the continuous $\mathbf{f}(\mathbf{x}, \mathbf{c}, t)$ into a finite set of distributions $\mathbf{f}_i(\mathbf{x}, t)$, and write the lattice-Boltzmann equation.
- **Section 17.4:** Return to the Gaussian equilibrium and ask: what constraints do equations (17.3)–(17.5) impose on the discrete lattice? This will determine the weights $\mathbf{w}_i$, velocity vectors $\mathbf{c}_i$, and the discrete equilibrium $\mathbf{f}_i^{\text{eq}}$.
- **Section 17.5:** Introduce moment space — a representation that reveals which physical laws the collision operator is actually enforcing, and how to control them independently.

17.2 The Boltzmann Equation and Kinetic Theory

The Boltzmann Equation: Fundamental Form

The Boltzmann equation describes how the distribution function $\mathbf{f}(\mathbf{x}, \mathbf{c}, t)$ — introduced in Section 17.1 — evolves in time. In the **Eulerian frame** (you stand at a fixed point in phase space and watch the distribution change around you), it reads:

$$\frac{\partial \mathbf{f}}{\partial t} + \mathbf{c} \cdot \nabla \mathbf{f} + \mathbf{F} \cdot \nabla_{\mathbf{c}} \mathbf{f} = \Omega(\mathbf{f}) \quad (17.7)$$

Let us parse each term on the left-hand side:

- $\frac{\partial \mathbf{f}}{\partial t}$: How the distribution changes in time at a fixed point in phase space.
- $\mathbf{c} \cdot \nabla \mathbf{f}$: Particles streaming through physical space at velocity $\mathbf{c}$. This is advection: a particle at $\mathbf{x}$ with velocity $\mathbf{c}$ will be at $\mathbf{x} + \mathbf{c}\Delta t$ after time Δt .
- $\mathbf{F} \cdot \nabla_{\mathbf{c}} \mathbf{f}$: External forces accelerating particles, changing their velocities. The $\nabla_{\mathbf{c}}$ means the derivative is in *velocity* space, not physical space. This term accounts for gravity, electric fields, or any body force that shifts the velocity distribution.

The first two terms together form the **material (total) derivative** familiar from fluid mechanics — the rate of change following a particle that streams at velocity $\mathbf{c}$. The third term is separate: it represents forces that *change* the particle's velocity, pushing it through velocity space. If we define the streaming derivative as

$$\frac{D\mathbf{f}}{Dt} \equiv \frac{\partial \mathbf{f}}{\partial t} + \mathbf{c} \cdot \nabla \mathbf{f}$$

then equation (17.7) can be written compactly as

$$\frac{Df}{Dt} + \mathbf{F} \cdot \nabla_{\mathbf{c}} f = \Omega(f) \quad (17.7a)$$

This **Lagrangian perspective** is often more intuitive: ride along with a particle that streams at constant velocity $\mathbf{c}$. From its frame, the distribution f changes for only two reasons — external forces push particles to different velocities, and collisions reshuffle the velocity population. If there were no forces and no collisions, f would be carried unchanged along the streaming trajectory, like a ball on a frictionless surface. For LBM applications (diffusion, monodomain), we will drop the external force term entirely (no body forces accelerate particles through velocity space). This makes the Eulerian form (17.7) the natural starting point for deriving the lattice-Boltzmann equation. **The right-hand side is the collision operator:** The symbol $\Omega(f)$ represents collisions. When particles collide, their velocities change. The full collision integral is:

$$\Omega(f) = \int \int (f' f'_1 - f f_1) g B(g, \theta) d\Omega d\mathbf{c}_1 \quad (17.8)$$

The notation is intimidating. Here is what it means:

- f and f_1 : the distribution functions of two particles BEFORE collision.
- f' and f'_1 : the distributions of those same two particles AFTER collision.
- $g = |\mathbf{c} - \mathbf{c}_1|$: the relative speed of the two particles.
- $B(g, \theta)$: the scattering cross-section, which depends on the relative velocity and scattering angle θ .
- $d\Omega$: integration over all possible scattering angles.
- $d\mathbf{c}_1$: integration over all velocities of the second particle.

The expression $(f' f'_1 - f f_1)$ is a balance: the gain term $f' f'_1$ counts particles scattered INTO the state $(\mathbf{x}, \mathbf{c})$ by collisions, while the loss term $f f_1$ counts particles scattered OUT.

The BGK Simplification

The collision integral (17.8) is a 5-dimensional integral — computationally intractable. For a coarse 3D grid with 100^3 points and even a modest 10^3 velocities per point, evaluating it requires $\sim 10^{15}$ operations per timestep. For most of the 20th century, the Boltzmann equation remained a theoretical tool, not a practical one. The breakthrough came from Bhatnagar, Gross, and Krook in 1954. They asked: **Can we replace the complex collision integral with something simple that still produces the correct macroscopic physics?** Their answer was to replace the collision integral with a single relaxation toward the equilibrium distribution we met in Section 17.1:

$$\Omega(f) \approx -\frac{1}{\tau}(f - f^{\text{eq}}) \quad (17.11)$$

where τ is a single relaxation time and f^{eq} is the Maxwell-Boltzmann equilibrium (equation 17.2). The idea is elegantly simple: the actual distribution f is nudged toward its local equilibrium at a rate determined by τ . No detailed calculation of scattering angles or cross-sections is needed. This is the **BGK approximation**. Notice the direct connection to Section 17.1: the f^{eq} that appears here is exactly the Gaussian equilibrium (17.2) — the bell curve in velocity space that encodes conservation, zero flux, and isotropic variance. The BGK operator says: whatever the current distribution looks like, push it toward that Gaussian. The three physical claims we established in Section 17.1 — equations (17.3), (17.4), and (17.5) — are the properties that f^{eq} carries into the collision step. Substituting (17.11) into (17.7), we get the **BGK Boltzmann equation**:

$$\frac{\partial f}{\partial t} + \mathbf{c} \cdot \nabla f = -\frac{1}{\tau}(f - f^{\text{eq}}) \quad (17.12)$$

where we have dropped the external force term $\mathbf{F} \cdot \nabla_{\mathbf{c}} \mathbf{f}$ (in LBM for diffusion, there are no external forces accelerating particles through velocity space).

Why BGK Works: Chapman-Enskog Analysis

Why should such a drastic simplification — replacing a 5-dimensional integral with a simple relaxation — give the right macroscopic physics? The answer comes from **Chapman-Enskog analysis**, one of the deepest results in statistical mechanics. The idea: assume the distribution function is close to equilibrium, with small corrections:

$$\mathbf{f} = \mathbf{f}^{\text{eq}} + \epsilon \mathbf{f}^{(1)} + \epsilon^2 \mathbf{f}^{(2)} + \dots$$

where ϵ is a small parameter measuring how far the system is from equilibrium (technically, the Knudsen number $\mathbf{Kn} = \lambda/L$, the ratio of mean free path to system size). Then expand the Boltzmann equation order by order. What emerges at each order is a specific piece of the macroscopic equations: **Zeroth order** (ϵ^0): the distribution equals the equilibrium $\mathbf{f} = \mathbf{f}^{\text{eq}}$. Taking the zeroth and first moments of $\mathbf{f}^{\text{eq}}$ (exactly the integrals from equations 17.3 and 17.4) gives the **conservation laws**:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0 \quad (\text{mass conservation})$$

These are the Euler equations — ideal fluid dynamics with no dissipation. **First order** (ϵ^1): the correction $\mathbf{f}^{(1)}$ introduces gradients of velocity and temperature. Taking moments of $\mathbf{f}^{(1)}$ produces the **dissipative terms** — this is where viscosity and diffusivity enter:

$$\rho \frac{D\mathbf{u}}{Dt} = -\nabla p + \underbrace{\eta \nabla^2 \mathbf{u}}_{\text{viscous stress}} + \mathbf{f} \quad (17.9)$$

for fluid flow (Navier-Stokes), and

$$\frac{\partial T}{\partial t} + \mathbf{u} \cdot \nabla T = \underbrace{D \nabla^2 T}_{\text{diffusion}} \quad (17.10)$$

for heat or mass diffusion. The underbraced terms are the ones that the first-order correction adds — they are absent in the Euler (zeroth-order) equations. The transport coefficients (viscosity η , diffusivity D) are determined by the relaxation time τ . Large τ (slow relaxation toward equilibrium) gives high viscosity or diffusivity; small τ gives low viscosity or diffusivity. The relationship is precise and will be derived explicitly in Section 17.4 as the τ - D relationship.

Intuition

The key insight: The macroscopic PDEs we have been solving throughout this textbook — the diffusion equation, the monodomain equation, even Navier-Stokes — are not fundamental. They are **emergent**. They arise as the zeroth- and first-order limits of the Boltzmann equation in the collision-dominated regime. The zeroth-order (equilibrium) contribution gives conservation laws. The first-order (near-equilibrium) correction gives the dissipative transport terms. The equilibrium $\mathbf{f}^{\text{eq}}$ carries the conservation physics (17.3)–(17.5); the relaxation rate $1/\tau$ sets the transport coefficients. Together, they recover the full macroscopic PDE.

Mathematical Note

Why Chapman-Enskog Works: The expansion assumes the distribution is locally near equilibrium, with small non-hydrodynamic corrections. The collision operator acts to restore equilibrium. By expanding around this equilibrium and using the fact that the collision operator has specific zero eigenvalues (corresponding to conserved quantities like mass and momentum), one can systematically extract the macroscopic equations order by order in the Knudsen number.

17.3 The Lattice-Boltzmann Equation: Single Relaxation Time (SRT/BGK)

The Core Strategy of LBM

1. **Start from the kinetic level:** the Boltzmann equation governs how $f(\mathbf{x}, \mathbf{c}, t)$ evolves in phase space (Section 17.2).
2. **Simplify the collision:** replace the intractable collision integral with BGK relaxation toward f^{eq} — the Maxwell-Boltzmann Gaussian from Section 17.1 (Section 17.2).
3. **Discretize velocity space:** replace the continuous $\mathbf{c}$ with a small finite set of lattice velocities $\mathbf{c}_i$ and write the discrete update rule (this section).
4. **Derive the lattice from the Gaussian:** the weights w_i , velocity sets $\mathbf{c}_i$, and discrete equilibrium f_i^{eq} all follow from quadrature on the Maxwell-Boltzmann distribution (Section 17.4).
5. **Trust Chapman-Enskog:** this analysis guarantees that the simplified, discretized scheme recovers the correct macroscopic PDE in the hydrodynamic limit.

The equilibrium f^{eq} is the thread that connects everything: it defines the target of relaxation (BGK), it carries the conservation laws (Section 17.1), and it determines which macroscopic equation the scheme recovers (Chapman-Enskog).

From Continuous to Discrete: Three Deliberate Choices

In Section 17.2 we arrived at the BGK Boltzmann equation (17.12):

$$\frac{\partial f}{\partial t} + \mathbf{c} \cdot \nabla f = -\frac{1}{\tau}(f - f^{\text{eq}})$$

This is still a continuous equation — f lives on a continuous position space, a continuous velocity space, and continuous time. To turn it into something a computer can execute, we make three deliberate discretization choices:

1. **Discretize velocity space.** Replace the continuous velocity $\mathbf{c}$ with a small finite set of lattice velocities $\{\mathbf{c}_0, \mathbf{c}_1, \dots, \mathbf{c}_{Q-1}\}$. The continuous distribution $f(\mathbf{x}, \mathbf{c}, t)$ becomes a finite collection of distributions $f_i(\mathbf{x}, t)$, one for each lattice direction i . This is the step where velocity space — the infinite-dimensional space from Section 17.1 — collapses to a handful of directions. The choice of which velocities to keep (and the associated weights) is governed by the Gaussian quadrature we will derive in Section 17.4.
2. **Discretize time.** Replace $\partial f / \partial t$ with a finite difference over a timestep Δt . This turns the differential equation into an update rule.
3. **Lock the grid to the velocities.** Choose the spatial grid spacing $\Delta \mathbf{x}$ and timestep Δt so that each lattice velocity $\mathbf{c}_i$ moves a particle from one grid node to an adjacent grid node in exactly one timestep: $\mathbf{c}_i \Delta t = \mathbf{c}_i \Delta \mathbf{x} / |\mathbf{c}_i|$. This eliminates interpolation entirely — streaming is exact.

Notice one term that is conspicuously absent: the external force $\mathbf{F} \cdot \nabla_{\mathbf{c}} f$ from equation (17.7). Recall that this term describes forces that accelerate particles through velocity space — gravity pulling molecules downward, electric fields pushing ions. For diffusion and the monodomain equation, there are no such body forces: particles stream freely and collide, but nothing pushes them to change velocity mid-flight. We therefore drop it. (For applications that do require forcing — buoyancy-driven flows, electrokinetics — the force re-enters as a source term added to the right-hand side, but that is beyond our scope.) With these three choices, the material derivative Df/Dt becomes a simple finite difference along the lattice direction i , and the BGK equation becomes:

The Discrete Lattice-Boltzmann Equation

$$f_i(\mathbf{x}, t + \Delta t) - f_i(\mathbf{x}, t) = -\frac{\Delta t}{\tau} [f_i(\mathbf{x}, t) - f_i^{\text{eq}}(\mathbf{x}, t)] \quad (17.13)$$

The left-hand side is the change in the distribution f_i at node $\mathbf{x}$ over one timestep. The right-hand side is the collision: a relaxation nudge toward the local equilibrium f_i^{eq} , scaled by $\Delta t / \tau$. This is the entire algorithm at a single node before streaming. After computing the post-collision state, we stream: each f_i is moved to the neighboring node in the direction $\mathbf{c}_i$. The combined collision-then-stream update gives the full lattice-Boltzmann equation in its standard form:

$$f_i(\mathbf{x} + \mathbf{c}_i \Delta t, t + \Delta t) = f_i(\mathbf{x}, t) - \frac{\Delta t}{\tau} [f_i(\mathbf{x}, t) - f_i^{\text{eq}}(\mathbf{x}, t)]$$

This single line says: compute the collision at $\mathbf{x}$, then place the result at the downstream neighbor $\mathbf{x} + \mathbf{c}_i \Delta t$. The equation is fully explicit — no matrix inversions, no implicit coupling between nodes, no iterative solves. You compute it one node at a time, and the entire simulation marches forward. In practice, we split this into two conceptual steps, which we now describe.

Step 1: Collision (Local, Pure Relaxation)

At each node, we compute:

$$f_i^*(\mathbf{x}, t) = f_i(\mathbf{x}, t) - \frac{\Delta t}{\tau} [f_i(\mathbf{x}, t) - f_i^{\text{eq}}(\mathbf{x}, t)] \quad (17.14)$$

This is the collision step. The superscript $*$ on f_i^* denotes the **post-collision** distribution — the state of the distribution function AFTER the collision relaxation has been applied but BEFORE streaming moves data to neighbors. You can read f_i^* as "f-i-star" or "f-i post-collision." It is an intermediate quantity: f_i is the pre-collision state, f_i^* is the post-collision state, and after streaming, f_i^* becomes the new f_i at the destination node. The collision step happens at a single node, purely local. The distributions f_i at node $\mathbf{x}$ are nudged toward their equilibrium values f_i^{eq} . The speed of this nudge is controlled by $\Delta t/\tau$:

- If τ is large, the nudge is small — the distribution relaxes slowly.
- If τ is small (but must remain $> \Delta t/2$ for stability), the nudge is large — the distribution relaxes quickly.

The parameter τ directly controls the transport properties (diffusivity, viscosity). This is the heart of the method: all the physics (how fast heat diffuses, how viscous the fluid is) is encoded in τ .

Step 2: Streaming (Pure Data Movement)

After collision, we shift the post-collision distributions to their neighboring nodes:

$$f_i(\mathbf{x} + \mathbf{c}_i \Delta t, t + \Delta t) = f_i^*(\mathbf{x}, t) \quad (17.15)$$

This is the streaming step. Each distribution f_i^* at node $\mathbf{x}$ slides to the neighboring node in direction $\mathbf{i}$. The destination is $\mathbf{x} + \mathbf{c}_i \Delta t$. If the lattice is designed so that $\mathbf{c}_i \Delta t = \Delta \mathbf{x}$ (one lattice spacing), then streaming is a simple data copy — a value at the center moves to a neighbor. Critically, streaming is not a collision. There is no physics in streaming — it is pure kinematics. A particle moving with velocity $\mathbf{c}_i$ will naturally travel a distance $\mathbf{c}_i \Delta t$ in time Δt . Streaming enforces this.

The Combined Form

Substituting the collision step (17.14) into the streaming step (17.15) — replacing f_i^* with its definition — gives the single combined equation:

$$f_i(\mathbf{x} + \mathbf{c}_i \Delta t, t + \Delta t) = f_i(\mathbf{x}, t) - \frac{\Delta t}{\tau} [f_i(\mathbf{x}, t) - f_i^{\text{eq}}(\mathbf{x}, t)] \quad (17.16)$$

This is the **lattice-Boltzmann equation in streaming form**. It is the same physics as equation (17.13), but written from the destination's perspective: the value arriving at node $\mathbf{x} + \mathbf{c}_i \Delta t$ at the next timestep equals the current value at node $\mathbf{x}$, corrected by the collision. You will encounter this form throughout the LBM literature — it is the most compact way to write the full collide-and-stream update.

Intuition

Locality and Parallelization: Both collision and streaming are local operations. Collision affects only one node at a time. Streaming is a data copy — the value $f_i^*(\mathbf{x}, t)$ moves to a neighbor, but it does not interact with neighbors' data. After collision, all the f_i at all nodes are independent. Streaming then redistributes them. This extreme locality means LBM is trivially parallelizable: each node (or

even each lattice direction at each node) can be processed on a separate processor with zero communication overhead during the step. Only a halo exchange is needed between processors after streaming. This is one reason LBM is popular on GPUs.

Variable Definitions and Component Form

To implement LBM, you must understand every variable. Here is a complete reference:

Symbol	Meaning	Units / Constraints
$f_i(\mathbf{x}, t)$	Distribution function for direction i at node $\mathbf{x}$, time t	Density (or concentration if diffusion)
$f_i^*(\mathbf{x}, t)$	Post-collision distribution for direction i (after relaxation, before streaming)	Same units as f_i
$f_i^{\text{eq}}(\mathbf{x}, t)$	Equilibrium distribution for direction i	Density (or concentration)
τ	Relaxation time	Must satisfy $\tau > \Delta t/2$
Δt	Timestep	In lattice units, $\Delta t = 1$
$\mathbf{c}_i$	Discrete velocity vector for direction i	$\mathbf{c}_0 = \mathbf{0}$ (rest); $ \mathbf{c}_i = c = \Delta x / \Delta t$ (moving)
w_i	Weight for direction i	$\sum_i w_i = 1$
$\phi(\mathbf{x}, t) = \sum_i f_i$	Macroscopic variable (temperature, voltage, density)	Physical units (K, mV, kg/m ³)
Q	Number of lattice directions (velocities)	i ranges from 0 to $Q - 1$

In detail: $f_i(\mathbf{x}, t)$ is the distribution function for direction i . Think of it as "the portion of the macroscopic quantity assigned to direction i at this node at this time." There are Q values of f_i stored at each node — one per lattice direction. The sum $\sum_i f_i = \phi$ recovers the total macroscopic quantity. $f_i^{\text{eq}}(\mathbf{x}, t)$ is the equilibrium distribution for direction i — the target that the collision step relaxes toward. Its exact form depends on the weights w_i and the macroscopic field, and will be derived in Section 17.4 from the Gaussian equilibrium we built in Section 17.1. τ is the relaxation time. It controls how quickly the distribution approaches equilibrium. The relationship between τ and the physical diffusivity D comes from Chapman-Enskog analysis and will be derived in Section 17.4. Δt is the timestep. In lattice units, we often set $\Delta t = 1$, but it is useful to keep Δt explicit when connecting lattice variables to physical ones. $\mathbf{c}_i$ is the discrete velocity vector for direction i . Each $\mathbf{c}_i$ points from a node to a neighbor (or stays put if $i = 0$). The specific velocity sets and their associated weights are derived in Section 17.4 from the Gaussian quadrature framework. w_i is the weight for direction i . Like $\mathbf{c}_i$, the weights are uniquely determined by the quadrature exactness conditions and will be derived in Section 17.4. $\phi(\mathbf{x}, t) = \sum_i f_i(\mathbf{x}, t)$ is the recovered macroscopic quantity. For diffusion of temperature, ϕ is temperature. For cardiac electrophysiology, ϕ is the voltage. The sum over all lattice directions gives the total.

17.4 From the Gaussian to the Lattice: Discretizing f^{eq} , w_i , and $\mathbf{c}_i$

From Integrals to Sums

In Section 17.1 we wrote down the Maxwell-Boltzmann equilibrium (equation 17.2) and showed that it satisfies three integral identities — conservation of mass (17.3), momentum (17.4), and isotropic variance (17.5). Each of these is an integral over velocity space: you integrate $f^{\text{eq}}(\mathbf{c})$ — or f^{eq} multiplied by some power of $\mathbf{c}$ — over *all directions and all magnitudes*. LBM replaces "all directions and all magnitudes" with **a few lattice vectors $\mathbf{e}_i$ and weights w_i** . The lattice vectors are dimensionless integer vectors — their components are just 0 and ± 1 . For example, in D2Q5:

$$\mathbf{e}_0 = (0, 0), \quad \mathbf{e}_1 = (1, 0), \quad \mathbf{e}_2 = (-1, 0), \quad \mathbf{e}_3 = (0, 1), \quad \mathbf{e}_4 = (0, -1)$$

Each lattice vector comes with a weight w_i that says “this sample point represents this much of the Gaussian bell curve.” Every continuous integral becomes a finite sum:

$$\int p(\mathbf{c}) \underbrace{\frac{1}{(2\pi c_s^2)^{D/2}} e^{-|\mathbf{c}|^2/2c_s^2}}_{\text{Gaussian kernel}} d\mathbf{c} \approx \sum_{i=0}^{Q-1} w_i p(\mathbf{e}_i) \quad (17.18)$$

The left-hand side integrates over the full continuous velocity space $\mathbf{c}$. The right-hand side evaluates at the lattice nodes $\mathbf{e}_i$ in dimensionless space — this is where the quadrature lives. Here p is a placeholder — it stands for whatever function of velocity you want to integrate against the Gaussian. Plug in $p = \mathbf{1}$ and you compute total mass. Plug in $p = \mathbf{e}_{i\alpha}$ (a single component of $\mathbf{e}_i$) and you compute momentum. Plug in $p = \mathbf{e}_{i\alpha}\mathbf{e}_{i\beta}$ and you compute the stress tensor. The nodes $\mathbf{e}_i$ and their weights w_i are chosen so that these sums are *exact* for all polynomials up to the degree you need. **This is where w_i and $\mathbf{e}_i$ come from.** They are not free parameters that we tune. They are the nodes and weights of a quadrature rule applied to the Gaussian — a married pair, born together from the bell curve’s shape. Once you choose a lattice geometry (how many sample points, and where), the weights are uniquely determined.

Intuition

How Can Five Points Replace an Entire Plane? In the continuous integral, $\mathbf{c}$ ranges over all directions and all magnitudes — an infinite velocity plane. On the lattice, we replace this with just 5 (or 9) sample points whose components are 0 and ± 1 . This seems like a drastic reduction. But remember what we are integrating: a Gaussian bell curve that decays rapidly away from the origin. Almost all of the integral’s value is concentrated within a few lattice spacings of the center. The quadrature exploits this — it places its sample points precisely where the Gaussian has significant weight, and the weights w_i encode how much of the bell curve each point represents. Five well-chosen points on a steep hill capture more information than a thousand points in the flat plains beyond it.

Physical Velocities

The $\mathbf{e}_i$ are dimensionless integers — they tell you which neighbor to hop to on the grid. But the Boltzmann equation and the physics it describes involve velocities with real units. To recover a **physical velocity $\mathbf{c}_i$** (length/time), scale by the lattice speed $\mathbf{c} = \Delta\mathbf{x}/\Delta t$:

$$\mathbf{c}_i = \mathbf{e}_i \frac{\Delta\mathbf{x}}{\Delta t}$$

So $\mathbf{e}_1 = (1, 0)$ is a one-node hop on the grid, and $\mathbf{c}_1 = (\Delta\mathbf{x}/\Delta t, 0)$ is the physical speed of that hop. Throughout this chapter, we use $\mathbf{e}_i$ when writing velocity tables, streaming rules, and the quadrature formula — anything that lives in lattice space. We use $\mathbf{c}_i$ when connecting to physical quantities: the speed of sound $\mathbf{c}_s$, the τ - D diffusion relationship, and the equilibrium distribution in fluid-flow problems. A natural worry: if $\mathbf{c}_i$ changes when you refine your grid (because $\Delta\mathbf{x}/\Delta t$ changes), do the weights w_i change too? They do not. The weights are married to $\mathbf{e}_i$, not to $\mathbf{c}_i$, and $\mathbf{e}_i$ is a fixed set of integer vectors that never changes. When you halve $\Delta\mathbf{x}$, every physical velocity $\mathbf{c}_i$ shrinks — but the lattice speed of sound $\mathbf{c}_s$ shrinks by the same factor, so the dimensionless ratio $\mathbf{c}_i/\mathbf{c}_s = \mathbf{e}_i/\sqrt{3}$ is unchanged. The Gaussian in lattice units always has the same shape, the sample points sit in the same places relative to its width, and the weights stay fixed. Grid refinement changes the physics that $\mathbf{c}_s$ maps to, not the quadrature itself.

The Discrete Equilibrium

For **pure diffusion** ($\mathbf{u} = 0$), the Gaussian is centered at the origin. When you evaluate it at the quadrature nodes and fold in the weights, all the velocity-dependent structure vanishes. The discrete equilibrium is simply:

$$f_i^{\text{eq}} = w_i \phi \quad (17.27)$$

where $\phi = \sum_j f_j$ is the local macroscopic quantity (temperature, voltage, concentration). Each direction carries a fraction w_i of the total field — the rest direction carries the most (it sits at the peak of the bell curve), and the moving directions carry the remainder. There is no preferred direction — the equilibrium is symmetric, just as the centered Gaussian is symmetric. This is the equilibrium we use for cardiac electrophysiology, since the monodomain and bidomain equations are diffusion equations with no bulk flow.

Gauss-Hermite Quadrature: Why These Specific Nodes and Weights

The quadrature rule in equation (17.18) has a classical name: **Gauss-Hermite quadrature**. It is the standard method for integrating polynomials multiplied by a Gaussian kernel, and it has been studied since the 19th century. The key result is this: given a Gaussian in D dimensions, there exist specific sets of nodes and weights such that the finite sum is *exact* (not approximate) for all polynomial integrands up to a certain degree.

Intuition

Nodes and Weights Are a Married Pair: In Gauss-Hermite quadrature, you do not get to choose the nodes and the weights independently. Once you pick a set of sample points on the bell curve, the weights are uniquely determined (and vice versa). This is why every lattice specification — D2Q5, D2Q9, D3Q7 — always lists both $\mathbf{e}_i$ AND $\mathbf{w}_i$ together. They are a package deal, born from the same Gaussian.

For **diffusion** (no bulk flow), the equilibrium is the Gaussian centered at zero. The macroscopic equations involve integrals of polynomials up to second degree in $\mathbf{c}$, so exactness through second order is sufficient. D2Q5 and D3Q7 satisfy this. For **fluid flow** (Navier-Stokes), the equilibrium is a shifted Gaussian, and higher-order polynomial terms appear. The quadrature must also be exact for fourth-degree polynomials, which requires more lattice directions (D2Q9 instead of D2Q5).

Discrete vs. Continuous: Lattice Naming Convention

Lattices are named using the notation $\mathbf{D}_d\mathbf{Q}_q$, where:

- $\mathbf{D}$ = spatial dimension (1, 2, or 3)
- $\mathbf{Q}$ = number of lattice directions (discrete velocities)

Common examples:

- **D1Q3:** 1D, 3 velocities (left, right, rest)
- **D2Q5:** 2D, 5 velocities (rest + 4 cardinal neighbors)
- **D2Q9:** 2D, 9 velocities (rest + 4 cardinal + 4 diagonal)
- **D3Q7:** 3D, 7 velocities (rest + 6 face-centered neighbors)
- **D3Q19:** 3D, 19 velocities (face-centered + edge-adjacent)
- **D3Q27:** 3D, 27 velocities (complete cubic stencil)

The choice of lattice is determined by the physics:

- **For diffusion** (no bulk flow), D2Q5 or D3Q7 suffice. These have only cardinal directions.
- **For fluid flow** (Navier-Stokes), diagonal directions are needed to represent the full stress tensor. D2Q9 or D3Q19 are required.

Since cardiac electrophysiology primarily solves diffusion problems (the monodomain and bidomain equations), **D2Q5 and D3Q7 are the natural choices**.

Key Insight

Memory vs. Physics: The number of distributions stored per node equals Q . A D2Q9 lattice requires storing 9 values per node; a D2Q5 requires only 5. For a $500 \times 500 \times 500$ 3D mesh, a D2Q5 saves a factor of $(7 \text{ directions}) / (27 \text{ directions}) \approx 4\times$ memory compared to D3Q27, even though physically, D3Q7 produces the same diffusion results. This is one reason LBM for cardiac problems uses minimal lattices.

The Discrete Velocity Set

With the quadrature framework in mind, here are the standard lattices. Each is a specific Gauss-Hermite quadrature rule on a regular grid — the nodes are placed at grid points so that streaming is a simple data shift. **D1Q3 (One-Dimensional):**

$$e_0 = 0, \quad e_1 = 1, \quad e_2 = -1$$

(17.17)

Three quadrature nodes in one dimension: one at the center of the bell curve (rest), one hop to the right, one hop to the left. The physical velocities are $\mathbf{c}_i = \mathbf{e}_i \cdot \Delta \mathbf{x} / \Delta t$. **D2Q5 (Two-Dimensional, Cardinal Directions):**

i	Direction	$\mathbf{e}_i$
0	Rest	(0, 0)
1	East	(1, 0)
2	West	(-1, 0)
3	North	(0, 1)
4	South	(0, -1)

Five quadrature nodes: one at the peak of the 2D bell curve (rest), four at one lattice spacing along each cardinal direction. These five sample points are sufficient to exactly integrate second-degree polynomials against the 2D Gaussian.

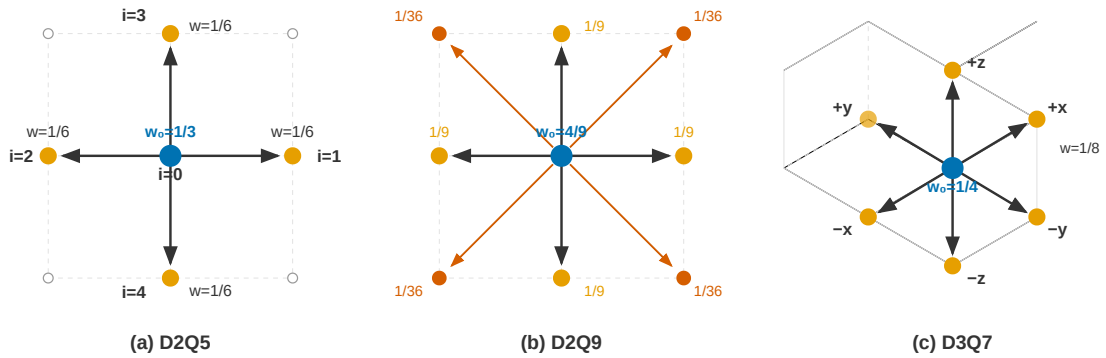


Figure 17.1: Common lattice stencils for LBM. **(a)** D2Q5: five velocities (rest + four cardinal) — sufficient for isotropic diffusion. **(b)** D2Q9: nine velocities (rest + cardinal + diagonal) — required for fluid flow to capture cross-velocity terms. **(c)** D3Q7: seven velocities in 3D (rest + six face-centered) — the minimal 3D stencil for diffusion. Blue center node = rest velocity (peak of the Gaussian); orange = cardinal neighbors; vermillion = diagonal neighbors. Arrows indicate streaming directions; weights w_i annotated at each node.

D2Q9 (Two-Dimensional, Cardinal + Diagonal): For fluid flow, the polynomial expansion of the shifted Gaussian contains terms like $e_{ix}e_{iy}$ (cross-products of velocity components). To integrate these exactly, we need diagonal quadrature nodes. D2Q9 adds four diagonal directions to D2Q5:

i	Direction	$\mathbf{e}_i$
0	Rest	(0, 0)
1	East	(1, 0)
2	West	(-1, 0)
3	North	(0, 1)
4	South	(0, -1)
5	NE	(1, 1)
6	NW	(-1, 1)
7	SE	(1, -1)
8	SW	(-1, -1)

Directions 1–4 are the same cardinal nodes as D2Q5. Directions 5–8 are the diagonal nodes. In lattice units, they sit at distance $\sqrt{2}$ from the center (vs. distance 1 for the cardinal nodes). They are further out on the bell curve, sampling the Gaussian's tails in

diagonal directions. **D3Q7 (Three-Dimensional, Face-Centered)**: The 3D analogue of D2Q5. One quadrature node at the peak, six at one lattice spacing along each coordinate axis:

i	Direction	$\mathbf{e}_i$
0	Rest	$(0, 0, 0)$
1	$+x$	$(1, 0, 0)$
2	$-x$	$(-1, 0, 0)$
3	$+y$	$(0, 1, 0)$
4	$-y$	$(0, -1, 0)$
5	$+z$	$(0, 0, 1)$
6	$-z$	$(0, 0, -1)$

All six moving nodes sit at the same distance (one lattice spacing) from the center — they sample the Gaussian at the same height on the bell curve, which is why they all receive the same weight.

The Weight Lattice

The weights paired with each velocity set are determined by Gauss-Hermite quadrature — equation (17.18). Each weight w_i represents how much of the Gaussian bell curve the quadrature node $\mathbf{e}_i$ is responsible for. Nodes near the peak of the Gaussian (where the bell curve is tallest) represent more of the total area and receive larger weights. Nodes further out (where the bell curve has decayed) represent less area and receive smaller weights. **D1Q3 weights**:

$$w_0 = \frac{2}{3}, \quad w_1 = w_2 = \frac{1}{6} \quad (17.19)$$

The rest node sits at the peak of the 1D bell curve — it captures two-thirds of the total area. The two moving nodes, one lattice spacing away from the peak, each capture one-sixth. Most of the Gaussian's area is concentrated near the center, which is why the rest weight dominates. **D2Q5 weights**:

$$w_0 = \frac{1}{3}, \quad w_1 = w_2 = w_3 = w_4 = \frac{1}{6} \quad (17.20)$$

The rest node captures one-third of the 2D bell curve. Each cardinal node captures one-sixth. The total is $1/3 + 4 \times 1/6 = 1$. All four cardinal nodes are at the same distance from the peak, so they receive the same weight — the bell curve is symmetric. **D2Q9 weights**:

$$w_0 = \frac{4}{9}, \quad w_{1,2,3,4} = \frac{1}{9} \text{ (cardinal)}, \quad w_{5,6,7,8} = \frac{1}{36} \text{ (diagonal)} \quad (17.21)$$

Three distinct weight classes, directly reflecting the Gaussian's shape. The rest node at the peak captures $4/9$ of the area — nearly half. The cardinal nodes at distance 1 (in lattice units) each capture $1/9$. The diagonal nodes at distance $\sqrt{2}$ each capture only $1/36$ — four times less than a cardinal node. This factor of four is the Gaussian at work: moving from distance c to $\sqrt{2}c$ from the peak, the bell curve has decayed, so those nodes represent less area. Check: $4/9 + 4 \times 1/9 + 4 \times 1/36 = 4/9 + 4/9 + 1/9 = 1 \checkmark$.

Intuition

Why Diagonal Weights Are Smaller: Picture a 2D bell curve — a smooth hill centered at the origin. The cardinal quadrature nodes sit at distance 1 from the peak (along the axes). The diagonal nodes sit at distance $\sqrt{2}$ (along the diagonals). Because the Gaussian decays exponentially with distance from the center, the bell curve is lower at the diagonal nodes than at the cardinal nodes. Lower height means less area to represent, which means a smaller quadrature weight. This is not a choice — it is a consequence of

the Gaussian's shape. The ratio $w_{\text{diagonal}}/w_{\text{cardinal}} = (1/36)/(1/9) = 1/4$ is determined by the exactness conditions, but the physical reason is the Gaussian decay.

D3Q7 weights:

$$w_0 = \frac{1}{4}, \quad w_1 = \dots = w_6 = \frac{1}{8} \quad (17.22)$$

The rest node captures one-quarter of the 3D bell curve. Each of the six face-centered nodes captures one-eighth. All moving nodes are at the same distance c from the peak (same height on the bell curve), so they receive equal weight. Check:

$$1/4 + 6 \times 1/8 = 1/4 + 3/4 = 1 \checkmark.$$

Mathematical Note

Deriving D2Q5 Weights from Quadrature: For D2Q5, we have two unknowns: the rest weight w_0 and the cardinal weight w_c (shared by directions 1–4 by symmetry). Two quadrature exactness conditions pin them down. Plugging $\mathbf{p} = \mathbf{1}$ into (17.18): $w_0 + 4w_c = 1$ (the weights must sum to one). Plugging $\mathbf{p} = \mathbf{e}_{ix}^2$ into (17.18): $\sum_i w_i e_{ix}^2 = 1/3$. The rest node ($\mathbf{e}_{0x} = 0$) contributes zero. Directions 1 and 2 ($\mathbf{e}_{ix} = \pm 1$) each contribute $w_c \cdot 1$. Directions 3 and 4 ($\mathbf{e}_{ix} = 0$) contribute nothing. This gives $2w_c = 1/3$, yielding $w_c = 1/6$. Substituting back: $w_0 = 1 - 4(1/6) = 1/3$. The weights are not chosen — they are uniquely determined by Gauss-Hermite quadrature on this lattice geometry.

The Navier-Stokes Equilibrium

For **fluid flow** (Navier-Stokes), the Gaussian is centered at the bulk velocity $\mathbf{u}$ rather than at zero. Expanding the exponential in (17.2) as a Taylor series in $\mathbf{u}/c_s$ (valid when the flow speed is much smaller than the speed of sound — the low Mach number regime), the discrete equilibrium becomes:

$$f_i^{\text{eq}} = w_i \rho \left(1 + \frac{\mathbf{u} \cdot \mathbf{c}_i}{c_s^2} + \frac{(\mathbf{u} \cdot \mathbf{c}_i)^2}{2c_s^4} - \frac{u^2}{2c_s^2} \right) \quad (17.28)$$

The terms beyond $w_i \rho$ are the polynomial corrections that arise from shifting the bell curve's center from zero to $\mathbf{u}$. These terms break the directional symmetry — they bias the distribution toward the direction of flow. Because these corrections involve products like $c_{ix}c_{iy}$, the quadrature must exactly integrate fourth-degree polynomials, which is why fluid flow requires more quadrature nodes (D2Q9 instead of D2Q5).

Lattice Requirements for Different Physics

The choice of lattice is determined by how many terms in the polynomial expansion of the Gaussian need to be captured exactly. For diffusion with the isotropic equilibrium (17.27), only terms up to second degree matter — the Gauss-Hermite quadrature (17.18) need only be exact through second order. D2Q5 (and D3Q7 in 3D) satisfy these, making them the most efficient choices for cardiac problems. For fluid flow with the velocity-dependent equilibrium (17.28), terms up to fourth degree appear. The quadrature must be exact for fourth-order products like $\sum_i w_i e_{ix} e_{iy} e_{iz} e_{it}$. D2Q5 does not have enough nodes, but D2Q9 does. In cardiac electrophysiology, we solve diffusion (not Navier-Stokes), so D2Q5 and D3Q7 are sufficient. Using a minimal lattice saves both memory (Q distributions per node) and computation (fewer collision and streaming operations per timestep).

Lattice Speed of Sound and the τ - D Relationship

For each lattice, there is a characteristic speed c_s called the **lattice speed of sound**. From the Gauss-Hermite exactness conditions and the lattice geometry, we have:

$$c_s^2 = \frac{c^2}{3} \quad (17.23)$$

The parameter c_s plays a dual role. In the continuous Gaussian of Section 17.1, c_s^2 is the **variance** — it controls the width of the bell curve. On the lattice, this same c_s becomes the **lattice speed of sound**, with the specific value $c_s^2 = c^2/3$ where $c = \Delta x/\Delta t$ is the lattice speed. This is not a coincidence: the lattice speed of sound is inherited directly from the variance of the Maxwellian that the lattice discretizes. Every time you see c_s^2 in an LBM formula — in the weights, in the equilibrium, in the τ - D relationship — it traces back to this Gaussian width. The speed of sound c_s is just a lattice constant — it emerges from the geometry and isotropy conditions. However, it plays a crucial role in relating the relaxation time τ to the physical diffusivity D . Through Chapman-Enskog analysis, one can show that for the BGK collision operator applied to a diffusion problem, the relationship is:

$$D = c_s^2 \left(\tau - \frac{\Delta t}{2} \right) \quad (17.24)$$

or, rearranging:

$$\tau = \frac{\Delta t}{2} + \frac{D}{c_s^2} \quad (17.25)$$

Substituting $c_s^2 = c^2/3 = \Delta x^2/(3\Delta t^2)$:

$$\tau = \frac{\Delta t}{2} + \frac{3D\Delta t}{\Delta x^2} \quad (17.26)$$

This is the fundamental bridge between the discrete lattice parameter τ and the physical diffusivity D . **Practical meaning:** To solve a problem with a given physical diffusivity D , you:

1. Choose your spatial and temporal resolution: Δx and Δt .
2. Compute τ from equation (17.26).
3. Use that τ in the collision step (equation 17.14).

Note the stability constraint: $\tau > \Delta t/2$ is required for stability. This places an upper bound on D for a given Δx and Δt . If you need very high diffusivity, you must either reduce Δx or accept a smaller Δt , which increases the total number of timesteps.

△ Key Insight

Stability Constraint: The requirement $\tau > \Delta t/2$ comes from the local truncation error analysis of the BGK collision operator. If $\tau \leq \Delta t/2$, the relaxation becomes too aggressive and unphysical oscillations grow. This is similar to the CFL condition in hyperbolic PDEs — it is a fundamental constraint on the simulation. For a fixed spatial grid, high diffusivity requires a small Δt , which can make the simulation expensive.

17.5 Moment Space: The Physical Decomposition of the Collision Operator

The Problem: What Does the Collision Operator Actually Conserve?

Look again at the BGK collision equation we derived in Section 17.3:

$$f_i^*(\mathbf{x}, t) = f_i(\mathbf{x}, t) - \frac{\Delta t}{\tau} [f_i(\mathbf{x}, t) - f_i^{\text{eq}}(\mathbf{x}, t)] \quad (17.29)$$

This equation relaxes each distribution f_i toward its equilibrium value f_i^{eq} , all at the same rate $1/\tau$. It works — we showed in Section 17.4 that the equilibrium was carefully designed to encode the right physics. But ask yourself: what *guarantees* that this relaxation conserves mass? What guarantees it produces the correct diffusion coefficient? What guarantees the stress tensor has the right

symmetry? The answers are hiding inside $\mathbf{f}^{\text{eq}}$, but equation (17.29) doesn't show them. Each $\mathbf{f}_i$ is a single number that mixes together information about density, flux, stress, and energy — all tangled up. When BGK relaxes $\mathbf{f}_1$ toward $\mathbf{f}_1^{\text{eq}}$, it is simultaneously relaxing *every* physical quantity that $\mathbf{f}_1$ carries, at the same rate. You can't see which conservation laws are being enforced, and you have no way to control them independently. Is there a way to **see** what physical laws the collision operator is actually enforcing — and to **verify** them directly?

The Macroscopic PDE Dictates Which Moments Matter

The answer starts not with the lattice, but with the PDE you are trying to solve. Consider the diffusion equation:

$$\frac{\partial \phi}{\partial t} = \nabla \cdot (D \nabla \phi) \quad (17.30)$$

This PDE makes three specific physical demands, and each one maps to a weighted sum of the distributions: **Demand 1** —

Conservation of mass (or charge, or heat): The total amount of ϕ at a node does not change during collision. Only streaming moves ϕ between nodes. This means:

$$\sum_{i=0}^{Q-1} f_i^* = \sum_{i=0}^{Q-1} f_i \quad (17.31)$$

The sum $\sum_i f_i$ is a weighted sum of the distributions with weight 1 for every direction. This is the **zeroth moment** — the simplest possible summary of the distributions. You take it because *mass conservation demands it*. **Demand 2 — Fick's law (diffusive flux):**

The macroscopic flux must satisfy $\mathbf{J} = -D \nabla \phi$. In the lattice-Boltzmann framework, the flux emerges from:

$$\mathbf{J}_\alpha = \sum_{i=0}^{Q-1} c_{i\alpha} f_i \quad (17.32)$$

These sums weight each distribution by its velocity component $c_{i\alpha}$. They are the **first moments**. You take them because *Fick's law demands them* — the diffusion coefficient is encoded in how fast these flux quantities relax toward equilibrium. **Demand 3 —**

Correct tensor structure: For the diffusion to be isotropic (or anisotropic in exactly the right way), the stress-like quantity:

$$\Pi_{\alpha\beta} = \sum_{i=0}^{Q-1} c_{i\alpha} c_{i\beta} f_i \quad (17.33)$$

must have the correct symmetry. These are the **second moments**. You take them because *the diffusion tensor structure demands them*. For isotropic diffusion, $\Pi_{\alpha\beta}$ must be proportional to $\delta_{\alpha\beta}$ at equilibrium. For anisotropic diffusion, its structure must match the physical diffusion tensor.

Intuition

The Rule: Each conservation law or transport property in your target PDE maps to a specific weighted sum of the $\mathbf{f}_i$. These weighted sums *are* the moments. You don't choose them for mathematical convenience — the physics tells you which ones matter. If you are solving a diffusion equation, you need the zeroth moment (conservation), the first moments (flux), and the second moments (tensor structure). If you were solving Navier-Stokes, the same zeroth and first moments would enforce mass and momentum conservation, but the second moments would need to capture the viscous stress tensor, and you might need third- and fourth-order moments for the energy equation.

What Are Moments?

The pattern is now clear. A **moment** is a weighted sum of the distribution functions, where the weights are chosen to extract a specific macroscopic quantity. In the most general form:

$$m_k = \sum_{i=0}^{Q-1} M_{ki} f_i \quad (17.34)$$

where M_{ki} is the weight that the k -th moment assigns to the i -th distribution. This is not unique to LBM — it is the same concept that appears across all of engineering:

- **Zeroth moment** ($M_{0i} = 1$ for all i): total density, total charge, total energy. The conserved quantity in any transport equation.
- **First moments** ($M_{ki} = c_{i\alpha}$): momentum flux in fluids, diffusive flux in electrophysiology, heat flux in conduction. The direction and rate at which "stuff" moves.
- **Second moments** ($M_{ki} = c_{i\alpha} c_{i\beta}$): the viscous stress tensor in fluid mechanics, the diffusion tensor in EP, the energy density in heat transfer. These control how transport varies with direction.
- **Higher moments** (M_{ki} involves cubic or higher products of $c_{i\alpha}$): in most diffusion and low-Mach flow problems, no conservation law requires them. They exist because the lattice has Q degrees of freedom but only a few physical constraints. These are the **ghost moments** — mathematically necessary to fill out the Q -dimensional space, but carrying no macroscopic physical content.

Building M: Each Row Is a Conservation Law

We can collect all the moment definitions into a single matrix equation:

$$\mathbf{m} = \mathbf{M} \mathbf{f} \quad (17.35)$$

where $\mathbf{m} = (m_0, m_1, \dots, m_{Q-1})^T$ is the vector of moments and $\mathbf{f} = (f_0, f_1, \dots, f_{Q-1})^T$ is the vector of distributions. The matrix $\mathbf{M}$ is the **transformation matrix** — each row is the recipe for extracting one physical quantity from the distributions. For **D2Q5**, we have $Q = 5$ distributions and therefore need a 5×5 matrix. The rows are constructed directly from the physics:

Row	Moment	Physical meaning	Why we need it
0	$\sum_i f_i$	Density (conserved scalar ϕ)	Mass/charge conservation
1	$\sum_i c_{ix} f_i$	x -flux	Fick's law in x
2	$\sum_i c_{iy} f_i$	y -flux	Fick's law in y
3	$\sum_i c_i ^2 f_i$	Energy	Ghost — no PDE requires it
4	$\sum_i (c_{ix}^2 - c_{iy}^2) f_i$	Energy difference (stress trace)	Ghost — no PDE requires it

The explicit 5×5 matrix, using the D2Q5 velocity vectors $(c_{ix}, c_{iy}) = (0, 0), (1, 0), (-1, 0), (0, 1), (0, -1)$:

$$\mathbf{M}_{\text{D2Q5}} = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 & -1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & -1 & -1 \end{pmatrix} \quad (17.36)$$

Read it row by row. Row 0: every entry is 1, so this row sums all f_i — that's the density. Row 1: the entries are the x -components of the velocity vectors, so this row computes the x -flux. Row 2: the y -components — the y -flux. Rows 3 and 4: combinations of c_{ix}^2 and c_{iy}^2 that complete the basis. The first three rows are *demanded by the physics*. The last two rows are chosen to make $\mathbf{M}$ invertible (so we can transform back), but any linearly independent choice would work. For **D3Q7**, the 7×7 transformation matrix from the Rapaka et al. LBM-EP paper follows the same logic. With velocity ordering $(0, 0, 0), (1, 0, 0), (-1, 0, 0), (0, 1, 0), (0, -1, 0), (0, 0, 1), (0, 0, -1)$:

$$\mathbf{M}_{D3Q7} = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & -1 \\ -6 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & -1 & -1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & -2 & -2 \end{pmatrix} \quad (17.37)$$

Rows 0–3: density and the three flux components (x, y, z) — required by mass conservation and Fick’s law in 3D. Rows 4–6: ghost moments with no macroscopic counterpart, chosen to complete the 7×7 basis.

Mathematical Note

Why the ghost rows look arbitrary: The first rows of $\mathbf{M}$ are dictated by physics — row 0 must be all ones (density), rows 1– D must use the velocity components (flux). The remaining $Q - 1 - D$ rows just need to be linearly independent from the physical rows. Different textbooks choose different ghost rows, and *all valid choices produce the same physical solution*. The ghost rows affect only numerical stability, not the macroscopic PDE that the scheme recovers.

The Round Trip: Rewriting the Collision Operator in Moment Space

Now we have the tool to see inside the collision operator. The key idea is simple: instead of relaxing the $\mathbf{f}_i$ directly (where every physical quantity is tangled together), we **decode** the distributions into moments, **relax** each moment separately, then **re-encode** back to distributions for streaming. **Step 1 — Decode:** Transform the distributions into moments:

$$\mathbf{m} = \mathbf{M} \mathbf{f} \quad (17.38)$$

Now $\mathbf{m}_0$ is the density, $\mathbf{m}_1$ is the x -flux, $\mathbf{m}_2$ is the y -flux, and so on. Each component has exactly one physical meaning. **Step 2 — Decode the equilibrium too:**

$$\mathbf{m}^{\text{eq}} = \mathbf{M} \mathbf{f}^{\text{eq}} \quad (17.39)$$

The equilibrium moments tell you the target state for each physical quantity. For the diffusion equilibrium $\mathbf{f}_i^{\text{eq}} = \mathbf{w}_i \phi$, the equilibrium density is $\mathbf{m}_0^{\text{eq}} = \phi$ (all the field), the equilibrium fluxes are $\mathbf{m}_1^{\text{eq}} = \mathbf{m}_2^{\text{eq}} = \mathbf{0}$ (no net flow at equilibrium), and the ghost moments take whatever values $\mathbf{w}_i$ and the lattice geometry dictate. **Step 3 — Relax in moment space:**

$$\mathbf{m}^* = \mathbf{m} - \mathbf{S}(\mathbf{m} - \mathbf{m}^{\text{eq}}) \quad (17.40)$$

where $\mathbf{S} = \text{diag}(s_0, s_1, \dots, s_{Q-1})$ is the **relaxation matrix**. Each diagonal entry s_k controls how fast moment k relaxes toward its equilibrium value. This is the step where the physics is enforced:

- For the conserved moment (density): $s_0 = \mathbf{0}$. The density is never changed by collision — it is a conserved quantity. Setting $s_0 = \mathbf{0}$ means $\mathbf{m}_0^* = \mathbf{m}_0$ exactly, regardless of what happens to the other moments.
- For the flux moments: s_1 and s_2 control how fast the flux relaxes, which determines the diffusion coefficient. In the simplest case (isotropic BGK), $s_1 = s_2 = \Delta t / \tau$.
- For the ghost moments: $s_3, s_4, \dots$ can be chosen freely — they do not affect the macroscopic PDE, only the numerical behavior of the scheme.

Step 4 — Re-encode: Transform back to distribution space:

$$\mathbf{f}^* = \mathbf{M}^{-1} \mathbf{m}^*$$

(17.41)

Now $\mathbf{f}^*$ is ready for streaming. The distributions have been updated in a way that *explicitly* conserves density, *explicitly* relaxes flux at a controlled rate, and handles the ghost moments separately.

Intuition

The destination vs. the journey: It is worth pausing to see how $\mathbf{f}^{\text{eq}}$ and moment space play complementary roles. The equilibrium $\mathbf{f}_i^{\text{eq}}$ defines the **destination** — the state the distributions are relaxing toward. It encodes *what* the physics looks like at equilibrium (isotropic for diffusion, velocity-biased for fluid flow). Moment space governs the **journey** — *how* the distributions get to that destination and *what is preserved along the way*. By decomposing the non-equilibrium deviation $\mathbf{f} - \mathbf{f}^{\text{eq}}$ into physically meaningful components, you can verify that conservation laws are respected, that transport coefficients are correct, and that numerical artifacts are controlled.

BGK Is a Special Case: All Dials Locked Together

Here is the crucial connection. Set $\mathbf{S} = \frac{\Delta t}{\tau} \mathbf{I}$ (the identity matrix times a single rate). Then the moment-space collision (17.40) becomes:

$$\mathbf{m}^* = \mathbf{m} - \frac{\Delta t}{\tau} (\mathbf{m} - \mathbf{m}^{\text{eq}})$$

and the full round trip is:

$$\mathbf{f}^* = \mathbf{M}^{-1} \left[\mathbf{M} \mathbf{f} - \frac{\Delta t}{\tau} \mathbf{M} (\mathbf{f} - \mathbf{f}^{\text{eq}}) \right] = \mathbf{f} - \frac{\Delta t}{\tau} (\mathbf{f} - \mathbf{f}^{\text{eq}})$$

The $\mathbf{M}^{-1}$ and $\mathbf{M}$ cancel. We recover exactly the BGK equation (17.29). **BGK was always operating in moment space** — it just didn't need to make the round trip explicit because every moment was being relaxed at the same rate. The single τ locks all the dials together: conservation, transport, and ghost moments all receive the same treatment. This is why BGK works for isotropic diffusion: when $\mathbf{s}_1 = \mathbf{s}_2$, the $\mathbf{x}$ - and $\mathbf{y}$ -flux moments relax at the same rate, producing equal diffusion in all directions. And this is why BGK *cannot* produce anisotropic diffusion: there is no way to set $\mathbf{s}_1 \neq \mathbf{s}_2$ when $\mathbf{S}$ is constrained to be a scalar multiple of the identity.

Key Insight

Conservation in BGK — a hidden accident: In BGK, the conserved moment (density) formally receives the same relaxation rate $\mathbf{s}_0 = \Delta t / \tau$ as every other moment. But conservation is not violated — because $\mathbf{f}^{\text{eq}}$ is designed so that $\mathbf{m}_0^{\text{eq}} = \mathbf{m}_0$ (the equilibrium density equals the actual density), the relaxation step computes $\mathbf{m}_0^* = \mathbf{m}_0 - \frac{\Delta t}{\tau} (\mathbf{m}_0 - \mathbf{m}_0) = \mathbf{m}_0$. Conservation holds, but only because the equilibrium was carefully constructed to make it so. In the moment-space view, you enforce conservation *explicitly* by setting $\mathbf{s}_0 = \mathbf{0}$ rather than relying on this cancellation.

What Each Moment Sees During Collision

To make this concrete, consider a D2Q5 node mid-simulation. The distributions $\mathbf{f}_i$ are some non-equilibrium state (a diffusion front is passing through). When we decode:

- $\mathbf{m}_0 = \sum_i \mathbf{f}_i = \phi$ — the local voltage. After collision with $\mathbf{s}_0 = \mathbf{0}$: $\mathbf{m}_0^* = \phi$. Unchanged. Mass conservation enforced.
- $\mathbf{m}_1 = \sum_i c_{1x} \mathbf{f}_i = J_x$ — the $\mathbf{x}$ -flux. Non-zero because a diffusion front creates a gradient. After collision:
 $\mathbf{m}_1^* = J_x - \mathbf{s}_1 (J_x - 0) = (1 - \mathbf{s}_1) J_x$. The flux decays toward zero (equilibrium has no net flux). The rate $\mathbf{s}_1$ determines how fast — which sets the diffusion coefficient $\mathbf{D}$.
- $\mathbf{m}_2 = \sum_i c_{1y} \mathbf{f}_i = J_y$ — same story for $\mathbf{y}$.
- $\mathbf{m}_3, \mathbf{m}_4$ — the ghost moments. They relax too, but toward values dictated by the lattice weights, not by any conservation law. Their rates $\mathbf{s}_3, \mathbf{s}_4$ are free parameters.

This decomposition reveals the internal structure that equation (17.29) hides. You can now **verify** conservation (is $\mathbf{m}_0$ really unchanged?), **measure** the flux relaxation rate (does it match your desired $\mathbf{D}$?), and **tune** the ghost moments independently of the

physics.

Worked Example: One D2Q5 Collision in Both Spaces

Let us make the round trip concrete with numbers. Consider a D2Q5 node with $\phi = 1.0$, $\Delta x = 0.025$ cm, $\Delta t = 0.01$ ms, and isotropic diffusivity $D = 0.001$ cm²/ms. From equation (17.26):

$$\tau = \frac{\Delta t}{2} + \frac{3D\Delta t}{\Delta x^2} = 0.005 + \frac{3(0.001)(0.01)}{0.000625} = 0.005 + 0.048 = 0.053 \text{ ms}$$

so $\omega = \Delta t/\tau = 0.01/0.053 \approx 0.189$. Suppose the current distributions are (a diffusion front has created a slight asymmetry):

$$\mathbf{f} = (0.340, 0.175, 0.155, 0.168, 0.162)^T$$

Check: $\sum f_i = 1.000 = \phi$. Good. The equilibrium is $f_i^{\text{eq}} = w_i \phi = (1/3, 1/6, 1/6, 1/6, 1/6) \approx (0.333, 0.167, 0.167, 0.167, 0.167)$.

BGK (distribution space):

$$f_i^* = f_i - 0.189(f_i - f_i^{\text{eq}})$$

Computing each component:

i	f_i	f_i^{eq}	$f_i - f_i^{\text{eq}}$	f_i^*
0	0.340	0.333	+0.007	0.3387
1	0.175	0.167	+0.008	0.1735
2	0.155	0.167	−0.012	0.1573
3	0.168	0.167	+0.001	0.1678
4	0.162	0.167	−0.005	0.1629

Check: $\sum f_i^* = 1.0002 \approx 1.000$. Conservation holds (the small deviation is rounding). **Moment space (same collision, decoded):**

Apply $\mathbf{m} = \mathbf{M}\mathbf{f}$:

k	Moment	m_k	m_k^{eq}	$m_k^* (s_k = \omega)$	Physical role
0	ρ	1.000	1.000	1.000	Conserved — unchanged
1	J_x	+0.020	0	+0.0162	Flux decays $\rightarrow$ sets D_x
2	J_y	+0.006	0	+0.0049	Flux decays $\rightarrow$ sets D_y
3	e	0.660	0.667	0.6613	Ghost — no PDE role
4	ϵ	−0.000	0	−0.000	Ghost — no PDE role

Now you can *see* the physics. The density ($m_0 = 1.000$) is untouched — conservation is enforced. The x -flux ($m_1 = 0.020$) relaxes toward zero at rate ω — this sets the diffusion coefficient. The ghost moments relax too, but they don't affect the macroscopic solution. In BGK, all moments happen to use the same ω . In the moment-space view, you can verify each constraint independently. Applying $\mathbf{f}^* = \mathbf{M}^{-1}\mathbf{m}^*$ gives back the same $\mathbf{f}^*$ as the BGK calculation above — the round trip is exact.

Intuition

What we have gained: The moment-space formulation does not change the physics — for BGK with a single τ , the result is identical. What it changes is **visibility**. You can now see which quantity is conserved, which quantities control transport, and which quantities are free parameters with no physical role. This visibility is the foundation for everything that follows: anisotropic diffusion (relaxing x - and y -flux at different rates), stability tuning (adjusting ghost-moment rates without affecting the solution), and the full MRT collision operator.

What Comes Next

We now have the complete theoretical framework for the lattice-Boltzmann method: the thermodynamic foundation (Section 17.1), the Boltzmann equation and BGK simplification (Section 17.2), the discrete lattice-Boltzmann equation (Section 17.3), the equilibrium distribution and its constraints on weights and velocities (Section 17.4), and the moment-space decomposition that

reveals the physical content of the collision operator (Section 17.5). In the next chapter, we will put this machinery to work — the f_i are our fundamental variables, and the BGK collision operator relaxes all of them at the same rate $1/\tau$. But there is a more revealing way to look at the same information. In Section 17.5 above, we transformed the distributions into **moment space**, — first by unlocking the moment-space relaxation rates to achieve anisotropic diffusion (Multiple Relaxation Time), then by applying LBM to the cardiac monodomain and bidomain equations.

Intuition

Complete Reference Tables: Appendix A collects the velocity vectors $\mathbf{c}_i$ and weights w_i for all standard lattices (D1Q3, D2Q5, D2Q9, D3Q7, D3Q19, D3Q27) in a single reference table. These are derived from Gauss-Hermite quadrature (17.18) applied to the lattice geometries described in this chapter.

Lattice-Boltzmann Methods for Monodomain

Chapter 17 built the LBM machinery from first principles: distribution functions, lattice geometry, collision and streaming, the equilibrium distribution born from Gauss-Hermite quadrature, and the moment-space decomposition that reveals the physical content of the collision operator. Now we put that machinery to work on the cardiac monodomain equation. The key additions are surprisingly mechanical: an ionic source term injected into the collision step, operator splitting to separate reaction from diffusion, and bounce-back boundary conditions to handle tissue edges. For anisotropic tissue, we unlock the moment-space relaxation rates derived in Section 17.5 — but now we must connect them quantitatively to the cardiac conductivity tensor. This chapter derives that connection, presents the complete algorithm, and discusses the implementation choices that make LBM fast on GPUs.

18.1 The Monodomain Equation as a Target PDE

Our target is equation (7.1), the monodomain PDE that governs wave propagation in cardiac tissue:

$$\chi C_m \frac{\partial V_m}{\partial t} = \nabla \cdot (\mathbf{D} \nabla V_m) - \chi I_{\text{ion}}(V_m, \mathbf{w}) + I_{\text{stim}}$$

The left-hand side is the rate of change of the transmembrane potential V_m , scaled by the surface-to-volume ratio χ and membrane capacitance C_m . On the right, three terms compete: diffusion ($\nabla \cdot (\mathbf{D} \nabla V_m)$) spreads the voltage through tissue, ionic currents (I_{ion}) from the cell membrane shape the action potential, and stimulus current (I_{stim}) initiates activation. Chapters 5 and 6 detailed the ionic models (TTP06 and ORd) that compute I_{ion} from V_m and the gating variables $\mathbf{w}$. Chapters 9 and 10 solved the diffusion side with explicit and implicit finite-difference methods. Now we solve it with LBM.

The diffusion tensor $\mathbf{D}$ encodes the anisotropy of cardiac tissue. In a 2D slab with fiber direction $\mathbf{a}_f = (\cos \theta, \sin \theta)$, the tensor takes the form:

$$\mathbf{D} = D_f \mathbf{a}_f \mathbf{a}_f^T + D_t (\mathbf{I} - \mathbf{a}_f \mathbf{a}_f^T) \quad (18.1)$$

where D_f is the along-fiber diffusivity and D_t is the cross-fiber diffusivity. Typical values are $D_f \approx 0.001 \text{ cm}^2/\text{ms}$ and $D_t \approx 0.00025 \text{ cm}^2/\text{ms}$ (a 4:1 ratio). In component form:

$$D_{xx} = D_f \cos^2 \theta + D_t \sin^2 \theta, \quad D_{yy} = D_f \sin^2 \theta + D_t \cos^2 \theta, \quad D_{xy} = (D_f - D_t) \sin \theta \cos \theta$$

For isotropic tissue ($D_f = D_t = D$), the tensor reduces to $D\mathbf{I}$ and the off-diagonal term vanishes. For aligned fibers ($\theta = 0$), we get $D_{xx} = D_f$, $D_{yy} = D_t$, $D_{xy} = 0$. The general case ($\theta \neq 0$, $D_f \neq D_t$) produces a full tensor with off-diagonal coupling.

Intuition

The two-part structure. The monodomain PDE has a clear two-part structure: *diffusion* (the divergence term, which spreads voltage through tissue) and *reaction* (the ionic and stimulus terms, which generate and shape the action potential locally). These two parts operate on vastly different scales — diffusion is a spatial coupling between neighboring nodes, while reaction is purely local (each node's ionic currents depend only on that node's voltage and gating variables). This separation is exactly what operator splitting exploits, and it maps perfectly onto the LBM framework: collision handles diffusion, and the ionic ODE handles reaction.

18.2 Operator Splitting: Reaction Then Diffusion

Following the Godunov splitting framework of Chapter 8, we solve the monodomain equation in two sequential substeps per timestep. The reaction substep is purely local — it updates V_m and $\mathbf{w}$ at each node independently. The diffusion substep couples neighbors through LBM collision and streaming.

Substep 1 — Reaction (ionic ODE): At each node, hold the diffusion term frozen and solve:

$$\chi C_m \frac{dV_m}{dt} = -\chi I_{\text{ion}}(V_m, \mathbf{w}) + I_{\text{stim}}, \quad \frac{d\mathbf{w}}{dt} = \mathbf{g}(V_m, \mathbf{w})$$

This is an ODE system at each node — identical to what we solve in Chapters 5 and 6. The Rush-Larsen method (equation 8.1) handles the gating variables $\mathbf{w}$, exploiting their exponential relaxation structure for accuracy and stability. After this substep, we have updated gating variables $\mathbf{w}^{n+1}$ and an intermediate voltage V_m^* that includes the ionic contribution but not yet the diffusion.

Substep 2 — Diffusion (LBM): Now solve the diffusion part $\partial V_m / \partial t = \nabla \cdot (\mathbf{D} \nabla V_m) / (\chi C_m)$ using the LBM collision-streaming cycle. The collision operator splits cleanly into two pieces that mirror the two-part physics of the monodomain equation:

- Ω^{NR} (**non-reactive collision**) — drives distributions toward equilibrium, producing diffusive flux. This is the LBM mechanism for the divergence term $\nabla \cdot (\mathbf{D} \nabla V_m)$.
- Ω^R (**reactive collision**) — injects the ionic source into the distribution. This is how the reaction substep's output couples back into the lattice populations.

The reactive collision needs a **source term** S that encapsulates the net effect of ionic currents and external stimulus:

$$S = -\frac{I_{\text{ion}}(V_m^n, \mathbf{w}^n) + I_{\text{stim}}}{\chi C_m} \quad (18.2)$$

This source has units of voltage per unit time (mV/ms). The sign convention follows equation (7.1): inward current (depolarizing) makes $S > 0$, outward current (repolarizing) makes $S < 0$. The precise definitions of Ω^{NR} and Ω^R — and the physical reasons they take the forms they do — are derived in Section 18.3 (BGK) and Section 18.4 (MRT).

△ Key Insight

Splitting order matters. Godunov splitting (reaction first, then diffusion) is first-order accurate in time. Strang splitting (half-reaction, full diffusion, half-reaction) is second-order but requires two ionic evaluations per timestep. For cardiac LBM, first-order Godunov splitting is standard because the timestep Δt is already small (typically 0.005–0.02 ms) due to the lattice resolution requirements, and the splitting error is dominated by the spatial discretization error. See Chapter 8 for the full analysis.

18.3 BGK Collision with Source Term

The diffusion substep is where LBM does its work. Recall from Chapter 17 that each lattice node carries Q distribution functions f_i (five for D2Q5, seven for D3Q7). The macroscopic voltage is their sum: $V_m = \sum_i f_i$. The equilibrium distribution for a pure diffusion problem, derived from Gauss-Hermite quadrature in Section 17.4, is:

$$f_i^{\text{eq}} = w_i V_m$$

This is equation (17.27) with the scalar field ϕ replaced by the transmembrane potential V_m , and no advection ($\mathbf{u} = \mathbf{0}$). The weights w_i are the Gauss-Hermite quadrature weights from Section 17.4: for D2Q5, $w_0 = 1/3$ and $w_{1,2,3,4} = 1/6$.

The BGK collision with ionic source combines two physically distinct operations. We define each as a **rate** (per unit time), consistent with the continuous collision operator $\Omega(f)$ from equation (17.11). The **non-reactive collision rate** Ω_i^{NR} is the BGK relaxation toward equilibrium:

$$\Omega_i^{NR} = -\frac{1}{\tau} (f_i - w_i V_m)$$

This is a direct transcription of equation (17.11) with $f^{\text{eq}} = w_i V_m$. Physically, Ω_i^{NR} drives each distribution toward the local equilibrium at rate $1/\tau$. When $f_i > w_i V_m$ (more particles than equilibrium in direction i), the rate is negative — particles are removed. When $f_i < w_i V_m$, particles are added. This redistribution of particles among directions is diffusion: non-equilibrium structure in the distribution function corresponds to flux, and relaxing that structure spreads voltage through tissue. The τ - D relationship (equation 17.26) quantifies exactly how the relaxation rate maps to the macroscopic diffusion coefficient.

The **reactive collision rate** Ω_i^R injects the ionic source into the distribution:

$$\Omega_i^R = w_i S$$

This is a pure source injection: the reaction term $\mathbf{S}$ (equation 18.2) — which encapsulates all ionic currents and external stimulus — is distributed among lattice directions with weights $\mathbf{w}_i$. Since $\sum_i \mathbf{w}_i = \mathbf{1}$, the total injected rate is $\sum_i \Omega_i^R = \mathbf{S}$, preserving the correct total source. The weights ensure isotropy: the source does not preferentially inject particles in any direction, because ionic currents have no spatial directionality — they flow across the cell membrane, not along any lattice vector.

The full collision update discretizes these rates with timestep Δt :

$$f_i^* = f_i + \Delta t(\Omega_i^{NR} + \Omega_i^R) = f_i - \frac{\Delta t}{\tau}(f_i - w_i V_m) + \Delta t w_i \mathbf{S} \quad (18.3)$$

The separation is physically exact, not merely algebraic convenience. Ω^{NR} handles **transport** — the spatial coupling between neighboring nodes that produces the $\nabla \cdot (\mathbf{D} \nabla V_m)$ term in the monodomain PDE. Ω^R handles **reaction** — the purely local ionic and stimulus currents that produce the $-\chi I_{\text{ion}} + I_{\text{stim}}$ terms. This mirrors the operator-splitting structure of Section 18.2, but now at the level of the collision operator itself. The decomposition carries directly into the MRT formulation (Section 18.4), where Ω^{NR} gains direction-dependent relaxation rates to model anisotropic diffusion, while Ω^R remains unchanged — because the ionic physics is isotropic.

The relaxation time τ is set by the $\tau\text{-}\mathbf{D}$ relationship (equation 17.26):

$$\tau = \frac{\Delta t}{2} + \frac{3\mathbf{D}\Delta t}{\Delta x^2}$$

For isotropic tissue with $\mathbf{D} = 0.001 \text{ cm}^2/\text{ms}$, $\Delta x = 0.025 \text{ cm}$, $\Delta t = 0.01 \text{ ms}$: $\tau = 0.005 + 0.048 = 0.053 \text{ ms}$. The collision rate is $\omega = \Delta t/\tau \approx 0.189$. Stability requires $\tau > \Delta t/2$, which holds for any positive diffusivity.

After collision, streaming (equation 17.15) shifts each post-collision distribution one lattice spacing in its designated direction:

$$f_i(\mathbf{x} + \mathbf{e}_i \Delta x, t + \Delta t) = f_i^*(\mathbf{x}, t)$$

No new mathematical machinery is required beyond what Chapter 17 established. The cardiac monodomain is solved by the same collide-and-stream cycle that solves any diffusion equation — the ionic model simply provides the source term.

Intuition

Physical interpretation. Think of the source $\mathbf{S}$ as a faucet (or drain) at each lattice node. When a stimulus depolarizes a cell ($\mathbf{S} > 0$), it adds voltage to the local particle population — distributed equally among all directions via the weights $\mathbf{w}_i$. The collision then relaxes the distribution toward equilibrium (spreading the perturbation), and streaming carries the information to neighboring nodes. This is how a depolarization wave propagates through the lattice: source injection $\rightarrow$ collision $\rightarrow$ streaming $\rightarrow$ recovery, repeated every timestep.

18.4 MRT for Anisotropic Tissue

Why BGK Fails for Anisotropy

Cardiac tissue conducts three to four times faster along fibers than across them. This anisotropy is encoded in the diffusion tensor $\mathbf{D}$ (equation 18.1), which has different eigenvalues along and perpendicular to the fiber direction. Can BGK handle this?

No. Recall from Section 17.5 that BGK uses a single relaxation rate $\omega = \Delta t/\tau$ for *all* moments. When we decoded the collision into moment space, we saw that the $\mathbf{x}$ -flux and $\mathbf{y}$ -flux moments both relax at the same rate $\mathbf{s}_1 = \mathbf{s}_2 = \omega$. Since the diffusion coefficient in each direction is determined by the corresponding flux relaxation rate, BGK forces $\mathbf{D}_x = \mathbf{D}_y$. It can only produce isotropic diffusion. For anisotropic cardiac tissue, we need to unlock those rates — relax the $\mathbf{x}$ -flux and $\mathbf{y}$ -flux at different speeds. This is exactly what MRT (Multiple Relaxation Time) does.

From the Conductivity Tensor to Relaxation Rates

Section 17.5 showed that MRT relaxes each moment independently: the conserved moment (density) at rate $\mathbf{s}_0 = 0$, the flux moments at rates $\mathbf{s}_1, \mathbf{s}_2$, and the ghost moments at freely chosen rates. The question is: what values of $\mathbf{s}_1$ and $\mathbf{s}_2$ reproduce a given anisotropic

diffusion tensor?

Start from the scalar τ - D relationship (equation 17.24), which Chapman-Enskog analysis derives for the BGK operator:

$$D = c_s^2 \left(\tau - \frac{\Delta t}{2} \right)$$

This relates a single diffusivity D to a single relaxation time τ . For anisotropic diffusion, we need a **tensor** generalization. The Chapman-Enskog expansion applied to the MRT collision operator (equation 17.40) shows that each component of the diffusion tensor maps to a corresponding relaxation parameter:

$$D_{\alpha\beta} = c_s^2 \left(\tau_{\alpha\beta} - \frac{\Delta t}{2} \delta_{\alpha\beta} \right) \quad (18.4)$$

where $\alpha, \beta \in \{x, y\}$ (or $\{x, y, z\}$ in 3D). This is the tensor τ - D relationship — the natural extension of equation (17.24) from a scalar to a matrix. Rearranging for the relaxation parameters:

$$\tau_{\alpha\beta} = \frac{\Delta t}{2} \delta_{\alpha\beta} + \frac{D_{\alpha\beta}}{c_s^2} \quad (18.5)$$

Substituting $c_s^2 = \Delta x^2 / (3\Delta t^2)$:

$$\tau_{\alpha\beta} = \frac{\Delta t}{2} \delta_{\alpha\beta} + \frac{3 D_{\alpha\beta} \Delta t}{\Delta x^2} \quad (18.6)$$

This is the tensor generalization of equation (17.26). Each entry of the τ matrix is determined by the corresponding entry of the diffusion tensor. The Kronecker delta $\delta_{\alpha\beta}$ ensures that the $\Delta t/2$ offset only appears on the diagonal.

For D2Q5 with axis-aligned fibers ($\theta = 0$), the diffusion tensor is diagonal ($D_{xx} = D_l$, $D_{yy} = D_t$, $D_{xy} = 0$), and the relaxation parameters simplify to:

$$\tau_{xx} = \frac{\Delta t}{2} + \frac{3D_l\Delta t}{\Delta x^2}, \quad \tau_{yy} = \frac{\Delta t}{2} + \frac{3D_t\Delta t}{\Delta x^2}, \quad \tau_{xy} = 0$$

These become the flux relaxation rates: $s_1 = \Delta t / \tau_{xx}$ controls the x -flux decay (and hence D_{xx}), while $s_2 = \Delta t / \tau_{yy}$ controls the y -flux decay (and hence D_{yy}). Since $D_l > D_t$, we have $\tau_{xx} > \tau_{yy}$, so the x -flux relaxes more slowly — it takes longer to reach equilibrium, which means more information persists in the x -direction, producing faster diffusion along the fiber.

Intuition

Why slower relaxation means faster diffusion. This seems backwards at first. A larger τ means *less* relaxation per timestep — the distribution retains more of its non-equilibrium structure. But non-equilibrium structure *is* the flux. A large departure from equilibrium means a large gradient, which drives faster diffusion. If τ were very small (rapid relaxation), the distribution would snap to equilibrium immediately — zero flux, zero diffusion. This is the lattice-Boltzmann version of "a stiff spring returns to rest quickly but doesn't transmit waves far."

The Off-Diagonal Problem: Rotated Fibers

When fibers are not aligned with the lattice axes ($\theta \neq 0$), the diffusion tensor has off-diagonal entries ($D_{xy} \neq 0$). Equation (18.6) then gives $\tau_{xy} \neq 0$, which means the relaxation matrix $\mathbf{S}$ in equation (17.40) is no longer diagonal in the Cartesian moment basis. There are two standard approaches:

Approach 1 — Rotate the moment basis. Construct $\mathbf{M}$ using the fiber-aligned coordinate system instead of the lattice axes. The flux moments then align with the fiber and cross-fiber directions, where $\mathbf{D}$ is diagonal, and $\mathbf{S}$ stays diagonal. This requires a different $\mathbf{M}$ at each node (since fiber angles vary spatially), which adds computational cost.

Approach 2 — Modify the equilibrium distribution. Keep the standard $\mathbf{M}$ (equation 17.36) but replace the isotropic equilibrium $f_i^{\text{eq}} = w_i V_m$ with an anisotropic equilibrium that encodes the full diffusion tensor:

$$f_i^{\text{eq}} = w_i V_m \left(1 + \frac{e_{i\alpha} e_{i\beta} D_{\alpha\beta}}{2 c_s^2 D_0} - \frac{1}{2} \frac{\text{tr}(\mathbf{D})}{D_0 \cdot d} \right) \quad (18.7)$$

where D_0 is a reference diffusivity and summation over α, β is implied. This approach absorbs the anisotropy into the equilibrium target rather than the relaxation rates, allowing a single scalar τ to be used. The implementation is simpler (no per-node $\mathbf{M}$ matrices) but the equilibrium is more complex.

For the remainder of this chapter, we use **Approach 1** (the MRT formulation with diagonal $\mathbf{S}$ in the principal-axis frame), which makes the connection to Chapter 17's moment-space machinery most transparent.

The Full MRT Collision with Source

Assembling the pieces from Section 17.5 (equations 17.38–17.41), the MRT collision with ionic source uses the same $\Omega^{NR} + \Omega^R$ decomposition from Section 18.3, but with the non-reactive part now operating in moment space. Defining the rate-based collision operators:

$$\Omega^{NR} = -\frac{1}{\Delta t} \mathbf{M}^{-1} \mathbf{S} (\mathbf{M} \mathbf{f} - \mathbf{M} \mathbf{f}^{\text{eq}}), \quad \Omega^R = \mathbf{w} S$$

where $\mathbf{S} = \Delta t \cdot \mathbf{\Lambda}$ is the dimensionless relaxation matrix (with entries $s_k = \Delta t / \tau_k$), the factor $1/\Delta t$ converts the finite increment back to a rate. The full MRT update is then:

$$\mathbf{f}^* = \mathbf{f} + \Delta t \left(\underbrace{\Omega^{NR}}_{\text{diffusion}} + \underbrace{\Omega^R}_{\text{reaction}} \right) = \mathbf{f} - \mathbf{M}^{-1} \mathbf{S} (\mathbf{M} \mathbf{f} - \mathbf{M} \mathbf{f}^{\text{eq}}) + \Delta t \mathbf{w} S \quad (18.8)$$

The expanded right-hand side is identical to the standard MRT form — the $1/\Delta t$ in Ω^{NR} cancels with Δt in the update. But the conceptual decomposition is preserved: Ω^{NR} is the rate at which moment-space relaxation produces diffusive flux (anisotropic, direction-dependent), while Ω^R is the rate at which ionic source is injected (isotropic, identical to the BGK case).

Here $\mathbf{M}$ is the transformation matrix (equation 17.36 for D2Q5, equation 17.37 for D3Q7), $\mathbf{S}$ is the diagonal relaxation matrix, $\mathbf{w} = (w_0, w_1, \dots, w_{Q-1})^T$ is the weight vector, and S is the ionic source (equation 18.2). The relaxation matrix for D2Q5 is:

$$\mathbf{S}_{\text{D2Q5}} = \text{diag} \left(0, \frac{\Delta t}{\tau_{xx}}, \frac{\Delta t}{\tau_{yy}}, s_3, s_4 \right) \quad (18.9)$$

The first entry is zero (density is conserved — never relaxed). Entries 1 and 2 are the flux relaxation rates that control D_{xx} and D_{yy} through equation (18.6). Entries 3 and 4 are the **ghost-moment rates** — the free parameters discussed in Section 17.5.

Ghost-Moment Rates and the Gram-Schmidt Construction

Section 17.5 stated that the ghost rows of $\mathbf{M}$ "just need to be linearly independent" from the physical rows to make the matrix invertible. But *how* do you construct them in practice?

The standard procedure is **Gram-Schmidt orthogonalization**. Start with the physical rows — the density row $(1, 1, 1, 1, 1)$ and the flux rows $(0, 1, -1, 0, 0)$, $(0, 0, 0, 1, -1)$ — which are dictated by conservation laws and Fick's law. Then construct the remaining rows by taking higher-order velocity products ($|\mathbf{e}_i|^2$, $e_{ix}^2 - e_{iy}^2$, etc.) and orthogonalizing them against the physical rows using the Gram-Schmidt process. This yields a well-conditioned $\mathbf{M}$ whose inverse is numerically stable.

For D2Q5, the two ghost rows in equation (17.36) correspond to the "energy" moment $\sum_i |\mathbf{e}_i|^2 f_i$ and the "energy difference" moment $\sum_i (e_{ix}^2 - e_{iy}^2) f_i$. These are second-order velocity products that are linearly independent of the zeroth- and first-order physical rows. The Gram-Schmidt process ensures they are also orthogonal, which makes $\mathbf{M}$ well-conditioned. For D3Q7 (equation 17.37), the same procedure generates four ghost rows from second-order products in three dimensions.

What about the ghost relaxation rates s_3 and s_4 ? Since no macroscopic conservation law constrains them, they are **free parameters**. The Chapman-Enskog analysis shows that they do not appear in the recovered PDE at leading order — they affect only the truncation error and numerical stability. In practice, the standard choice is:

$$s_3 = s_4 = 1.0 \quad (\text{full relaxation to equilibrium each timestep})$$

Setting a ghost rate to 1.0 means the corresponding moment is driven entirely to its equilibrium value in a single collision step — maximum damping of non-physical modes. This is the most stable choice. Some authors optimize ghost rates for specific accuracy targets (for example, setting $s_3 = s_4 = 8(2 - s_1)/(8 - s_1)$ to cancel certain third-order error terms), but for cardiac simulations where stability matters more than squeezing out extra order of accuracy, $s_3 = s_4 = 1.0$ is the standard.

Mathematical Note

Why ghost rates affect stability but not the PDE. The Chapman-Enskog expansion recovers the macroscopic PDE from the lattice-Boltzmann equation order by order in the Knudsen number $\epsilon = \Delta x/L$ (where L is a macroscopic length scale). The zeroth-order recovery gives mass conservation. The first-order recovery gives the diffusion equation, with D determined by the flux relaxation rates s_1, s_2 . The ghost rates s_3, s_4 first appear at *second* order, in the truncation error. They modify the coefficient of the $O(\Delta x^2)$ error term but leave the leading-order PDE untouched. This is why different textbooks use different ghost rows and different ghost rates yet recover the same diffusion equation — they differ only in their numerical error properties.

Worked Example: MRT Collision with Anisotropic Tissue

Consider D2Q5 with $D_t = 0.001 \text{ cm}^2/\text{ms}$, $D_\perp = 0.00025 \text{ cm}^2/\text{ms}$, fiber angle $\theta = 0$ (aligned with x), $\Delta x = 0.025 \text{ cm}$, $\Delta t = 0.01 \text{ ms}$.

From equation (18.6):

$$\begin{aligned}\tau_{xx} &= 0.005 + \frac{3 \times 0.001 \times 0.01}{0.000625} = 0.005 + 0.048 = 0.053 \text{ ms} \\ \tau_{yy} &= 0.005 + \frac{3 \times 0.00025 \times 0.01}{0.000625} = 0.005 + 0.012 = 0.017 \text{ ms}\end{aligned}$$

The relaxation rates:

$$s_1 = \Delta t / \tau_{xx} = 0.01 / 0.053 = 0.189, \quad s_2 = \Delta t / \tau_{yy} = 0.01 / 0.017 = 0.588$$

As expected, the cross-fiber direction (y) relaxes faster ($s_2 > s_1$), producing slower diffusion. The full relaxation matrix is:

$$\mathbf{S} = \text{diag}(0, 0.189, 0.588, 1.0, 1.0)$$

Now suppose $V_m = -85 \text{ mV}$, with the same non-equilibrium distributions from the Chapter 17 worked example: $\mathbf{f} = (0.340, 0.175, 0.155, 0.168, 0.162)^T \times (-85)$ (scaled to voltage), and a stimulus source $\mathcal{S} = 0.0371 \text{ mV/ms}$. The MRT collision proceeds as:

1. Decode: $\mathbf{m} = \mathbf{M}\mathbf{f}$ — extract density, x -flux, y -flux, and ghost moments.
2. Relax: $m_0^* = m_0$ (conserved), $m_1^* = m_1 - 0.189(m_1 - 0)$, $m_2^* = m_2 - 0.588(m_2 - 0)$, $m_3^* = 0$ (full relaxation), $m_4^* = 0$ (full relaxation).
3. Re-encode: $\mathbf{f}^* = \mathbf{M}^{-1}\mathbf{m}^*$.
4. Add source: $f_i^* \leftarrow f_i^* + \Delta t w_i \mathcal{S}$.

The x -flux decays by 18.9% (slow — high diffusivity along fibers), while the y -flux decays by 58.8% (fast — low diffusivity across fibers). The ghost moments snap entirely to equilibrium. This is the same collision from the Chapter 17 worked example, but now with the x and y dials set to different values.

18.5 Boundary Conditions for LBM Electrophysiology

In classical methods (FEM, FDM), boundary conditions are applied by modifying the global stiffness matrix or by constraining node values. In LBM, boundaries are handled at the *distribution level* — you modify individual f_i at boundary nodes, one direction at a time. This local, direction-by-direction approach is one of LBM's most elegant features: no global matrix is ever touched, and complex geometries require no special mesh generation.

For cardiac electrophysiology, the dominant boundary condition is **no-flux Neumann**: no transmembrane current leaves the tissue domain. Physically, the heart is electrically insulated — ionic currents flow within the myocardium but cannot escape through the

epicardial surface. Mathematically, this is $\nabla V_m \cdot \hat{n} = 0$ at every tissue boundary, where $\hat{n}$ is the outward normal.

18.5.1 Full-Way Bounce-Back

Imagine a ball thrown at a wall: it bounces straight back. **Full-way bounce-back** works the same way. When a distribution f_i at a boundary node would stream into a solid (non-tissue) region, it reverses direction — it becomes $f_{\bar{i}}$, where $\bar{i}$ denotes the opposite direction ($\mathbf{e}_{\bar{i}} = -\mathbf{e}_i$).

Concretely, at a boundary node $\mathbf{x}_b$, after the collision step produces $f_i^*(\mathbf{x}_b)$, instead of streaming f_i^* to the neighbor $\mathbf{x}_b + \mathbf{e}_i \Delta x$ (which lies outside the domain), we apply:

$$f_i(\mathbf{x}_b, t + \Delta t) = f_i^*(\mathbf{x}_b, t) \quad (18.10)$$

The post-collision distribution that *would have* left the domain in direction i is redirected to arrive at the same node in the opposite direction $\bar{i}$. Every unit of “flux” heading outward is perfectly reflected inward, so the net flux at the boundary is zero — exactly the no-flux Neumann condition.

For the D2Q9 lattice used in this chapter, the opposite pairs are:

Type	Direction i	Velocity $\mathbf{e}_i$	Opposite $\bar{i}$	Velocity $\mathbf{e}_{\bar{i}}$
Axial	1	(+1, 0)	3	(−1, 0)
	2	(0, +1)	4	(0, −1)
Diagonal	5	(+1, +1)	7	(−1, −1)
	6	(−1, +1)	8	(+1, −1)
Rest	0	(0, 0)	No partner — never streamed	

Intuition

Corner nodes. At a corner where two boundaries meet (say, the bottom-left corner of a rectangular domain), bounce-back applies to *both* boundary directions independently. If $\mathbf{e}_3 = (-1, 0)$ and $\mathbf{e}_4 = (0, -1)$ would stream out of the domain, both are reflected: $f_1 \leftarrow f_3^*$ and $f_2 \leftarrow f_4^*$. The diagonal direction $\mathbf{e}_7 = (-1, -1)$ is also reflected: $f_8 \leftarrow f_7^*$. No special corner treatment is needed — bounce-back handles it automatically, direction by direction.

18.5.2 Half-Way Bounce-Back

Full-way bounce-back places the effective wall *at* the boundary node itself. **Half-way bounce-back** places the wall halfway between the last fluid node and the first solid node. This seemingly small shift has an important consequence: the boundary condition becomes **second-order accurate** in space, matching the second-order accuracy of the interior LBM scheme.

The rule is almost identical to full-way bounce-back, but the timing changes. Instead of reflecting at the same timestep, the reflected distribution arrives one timestep later:

$$f_i(\mathbf{x}_b, t + \Delta t) = f_i^*(\mathbf{x}_b, t) \quad (18.10b)$$

The formula *looks* the same as full-way bounce-back (eq. 18.10), but the interpretation differs: the wall is now located at $\mathbf{x}_b + \frac{1}{2}\mathbf{e}_i \Delta x$, not at $\mathbf{x}_b$. On a uniform Cartesian grid, half-way bounce-back is the standard choice because the wall sits exactly at the cell face — which is where finite-difference methods also place the boundary.

Mathematical Note

Why second-order? The key insight is that the streaming step in LBM is *exact* — distributions travel exactly one lattice spacing per timestep. When the wall is at the half-way point, the round-trip distance (fluid node $\rightarrow$ wall $\rightarrow$ fluid node) is exactly $\Delta\mathbf{x}$, which takes exactly Δt . This geometric coincidence eliminates the first-order error that arises when the wall does not sit at a half-integer position. For regular Cartesian grids (as used in cardiac LBM), this condition is automatically satisfied.

18.5.3 Identifying Boundary Nodes

Before applying bounce-back, we must know *which* nodes are boundary nodes and *which directions* need reflection. In a cardiac simulation on a regular grid, the tissue geometry is typically defined by a binary mask: each grid point is either “tissue” (active) or “non-tissue” (inactive).

A fluid node $\mathbf{x}$ is a **boundary node** if at least one of its lattice neighbors $\mathbf{x} + \mathbf{e}_i \Delta\mathbf{x}$ is non-tissue. The specific directions i that point into non-tissue are the **boundary links** — these are the directions where bounce-back must be applied. In practice, this is precomputed once at initialization:

```
for each fluid node x:
  for each direction i = 1, ..., Q-1:
    neighbor = x + e_i * dx
    if neighbor is outside domain or non-tissue:
      mark (x, i) as a boundary link
```

At runtime, the streaming step skips boundary links (no data is sent to non-tissue nodes), and the bounce-back rule (eq. 18.10) is applied instead. This keeps the boundary treatment local and branchless — ideal for GPU execution.

18.5.4 Handling Irregular Tissue Geometries

Real cardiac tissue is not rectangular. The left ventricle is roughly ellipsoidal, the atria have complex curved surfaces, and scar tissue creates irregular internal boundaries. On a regular Cartesian grid, these curved boundaries are approximated by a **staircase** — the tissue mask follows the grid lines, creating jagged edges where the true boundary is smooth.

For cardiac electrophysiology, staircase boundaries are generally acceptable for two reasons:

- Diffusion dominates geometry.** The monodomain equation is a reaction-diffusion PDE, not a fluid-flow problem. Small geometric errors at the boundary (on the order of $\Delta\mathbf{x}$) produce errors in the diffusion flux that are $O(\Delta\mathbf{x})$ — the same order as the spatial discretization error. Refining the grid improves both simultaneously.
- No pressure or velocity fields.** In fluid LBM, staircase boundaries create spurious pressure oscillations and inaccurate drag forces. These problems do not arise in diffusion-only LBM because there is no momentum equation — only a scalar field V_m that diffuses smoothly.

⚠ Key Insight

When staircases fail. If you need sub-grid accuracy at the boundary (e.g., modeling the exact endocardial surface for defibrillation electrode placement), interpolated bounce-back methods exist. The simplest is **linear interpolation bounce-back**, which uses the fractional distance q from the fluid node to the actual wall: $f_i^*(\mathbf{x}_b, t + \Delta t) = 2q f_i^*(\mathbf{x}_b, t) + (1 - 2q) f_i^*(\mathbf{x}_b - \mathbf{e}_i \Delta\mathbf{x}, t)$ for $q < 1/2$, with a complementary formula for $q \geq 1/2$. This restores second-order accuracy on curved boundaries but adds implementation complexity. For most cardiac EP applications, standard bounce-back on a sufficiently fine grid ($\Delta\mathbf{x} \leq 0.2$ mm) is adequate.

18.5.5 Comparison with Classical BC Methods

Aspect	LBM (Bounce-Back)	FDM (Ghost Nodes)	FEM (Natural BC)
No-flux Neumann	Reflect f_i^*	Mirror V_m across boundary	Omit boundary integral (automatic)
Implementation	Local, per-link	Ghost node layer	Modify stiffness matrix
Curved geometry	Staircase or interpolated BB	Immersed boundary	Unstructured mesh (exact)
Accuracy	1st order (full-way) / 2nd order (half-way)	2nd order	Arbitrary (mesh-dependent)

Aspect	LBM (Bounce-Back)	FDM (Ghost Nodes)	FEM (Natural BC)
GPU friendliness	Excellent (local, branchless)	Good	Moderate (sparse matrix ops)

⚙️ Engine V5.4 Connection

Engine V5.4 implementation. The LBM solver uses half-way bounce-back on a Cartesian grid. Boundary links are precomputed during initialization into a compressed list `[(node_idx, direction)]`. During the streaming step, boundary links are processed separately: instead of writing to the neighbor, the post-collision value is written back to the opposite direction at the same node. The tissue mask is read from the geometry input (same binary mask used by the FDM solver) and converted to boundary links automatically.

18.6 The Complete LBM-EP Algorithm

Assembling all the pieces — operator splitting (Section 18.2), BGK or MRT collision with source (Sections 18.3–18.4), streaming (Chapter 17), and bounce-back boundaries (Section 18.5) — the complete algorithm for one timestep is:

Complete LBM-EP step (BGK or MRT):

(18.11)

- 1. Ionic:** $I_{\text{ion}} = \text{IonicModel.compute_Iion}(V_m, \mathbf{w});$ **update $\mathbf{w}$ via Rush-Larsen**
- 2. Source:** $S = -\frac{I_{\text{ion}} + I_{\text{stim}}}{\chi C_m}$
- 3. Collide:** $f_i^* = f_i + \Delta t(\Omega_i^{NR} + \Omega_i^R)$ (BGK eq. 18.3, or MRT eq. 18.8)
- 4. Stream:** $f_i(\mathbf{x} + \mathbf{e}_i \Delta x) \leftarrow f_i^*(\mathbf{x}) \quad \forall i$
- 5. Bounce-back:** $f_i(\mathbf{x}_b) \leftarrow f_i^*(\mathbf{x}_b)$ at boundary nodes
- 6. Recover:** $V_m^{n+1} = \sum_i f_i$

Steps 3–6 constitute the diffusion solver. Note the complete absence of matrix assembly, matrix-vector products, or iterative convergence checks. Every operation is either purely local (collision, source injection, recovery) or a nearest-neighbor shift (streaming). This is why LBM achieves its speed advantage on GPUs.

Intuition

Worked example. For cardiac tissue with $D = 0.001 \text{ cm}^2/\text{ms}$, $\Delta x = 0.025 \text{ cm}$, $\Delta t = 0.01 \text{ ms}$: $\tau = 0.053 \text{ ms}$, so $\omega = \Delta t/\tau = 0.189$. Consider a node with $V_m = -85 \text{ mV}$, all distributions at equilibrium, and a stimulus $I_{\text{stim}} = -52 \mu\text{A}/\mu\text{F}$ (with $I_{\text{ion}} = 0$, $\chi = 1400 \text{ cm}^{-1}$, $C_m = 1 \mu\text{F}/\text{cm}^2$).

Then $S = 52/1400 = 0.0371 \text{ mV/ms}$.

After collision: $f_0^* = f_0 + 0.01 \times \frac{1}{3} \times 0.0371 \approx f_0 + 1.24 \times 10^{-4}$.

After streaming and recovery: $V_m^{n+1} = \sum f_i \approx -85 + 0.01 \times 0.0371 = -84.9996 \text{ mV}$. The stimulus produces a tiny depolarization in one 0.01 ms step, as expected.

18.7 Implementation: Memory Layout and GPU Acceleration

LBM is a textbook example of SIMD parallelism: every node performs the same collision and streaming operations in lockstep. There are no global data dependencies, no reduce operations, no convergence criteria — each GPU thread is completely independent. On modern GPUs, this translates to near-peak memory bandwidth utilization and minimal synchronization overhead.

The Swap-Streaming Optimization

The naive implementation of Algorithm (18.11) uses two arrays: a pre-collision array $\mathbf{f}$ and a post-collision array $\mathbf{f}^*$. After streaming populates $\mathbf{f}^*$, the arrays are swapped. This doubles memory usage. The **swap-streaming** (or "AA-pattern") optimization fuses collision and streaming into a single pass:

On even timesteps, read $\mathbf{f}_i(\mathbf{x})$ from the source node and write $\mathbf{f}_i^*$ directly to the destination node $\mathbf{x} + \mathbf{e}_i \Delta \mathbf{x}$ (scatter pattern). On odd timesteps, reverse: read $\mathbf{f}_i$ from the neighbor $\mathbf{x} - \mathbf{e}_i \Delta \mathbf{x}$ and write the post-collision value to $\mathbf{x}$ (gather pattern). By alternating read/write patterns every other timestep, a single array suffices — halving memory and improving cache performance.

Memory Layout: Structure of Arrays

For GPU kernels, the distributions are stored in **Structure of Arrays** (SoA) format: one contiguous array per direction i , each of size $N_x \times N_y$ (or $N_x \times N_y \times N_z$ in 3D). The node at position $(\mathbf{x}, \mathbf{y})$ has its $\mathbf{f}_i$ stored at index $i \times N_x N_y + \mathbf{y} \times N_x + \mathbf{x}$. This layout ensures that when a GPU warp processes adjacent nodes, it accesses contiguous memory locations within each direction — maximizing memory coalescing.

Sparse/Indirect Addressing for Irregular Geometries

Cardiac tissue is not rectangular. A ventricular slab, a cross-section of the septum, or a full biventricular geometry all have irregular boundaries. Storing distributions on a full Cartesian grid wastes memory on nodes outside the tissue. The solution is **indirect addressing**: an adjacency array maps each active node to its neighbors. Instead of computing neighbor positions from $(\mathbf{x} \pm 1, \mathbf{y} \pm 1)$, the streaming step looks up the neighbor index from a precomputed table. This adds one integer load per direction per node but eliminates wasted storage on inactive nodes.

⚙️ Engine V5.4 Connection

In Engine V5.4, the LBM monodomain solver is in the `cardiac_sim.simulation.lbm` package. The lattice definitions (`D2Q5`, `D3Q7`) are frozen dataclasses holding weights, velocity vectors, and the $\tau\text{-}\mathbf{D}$ conversion formula. Collision operators (`BGKCollision`, `MRTCollision`) inherit from `CollisionOperator(ABC)` and implement the update step (equation 18.3 or 18.8). The `LBMState` container holds the distribution tensor $\mathbf{f}$ (shape $Q \times N_x \times N_y$ for 2D), voltage $\mathbf{V}_m$, ionic states $\mathbf{w}$, and the domain mask. The `LBMSimulation` class orchestrates the six-step loop (18.11): call `collision.collide(f, v, source, dt)`, apply `state.stream()` (one `torch.roll` per direction), `state.apply_bounce_back()`, and finally `state.recover_voltage()` to compute $\mathbf{V}_m = \sum \mathbf{f}_i$. The ionic substep is identical to the classical path — only the diffusion step changes. Factory functions `create_isotropic_bgk(D, dx, dt)` and `create_anisotropic_mrt(D_fiber, D_cross, fiber_angle, dx, dt)` handle initialization.

18.8 LBM vs. Classical Methods: When to Choose What

LBM, explicit methods (Chapter 9), and implicit methods (Chapter 10) each have distinct trade-offs:

Aspect	Explicit (Ch 9)	Implicit (Ch 10)	LBM-BGK/MRT
Linear solve?	No	Yes (PCG/FFT)	No
Stability limit?	Yes ($\Delta t \leq h^2/2dD$)	No (unconditional)	$\tau > \Delta t/2$ (always satisfied)
Anisotropy?	Via diffusion tensor	Via diffusion tensor	BGK = isotropic only; MRT = full tensor
Unstructured mesh?	Yes (FEM)	Yes (FEM)	No (Cartesian only)
GPU suitability	Good (SpMV)	Fair (global reductions)	Excellent (zero global sync)
Memory per node	1 float ($\mathbf{V}_m$)	1 float + sparse $\mathbf{A}$	Q floats ($\mathbf{f}_i$): 5 (D2Q5) or 7 (D3Q7)
Typical use	Small problems, rapid prototyping	Production FEM, large Δt	GPU throughput, parameter sweeps

When to use LBM: GPU hardware, Cartesian grids, and when throughput (simulations per unit wall-clock time) matters more than memory. The total absence of global synchronization makes LBM ideal for multi-GPU scaling and massive parameter sweeps. **When to use classical methods:** unstructured meshes (patient-specific geometries from MRI), problems where memory is the bottleneck (very large 3D domains), or when existing FEM infrastructure is already validated and optimized.

With the monodomain LBM now complete, we turn to its greatest challenge: the bidomain system. The parabolic intracellular equation can be solved by LBM, but the elliptic extracellular equation has no time dependence — it is a constraint that must be

satisfied at every instant. Chapter 19 develops three strategies to overcome this fundamental obstacle: pseudo-time relaxation, hybrid LBM–classical solvers, and the dual-lattice architecture.

Lattice-Boltzmann Methods for Bidomain

Engine V5.4's LBM solver achieves 10–45× speedups over FEM for the monodomain equation. Can this advantage be preserved for bidomain? Yes — but the elliptic equation requires creative treatment. This chapter develops the core strategies: pseudo-time relaxation, hybrid LBM–classical solvers, and the dual-lattice architecture that couples two independent distribution systems. We show how to extend the computational fiction of particle distributions to solve problems with fundamentally different mathematical structure.

19.1 The Bidomain Challenge: One Parabolic + One Elliptic

Imagine two fish tanks connected by pipes: an intracellular tank (inside the cells) and an extracellular tank (outside the cells). Water flows from one tank to the other through countless tiny pipes — the ion channels. The voltage across the cell membrane is the water-level difference. Now here is the key: the intracellular voltage evolves over time (it changes as currents flow), so it satisfies a time-dependent equation. But the extracellular voltage is constrained to always satisfy a balance — at every instant, whatever current flows out of the intracellular space must flow through the external medium. This constraint is instantaneous, with no time dependence. One tank's level evolves; the other's is a slave variable determined by the constraint.

The bidomain equations (from Chapter 12) are:

$$\chi C_m \frac{\partial V_m}{\partial t} + I_{\text{ion}} = \nabla \cdot (\mathbf{D}_i \nabla V_m) + \nabla \cdot (\mathbf{D}_e \nabla \phi_e)$$

$$0 = \nabla \cdot [(\mathbf{D}_i + \mathbf{D}_e) \nabla \phi_e] + \nabla \cdot (\mathbf{D}_i \nabla V_m) + I_{\text{stim}}$$

The first equation is *parabolic* — it has a time derivative, so it describes an evolution. The second is *elliptic* — no time derivative at all. It is a constraint that must be satisfied at every instant. LBM, by its nature, is a time-marching method: it evolves distributions forward in time. This is perfect for the parabolic equation but fundamentally at odds with the elliptic constraint.

The core mismatch is this: LBM asks "How do the distributions evolve?" But the elliptic equation says "The distributions must be such that this constraint is satisfied right now." These are two different questions. Resolving the tension between them — evolution versus constraint — is the central challenge of bidomain LBM.

Intuition

Think of the monodomain equation (Chapter 18) as a single fish tank with a changing water level. LBM marches the level forward in time with each step. The bidomain is two tanks. The intracellular tank (V_m) still evolves. But the extracellular tank (ϕ_e) is not free to evolve — its level is pinned down by the requirement that water conservation is always satisfied. This is why ϕ_e is an elliptic equation: it is an instantaneous constraint, not a time-evolution equation.

19.2 Strategy 1: Pseudo-Time Relaxation

If you heat a cold metal plate in a furnace with a fixed pattern of heat sources, the temperature rises over time. Eventually it reaches a steady state where the heat input exactly balances the heat loss through the edges. The temperature at that steady state is determined by the heat sources alone — it is independent of the initial condition or the "time" you waited. In mathematics, finding this steady state is the same as solving the elliptic equation that governs heat diffusion in equilibrium.

Pseudo-time relaxation exploits this observation. We cannot use LBM directly on an elliptic equation because there is no time-derivative to march. But we can invent a fictitious time — call it τ (not to be confused with the relaxation time parameter) — and add

an artificial time derivative to the elliptic equation:

$$\frac{\partial \phi_e}{\partial \tau} = \nabla \cdot [(\mathbf{D}_i + \mathbf{D}_e) \nabla \phi_e] + \nabla \cdot (\mathbf{D}_i \nabla V_m) + I_{\text{stim}} \quad (19.1)$$

This is now a parabolic equation in the pseudo-time τ . We march it with LBM in pseudo-time for many sub-iterations until the solution settles to a steady state, meaning $\partial \phi_e / \partial \tau$ becomes negligibly small. At that point, the solution is the same as what the elliptic equation demands — the constraint is satisfied.

The beauty of this approach: LBM works just as well on the pseudo-time equation as on the physical parabolic equation. The lattice parameter τ_e is set by the same Chapman–Enskog relationship (equation 17.26):

$$\tau_e = \frac{1}{2} + \frac{3(\mathbf{D}_i + \mathbf{D}_e) \Delta \tau}{\Delta x^2} \quad (19.2)$$

where $\Delta \tau$ is the pseudo-time step size and the conductivity is $\mathbf{D}_i + \mathbf{D}_e$ (the sum of intracellular and extracellular tensors, as required by the elliptic equation).

Intuition

The convergence speed depends on τ_e (which determines the collision rate) and on the geometry of the domain. A larger τ_e means gentler relaxation and slower convergence. Typical practice is to use a moderately large τ_e (around 0.8–1.5) to balance stability with speed. The number of pseudo-time sub-iterations per physical time step is usually 10–100, depending on the desired residual reduction. With multigrid acceleration (not detailed here), this can be reduced to 5–10 iterations.

19.3 Strategy 2: Hybrid LBM–Classical Solver

You would not use a hammer to write on paper — you use a pen. And you would not use a pen to drive nails — you use a hammer. The principle is simple: use the right tool for the job. LBM is superb at time-marching parabolic equations on Cartesian grids with GPUs. Classical solvers (conjugate gradient with multigrid preconditioning) are superb at solving elliptic systems exactly (or nearly exactly) without iteration over a fictitious time.

The hybrid approach splits the labor:

- **V_m equation (parabolic):** Use LBM with MRT collision (as in monodomain). Advance it by one physical time step.
- **ϕ_e equation (elliptic):** Use a classical solver — Conjugate Gradient (CG) preconditioned by Algebraic Multigrid (AMG). Solve it directly to a tight tolerance.

At each physical time step:

1. Compute ionic currents and update gates (Rush–Larsen).
2. March V_m forward one LBM step using the V_m distribution lattice.
3. Assemble the right-hand side for the elliptic ϕ_e equation, which includes the V_m gradient (now known from step 2) and the stimulus term.
4. Solve the elliptic system $\nabla \cdot [(\mathbf{D}_i + \mathbf{D}_e) \nabla \phi_e] = \dots$ using CG+AMG, yielding ϕ_e .
5. Feed ϕ_e back as a source term for the next step's V_m equation.

The key advantage: avoiding the pseudo-time convergence loop. The classical solver reaches machine precision in 10–30 iterations (typically much faster than 100 pseudo-time iterations). The tradeoff: data must be transferred between the LBM lattice (which lives on a Cartesian GPU grid) and the classical solver's matrix assembly (which may involve FEM or FDM operators).

Mathematical Note

The hybrid approach requires careful bookkeeping of the gradient ∇V_m on the LBM lattice and its transfer to the classical solver. One way to do this: compute ∇V_m on the Cartesian grid using finite differences (natural in LBM since we have nearest-neighbor data), then pass this to the classical solver's RHS assembly. The solver can run on the same grid (FDM) or on a finer/coarser grid (via interpolation operators) to trade memory for accuracy.

19.4 The Dual-Lattice Architecture

Instead of mixing tools, use two of the same tool — but in different modes. Imagine having two bookkeepers, each tracking inventory for different tanks. Tank 1's bookkeeper (Lattice 1) evolves the intracellular voltage. Tank 2's bookkeeper (Lattice 2) repeatedly adjusts the extracellular potential until the water-balance constraint is met. They hand notes to each other at the end of each physical time step, telling each other what changed.

The dual-lattice architecture maintains two independent LBM lattices, each solving its own equation:

- **Lattice 1:** Distribution functions $f_i^{(m)}(\mathbf{x}, t)$ solving the parabolic V_m equation. Relaxation time τ_m encodes the intracellular diffusivity D_i .
- **Lattice 2:** Distribution functions $f_i^{(e)}(\mathbf{x}, t)$ solving the pseudo-time-augmented ϕ_e equation. Relaxation time τ_e encodes the sum $D_i + D_e$.

The two lattices are coupled through source terms. The source for Lattice 1 includes the gradient of ϕ_e (the extracellular potential) plus the ionic current. The source for Lattice 2 includes the gradient of V_m plus the stimulus:

$$f_i^{(m)*} = f_i^{(m)} - \omega_m (f_i^{(m)} - f_i^{(m)eq}) + \Delta t w_i [\text{ionic term} + \text{coupling to } \phi_e] \quad (19.3)$$

$$f_i^{(e)*} = f_i^{(e)} - \omega_e (f_i^{(e)} - f_i^{(e)eq}) + \Delta \tau w_i [\text{coupling to } V_m + \text{stimulus}] \quad (19.4)$$

The algorithm at each physical time step is:

Dual-Lattice Bidomain LBM Timestep:

- | | |
|--|---|
| 1. Reaction: | Ionic currents and state updates (Rush–Larsen) |
| 2. Vm update: | One LBM step on Lattice 1 (physical time step Δt) |
| 3. ϕ_e convergence: | Iterate Lattice 2 until $\frac{\partial \phi_e}{\partial \tau}$ is negligible |
| 4. Source exchange: | Compute coupling terms for the next physical step |

Intuition

Worked example outline. Suppose at the start of a timestep: V_m is known (say, -85 mV at most nodes), and ϕ_e is at its previous steady state. We compute ionic currents (depends on V_m and gates). Lattice 1 takes one LBM step, advancing V_m to -84 mV (approximately). Now V_m has changed, so the constraint for ϕ_e has changed (because the RHS includes $\nabla \cdot (D_i \nabla V_m)$). Lattice 2 then iterates in pseudo-time, re-adjusting ϕ_e at every node until the new steady state is reached. This typically takes 10–50 pseudo-time iterations. Once converged, Lattice 1 knows the new ϕ_e value for the next physical step's source term.

How do the three strategies compare?

Architecture	$\mathbf{V_m}$ Solver	ϕ_e Solver	Pros	Cons
A: Dual-Lattice	LBM	LBM (pseudo-time)	Pure LBM; maximal GPU parallelism; unified code path	Pseudo-time convergence overhead (typically 20–100 sub-iterations)
B: Hybrid	LBM	CG+AMG	Optimal solver for each PDE type; fewer iterations for ϕ_e	Data transfer overhead between LBM and classical grids
C: Full Classical	FEM/FDM	FEM/FDM	Mature solver ecosystem; handles unstructured meshes	Slower than LBM for parabolic; matrix assembly overhead

Architecture A (dual-lattice) is the most elegant and maximizes GPU utilization. It is the natural choice for research and exploration. Architecture B (hybrid) trades elegance for robustness: it leverages decades of optimization in multigrid solvers and avoids the pseudo-time convergence cost. In practice, Architecture A is recommended as the starting point for Engine V5.4, with Architecture B as a fallback.

⚙ Engine V5.4 Connection

Engine V5.4 already has the infrastructure for Lattice 1: the D3Q7 lattice definitions, the MRT collision operator, and the torch-based streaming and recovery. For Lattice 2, you need: (1) a second set of distribution arrays $\mathbf{f}_i^{(e)}$, (2) a pseudo-time loop that iterates until convergence (checking $\|\partial\phi_e/\partial\tau\|$ after each step), (3) source term construction that couples $\mathbf{V_m}$ gradients from Lattice 1 into Lattice 2's collision. The factory functions can be extended: `create_dual_lattice_bidomain(D_i, D_e, dx, dt)` would initialize both lattices and return a `DualLatticeBidomainSolver` object with methods `step_reaction()`, `step_vm()`, `converge_phi_e(residual_tol)`.

19.5 Performance and Implementation Outlook

How much faster is bidomain LBM than classical bidomain solvers? That depends on how much time the classical solver spends on the elliptic solve. If the elliptic solve is fast (because the domain is simple or multigrid works well), then the speedup is modest. If the elliptic solve dominates, then LBM's advantage is large.

Based on the literature for monodomain LBM (which achieves 10–45× speedup over FEM) and accounting for the added cost of the elliptic ϕ_e equation, estimated performance ranges for bidomain LBM are:

- **Dual-Lattice (Architecture A):** 5–20× speedup over classical FEM bidomain, depending on the pseudo-time convergence rate and domain geometry. The lower end occurs when pseudo-time takes many iterations; the upper end occurs on simple geometries or with multigrid acceleration of the pseudo-time loop.
- **Hybrid (Architecture B):** 3–10× speedup. The elliptic solve using AMG is already near-optimal, so gains come mainly from the LBM parabolic solve and from avoiding the global reductions (inner products) required by iterative classical solvers.

Memory overhead: Each architecture stores two voltage fields ($\mathbf{V_m}$ and ϕ_e) plus either one or two lattices. The dual-lattice uses $2Q$ floats per node (two sets of Q distributions); the hybrid uses Q floats plus the classical solver's matrix storage. For a 3D grid with D3Q7 ($Q = 7$), the dual-lattice costs 14 floats per node — manageable on modern GPUs with 24–80 GB.

The recommended implementation path for Engine V5.4:

1. **Phase 1:** Extend the existing LBM infrastructure to support dual lattices. Test on a simple 2D domain with uniform conductivity.
2. **Phase 2:** Implement pseudo-time convergence checks and convergence acceleration (e.g., line-by-line SOR, incomplete factorization).
3. **Phase 3:** Add the hybrid solver as an alternative path: call external AMG libraries (AMGX on GPU, Hypre on CPU).
4. **Phase 4:** Benchmark against classical FEM bidomain solvers on realistic geometries and compare accuracy and wall-clock time.
5. **Phase 5:** Extend to anisotropic conductivity (different $\mathbf{D_f}$ and $\mathbf{D_e}$ for intracellular and extracellular spaces) using MRT on both lattices.

⚙ Engine V5.4 Connection

Recommended starting point for Engine V5.4: Implement Architecture A (dual-lattice) first. The pseudocode:

```
class DualLatticeBidomainLBMSolver(ABC):
    def __init__(self, D_i, D_e, dx, dt, ionic_model):
        self.lattice_vm = LBMLattice(D_i, dx, dt)
        self.lattice_phi_e = LBMLattice(D_i + D_e, dx, dt_pseudo)
        self.ionic = ionic_model

    def step(self, V_m, phi_e, I_stim, residual_tol=1e-6):
        # Step 1: Ionic
        I_ion, gates = self.ionic.step(V_m)

        # Step 2: V_m LBM
        source_vm = -(I_ion + I_stim) / (chi * Cm)
        source_vm += gradient_phi_e # coupling term
        V_m = self.lattice_vm.step(source_vm)

        # Step 3: phi_e convergence (pseudo-time loop)
        source_phi_e = -gradient(D_i @ gradient(V_m)) + I_stim_e
        phi_e = self.lattice_phi_e.converge(source_phi_e, tol=residual_tol)

        return V_m, phi_e, gates
```

Extend `LBMLattice` (already in the codebase for monodomain) to support pseudo-time iteration with convergence checks. Reuse the collision operator; only the time step size and convergence criterion change.

With the completion of Part IV (Chapters 17–19), we have the full mathematical and algorithmic foundation for bidomain electrophysiology. From the ionic model (Part I) through the monodomain and bidomain frameworks (Parts II and III), every piece is in place for a production cardiac simulator. The final part of this textbook turns to validation, performance optimization, and the art of choosing the right solver for your problem — the practical craft of turning equations into fast, accurate simulations.

APPENDICES

Reference Material

Key references, a visual guide to differential equations, and an introduction to PyTorch for scientific computing.

Key References

Core Bidomain Theory

1. Tung, L. (1978). *A bidomain model for describing ischemic myocardial D-C potentials*. PhD thesis, MIT.
2. Henriquez, C.S. (1993). Simulating the electrical behavior of cardiac tissue using the bidomain model. *Crit. Rev. Biomed. Eng.*, 21(1):1–77.
3. Sundnes, J., Lines, G.T., Cai, X., et al. (2006). *Computing the Electrical Activity in the Heart*. Springer.

Numerical Methods and Solvers

4. Vigmond, E.J., Weber dos Santos, R., Prassl, A.J., et al. (2008). Solvers for the cardiac bidomain equations. *Prog. Biophys. Mol. Biol.*, 96(1-3):3–18.
5. Pennacchio, M., Simoncini, V. (2002). Efficient algebraic solution of reaction–diffusion systems for the cardiac excitation process. *J. Comput. Appl. Math.*, 145(1):49–70.
6. Plank, G., Liebmann, M., Weber dos Santos, R., et al. (2007). Algebraic multigrid preconditioner for the cardiac bidomain model. *IEEE Trans. Biomed. Eng.*, 54(4):585–596.

Lattice-Boltzmann Methods for Cardiac EP

7. Rapaka, S., et al. (2012). LBM-EP: Lattice-Boltzmann method for fast cardiac electrophysiology simulation. *MICCAI 2012*.
8. Belmiloudi, A., et al. (2015). Coupled lattice Boltzmann modeling of bidomain type models in cardiac electrophysiology. *Springer*.
9. Belmiloudi, A., et al. (2019). Coupled lattice Boltzmann simulation method for bidomain type models with multiple time-delays. *Math. Model. Nat. Phenom.*

Software Frameworks

10. Africa, P.C., et al. (2023). lifex-ep: A flexible framework for cardiac electrophysiology. *BMC Bioinformatics*, 24:389.
11. Plank, G., et al. (2021). The openCARP simulation environment. *Comput. Methods Programs Biomed.*

This document synthesizes research from the four comprehensive research documents in the `Research/Bidomain/` folder: Discretization, Solver Methods, Linear Solvers, and LBM Bidomain. Code examples and implementation guides accompany this textbook in the `Code_Examples/` subfolder.

A Visual Guide to Differential Equations

This appendix steps back from the bidomain model to build a deep, visual intuition for the three fundamental types of partial differential equations: parabolic, elliptic, and hyperbolic. We begin with the heat equation — the simplest and most intuitive PDE — and use it as a lens to understand all three families. The style here follows the philosophy of 3Blue1Brown: start with a concrete picture, build understanding layer by layer, and let the math emerge naturally from the intuition.

A.1 The Heat Equation: Where It All Begins

Imagine you have a thin metal rod. You heat one spot in the middle with a blowtorch, creating a sharp temperature peak, and then you turn off the blowtorch. What happens?

You already know the answer intuitively: the hot spot cools down, and the warmth spreads outward into the cooler parts of the rod. Over time, the temperature profile gets smoother and flatter, until eventually the whole rod reaches a uniform temperature. This simple physical process — heat spreading from hot regions to cold regions — is described by one of the most important equations in all of mathematics:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2} \quad (A.1)$$

This is the **heat equation** in one dimension. Here $u(x, t)$ is the temperature at position x and time t , and $\alpha > 0$ is the *thermal diffusivity* — a material property that tells you how quickly heat spreads (copper has high α ; wood has low α).

But what does this equation actually *say*? Let us read it like a sentence.

Intuition

Reading the heat equation as English: "The rate at which the temperature changes at a point ($\partial u / \partial t$) is proportional to how much that point's temperature differs from the average of its neighbors ($\partial^2 u / \partial x^2$)." If a point is hotter than its surroundings, it cools down. If it is cooler, it warms up. The equation says nothing more — and nothing less — than this.

This is the fundamental insight, and it is worth dwelling on. The second derivative $\partial^2 u / \partial x^2$ is not just a mathematical abstraction — it measures *concavity*. At a temperature peak (concave down), the second derivative is negative, so $\partial u / \partial t < 0$: the temperature decreases. At a temperature valley (concave up), the second derivative is positive, so $\partial u / \partial t > 0$: the temperature increases. The heat equation is a machine that relentlessly destroys peaks and fills valleys.

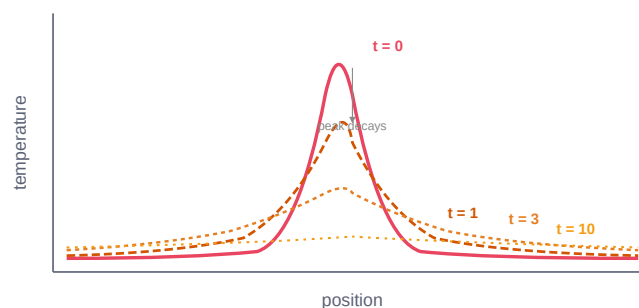


Figure A.1. A sharp temperature peak (red) diffuses over time, becoming broader and flatter. The heat equation smooths all discontinuities — information about the initial sharp shape is irreversibly lost.

Notice three properties that are visible even from the picture:

1. **Smoothing.** The solution gets smoother over time. A sharp corner at $t = 0$ becomes an infinitely smooth curve at any $t > 0$. The heat equation is a "smoothing machine."
2. **Irreversibility.** You cannot run time backwards and recover the sharp peak from the flat profile. Information is destroyed. (Mathematically: the backwards heat equation is ill-posed.)
3. **Maximum principle.** The hottest point at any later time is never hotter than the hottest point at $t = 0$. Diffusion can only reduce extremes, never amplify them.

These three properties — smoothing, irreversibility, and a maximum principle — are the *defining fingerprint* of **parabolic** equations. Every parabolic PDE shares them.

A.2 What the Laplacian Really Means

Before we classify PDEs, we need to build intuition for the single most important differential operator in physics: the **Laplacian** $\nabla^2 u$ (also written Δu).

In one dimension, $\nabla^2 u = \partial^2 u / \partial x^2$ — just the second derivative. In two or three dimensions:

$$\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2}$$

But what does it *mean*? Here is the deepest and most useful interpretation:

△ Key Insight

The Laplacian measures the difference between a point's value and the average of its neighbors. Specifically, for a function u that is smooth enough:

$$\nabla^2 u(\mathbf{x}) = \lim_{r \rightarrow 0} \frac{2d}{r^2} [\bar{u}_r(\mathbf{x}) - u(\mathbf{x})]$$

where $\bar{u}_r(\mathbf{x})$ is the average of u on a sphere of radius r centered at $\mathbf{x}$, and d is the spatial dimension. If $u(\mathbf{x})$ is *less* than its neighborhood average, then $\nabla^2 u > 0$. If *more*, then $\nabla^2 u < 0$.

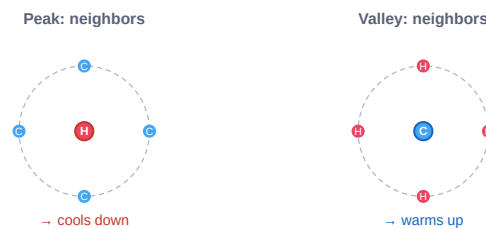


Figure A.2. The Laplacian as a "disagreement detector." Left: a hot point (H) surrounded by cool neighbors (C) has $\nabla^2 u < 0$. Right: a cool point surrounded by hot neighbors has $\nabla^2 u > 0$. The heat equation says: resolve the disagreement.

This is why the Laplacian appears in so many equations across physics. Any time nature wants to smooth out differences — heat, chemical concentration, electrical potential — the Laplacian is doing the work. The heat equation $\partial u / \partial t = \alpha \nabla^2 u$ simply says: *evolve toward the local average*.

Think About It

A discrete analogy. On a grid, approximate $\nabla^2 u$ at node i as $\nabla^2 u_i \approx \frac{1}{h^2} (\bar{u}_{\text{neighbors}} - u_i)$ where $\bar{u}_{\text{neighbors}}$ is the average of the neighboring values. The heat equation on a grid becomes: each node moves a small fraction toward the average of its neighbors at each time step. This is literally a smoothing filter — the same operation used in image processing.

A.3 The Three Families of PDEs

Now we are ready for the big classification. Consider the most general second-order linear PDE in two variables:

$$A \frac{\partial^2 u}{\partial x^2} + 2B \frac{\partial^2 u}{\partial x \partial y} + C \frac{\partial^2 u}{\partial y^2} + (\text{lower-order terms}) = 0 \quad (A.2)$$

The character of this equation — its fundamental behavior, the types of solutions it admits, the numerical methods that work — is determined by a single number: the **discriminant**

$$\Delta = B^2 - AC$$

This is exactly the same discriminant from the quadratic formula, and the analogy to conic sections is deep. Just as $B^2 - 4AC$ classifies conics into ellipses, parabolas, and hyperbolas, this discriminant classifies PDEs into three families:

Parabolic ($\Delta = 0$)

Prototype: Heat equation

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u$$

Diffusion, smoothing, irreversible

Elliptic ($\Delta < 0$)

Prototype: Laplace equation

$$\nabla^2 u = 0$$

Equilibrium, steady-state, no time

Hyperbolic ($\Delta > 0$)

Prototype: Wave equation

$$\frac{\partial^2 u}{\partial t^2} = c^2 \nabla^2 u$$

Propagation, waves, reversible

Think About It

Why does the name "parabolic" come from the degenerate case $\Delta = 0$? Think of the conic $Ax^2 + 2Bxy + Cy^2 = 1$. When $\Delta = 0$, this degenerates from a closed curve (ellipse) to an open, unbounded shape (parabola). Similarly, the PDE loses one "degree of freedom" — information propagates in one direction only (forward in time), just as a parabola is open in one direction.

Let us now build deep intuition for each family.

A.4 Parabolic Equations: Diffusion and Smoothing

The heat equation is the prototype, but parabolic equations describe far more than heat. Any process where "stuff" spreads out from high-concentration regions to low-concentration regions is parabolic: chemical diffusion (Fick's law), financial option pricing (Black–Scholes), probability spreading (Fokker–Planck), and — most relevant to us — the evolution of the transmembrane potential V_m in the monodomain equation.

The unifying feature is the structure: **one time derivative balanced against two spatial derivatives**. This mismatch (∂_t vs. ∂_{xx}) dictates everything:

$$\underbrace{\frac{\partial u}{\partial t}}_{\text{1st order in } t} = \alpha \underbrace{\frac{\partial^2 u}{\partial x^2}}_{\text{2nd order in } x}$$

Because there is only a *first* derivative in time, you need only *one* initial condition: $u(x, 0) = u_0(x)$. Time marches in one direction only. There is no "bouncing back." This is why parabolic equations model irreversible processes.

The smoothing property, precisely stated. If you start with any initial condition — even one with sharp corners or discontinuities — the solution $u(x, t)$ is infinitely smooth for all $t > 0$. This is remarkable. A square-wave initial condition instantly (at any positive time, no matter how small) becomes a C^∞ function. Intuitively: diffusion smears out all sharp features immediately.

Infinite propagation speed. Here is a subtle and often confusing point. Heat propagates at *infinite speed* in the mathematical heat equation — if you heat one end of an infinitely long rod, the other end feels it instantly (though the effect is exponentially small). This is physically unrealistic but mathematically convenient. The parabolic monodomain equation inherits this property: a stimulus at one point technically affects the entire domain immediately, though the practical effect dies off exponentially fast.

△ Key Insight

The parabolic CFL condition. When you discretize a parabolic equation with an explicit time stepper (like forward Euler), stability demands $\Delta t \leq C \Delta x^2$. The *quadratic* dependence on Δx is severe: halving the mesh spacing requires quartering the time step. This is why implicit methods (Crank–Nicolson, BDF, SDIRK) are essential for parabolic problems — they are unconditionally stable, allowing Δt dictated by accuracy rather than stability.

A.5 Elliptic Equations: Equilibrium and Constraints

Now remove time from the picture entirely. Ask: *what does the temperature look like after the system has fully equilibrated?* At equilibrium, $\partial u / \partial t = 0$, and the heat equation becomes:

$$\nabla^2 u = 0 \quad (\text{Laplace's equation})$$

(A.3)

This is the prototypical **elliptic** equation. It says: at every point, the value of u equals the average of its neighbors. There is no time. There is no evolution. It is a pure *constraint*.

Intuition

The rubber sheet analogy. Imagine a flat rubber membrane stretched over a wire frame. The frame is bent into some shape (the boundary condition). Where does the membrane settle? At every interior point, the height of the membrane equals the average height of the surrounding points — because the elastic forces balance in all directions. The equilibrium shape satisfies Laplace's equation.

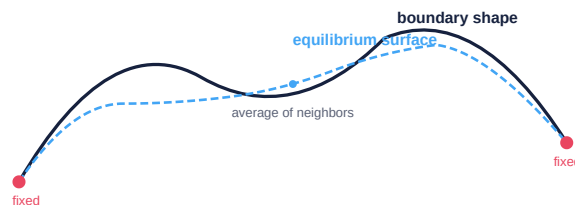


Figure A.3. The rubber sheet analogy for Laplace's equation. The boundary conditions (wire frame) determine the shape; the interior settles into an equilibrium where each point's value equals the average of its neighbors.

The key properties of elliptic equations mirror the equilibrium intuition:

1. **Mean value property.** The value at any interior point equals the average over any surrounding sphere. This is the deepest characterization of harmonic functions (solutions to $\nabla^2 u = 0$).
2. **Maximum principle.** The maximum and minimum of u occur on the *boundary*, never in the interior. The rubber sheet cannot bulge above or below the wire frame.
3. **Global coupling.** Changing the boundary condition at one point affects the solution *everywhere* in the interior. Information propagates in all directions simultaneously. There is no concept of "marching" from one end to the other.
4. **No time, no initial conditions.** You need only boundary conditions. The equation determines the interior values uniquely from the boundary.

Think About It

Why you cannot "march" an elliptic equation. In a parabolic equation, you know u at $t = 0$ and march forward to $t = \Delta t$, then to $t = 2\Delta t$, and so on. In an elliptic equation, there is no "forward." Changing one boundary value ripples through the *entire* domain simultaneously. This is why elliptic equations always require solving a **global linear system** — you must find all unknowns at once.

If there is a source term, Laplace's equation becomes **Poisson's equation**: $\nabla^2 u = f$. This is still elliptic. It says: the difference between a point's value and its neighborhood average is proportional to the local source strength f . This is exactly the form of the bidomain elliptic constraint.

A.6 Hyperbolic Equations: Waves and Propagation

The third family is fundamentally different from the first two. Consider the **wave equation**:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2} \quad (A.4)$$

Notice the crucial structural difference: the *second* derivative in time. Where the heat equation had ∂_t (first order), the wave equation has ∂_{tt} (second order). This single change transforms the physics completely.

With a first-order time derivative, you need one initial condition (u at $t = 0$). With a second-order time derivative, you need *two*: the initial shape $u(x, 0)$ and the initial velocity $\partial u / \partial t|_{t=0}$. This is like specifying both the position and velocity of a particle — and it means the equation is *reversible*. You can run time forward or backward and get valid solutions.

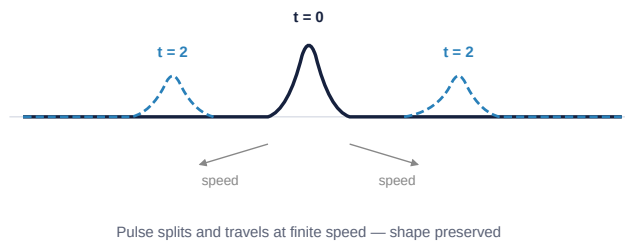


Figure A.4. The wave equation splits an initial pulse into two copies traveling in opposite directions at speed c . Unlike diffusion, the pulse shape is preserved — no smoothing occurs.

The contrast with the heat equation could not be sharper:

Property	Parabolic (heat)	Hyperbolic (wave)
Time derivative order	1st (∂_t)	2nd (∂_{tt})
Initial conditions needed	1 (value)	2 (value + velocity)
Reversible?	No	Yes
Propagation speed	Infinite (exponential decay)	Finite (c)
Smoothing?	Yes — destroys sharp features	No — preserves sharp features
Energy	Dissipated over time	Conserved

The finite propagation speed is the hallmark of hyperbolic equations. A disturbance at point x_0 at time t_0 can only be felt at point x after time $(x - x_0)/c$. This region of influence is called the **light cone** (even for non-relativistic waves), and the boundary of this cone consists of **characteristics** — curves in space-time along which information travels.

Intuition

Why cardiac EP is NOT hyperbolic. At first glance, the action potential "wavefront" sweeping through the heart looks like a traveling wave — so you might expect a wave equation. But it is not a wave in the hyperbolic sense. The action potential front is driven by *diffusion* (current spreading from depolarized cells to resting cells) plus a *nonlinear reaction* (ion channels opening in a cascade). There is no second time derivative. There is no energy conservation. And the wavefront *does* smooth out over time. The monodomain and bidomain equations are parabolic, not hyperbolic — the traveling wavefront emerges from the interplay of diffusion and reaction, not from wave mechanics.

A.7 Why the Bidomain Model Has Two Types

We can now understand the bidomain model's structure at a much deeper level. The bidomain system is not a single PDE — it is a *coupled pair*, and each equation belongs to a different family:

$$\underbrace{\chi C_m \frac{\partial V_m}{\partial t} = -\chi I_{\text{ion}} + \nabla \cdot (\mathbf{D}_i \nabla V_m) + \nabla \cdot (\mathbf{D}_i \nabla \phi_e)}_{\text{Parabolic: has } \partial V_m / \partial t} \quad (A.5)$$

$$\underbrace{0 = \nabla \cdot (\mathbf{D}_i \nabla V_m) + \nabla \cdot [(\mathbf{D}_i + \mathbf{D}_e) \nabla \phi_e]}_{\text{Elliptic: no time derivative at all}} \quad (A.6)$$

This mixed parabolic–elliptic structure is the source of both the model's physical richness and its computational difficulty:

1. **The parabolic equation** governs the evolution of the transmembrane potential V_m . It is a diffusion equation — with all the properties we discussed: smoothing, irreversibility, CFL restrictions for explicit methods. The "stuff" being diffused is electrical charge, and the diffusion coefficient is determined by the intracellular conductivity tensor $\mathbf{D}_i$. It can in principle be marched forward in time.
2. **The elliptic equation** constrains the extracellular potential ϕ_e . It has no time derivative — it is an *equilibrium condition* that must hold at every instant, like the rubber sheet analogy. At every moment, the extracellular potential adjusts *instantly* so that the total current in and out of every point balances. It cannot be marched — it must be solved as a global linear system.

△ Key Insight

The fundamental computational consequence. The parabolic equation can (in theory) be treated explicitly. The elliptic equation *cannot* — ever. This is why every bidomain solver requires at least one implicit linear solve per time step: the elliptic constraint demands it. This is not a choice of method; it is a consequence of the equation's type. No clever algorithm can avoid it, because the elliptic equation couples all spatial points simultaneously.

The monodomain equation, by contrast, is purely parabolic — a single equation with a single time derivative. This is why it is so much cheaper: you never need to solve an elliptic constraint. The LBM methods in Chapter 17 exploit this: a lattice-Boltzmann scheme can handle a parabolic PDE naturally (the collision-streaming cycle is essentially an explicit time stepper), but it has no mechanism for enforcing a global elliptic constraint.

Understanding the type of each equation also tells you *which numerical methods to reach for*:

Equation type	Natural solvers	Avoid
Parabolic (V_m)	Implicit time steppers (CN, BDF, SDIRK), operator splitting, IMEX, LBM	Fully explicit on fine meshes (CFL: $\Delta t \propto \Delta x^2$)

Elliptic (ϕ_e)	Direct solvers, CG/MINRES with AMG preconditioner, multigrid	Explicit time stepping (impossible — no time derivative)
-----------------------	--	--

The classification of PDEs into parabolic, elliptic, and hyperbolic is not merely academic taxonomy. It determines which initial/boundary conditions are needed, which numerical methods will work, what stability restrictions apply, and whether information flows forward only or in all directions simultaneously. For the bidomain model, this classification is the key to understanding why the computational challenge is fundamentally harder than monodomain — and why the solver strategies of Chapter 15 are not optional refinements but structural necessities.

The treatment in this appendix follows the pedagogical philosophy of Grant Sanderson's *3Blue1Brown* series on differential equations. For animated visualizations of these concepts, the reader is encouraged to watch the "Differential Equations" playlist on the 3Blue1Brown YouTube channel.

APPENDIX B

Introduction to PyTorch

This appendix is a self-contained tutorial on PyTorch, written for scientists who know Python and perhaps some NumPy but have never touched PyTorch. We start at the very beginning — creating a single number — and build up through arithmetic, matrices, linear algebra, GPU computing, automatic differentiation, and neural networks. By the final section, every PyTorch concept you have learned will map directly onto a piece of Engine V5.4's source code, and you will realize you already understand the entire computational backend of this textbook's cardiac simulation engine.

B.1 Installation and Your First Tensor

Install PyTorch with a single command:

```
pip install torch
```

Now open a Python shell and create your first tensor:

```
import torch

x = torch.tensor(3.14)
print(x)           # tensor(3.1400)
print(x.shape)     # torch.Size([])      ← a scalar has an empty shape
print(x.dtype)     # torch.float32      ← default floating-point type
print(x.item())    # 3.14               ← extract the plain Python number
```

A **tensor** is just a number — or a collection of numbers — that PyTorch knows how to do math on. A scalar (single number) is a 0-dimensional tensor. A list of numbers is a 1-D tensor (a vector). A grid of numbers is a 2-D tensor (a matrix). And it keeps going: 3-D, 4-D, any number of dimensions. But at the end of the day, it is just numbers in a box with a shape.

```
scalar = torch.tensor(42.0)           # shape: []      (0-D)
vector = torch.tensor([1.0, 2.0, 3.0]) # shape: [3]     (1-D)
matrix = torch.tensor([[1, 2], [3, 4], [5, 6]]) # shape: [3, 2] (2-D)
cube = torch.zeros(2, 3, 4)           # shape: [2, 3, 4] (3-D)

print(scalar.ndim, vector.ndim, matrix.ndim, cube.ndim) # 0 1 2 3
```

B.2 Arithmetic: Scalars, Vectors, and Matrices

PyTorch arithmetic works the same way as Python arithmetic, but it operates on entire tensors at once — no loops required.

Scalar arithmetic.

```
a = torch.tensor(3.0)
b = torch.tensor(2.0)

print(a + b)    # tensor(5.)
print(a - b)    # tensor(1.)
print(a * b)    # tensor(6.)
print(a / b)    # tensor(1.5000)
print(a ** b)   # tensor(9.)      ← 3^2
```

Vector arithmetic. Every operation applies element-by-element:

```
u = torch.tensor([1.0, 2.0, 3.0])
v = torch.tensor([4.0, 5.0, 6.0])

print(u + v)    # tensor([5., 7., 9.]
```

```
print(u * v)      # tensor([ 4., 10., 18.]) ← element-wise, NOT dot product
print(2.0 * u)    # tensor([2., 4., 6.])   ← scalar × vector
```

The **dot product** (inner product) of two vectors is a separate operation:

```
print(torch.dot(u, v)) # tensor(32.) ← 1·4 + 2·5 + 3·6 = 32
```

Matrix arithmetic follows the same pattern. Element-wise operations use `+`, `*`, etc. Matrix multiplication uses the `@` operator (or `torch.matmul`):

```
A = torch.tensor([[1., 2.], [3., 4.]])
B = torch.tensor([[5., 6.], [7., 8.]])

print(A * B)      # element-wise: tensor([[ 5., 12.], [21., 32.]])
print(A @ B)      # matrix multiply: tensor([[19., 22.], [43., 50.]])
```

If you have used NumPy before, this should feel identical — because it is. PyTorch deliberately mirrors the NumPy API. The only visible differences appear when we start using GPUs (B.8) and gradients (B.9).

B.3 Tensors and Shapes

The word “tensor” sounds intimidating, but the idea is simple: a tensor is a multi-dimensional array. You already know the special cases:

Dimensions	Name	Shape example	What it holds
0	Scalar	<code>[]</code>	A single number
1	Vector	<code>[3]</code>	A list of numbers
2	Matrix	<code>[3, 4]</code>	A table (rows × columns)
3	3-D tensor	<code>[2, 3, 4]</code>	A stack of matrices (e.g., RGB image)
4	4-D tensor	<code>[8, 3, 256, 256]</code>	A batch of images

Reshaping. You can rearrange the elements of a tensor without copying the data:

```
x = torch.arange(12)      # tensor([ 0,  1,  2, ..., 11]) shape [12]
y = x.view(3, 4)          # shape [3, 4] — same data, different layout
z = x.view(2, 2, 3)       # shape [2, 2, 3]

print(y)
# tensor([[ 0,  1,  2,  3],
#         [ 4,  5,  6,  7],
#         [ 8,  9, 10, 11]])
```

Slicing and indexing works like Python lists and NumPy arrays:

```
A = torch.tensor([[10, 20, 30],
                  [40, 50, 60]])

print(A[0, :])           # tensor([10, 20, 30]) ← first row
print(A[:, 1])           # tensor([20, 50])    ← second column
print(A[1, 2])           # tensor(60)         ← single element
```

Broadcasting. When two tensors have different shapes, PyTorch automatically expands the smaller one to match — the same rules as NumPy. The key rule: dimensions are compared from the right, and a dimension of size 1 can stretch to match any size.

```
v = torch.tensor([[1.0], [2.0], [3.0]]) # shape [3, 1]
w = torch.tensor([10.0, 20.0, 30.0])    # shape [3]
```

```
# V + w broadcasts: [3,1] + [3] → [3,1] + [1,3] → [3,3]
print(V + w)
# tensor([[11., 21., 31.],
#         [12., 22., 32.],
#         [13., 23., 33.]])
```

B.4 Data Types and Precision

Every tensor has a **dtype** that controls how much memory each number uses and how precisely it is stored:

dtype	Bits	Significant digits	Typical use
<code>torch.float32</code>	32	~7	Machine learning (default)
<code>torch.float64</code>	64	~16	Scientific computing, PDE solvers
<code>torch.int64</code>	64	exact (integers)	Indices, counts
<code>torch.bool</code>	8	True / False	Masks, conditions

Machine learning uses `float32` because neural networks are tolerant of rounding — a gradient that is off by 0.001% still points in the right direction. PDE solvers are not tolerant. Consider a single forward-Euler step of the monodomain equation: $\mathbf{V}^{n+1} = \mathbf{V}^n + \Delta t \cdot \mathbf{K} \cdot \mathbf{V}^n$. The stiffness matrix $\mathbf{K}$ has entries of order $D/\Delta x^2 \sim 10^4 \text{ s}^{-1}$, and $\Delta t \sim 10^{-5} \text{ s}$. The product $\Delta t \cdot \mathbf{K} \cdot \mathbf{V}$ requires at least 10 significant digits to avoid catastrophic cancellation when subtracted from $\mathbf{V}^n$. Seven digits (`float32`) is not enough.

```
# The default is float32:
x = torch.randn(5)
print(x.dtype)           # torch.float32

# For scientific computing, use float64:
V = torch.zeros(1000, dtype=torch.float64)

# Or change the global default:
torch.set_default_dtype(torch.float64)
V = torch.zeros(1000)     # now float64 by default
```

⚠ Key Insight

Rule of thumb. If your code involves physical units, differential equations, or any computation where small differences between large numbers matter, use `float64`. If you are training a neural network, `float32` (or even `float16`) is fine.

B.5 Common Operations Cheat Sheet

This section is a quick reference. Skim it now; come back when you need a specific operation.

Creating tensors.

```
torch.zeros(3, 4)           # 3x4 matrix of zeros
torch.ones(5)               # vector of ones
torch.arange(0, 10, 2)      # tensor([0, 2, 4, 6, 8])
torch.linspace(0, 1, 11)    # 11 points from 0 to 1 inclusive
torch.randn(3, 3)          # 3x3 matrix of standard-normal random numbers
torch.eye(4)                # 4x4 identity matrix
```

Reductions.

```
x = torch.tensor([3.0, 1.0, 4.0, 1.0, 5.0])

x.sum()                     # tensor(14.)
x.mean()                    # tensor(2.8000)
```

```
x.max()          # tensor(5.)
x.min()          # tensor(1.)
x.argmax()       # tensor(4)      ← index of the maximum
x.std()          # tensor(1.7889) ← standard deviation
```

Conditional and clamping.

```
V = torch.tensor([-120.0, -40.0, 20.0, 80.0])

torch.clamp(V, min=-85.0, max=40.0) # tensor([-85., -40., 20., 40.])
torch.where(V > 0, V, torch.zeros_like(V)) # ReLU: tensor([0., 0., 20., 80.])
```

Stacking and concatenating.

```
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])

torch.cat([a, b])          # tensor([1, 2, 3, 4, 5, 6])      ← concatenate
torch.stack([a, b])        # tensor([[1, 2, 3], [4, 5, 6]]) ← stack into new dimension
```

In-place operations. Any operation with an underscore suffix modifies the tensor directly, saving memory:

```
x = torch.zeros(5)
x.add_(1.0)      # x is now tensor([1., 1., 1., 1., 1.]) – no copy made
x.mul_(3.0)      # x is now tensor([3., 3., 3., 3., 3.])
```

Math functions.

```
x = torch.tensor([0.0, 1.0, 2.0])

torch.exp(x)      # tensor([1.0000, 2.7183, 7.3891])
torch.log(x + 1)  # tensor([0.0000, 0.6931, 1.0986])
torch.sin(x)      # tensor([0.0000, 0.8415, 0.9093])
torch.abs(x - 1)  # tensor([1., 0., 1.])
```

B.6 Linear Algebra

PyTorch includes a full linear algebra library in `torch.linalg`. If you have used `numpy.linalg`, the API is nearly identical.

Solving $Ax = b$.

```
A = torch.tensor([[2., 1., -1.],
                  [-3., -1., 2.],
                  [-2., 1., 2.]], dtype=torch.float64)
b = torch.tensor([8., -11., -3.], dtype=torch.float64)

x = torch.linalg.solve(A, b)
print(x)    # tensor([ 2.,  3., -1.]) ← the solution

# Verify: A @ x should equal b
print(A @ x) # tensor([ 8., -11., -3.])
```

Norms, determinants, eigenvalues.

```
M = torch.tensor([[4., 2.], [1., 3.]], dtype=torch.float64)

torch.linalg.norm(M)      # Frobenius norm: tensor(5.4772)
torch.linalg.det(M)       # determinant: tensor(10.)
torch.linalg.inv(M)       # inverse: tensor([[ 0.3, -0.2], [-0.1, 0.4]])

eigvals, eigvecs = torch.linalg.eig(M)
print(eigvals)    # tensor([5.+0.j, 2.+0.j]) ← eigenvalues (complex)
```

Intuition

When to use `linalg.solve` vs. `linalg.inv`. Never compute $\mathbf{A}^{-1}\mathbf{b}$ by inverting the matrix first. `linalg.solve(A, b)` is faster, more numerically stable, and uses less memory. The explicit inverse is only useful if you need $\mathbf{A}^{-1}$ for a different reason.

B.7 Sparse Matrices

In scientific computing, most large matrices are **sparse**: the vast majority of entries are zero. The stiffness matrix for the monodomain equation on a 512×512 grid is $262,144 \times 262,144$ — over 68 billion entries — but only about 1.3 million are nonzero (five per row for the 5-point stencil). Storing it dense would need 512 GB; storing it sparse needs about 30 MB.

PyTorch's native sparse format is **COO** (Coordinate): store each nonzero as a triplet (i, j, v) — row index, column index, value. You build a COO tensor by providing two arrays (row indices and column indices) and one array (values):

```
import torch

# A simple 4x4 sparse matrix with 6 nonzero entries
rows = [0, 0, 1, 1, 2, 3]
cols = [0, 1, 0, 1, 2, 3]
vals = [4.0, -1.0, -1.0, 4.0, 4.0, 4.0]

indices = torch.tensor([rows, cols], dtype=torch.long)
values = torch.tensor(vals, dtype=torch.float64)
S = torch.sparse_coo_tensor(indices, values, size=(4, 4)).coalesce()

print(S.to_dense())
# tensor([[ 4., -1.,  0.,  0.],
#         [-1.,  4.,  0.,  0.],
#         [ 0.,  0.,  4.,  0.],
#         [ 0.,  0.,  0.,  4.]])
```

The `.coalesce()` call merges any duplicate entries and sorts the indices — required before most sparse operations.

Worked example: the 5-point Laplacian. The discrete Laplacian $\nabla^2 V \approx (V_{i-1,j} + V_{i+1,j} + V_{i,j-1} + V_{i,j+1} - 4V_{i,j})/\Delta x^2$ produces a sparse matrix where each row has at most 5 nonzero entries. Here is how to assemble it on a small 4×4 grid:

```
N = 4          # 4x4 grid → 16 unknowns
dx = 1.0
rows, cols, vals = [], [], []

for j in range(N):
    for i in range(N):
        node = j * N + i
        nbrs = 0
        for (di, dj) in [(1,0), (-1,0), (0,1), (0,-1)]:
            ni, nj = i + di, j + dj
            if 0 <= ni < N and 0 <= nj < N:
                rows.append(node); cols.append(nj * N + ni)
                vals.append(1.0 / dx**2)
                nbrs += 1
        rows.append(node); cols.append(node)
        vals.append(-nbrs / dx**2)

indices = torch.tensor([rows, cols], dtype=torch.long)
values = torch.tensor(vals, dtype=torch.float64)
K = torch.sparse_coo_tensor(indices, values, size=(N*N, N*N)).coalesce()
```

Sparse matrix-vector multiplication is a single call:

```
V = torch.randn(N*N, 1, dtype=torch.float64)
KV = torch.sparse.mm(K, V)      # sparse × dense → dense column vector
```

PyTorch also supports scalar-sparse arithmetic, which is essential for building implicit time-stepping matrices like $A = \chi C_m I - \frac{1}{2} \Delta t K$:

```
# Sparse identity matrix
def speye(n, dtype=torch.float64):
    idx = torch.arange(n)
    return torch.sparse_coo_tensor(
        torch.stack([idx, idx]),
        torch.ones(n, dtype=dtype),
        size=(n, n)
    ).coalesce()

I = speye(N*N)
chi_Cm, dt = 1.0, 0.01
A = (chi_Cm * I - 0.5 * dt * K).coalesce() # scalar x sparse works naturally
```

B.8 GPU Acceleration

Everything so far has run on your CPU. Moving to a GPU requires exactly three changes: (1) choose a device, (2) place your tensors there, (3) ... there is no step 3. The math is the same.

```
# Step 1: choose a device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(device) # 'cuda' if you have an NVIDIA GPU, else 'cpu'

# Step 2: create tensors on that device (or move existing ones)
V = torch.zeros(1000, dtype=torch.float64, device=device)
K = build_laplacian(N=32, device=device) # your sparse matrix, on GPU

# All operations now run on the GPU automatically:
KV = torch.sparse.mm(K, V.unsqueeze(1)).squeeze(1)
V_new = V + dt * KV
```

The algorithm reads identically whether `device` is `'cpu'` or `'cuda'`. This is PyTorch's key selling point for scientific computing: **write once, run anywhere**.

To move data between CPU and GPU:

```
x_gpu = x_cpu.to('cuda') # CPU → GPU (copies data across PCIe bus)
x_cpu = x_gpu.cpu() # GPU → CPU (copies data back)
x_gpu = x_gpu.to('cuda') # already on GPU – no-op, no copy
```

⚠ Key Insight

Minimise transfers. The PCIe bus between CPU and GPU is slow compared to GPU computation. The best practice is to move your data to the GPU once at the start, do all computation there, and move results back only when you need to save or plot. Avoid calling `.cpu()` inside inner loops.

B.9 Automatic Differentiation (Autograd)

Automatic differentiation is PyTorch's signature feature and the reason it dominates machine learning. The idea: PyTorch can record every arithmetic operation you perform on a tensor, building an invisible **computation graph**. When you ask for gradients, it walks backward through the graph applying the chain rule — automatically, exactly, and efficiently.

Think of it like a receipt tape at a cash register. Every operation (add, multiply, exp, matmul) prints a line on the tape. When you call `.backward()`, PyTorch reads the tape in reverse, computing $\partial(\text{output})/\partial(\text{each input})$ at every step.

```
import torch

# Tell PyTorch to track operations on w
```

```

w = torch.tensor(2.0, requires_grad=True)
x = torch.tensor(3.0)
y = torch.tensor(5.0)

L = (w * x - y) ** 2          # L = (2*3 - 5)^2 = 1.0

L.backward()                  # walk the tape backward

print(w.grad)                 # tensor(6.)
# Check by hand: dL/dw = 2(wx - y) * x = 2(6-5)(3) = 6 ✓

```

The `requires_grad=True` flag turns on recording. The `.backward()` call traverses the graph and deposits the gradient in `w.grad`. This is the same mechanism that trains neural networks with millions of parameters — the only difference is scale.

When you do *not* need gradients (e.g., running a forward simulation), you can disable the tape entirely to save memory and speed:

```

with torch.no_grad():
    # Everything inside this block runs without recording the computation graph.
    V_new = V + dt * KV      # fast, no tape overhead

```

B.10 Building Neural Networks with `nn.Module`

PyTorch's neural network library lives in `torch.nn`. The core abstraction is `nn.Module`: a class where you define your network's layers in `__init__` and the forward pass in `forward`.

```

import torch
import torch.nn as nn

class TinyNet(nn.Module):
    def __init__(self, n_in, n_hidden, n_out):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_in, n_hidden),      # first layer: n_in → n_hidden
            nn.Tanh(),                      # activation function
            nn.Linear(n_hidden, n_hidden),  # second layer
            nn.Tanh(),
            nn.Linear(n_hidden, n_out),     # output layer
        )

    def forward(self, x):
        return self.net(x)

```

`nn.Linear(m, n)` is a fully connected layer: it stores an $n \times m$ weight matrix $\mathbf{W}$ and an n -vector bias $\mathbf{b}$, and computes $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$. `nn.Tanh()` applies the hyperbolic tangent element-wise — this introduces the nonlinearity that lets a network approximate complex functions.

The training loop. Training a neural network is a four-step cycle repeated many times:

1. **Forward pass.** Feed inputs through the network, get predictions.
2. **Loss.** Measure how wrong the predictions are (e.g., mean squared error).
3. **Backward pass.** Compute gradients of the loss with respect to every weight (autograd).
4. **Optimizer step.** Nudge every weight in the direction that reduces the loss.

Here is a complete worked example: training a small network to approximate the TTP06 sodium current $I_{Na}(V, m, h, j) = G_{Na} \cdot m^3 \cdot h \cdot j \cdot (V - E_{Na})$ from Chapter 5:

```

# The "true" function we want the network to learn
def true_INa(V, m, h, j):
    G_Na = 14.838      # nA/pF
    E_Na = 67.0        # mV
    return G_Na * m**3 * h * j * (V - E_Na)

```



```
# Generate 10,000 random training samples
N = 10000
V = torch.rand(N, 1) * 160 - 100      # V in [-100, 60] mV
m = torch.rand(N, 1)                  # gating variables in [0, 1]
h = torch.rand(N, 1)
j = torch.rand(N, 1)
X = torch.cat([V, m, h, j], dim=1)    # shape [10000, 4]
y = true_INa(V, m, h, j)              # shape [10000, 1]

# Create network, loss function, and optimizer
model = TinyNet(n_in=4, n_hidden=64, n_out=1)
loss_fn = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

# Training loop
for epoch in range(2000):
    y_pred = model(X)                  # 1. forward
    loss = loss_fn(y_pred, y)          # 2. loss
    optimizer.zero_grad()              # clear old gradients
    loss.backward()                    # 3. backward
    optimizer.step()                   # 4. update weights

    if epoch % 500 == 0:
        print(f"Epoch {epoch:4d} Loss: {loss.item():.6f}")
# After ~2000 epochs: loss ≈ 0.001, relative error < 1%
```

△ Key Insight

Surrogates are approximations, not replacements. A trained neural network evaluates in microseconds per batch — much faster than the original formula when that formula involves expensive sub-expressions (like the GHK flux in I_{CaL}). But the surrogate introduces approximation error that accumulates over time. For production simulations where accuracy matters, always validate against the original model.

B.11 Connecting to Engine V5.4

If you have followed sections B.1 through B.10, you now understand every PyTorch concept that Engine V5.4 uses. The engine does not use neural networks or automatic differentiation — it uses PyTorch purely as a **GPU-accelerated numerical backend**. Here is the map from what you just learned to the engine’s source code:

You learned	Section	Engine V5.4 uses it in	What it does there
Tensors, dtypes	B.1–B.4	Everywhere	All state variables (V_m , gating variables, ionic currents) are <code>float64</code> tensors
Element-wise ops	B.2, B.5	Ionic model solvers	Gate updates: $m^{n+1} = m_\infty + (m^n - m_\infty)e^{-\dots}$ — all element-wise
Reductions	B.5	LBM collision step	$V_m = \sum_i f_i$ — summing distribution functions to recover voltage
<code>linspace</code> , <code>clamp</code>	B.5	<code>cardiac_sim/ionic/lut.py</code>	Lookup tables: precompute $m_\infty(V)$ on a fine grid, interpolate at runtime via <code>clamp</code> + indexing
Sparse COO tensors	B.7	<code>cardiac_sim/simulation/classical/discretization_scheme/fdm.py</code>	Stiffness matrix K assembled as COO, implicit matrices $A = \chi C_m I - \frac{1}{2} \Delta t K$

You learned	Section	Engine V5.4 uses it in	What it does there
<code>torch.sparse.mm</code>	B.7	All diffusion solvers	The sparse matrix-vector product $\mathbf{K} \cdot \mathbf{V}$ at every timestep
Device management	B.8	<code>cardiac_sim/utils/backend.py</code>	The <code>Backend</code> class auto-detects GPU/CPU and passes <code>device</code> to all components
<code>torch.no_grad()</code>	B.9	All solver loops	Disables autograd tape — the engine has no learnable parameters so recording wastes memory

Notice what is *missing* from the engine: `nn.Module`, `nn.Linear`, `loss.backward()`, optimizers. Engine V5.4 solves the monodomain and bidomain equations with classical numerical methods (finite differences, finite elements, Crank-Nicolson, LBM). Every coefficient comes from the physics, not from training data. PyTorch is used as a numerical computing library, not a machine-learning framework — and that is perfectly fine. It is, in fact, the cleanest way to get GPU-accelerated sparse linear algebra in Python.

⚙ **Engine V5.4 Connection**

The `Backend` class. Every simulation in Engine V5.4 begins by instantiating `Backend(device='auto')`. This checks `torch.cuda.is_available()` and stores a `torch.device`. A helper function `ensure_tensor(x, device, dtype)` converts NumPy arrays, Python lists, and existing tensors to the correct device and dtype in a single call — this is the most-called utility in the engine. When you see `.to(device)` in the source, you now know exactly what it does.

⚙ **Engine V5.4 Connection**

The `LookupTable` class. The ionic solvers in `cardiac_sim/ionic/lut.py` use `torch.linspace` to create a voltage grid of 16,001 points from -100 to $+60$ mV, then precompute all gate steady-states and time constants on that grid. At each timestep, a single vectorized lookup (using `clamp` + integer indexing + linear interpolation) retrieves all 18 parameters for every node — replacing 4.7 million transcendental function evaluations per timestep with memory reads.

B.12 Where Machine Learning Meets Cardiac Modeling

The neural network in B.10 was a toy example, but the idea of combining machine learning with physics-based simulation is an active and rapidly growing research area. Three directions are particularly relevant to this textbook:

Physics-Informed Neural Networks (PINNs). Instead of training on data alone, a PINN embeds the PDE itself into the loss function. For the monodomain equation (eq. 7.1), the loss penalizes violations of $\chi \mathbf{C}_m \partial \mathbf{V}_m / \partial t = \nabla \cdot (D \nabla \mathbf{V}_m) - \chi \mathbf{I}_{ion}$. The network learns a solution $\mathbf{V}_m(\mathbf{x}, \mathbf{y}, t)$ that approximately satisfies the PDE without ever assembling a stiffness matrix. PINNs are mesh-free and handle irregular geometries naturally, but they struggle with the sharp wavefronts and fast timescales of cardiac action potentials.

Neural ODE surrogates. The most expensive part of a cardiac simulation is the ionic model — eighteen coupled ODEs (TTP06) or forty-one (ORd), evaluated at every node at every timestep. A neural ODE replaces this system with a learned vector field $d\mathbf{s}/dt = \mathbf{f}_\theta(\mathbf{s}, \mathbf{V})$ trained on trajectories from the original model, yielding 10–50× speedups while preserving action-potential morphology to within a few percent.

ML-based classification. Given simulation output (or clinical recordings), machine learning can classify arrhythmia mechanisms, predict reentry vulnerability, or identify optimal pacing sites. The simulation engine generates training data; the ML model learns the map from tissue parameters to clinical outcomes.

Intuition

When to use ML vs. when to solve the equations. Solve the equations when you need quantitative accuracy for a specific case (drug safety, patient-specific planning). Use ML when you need to explore a vast parameter space quickly (sensitivity analysis, real-

time tools) or when the forward model is too expensive to run thousands of times (uncertainty quantification, Bayesian inference). The two approaches are complementary: physics generates ground truth, ML learns to interpolate at a fraction of the cost.

For PyTorch tutorials, see pytorch.org/tutorials. For PINNs in cardiac EP, see Sahli Costabal et al. (2020). For neural ODE surrogates, see Herrero Martin et al. (2022).